



习题1.1

- 将下面的C++程序划分成正确的词素（token）序列。哪些词素应该有相关联的属性值？应该具有什么值？

```
float limitedSquare(x) {float x ;  
/* returns x-squared, but never more than 100 */  
return (x<=-10.01 || x>=10.0)?100:x*x;  
}
```

习题1.1

➤ 解:

- | | |
|--------------------------------------|-----------------------------|
| 1、float <FLOAT, -> | 16、x <IDN, x> |
| 2、limitedSquare <IDN, limitedSquare> | 17、>= <COMPARISON, >= > |
| 3、(<SLP, -> | 18、10.0 <CONST_FLOAT, 10.0> |
| 4、x <IDN, x> | 19、) <SRP, -> |
| 5、) <SRP, -> | 20、? <CONP, ?> |
| 6、{ <LP, -> | 21、100 <CONST_INT, 100> |
| 7、float <FLOAT, -> | 22、: <CONP, :=> |
| 8、x <IDN, x> | 23、x <IDN x> |
| 9、; <SEMI, -> | 24、* <ARITHMETIC, *> |
| 10、return <RETURN, -> | 25、x <IDN, x> |
| 11、(<SLP, -> | 26、; <SEMI, -> |
| 12、x <IDN, x> | 27、} <RP, -> |
| 13、<= <COMPARISON, <= > | |
| 14、-10.01 <CONST_FLOAT, -10.01> | |
| 15、 <LOGICAL, > | |

将14行分成两行，负数分开为两部分，负号和常量



习题1.1

➤ 解：

以下词素具有属性值，有过程名，浮点型变量名，关系运算符，浮点型常量值、整型常量值，算术运算符、条件运算符

limitedSquare <IDN, limitedSquare>

x <IDN, x>

<= <COMPARISON, <=>

-10.01 <CONST_FLOAT, -10.01>

|| <LOGICAL, ||>

>= <COMPARISON, >=>

10.0 <CONST_FLOAT, 10.0>

? <CONP, ?>

100 <CONST_INT, 100>

: <CONP :>

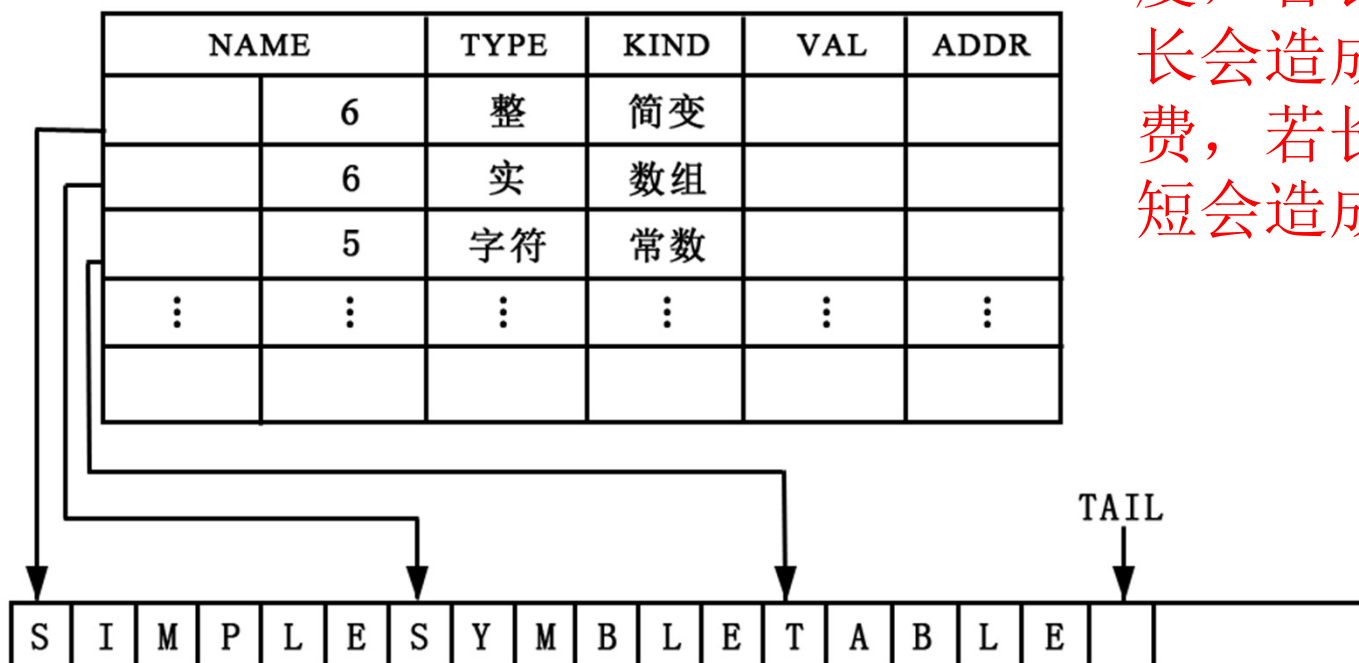
***** <ARITHMETIC, *>



习题1.2

- 符号表中**NAME**字段为什么要设计字符串表这样一种数据结构？而不是把标识符对应的字符串直接存放到**NAME**字段

符号表 (Symbol Table)



如果选择固定长度，若长度设置长会造成空间浪费，若长度设置短会造成溢出



习题2.1

➤ 考虑上下文无关文法：

$$S \rightarrow S S + \mid S S * \mid a$$

以及串 $aa+a^*$ 。

- ① 给出这个串的一个最左推导。
- ② 给出这个串的一个最右推导。
- ③ 给出这个串的一棵语法分析树。
- ④ 该文法生成的语言是什么？
- ⑤ 这个文法是否是二义性的？



习题2.2

➤对下列各文法重复习题2.1

➤ (1) $S \rightarrow 0 S 1 \mid 0 1$

和串 000111

➤ (2) $S \rightarrow + S S \mid * S S \mid a$

和串 $+ * a a a$

➤ (3) $S \rightarrow S (S) S \mid \varepsilon$

和串 $(() ())$

➤ (4) $S \rightarrow a \mid S + S \mid S S \mid S * \mid (S)$

和串 $(a + a) * a$

➤ (5) $S \rightarrow (L) \mid a$ 以及 $L \rightarrow L, S \mid S$ 和串 $((a, a), a, (a))$

➤ (6) $S \rightarrow a S b S \mid b S a S \mid \varepsilon$

和串 aabbab

➤ (7) $E \rightarrow E \text{ or } T \mid T$

$$T \rightarrow T \text{ and } F \mid F$$

$$F \rightarrow \text{not } F \mid (E) \mid \text{true} \mid \text{false} \quad (\text{只需完成第④⑤两题})$$



习题3.1

➤ 写出“注释”的正则定义，并为其设计一个DFA。

注释是/*和*/之间的串，且串中没有不在双引号（“）中的*/（不考虑奇数个双引号，不配对的情况）

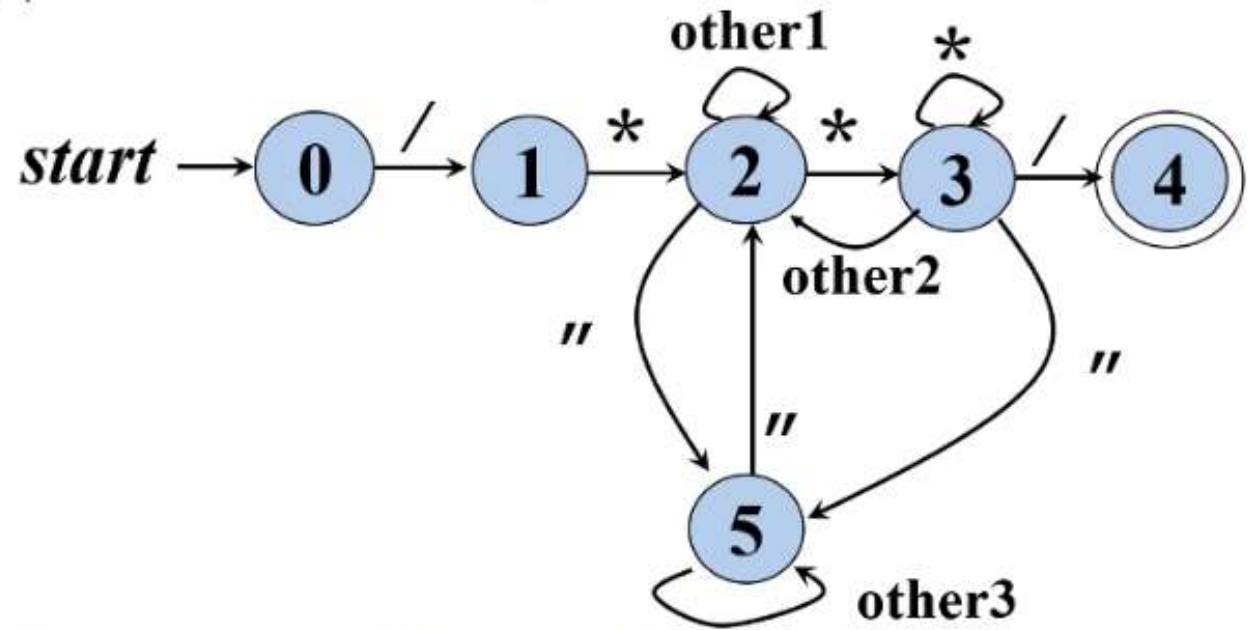
串中没有不在双引号（"）中的*/

可以有在双引号（"）中的*/，即"*/"

$$\underbrace{/* \dots "*/" \dots */}$$

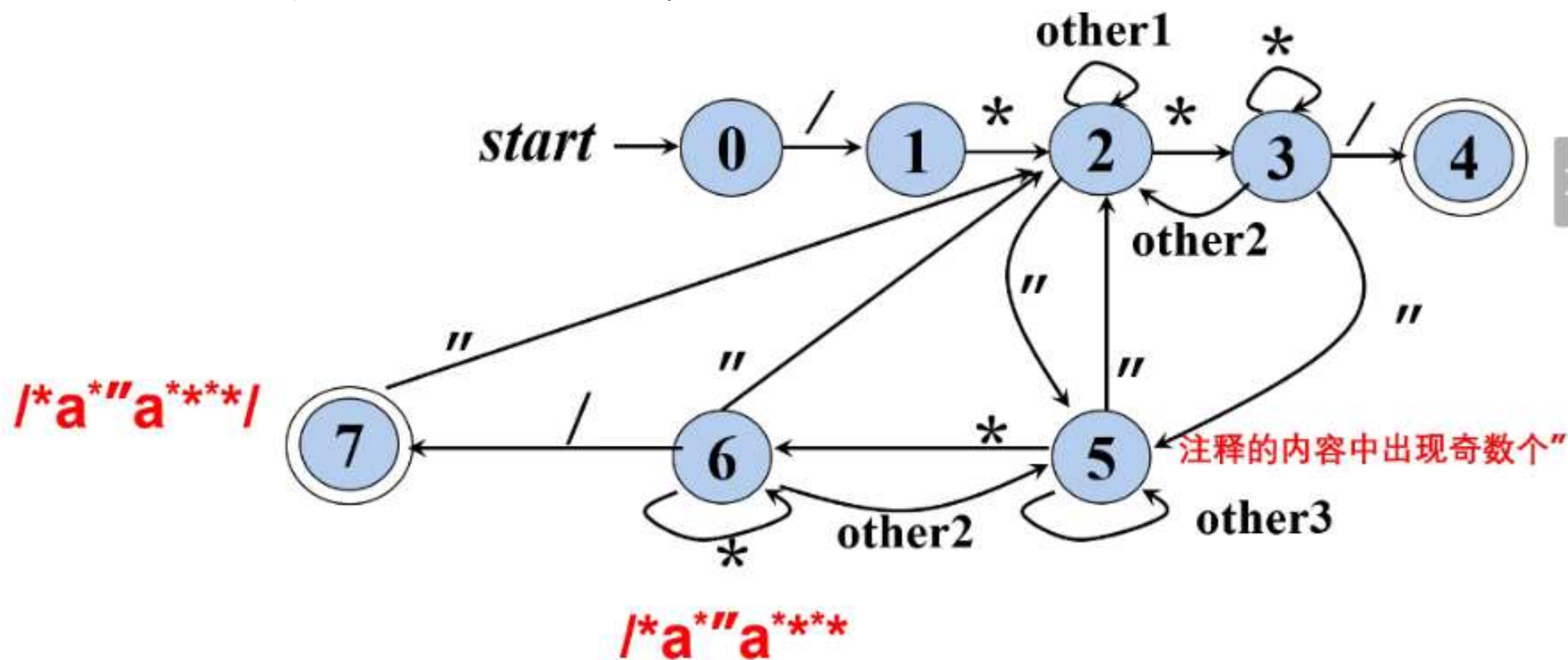


习题3.1



`comment` → `/ * (other1| "other3*"|***other2|***"other3*")** */`

■ 考虑奇数个双引号的情况





习题4.1

➤ 为下面的每一个文法设计一个预测分析器，并给出预测分析表。你可能先要对文法进行提取左公因子或消除左递归的操作。计算各文法的**FOIRST**和**FOLLOW**集合。

➤ (1) $S \rightarrow 0 S 1 \mid 0 1$

➤ (2) $S \rightarrow + S S \mid * S S \mid a$

➤ (3) $S \rightarrow S (S) S \mid \varepsilon$

➤ (4) $S \rightarrow a \mid S + S \mid S S \mid S * \mid (S)$

➤ (5) $S \rightarrow (L) \mid a$ 以及 $L \rightarrow L , S \mid S$

➤ (6) $E \rightarrow E \text{ or } T \mid T$

$T \rightarrow T \text{ and } F \mid F$

$F \rightarrow \text{not } F \mid (E) \mid \text{true} \mid \text{false}$



(1)

1.消除左递归

2.提取左公因子

$$S \rightarrow 0T$$

$$T \rightarrow S1$$

$$T \rightarrow 1$$

5.产生式的SELECT集

$$1.S \rightarrow 0T$$

$$2.T \rightarrow S1$$

$$3.T \rightarrow 1$$

$$FIRST(S) = \{0\}$$

3.计算FIRST集 (FIRST (T) 可以不计算)

$$FIRST(S) = \{0\}$$

$$FIRST(T) = \{1,0\}$$

4.计算FOLLOW集 (都可不计算)

$$FOLLOW(S) = \{\$,1\}$$

$$FOLLOW(T) = \{\$,1\}$$

6.预测分析表

$$SELECT(1) = \{0\}$$

$$SELECT(2) = \{0\}$$

$$SELECT(3) = \{1\}$$

$$S \rightarrow 0T$$

$$T \rightarrow S1$$

$$T \rightarrow 1$$

非终结符	输入		
	0	1	\$
S	$S \rightarrow 0T$		
T	$T \rightarrow S1$	$T \rightarrow 1$	



(2)

■ **First(S)={+,*,a}**

■ **计算过程**

■ S->+SS

■ S->*SS

■ S->a

显然S是一个非终结符，且 $S \rightarrow Y_1 \dots Y_k (k \geq 1)$ ，对于某个 i (这里是1)， $a|+|*$ 在 **FIRST** (Y_1)，其更左边没有符号就把 $a, +, *$ 加入到 **FIRST** (S) 中

即：

First(S)={+,*,a}

■ **Follow(S)={\$,+,* ,a}**

■ **计算过程**

■ S->+SS

■ S->*SS

■ S->a

■ **First(S)={+,*,a}**

由于S是开始符号，故加入{ $\$$ } 进入其**Follow**集。

注意到S->*SS

故而，**First(S) $\subseteq$ Follow(S)**

有：

Follow(S)={\$,+,* ,a}

	First(S)	Fellow(S)
S	+ * a	\$ + * a

■ (1) S->+SS **Select(1)={+}**

■ (2) S->*SS **Select(2)={*}**

■ (3) S->a **Select(3)={a}**



预测分析表

	产生式	Select
S	S->+SS	+
	S->*SS	*
	S->a	a

非终结符	输入符号			
	+	*	a	\$
S	S->+SS	S->*SS	S->a	



(3)

- 1. 观察题目，进行消除左递归
- 2. 式子等价形式：
 - $S \rightarrow S'$
 - $S' \rightarrow (S) S S' \mid \varepsilon$
- 3. 分析follow集：
 - $FOLLOW(S) = \{ \$,), (\}$
 - $FOLLOW(S') = \{ \$,), (\}$
- 4. 分析FIRST集：
 - $FIRST(S) = \{ \varepsilon, (\}$
 - $FIRST(S') = \{ \varepsilon, (\}$
- 5. 求得SELECT集：
 - $SELECT(S \rightarrow S') = \{ \$,), (\}$
 - $SELECT(S' \rightarrow (S) S S') = \{ (\}$
 - $SELECT(S' \rightarrow \varepsilon) = \{ \$,), (\}$

	产生式	SELECT
S	$S \rightarrow S'$	$\$,), ($
S'	$S' \rightarrow (S) S S'$	$($
	$S' \rightarrow \varepsilon$	$\$,), ($

非终结符	输入符号		
	$($	$)$	$\$$
S	$S \rightarrow S'$	$S \rightarrow S'$	$S \rightarrow S'$
S'	$S' \rightarrow (S) S S' \text{ 或 } S' \rightarrow \varepsilon$	$S' \rightarrow \varepsilon$	$S' \rightarrow \varepsilon$



(4)

原式:
 $S \rightarrow a \mid S + S \mid S \underline{S} \mid S^* \mid (S)$

提取左公因子:

$S \rightarrow SA \mid (S) \mid a$
 $A \rightarrow +S \mid S \mid ^*$

提取终结符:

$S \rightarrow SA \mid T$
 $A \rightarrow +S \mid S \mid ^*$
 $T \rightarrow (S) \mid a$

消除直接左递归:

$S \rightarrow TB$
 $B \rightarrow AB \mid \epsilon$
 $A \rightarrow +S \mid S \mid ^*$
 $T \rightarrow (S) \mid a$

消除间接左递归:

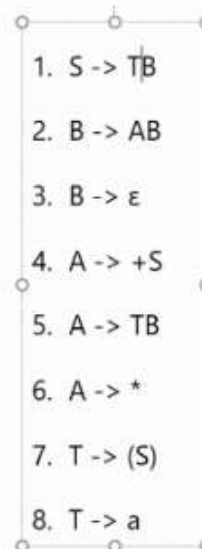
$S \rightarrow TB$
 $B \rightarrow AB \mid \epsilon$
 $A \rightarrow +S \mid TB \mid ^*$
 $T \rightarrow (S) \mid a$

$\text{first}(S) = \text{first}(T) = \{(, a)$
 $\text{first}(B) = \{\epsilon\} \cup \text{first}(A) = \{\epsilon, +, *, (, a)$
 $\text{first}(A) = \{+, *\} \cup \text{first}(T) = \{+, *, (, a)$
 $\text{first}(T) = \{(, a)$

$\text{follow}(S) = \{), \$\} \cup \text{follow}(A)$
 $\text{follow}(B) = \text{follow}(S) \cup \text{follow}(A)$
 $\text{follow}(A) = \text{first}(B) - \{\epsilon\} \cup \text{follow}(B)$
 $\text{follow}(T) = \text{first}(B) - \{\epsilon\} \cup \text{follow}(S) \cup \text{follow}(A)$

$\text{follow}(S) = \{ \$, +, *, (,), a \}$
 $\text{follow}(B) = \{ \$, +, *, (,), a \}$
 $\text{follow}(A) = \{ \$, +, *, (,), a \}$
 $\text{follow}(T) = \{ \$, +, *, (,), a \}$

X	First(x)	Follow(x)
S	(, a	\$, +, *, (,), a
B	$\epsilon, +, *, (, a$	$\$, +, *, (,), a$
A	$+, *, (, a$	$\$, +, *, (,), a$
T	(, a	$\$, +, *, (,), a$



Select(1) = {(, a}
 Select(2) = {+, *, (, a}
 Select(3) = {\$, +, *, (,), a}
 Select(4) = {+}
 Select(5) = {(, a}
 Select(6) = {*}
 Select(7) = {(}
 Select(8) = {a}

非终结符	输入符号					
	a	+	*	(	)	\$
S	$S \rightarrow TB$			$S \rightarrow TB$		
B	$B \rightarrow AB$ $B \rightarrow \epsilon$	$B \rightarrow AB$ $B \rightarrow \epsilon$	$B \rightarrow AB$ $B \rightarrow \epsilon$	$B \rightarrow AB$ $B \rightarrow \epsilon$	$B \rightarrow \epsilon$	$B \rightarrow \epsilon$
A	$A \rightarrow TB$	$A \rightarrow +S$	$A \rightarrow ^*$	$A \rightarrow TB$		
T	$T \rightarrow a$			$T \rightarrow (S)$		



(5)

➤ $S \rightarrow (L) \mid a$

$L \Rightarrow L, S$
 $\Rightarrow L, S, S$

➤ $L \rightarrow L, S \mid S$

➤ 消除 L -产生式的直接左递归，得：

$L \rightarrow SL'$
 $L' \rightarrow ,SL' \mid \epsilon$

$S \rightarrow (L) \mid a$
 $L \rightarrow SL'$
 $L' \rightarrow ,SL' \mid \epsilon$

$\text{FIRST}(S) = \{ (, a \}$
 $\text{FIRST}(L) = \{ (, a \}$
 $\text{FIRST}(L') = \{ , , \epsilon \}$

$\text{FOLLOW}(S) = \{) , , , \$ \}$
 $\text{FOLLOW}(L) = \{) \}$
 $\text{FOLLOW}(L') = \{) \}$

	产生式	SELECT
S	$S \rightarrow (L)$	(
	a	a
L	$L \rightarrow SL'$	(a
L'	$L' \rightarrow , SL'$	,
	$L' \rightarrow \epsilon$	)

预测分析表

非终结符	输入符号				
	(	)	a	,	\$
S	$S \rightarrow (L)$		$S \rightarrow a$		
L	$L \rightarrow SL'$		$L \rightarrow SL'$		
L'		$L' \rightarrow \epsilon$		$L' \rightarrow , SL'$	



(6)

消除左递归

① $E \rightarrow TE'$

② $E' \rightarrow or TE' \mid \varepsilon$

③ $T \rightarrow FT'$

④ $T' \rightarrow and FT' \mid \varepsilon$

⑤ $F \rightarrow not F \mid (E) \mid true \mid false$

■ $FIRST(E) = \{not (true false\}$

■ $FIRST(E') = \{or \ \varepsilon\}$

■ $FIRST(T) = \{not (true false\}$

■ $FIRST(T') = \{and \ \varepsilon\}$

■ $FIRST(F) = \{not (true false\}$

■ $FOLLOW(E) = \{ \$) \}$

■ $FOLLOW(E') = \{ \$) \}$

■ $FOLLOW(T) = \{ or \$) \}$

■ $FOLLOW(T') = \{ or \$) \}$

■ $FOLLOW(F) = \{ and or \$) \}$

	产生式	SELECT
E	$E \rightarrow TE'$	not (true false
E'	$E' \rightarrow or TE'$	or
	$E' \rightarrow \varepsilon$	\$)
T	$T \rightarrow FT'$	not (true false
T'	$T' \rightarrow and FT'$	and
	$T' \rightarrow \varepsilon$	or) \$
F	$F \rightarrow not F$	not
	$F \rightarrow (E)$	(
	$F \rightarrow true$	true
	$F \rightarrow false$	false

非终结符	输入符号							
	or	and	not	(	)	true	false	\$
E			$E \rightarrow TE'$	$E \rightarrow TE'$		$E \rightarrow TE'$	$E \rightarrow TE'$	
E'	$E' \rightarrow or TE'$				$E' \rightarrow \varepsilon$			$E' \rightarrow \varepsilon$
T			$T \rightarrow FT'$	$T \rightarrow FT'$		$T \rightarrow FT'$	$T \rightarrow FT'$	
T'	$T' \rightarrow \varepsilon$	$T' \rightarrow and FT'$			$T' \rightarrow \varepsilon$			$T' \rightarrow \varepsilon$
F			$F \rightarrow not F$	$F \rightarrow (E)$		$F \rightarrow true$	$F \rightarrow false$	



习题4.2

- 为下面的文法设计一个预测分析器，并给出预测分析表。你可能先要对文法进行提取左公因子或消除左递归的操作。计算文法的**FOIRST**和**FOLLOW**集合。

$$S \rightarrow S S + \mid S S * \mid a$$



$S \rightarrow a S'$ $\text{FIRST}(S)=\{ a \}$ $\text{FOLLOW}(S)=\{ \$ + * \}$
 $S' \rightarrow S S'' \mid \varepsilon$ $\text{FIRST}(S')=\{ a \varepsilon \}$ $\text{FOLLOW}(S')=\{ \$ + * \}$
 $S'' \rightarrow + S' \mid * S'$ $\text{FIRST}(S'')=\{ + * \}$ $\text{FOLLOW}(S'')=\{ \$ + * \}$

(1) $S \rightarrow a S'$ $\text{SELECT}(1)=\{ a \}$
(2) $S' \rightarrow S S''$ $\text{SELECT}(2)=\{ a \}$
(3) $S' \rightarrow \varepsilon$ $\text{SELECT}(3)=\{ \$ + * \}$
(4) $S'' \rightarrow + S'$ $\text{SELECT}(4)=\{ + \}$
(3) $S'' \rightarrow * S'$ $\text{SELECT}(3)=\{ * \}$

非终结符	输入符号			
	a	+	*	\$
S	$S \rightarrow a S'$			
S'	$S' \rightarrow S S''$	$S' \rightarrow \varepsilon$	$S' \rightarrow \varepsilon$	$S' \rightarrow \varepsilon$
S''		$S'' \rightarrow + S'$	$S'' \rightarrow * S'$	



习题5.1

- 为下列文法构造递归下降语法分析器（参见SPOC讲义“第4章 语法分析 - 上.pdf” 第42~48页）
 - (1) $S \rightarrow + S S \mid - S S \mid a$
 - (2) $S \rightarrow S (S) S \mid \varepsilon$
 - (3) $S \rightarrow 0 S 1 \mid 0 1$



(1)

- $S \rightarrow +SS$
 - $S \rightarrow -SS$
 - $S \rightarrow a$
- $FIRST(S) = \{+, -, a\}$
- $FOLLOW(S) = \{+, -, a, \$\}$

	产生式	SELECT
S	$S \rightarrow +SS$	+
	$S \rightarrow -SS$	-
	$S \rightarrow a$	a

预测分析表

非终结符	输入符号		
	+	-	a
S	$S \rightarrow +SS$	$S \rightarrow -SS$	$S \rightarrow a$

```
void S(TOKEN){
    if(TOKEN == 'a') GETNEXT(TOKEN);
    else if(TOKEN == '+' or TOKEN == '-'){
        GETNEXT(TOKEN);
        S(TOKEN);
        GETNEXT(TOKEN);
        S(TOKEN);
        GETNEXT(TOKEN);
    }
    else error;
    // if(TOKEN != '$') error;
}

int main(){
    GETNEXT(TOKEN);
    S(TOKEN);
    // GETNEXT(TOKEN);
    if (TOKEN != '$') ERROR;
}
```




(2)

$S \rightarrow S'$

$S' \rightarrow (S)SS' | \epsilon$

- 求每个变量的**FIRST**集和**FOLLOW**集

FIRST(S) = {(, ϵ }

FOLLOW(S) = {\$,), { }

FIRST(S') = {(, ϵ }

FOLLOW(S') = {\$,), { }

- 求得每个候选式的**SELECT**集

SELECT ($S \rightarrow S'$) = {\$,), { }

SELECT ($S' \rightarrow (S)SS'$) = {(}

SELECT ($S' \rightarrow \epsilon$) = {\$,), { }

构建预测分析表

非终结符	产生式	SELECT
S	$S \rightarrow S'$	\$(,)
S'	$S' \rightarrow (S)SS'$	(
	$S' \rightarrow \epsilon$	\$(,)

非终结符	输入符号		
	(	)	\$
S	$S \rightarrow S'$	$S \rightarrow S'$	$S \rightarrow S'$
S'	$S' \rightarrow (S)SS'$ or $S' \rightarrow \epsilon$	$S' \rightarrow \epsilon$	$S' \rightarrow \epsilon$

不是LL(1)文法，不可以采用递归的预测分析，
无法构造分析器



(3) 分析器代码略有问题

	产生式	<i>SELECT</i>
<i>S</i>	$S \rightarrow 0T$	0
<i>T</i>	$T \rightarrow S1$	0
	$T \rightarrow 1$	1

非终结符	输入符号	
	0	1
<i>S</i>	$S \rightarrow 0T$	
<i>T</i>	$T \rightarrow S1$	$T \rightarrow 1$

```
program DESCENT
begin
    GETNEXT(TOKEN);
    S(TOKEN);
    GETNEXT(TOKEN);
    if TOKEN ≠ '$' then ERROR;
end
procedure S(TOKEN);
begin
    if TOKEN ≠ '0' then ERROR;
    GETNEXT(TOKEN);
    T(TOKEN);
end
procedure T(TOKEN);
begin
    if TOKEN = '0'
    then
        S(TOKEN);
        GETNEXT(TOKEN);
        if TOKEN ≠ '1' then ERROR;
    end
    else if TOKEN = '1'
    then end
    else if TOKEN ≠ '1' then ERROR;
end
```



习题6.1

➤ 对于文法 $S \rightarrow 0 S 1 \mid 0 1$ ，指出下面各个最右句型的句柄：

➤ (1) 000111

➤ (2) 00S11

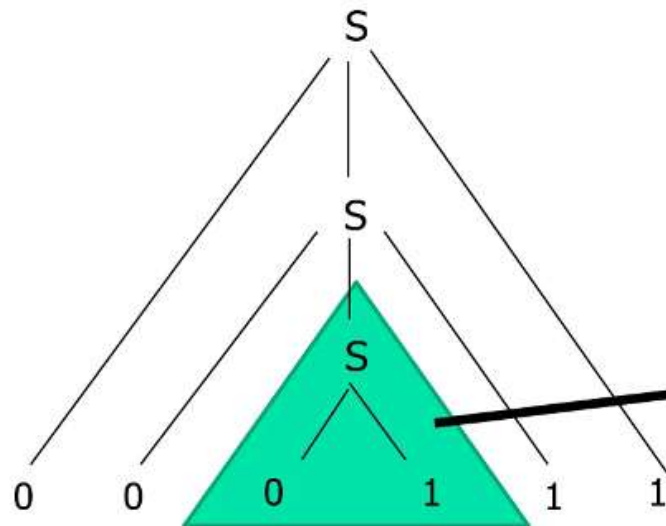


(1)



构造分析树，选取最左直接短语

$S \rightarrow 0S1 \mid 01$



栈

\$
\$ 0
\$ 00
\$ 000
\$ 0001
\$ 00S
\$ 00S1
\$ 0S
\$ 0S1
\$

剩余输入

000111\$
00111\$
0111\$
111\$
11\$
11\$
1\$
1\$
\$
\$

动作

移入
移入
归约
移入
规约
移入
规约
移入
归约

句柄为 01



(2)

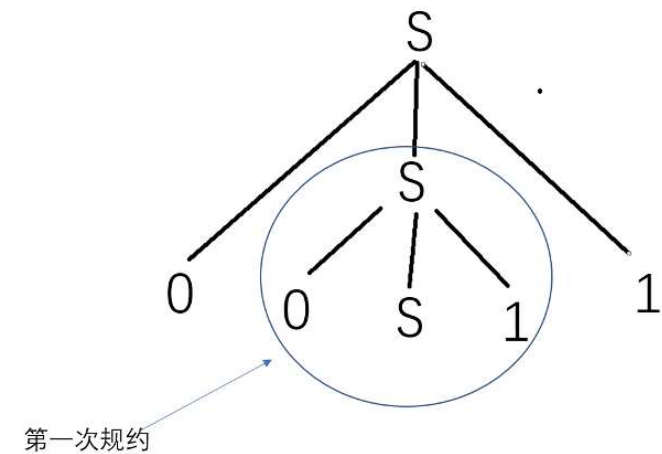
文法:

R1: $S \rightarrow 0S1 \mid 01$

栈	剩余输入	动作
\$	00S11\$	移入
\$0	0S11\$	移入
\$00	S11\$	移入
\$00S	11\$	移入
\$00S1	1\$	归约R1
\$0S	1\$	移入
\$0S1	\$	规约R1
\$S	\$	完成

对应的语法树

结论: 最左直接短语: 0S1, 即是句柄。





习题6.2

➤ 对于文法 $S \rightarrow SS+ | SS* | a$ ，指出下面各个最右句型的句柄：

➤ (1) $SSS+a*+$

➤ (2) $SS+a*a+$

➤ (3) $aaa*a++$



(1)

■ 移入-归约分析: $SSS+a^*+$

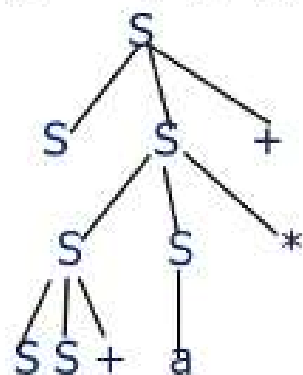
文法

$S \rightarrow SS+$

$S \rightarrow SS^*$

$S \rightarrow a$

语法分析树



栈	剩余输入	动作
\$	SSS+a*+\$	
\$ \$	SS+a*+\$	移入
\$ \$ \$	S+a*+\$	移入
\$ \$ \$ \$	+a*+\$	移入
\$ \$ \$ \$ +	a*+\$	移入
\$ \$ \$	a*+\$	归约: $S \rightarrow SS+$
\$ \$ \$ a	*+\$	移入
\$ \$ \$ \$	*+\$	归约: $S \rightarrow a$
\$ \$ \$ \$ *	+\$	移入
\$ \$ \$	+\$	归约: $S \rightarrow SS^*$
\$ \$ \$ +	\$	移入
\$ \$	\$	归约: $S \rightarrow SS+$

根据语法树可以看出最右句型的句柄为 $SS+$ 。



(2)



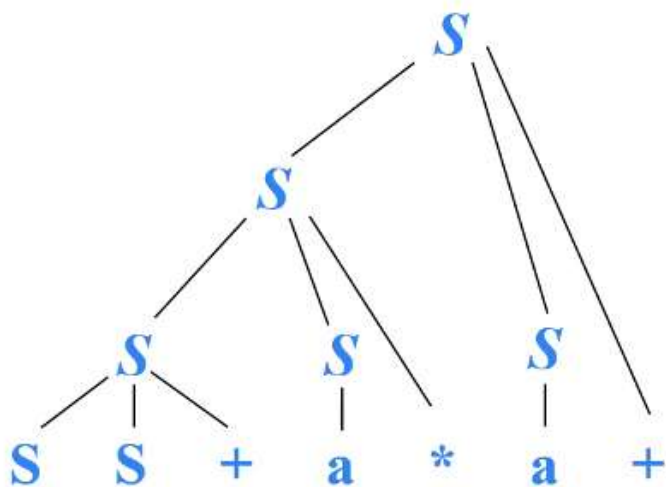
移入-归约分析

文法

① $S \rightarrow SS+$

② $S \rightarrow SS^*$

③ $S \rightarrow a$



栈

\$ SS

\$ SS+

\$ S

\$ Sa

\$ SS

\$ SS *

\$ S

\$ Sa

\$ SS

\$ SS+

\$ S

剩余输入

+a*a+\$

a*a+\$

a*a+\$

*a+\$

*a+\$

a+\$

a+\$

+\$

+\$

\$

\$

动作

移入

归约: $S \rightarrow SS+$

移入

归约: $S \rightarrow a$

移入

归约: $S \rightarrow SS^*$

移入

归约: $S \rightarrow a$

移入

归约: $S \rightarrow SS+$

(3)

栈	剩余输入	动作
\$	aaa*a++\$	
\$a	aa*a++\$	移入
\$S	aa*a++\$	归约
\$Sa	a*a++\$	移入
\$SS	a*a++\$	归约
\$SSa	*a++\$	移入
\$SSS	*a++\$	归约
\$SSS*	a++\$	移入
\$SS	a++\$	归约
\$SSa	++\$	移入
\$SSS	++\$	归约
\$SSS+	+\$	移入
\$SS	+\$	归约
\$SS+	\$	移入
\$S	\$	归约

短语:

- a
- aa*
- aa*a+
- aaa*a++

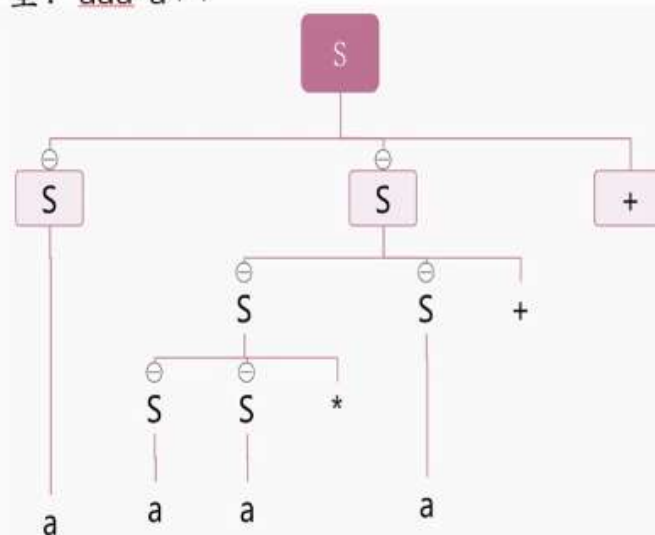
直接短语:

- a

句柄:

- a

句型: aaa*a++



定义: 一个句型的最左直接短语被称为该句型的句柄



习题6.3

- 我们可以根据语法分析栈中的**LR**状态来推断出这个状态表示了什么文法符号。我们如何推导出这个信息？

通过分析表推导出的，

ACTION: 状态这一列的终结符

GOTO: 状态这一列的非终结符

例：只有s...算，r...不算

状态3表示的是a，状态4表示的是b

状态1表示的是S

状态2，5，6表示的是B

*LR(0)*分析表

状态	ACTION			GOTO	
	a	b	\$	S	B
0	s3	s4		1	2
1			acc		
2	s3	s4			5
3	s3	s4			6
4	r3	r3	r3		
5	r1	r1	r1		
6	r2	r2	r2		



习题7.1

➤ 为（增广）文法

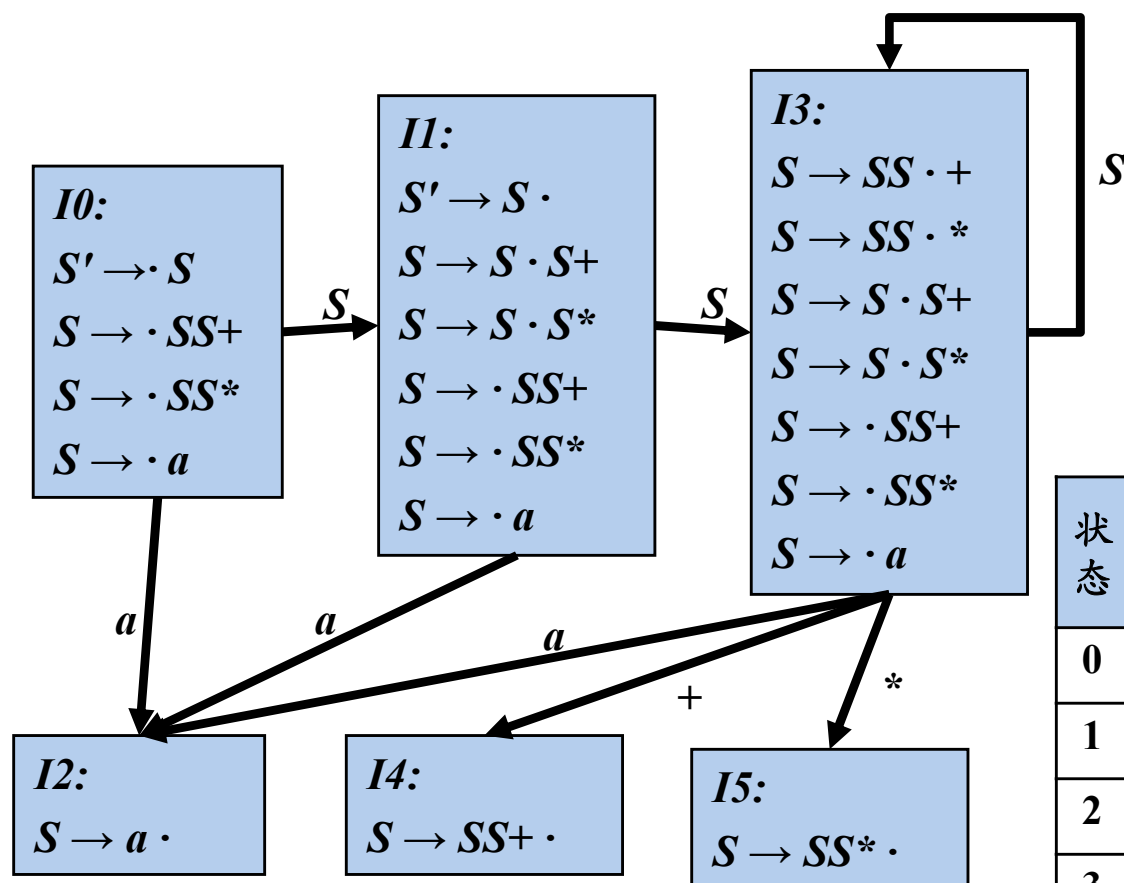
$$S \rightarrow S S + \mid S S * \mid a$$

构造SLR项集。计算这些项集的GOTO函数。给出这个文法的语法分析表。这个文法是SLR文法吗？如果是，利用得到的分析表，给出处理输入 $aa*a+$ 时的各个动作。

文法:

- (0) $S' \rightarrow S$
- (1) $S \rightarrow SS+$
- (2) $S \rightarrow SS^*$
- (3) $S \rightarrow a$

X	$FOLLOW(X)$
S'	$\$$
S	$+, *, \$, a$



结论: 分析表中无冲突动作, 为**SLR**文法

状态	ACTION				GOTO
	a	$*$	$+$	$\$$	
0	s2				1
1	s2			acc	3
2	r3	r3	r3	r3	
3	s2	s5	s4		3
4	r1	r1	r1	r1	
5	r2	r2	r2	r2	

文法:

(0) $S' \rightarrow S$

(1) $S \rightarrow SS+$

(2) $S \rightarrow SS^*$

(3) $S \rightarrow a$

X	$FOLLOW(X)$
S'	$\$$
S	$+, *, \$, a$

状态	$ACTION$				$GOTO$
	a	$*$	$+$	$\$$	S
0	s2				1
1	s2			acc	3
2	r3	r3	r3	r3	
3	s2	s5	s4		3
4	r1	r1	r1	r1	
5	r2	r2	r2	r2	

栈	剩余输入	动作
0 \$	aa*a+\$	
02 \$a	a*a+\$	移入a
0 \$\$	a*a+\$	$S \rightarrow a$
01 \$\$	a*a+\$	
012 \$\$a	*a+\$	移入a
01 \$\$\$	*a+\$	$S \rightarrow a$
013 \$\$\$	*a+\$	
0135 \$\$\$*	a+\$	移入*
0 \$\$	a+\$	$S \rightarrow SS^*$

栈	剩余输入	动作
01 \$\$	a+\$	
012 \$\$a	+\$	移入a
01 \$\$\$	+\$	$S \rightarrow a$
013 \$\$\$	+\$	
0134 \$\$\$+	\$	移入+
0 \$\$	\$	$S \rightarrow SS+$
01 \$\$	\$	



习题7.2

➤ 对于下列各（增广）文法：

① 构造SLR项集和它们的GOTO函数。

② 指出你的项集中有没有动作冲突。

③ 如果存在SLR语法分析表，构造出这个语法分析表。

➤ (1) $S \rightarrow 0 S 1 \mid 0 1$

输入样例：000111

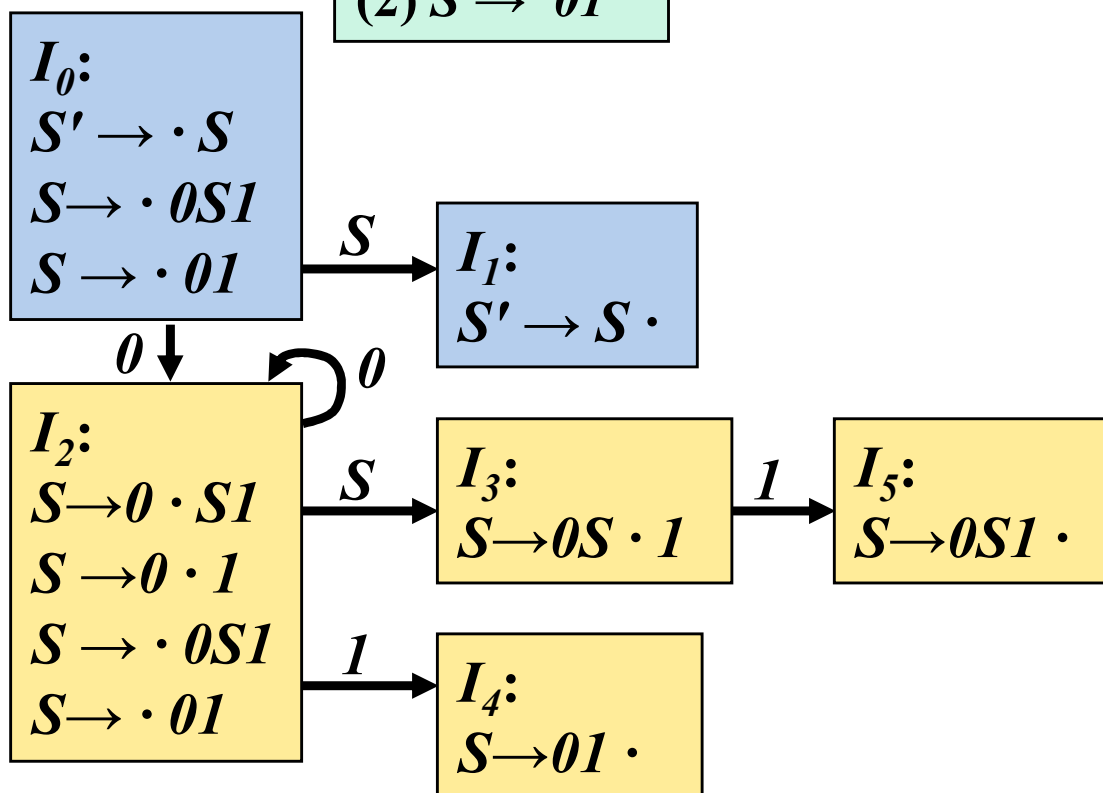
例

文法

- (0) $S' \rightarrow S$
- (1) $S \rightarrow 0S1$
- (2) $S \rightarrow 01$

$FOLLOW(S) = \{ \$, 1 \}$

*SLR*分析表



状态	ACTION			GOTO
	0	1	\$	
0	s2			1
1			acc	
2	s2	s4		3
3		s5		
4		r2	r2	
5		r1	r1	



栈 剩余输入

0

\$

000111\$

0 2

\$ 0

00111\$

0 2 2

\$ 0 0

0111\$

S
/ \

0 0 0 1 1 1

0 2 2 2

\$ 0 0 0

111\$

0 2 2 2 4

\$ 0 0 0 1

11\$

0 2 2

\$ 0 0 S

11\$

$S \rightarrow 01$

0 2 2 3

\$ 0 0 S

11\$

0 2 2 3 5

\$ 0 0 S 1

1\$

0 2

\$ 0 S

1\$

$S \rightarrow 0S1$

0 2 3

\$ 0 S

1\$

S
|
S
/ \

0 0 0 1 1 1

0 2 3 5

\$ 0 S 1

\$

0

\$ S

\$

$S \rightarrow 0S1$

0 1

\$ S

\$

0 1

\$ S

\$

acc

S
|
S
|
S
/ \

0 0 0 1 1 1



习题7.2

➤ 对于下列各（增广）文法：

① 构造SLR项集和它们的GOTO函数。

② 指出你的项集中有没有动作冲突。

③ 如果存在SLR语法分析表，构造出这个语法分析表。

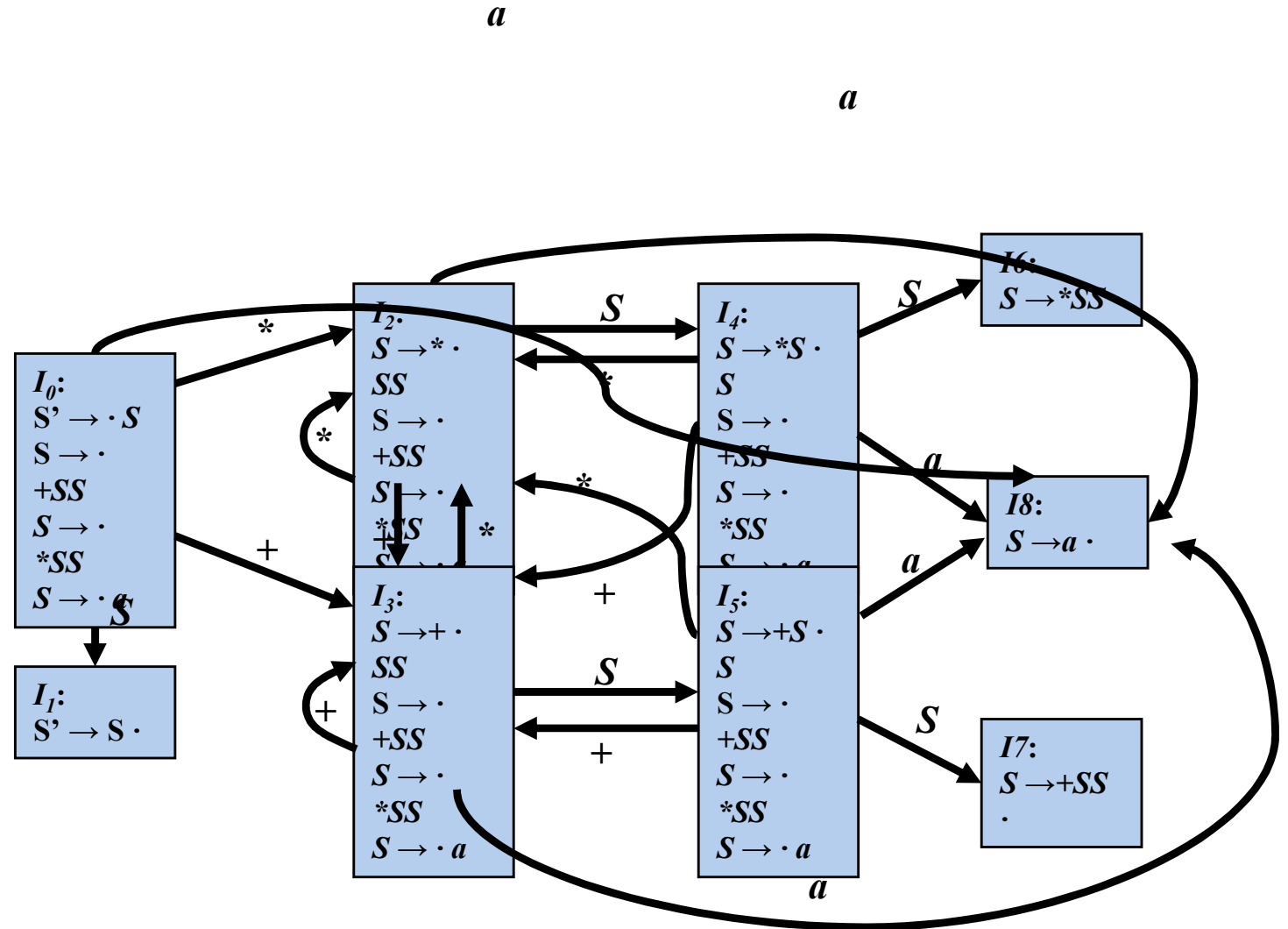
➤ (2) $S \rightarrow + S S \mid * S S \mid a$ 输入样例：+ * a a a



SLR分析

文法:

- (0) $S' \rightarrow S$
- (1) $S \rightarrow +SS$
- (2) $S \rightarrow *SS$
- (3) $S \rightarrow a$





SLR语法分析表

状态	ACTION				GOTO
	+	*	a	\$	S
0	s3	s2	s8		1
1				acc	
2	s3	s2	s8		4
3	s3	s2	s8		5
4	s3	s2	s8		6
5	s3	s2	s8		7
6	r2	r2	r2	r2	
7	r1	r1	r1	r1	
8	r3	r3	r3	r3	

栈

剩余输入

0
\$ +*aaa\$

03
\$+ *aaa\$

0328
\$+*a aa\$

0324
\$+*S aa\$

03248
\$+*Sa a\$

03246
\$+*SS a\$

0358
\$+Sa \$

0357
\$+SS \$

01
\$\$ \$



习题7.2

➤ 对于下列各（增广）文法：

① 构造SLR项集和它们的GOTO函数。

② 指出你的项集中有没有动作冲突。

③ 如果存在SLR语法分析表，构造出这个语法分析表。

➤ (3) $S \rightarrow S(S)S \mid \varepsilon$

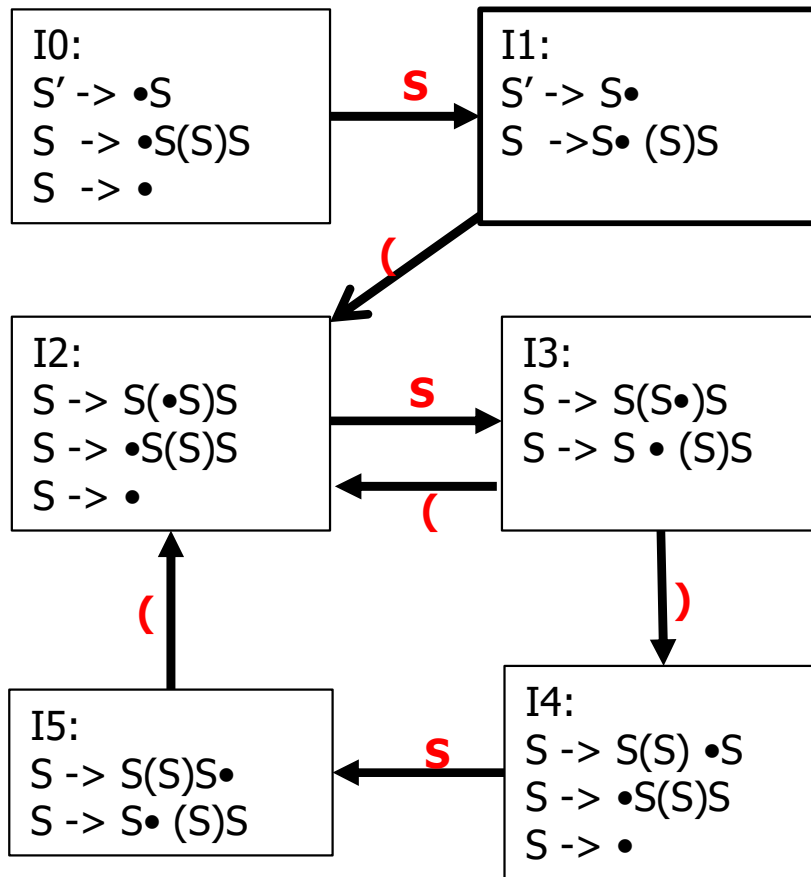
输入样例：(())



$\text{follow}(S) = \{ (,), \$ \}$

$\text{follow}(S') = \{ \$ \}$

自动机



文法: (0) $S' \rightarrow S$

(1) $S \rightarrow S(S)S$

(2) $S \rightarrow \epsilon$

语法分析表

	ACTION			GOTO
	(	)	\$	
				S
0	r2	r2	r2	1
1	s2		acc	
2	r2	r2	r2	3
3	s2	s4		
4	r2	r2	r2	5
5	s2/r1	r1	r1	

发现存在移入-归约冲突



习题7.2

➤ 对于下列各（增广）文法：

① 构造SLR项集和它们的GOTO函数。

② 指出你的项集中有没有动作冲突。

③ 如果存在SLR语法分析表，构造出这个语法分析表。

➤ (4) $S \rightarrow a \mid S + S \mid S S \mid S * \mid (S)$ 输入样例： $(a + a) * a$



问题求解

➤ 将产生式变换为增广文法，并求出各非终结符的Follow集

文法

(0) $S' \rightarrow S$

(1) $S \rightarrow a$

(2) $S \rightarrow S+S$

(3) $S \rightarrow SS$

(4) $S \rightarrow S^*$

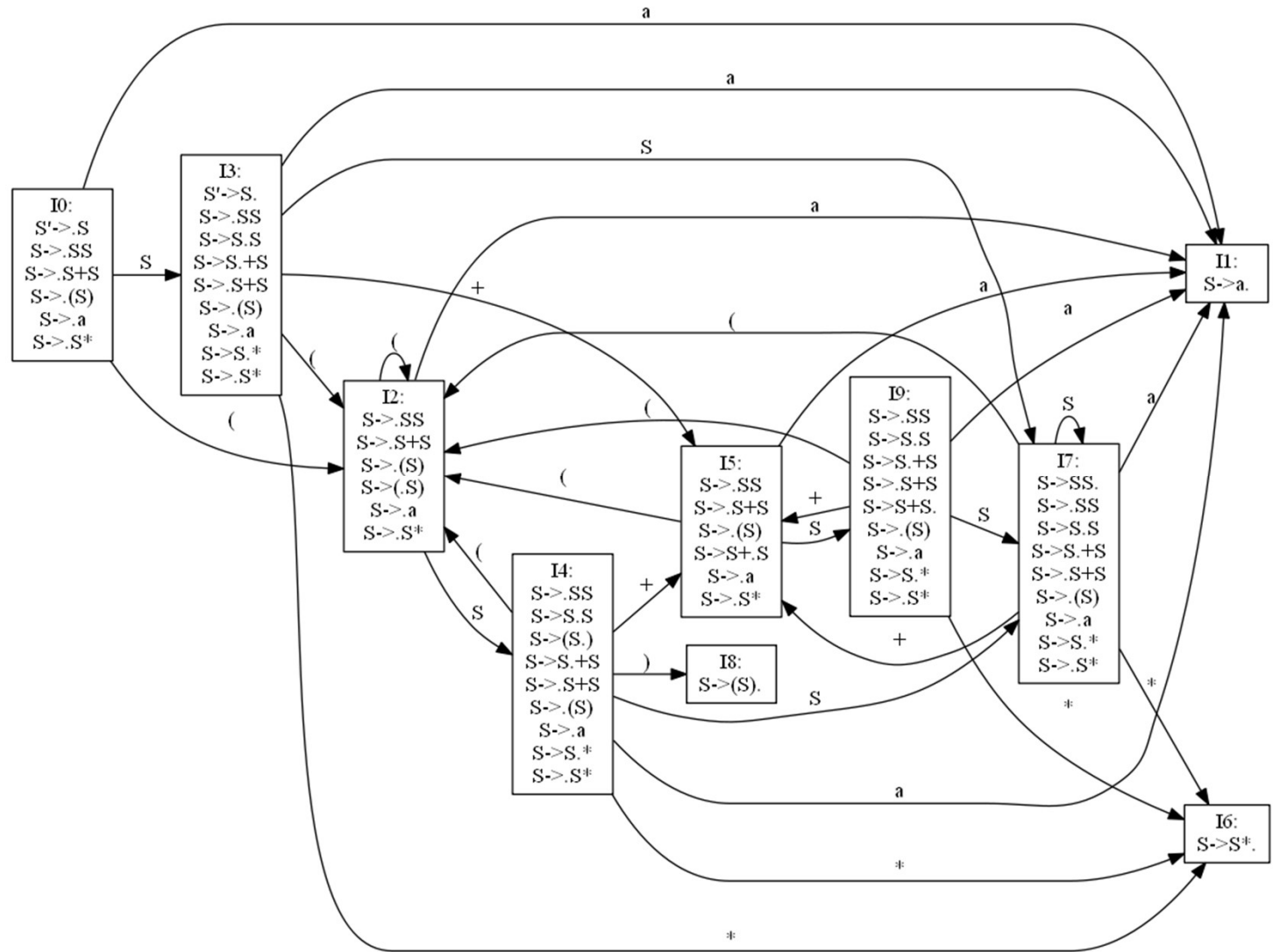
(5) $S \rightarrow (S)$

$\text{Follow}(S) = \{\$, a, +, *, (,)\}$



问题求解

➤ 文法的LR(0) 自动机





获得语法分析表

语法分析表							
状态	ACTION						GOTO
	<i>a</i>	+	*	(	)	\$	<i>S</i>
0	s1			s2			3
1	r1	r1	r1	r1	r1	r1	
2	s1			s2			4
3	s1	s5	s6	s2		acc	7
4	s1	s5	s6	s2	s8		7
5	s1			s2			9
6	r4	r4	r4	r4	r4	r4	
7	s1 r3	s5 r3	s6 r3	s2 r3	r3	r3	7
8	r5	r5	r5	r5	r5	r5	
9	s1 r2	s5 r2	s6 r2	s2 r2	r2	r2	7

冲突



习题7.2

➤ 对于下列各（增广）文法：

① 构造SLR项集和它们的GOTO函数。

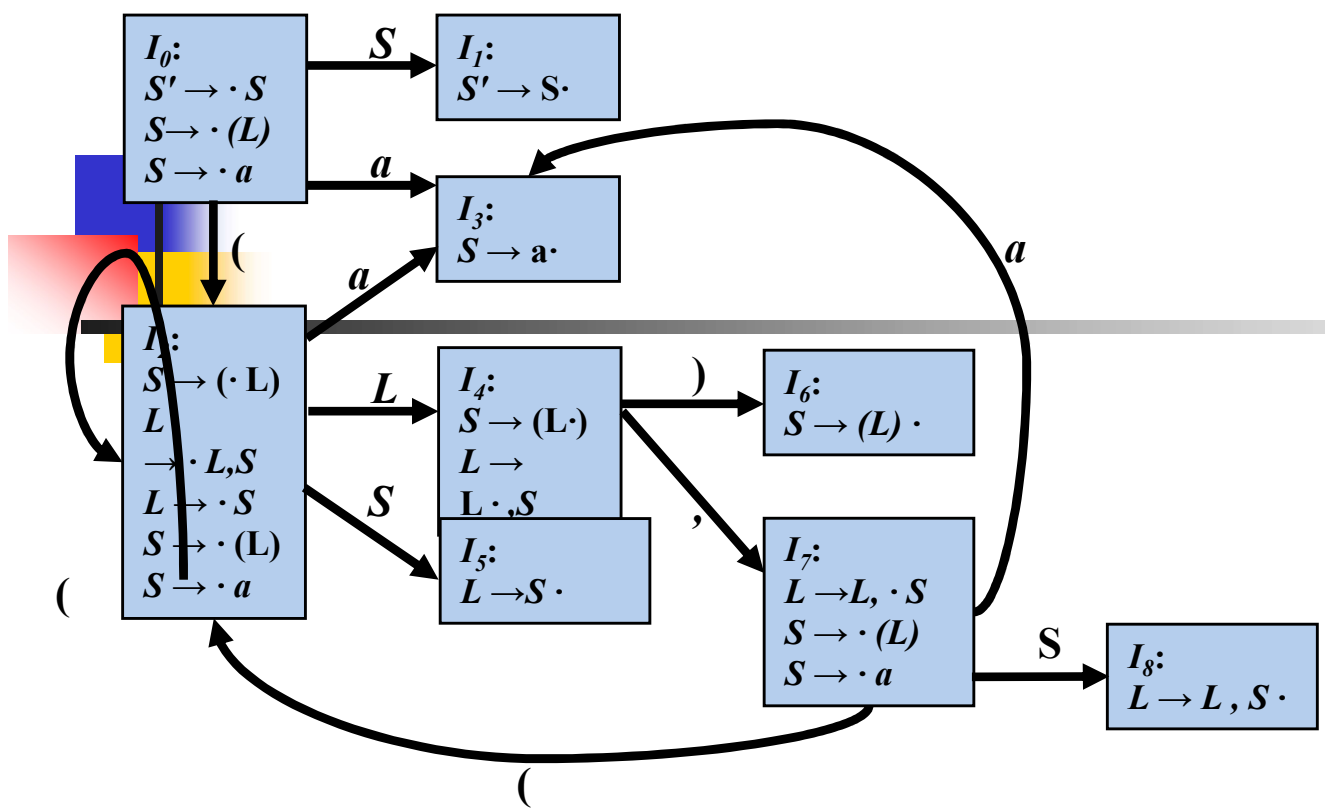
② 指出你的项集中有没有动作冲突。

③ 如果存在SLR语法分析表，构造出这个语法分析表。

➤ (5) $S \rightarrow (L) \mid a$ 以及 $L \rightarrow L, S \mid S$

输入样例：((a, a), a, (a))

2. SLR项集以及GOTO函数



文法:
(0) $S' \rightarrow S$
(1) $S \rightarrow (L)$
(2) $S \rightarrow a$
(3) $L \rightarrow L, S$
(4) $L \rightarrow S$

X	$FOLLOW(X)$
S'	$\$$
S	$\$,)$
L	$),$

X	$FIRST(X)$
S'	$(a$
S	$(a$
L	$(a$

没有动作冲突



3. SLR语法分析表

输入样例: ((a,a),a,(a))

0 0 2 2 5
\$ ((a,a),a,(a))\$ ((S
0 2 0 2 2
\$((a,a),a,(a))\$\$((L
0 2 2 0 2 2 4
\$((a,a),a,(a))\$ \$((L
0 2 2 3 0 2 2 3 7
\$((a ,a),a,(a))\$ \$((L,
0 2 2 0 2 2 3 7 3
\$((S ,a),a,(a))\$ \$((L, a),a,(a))\$
0 2 2 5 0 2 2 3 7
\$((S ,a),a,(a))\$ \$((L, S),a,(a))\$

状态	ACTION					GOTO	
	(	)	,	a	\$	S	L
0	s2			s3		1	
1					acc		
2	s2			s3		5	4
3		r2	r2		r2		
4		s6	s7				
5		r4	r4				
6		r1	r1		r1		
7	s2			s3		8	
8		r3	r3				

文法:
(0) $S' \rightarrow S$
(1) $S \rightarrow (L)$
(2) $S \rightarrow a$
(3) $L \rightarrow L, S$
(4) $L \rightarrow S$



0 2 2 3 7
\$((L , S) , a , (a))\$

0 2 2 3 7 8
\$((L , S) , a , (a))\$

0 2 2 4
\$((L) , a , (a))\$

0 2 2 4 6
\$((L) , a , (a))\$

0 2
\$(S , a , (a))\$

0 2 5
\$(S , a , (a))\$

0 2 4 7
\$(L , S , (a))\$

0 2 4 7 8
\$(L , S , (a))\$

0 2
\$(L , (a))\$

0 2 4
\$(L , (a))\$

0 2 4 7
\$(L , (a))\$

0 2 4 7 2
\$(L , (a))\$

0 2 5
\$(S , a , (a))\$

0 2
\$(L , a , (a))\$

0 2 4
\$(L , a , (a))\$

0 2 4 7
\$(L , a , (a))\$

0 2 4 7 3
\$(L , a , (a))\$

0 2 4 7
\$(L , S , (a))\$

0 2 4 7 2
\$(L , (a))\$

0 2 4 7 2 3
\$(L , (a))\$

0 2 4 7 2
\$(L , (S))\$

0 2 4 7 2 5
\$(L , (S))\$

0 2 4 7 2
\$(L , (L))\$

0 2 4 7 2 4
\$(L , (L))\$

0 2 4 7 2 4
\$(L , (L))\$

0 2 4 7 2 4 6
\$(L , (L))\$

0 2 4 7
\$(L , S)\$

0 2 4 7 8
\$(L , S)\$

0 2
\$(L)\$

0 2 4
\$(L)\$

0 2 4
\$(L)\$

0 2 4 6
\$(L)\$

0
\$ S\$

0 1
\$ S\$

0 1
\$ acc\$



习题7.2

➤ 对于下列各（增广）文法：

① 构造SLR项集和它们的GOTO函数。

② 指出你的项集中有没有动作冲突。

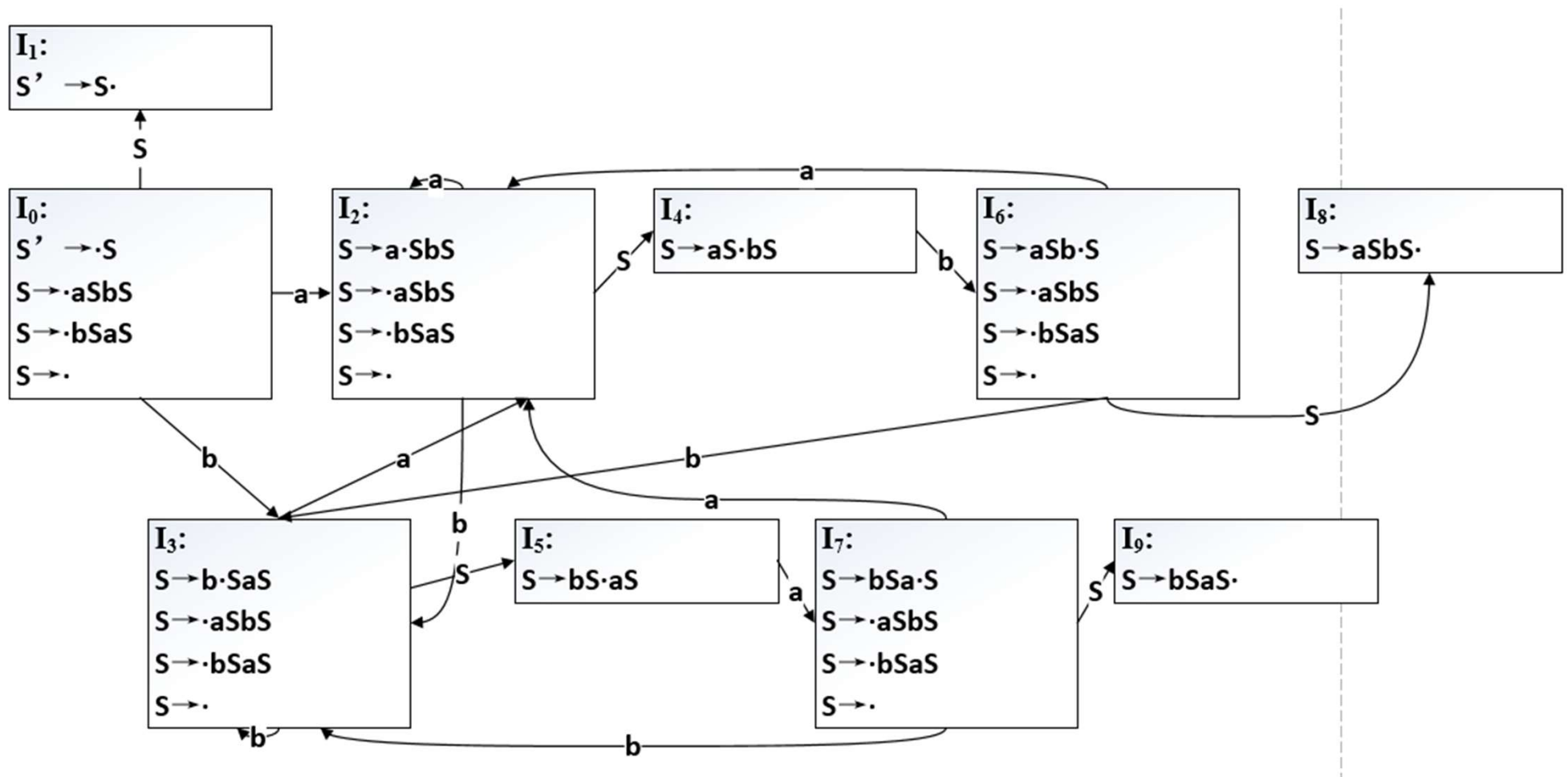
③ 如果存在SLR语法分析表，构造出这个语法分析表。

➤ (6) $S \rightarrow a S b S \mid b S a S \mid \epsilon$

输入样例：aabbab



➤ 项目集和GOTO函数





存在动作冲突

$\text{FOLLOW}(S') = \{\$\}$

$\text{FOLLOW}(S) = \{a, b, \$\}$

I_0, I_2, I_3, I_6, I_7 存在移入-归约冲突。

例

$I_0:$
$S' \rightarrow \cdot S$
$S \rightarrow \cdot aSbS$
$S \rightarrow \cdot bSaS$
$S \rightarrow \cdot$

$S \rightarrow \cdot aSbS$ 和 $S \rightarrow \cdot$

面对 a 时一个进行移入，一个进行归约。

状态	ACTION			GOTO
	a	b	\$	
0	s2/r3	s3/r3	r3	1
1			acc	
2	s2/r3	s3/r3	r3	4
3	s2/r3	s3/r3	r3	5
4		s6		
5	s7			
6	s2/r3	s3/r3	r3	8
7	s2/r3	s3/r3	r3	9
8	r3	r3	r3	
9	r3	r3	r3	



习题7.2

➤ 对于下列各（增广）文法：

① 构造SLR项集和它们的GOTO函数。

② 指出你的项集中有没有动作冲突。

③ 如果存在SLR语法分析表，构造出这个语法分析表。

➤ (7) $E \rightarrow E \text{ or } T \mid T$

$T \rightarrow T \text{ and } F \mid F$

$F \rightarrow \text{not } F \mid (E) \mid \text{true} \mid \text{false}$

文法增广及FOLLOW集

0) $S \rightarrow E$

1) $E \rightarrow E \text{ or } T$

2) $E \rightarrow T$

3) $T \rightarrow T \text{ and } F$

4) $T \rightarrow F$

5) $F \rightarrow \text{not } F$

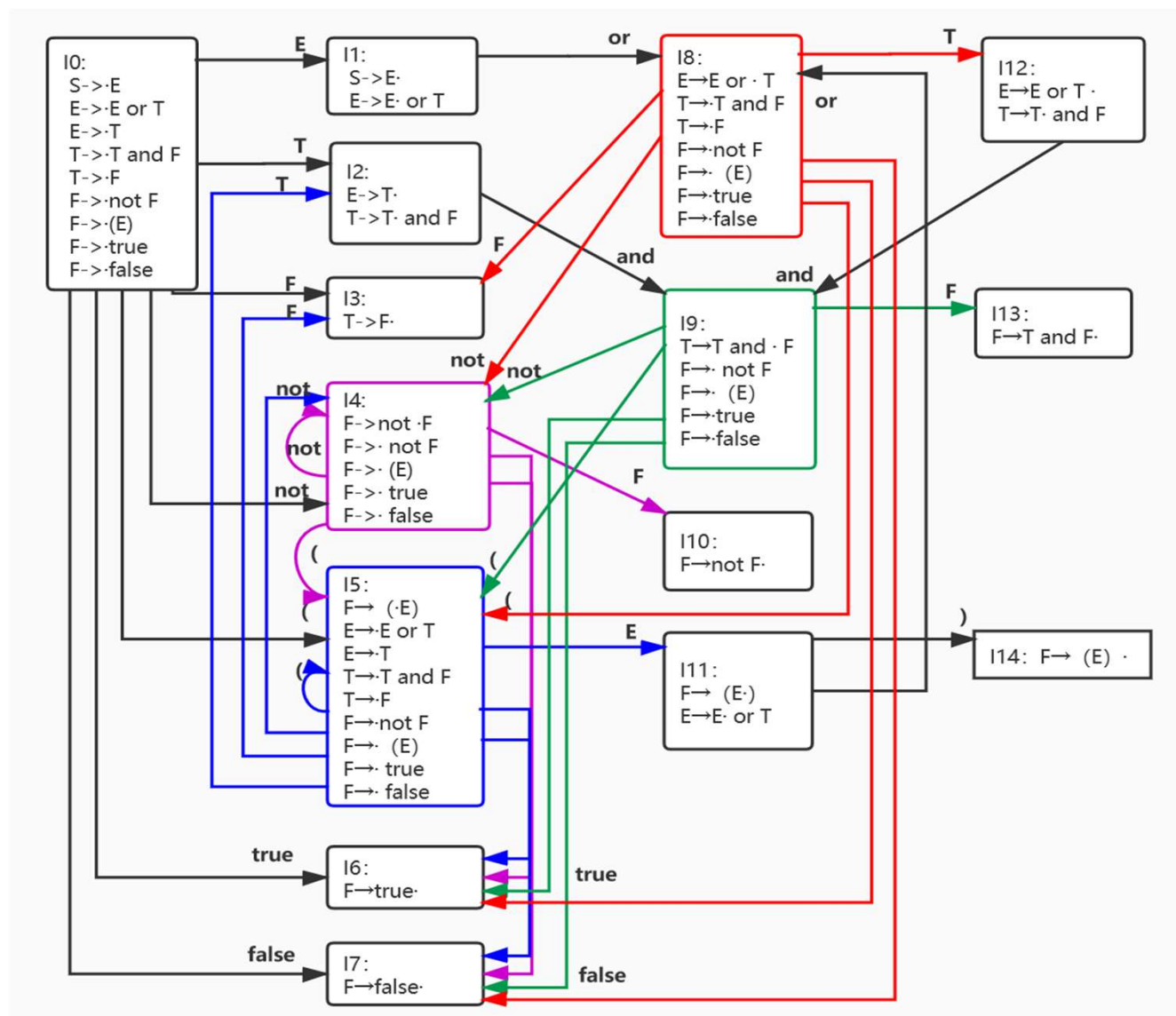
6) $F \rightarrow (E)$

7) $F \rightarrow \text{true}$

8) $F \rightarrow \text{false}$

非终结符	FIRST集	FOLLOW集
S	not, (, true, false	\$
E	not, (, true, false	or, \$,)
T	not, (, true, false	or, \$, and ,)
F	not, (, true, false	or, \$, and,)

构造自动机



LR(0)分析表

[illegible]

SLR分析表

0) $S \rightarrow E$

1) $E \rightarrow E \text{ or } T$

2) $E \rightarrow T$

3) $T \rightarrow T \text{ and } F$

4) $T \rightarrow F$

5) $F \rightarrow \text{not } F$

6) $F \rightarrow (E)$

7) $F \rightarrow \text{true}$

8) $F \rightarrow \text{false}$

没有冲突，是SLR文法

状态	ACTION								GOTO		
	or	and	not	(	)	true	false	\$	E	T	F
0			s4	s5		s6	s7		1	2	3
1	s8							acc			
2	r2	s9			r2			r2			
3	r4	r4			r4			r4			
4			s4	s5		s6	s7				10
5			s4	s5		s6	s7		11	2	3
6	r7	r7			r7			r7			
7	r8	r8			r8			r8			
8			s4	s5		s6	s7			12	3
9			s4	s5		s6	S7				13
10	r5	r5			r5			r5			
11	s8				s14						
12	r1	s9			r1			r1			
13	r3	r3			r3			r3			
14	r6	r6			r6			r6			



习题7.3

➤说明下面的文法

$$S \rightarrow A a A b \mid B b B a$$

$$A \rightarrow \varepsilon$$

$$B \rightarrow \varepsilon$$

是LL(1)的，LR (1) 的，但不是SLR(1)的



计算每个产生式的SELECT集

$$\text{FIRST}(S) = \{ a, b \}$$

$$\text{FIRST}(AaAb) = \{ a \}$$

$$\text{FIRST}(BbBa) = \{ b \}$$

$$\text{FIRST}(A) = \{ \epsilon \}$$

$$\text{FIRST}(B) = \{ \epsilon \}$$

$$\text{FOLLOW}(S) = \{ \$ \}$$

$$\text{FOLLOW}(A) = \{ a, b \}$$

$$\text{FOLLOW}(B) = \{ a, b \}$$

$$\text{SELECT}(S \rightarrow A a A b) = \{ a \}$$

$$\text{SELECT}(S \rightarrow B b B a) = \{ b \}$$

$$\text{SELECT}(A \rightarrow \epsilon) = \{ a, b \}$$

$$\text{SELECT}(B \rightarrow \epsilon) = \{ a, b \}$$

该文法不包含左递归且
具有相同左部的产生式的
select集并不相交,
所以该文法是**LL (1)**
文法

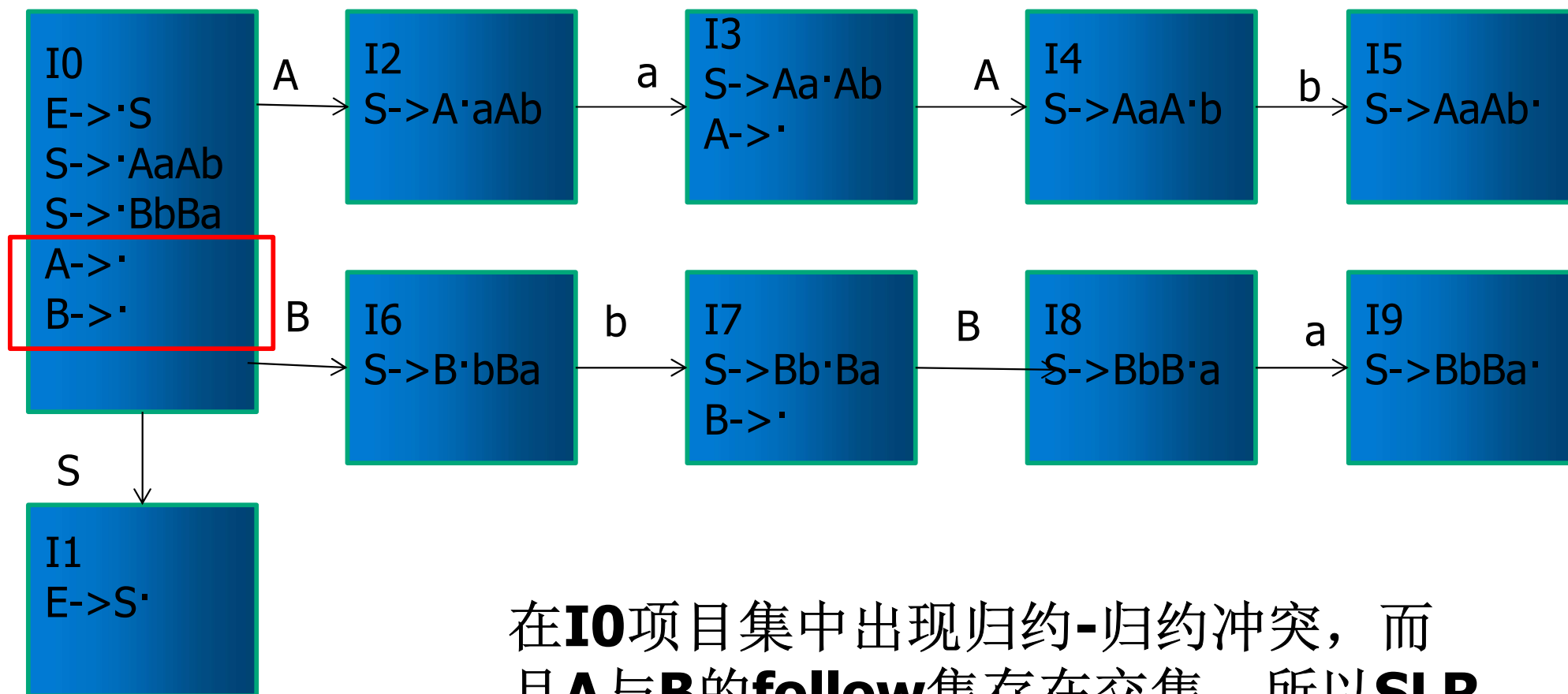


构造自动机

$\text{FOLLOW}(S) = \{ \$ \}$

$\text{FOLLOW}(A) = \{ a, b \}$

$\text{FOLLOW}(B) = \{ a, b \}$



在**I0**项目集中出现归约-归约冲突，而且**A**与**B**的**follow**集存在交集，所以**SLR**分析法无法解决该冲突，因此该文法不是**SLR(1)**文法。



习题7.4

➤说明下面的文法

$$S \rightarrow S A \mid A$$

$$A \rightarrow a$$

是SLR(1)的，但不是LL(1)的

SLR(1)的判定

$$\begin{aligned} S &\rightarrow SA|A \\ A &\rightarrow a \end{aligned}$$

改为增广文法

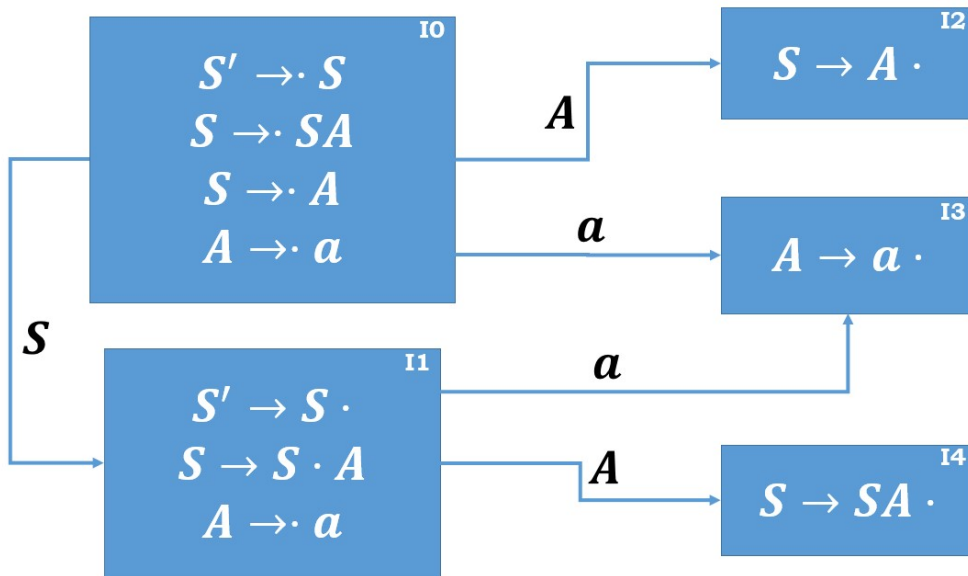
$$\begin{aligned} S' &\rightarrow S \\ S &\rightarrow SA|A \\ A &\rightarrow a \end{aligned}$$

	FIRST	FOLLOW
S'	a	$\$$
S	a	$\$ a$
A	a	$\$ a$



SLR(1)的判定

结论2: SLR表不存在冲突, 所以该文法是SLR(1)的

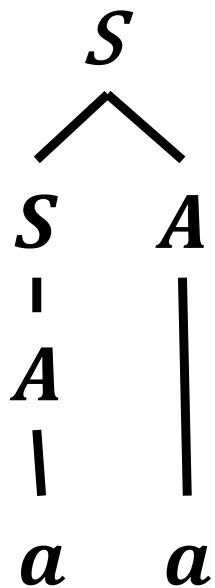


状态	ACTION		GOTO	
	a	$\$$	A	S
0	s3		2	1
1	s3	acc	4	
2	r2	r2		
3	r3	r3		
4	r1	r1		

LL(1)的判定

$$\begin{aligned} S &\rightarrow SA|A \\ A &\rightarrow a \end{aligned}$$

对于产生式 $S \rightarrow SA$ 和 $S \rightarrow A$,当选择终结符 a 时,二者都可以推出以 a 开头的串



LL(1)文法

- 文法 G 是 $LL(1)$ 的，当且仅当 G 的任意两个具有相同左部的产生式 $A \rightarrow \alpha \mid \beta$ 满足下面的条件：
 - 不存在终结符 a 使得 α 和 β 都能够推导出以 a 开头的串
 - α 和 β 至多有一个能推导出 ε
 - 如果 $\beta \Rightarrow^* \varepsilon$ ，则 $FIRST(\alpha) \cap FOLLOW(A) = \Phi$ ；
如果 $\alpha \Rightarrow^* \varepsilon$ ，则 $FIRST(\beta) \cap FOLLOW(A) = \Phi$ ；

结论1：不满足LL(1)文法条件，故该文法不是LL(1)的

习题7.5

➤ 下面是一个二义性文法：

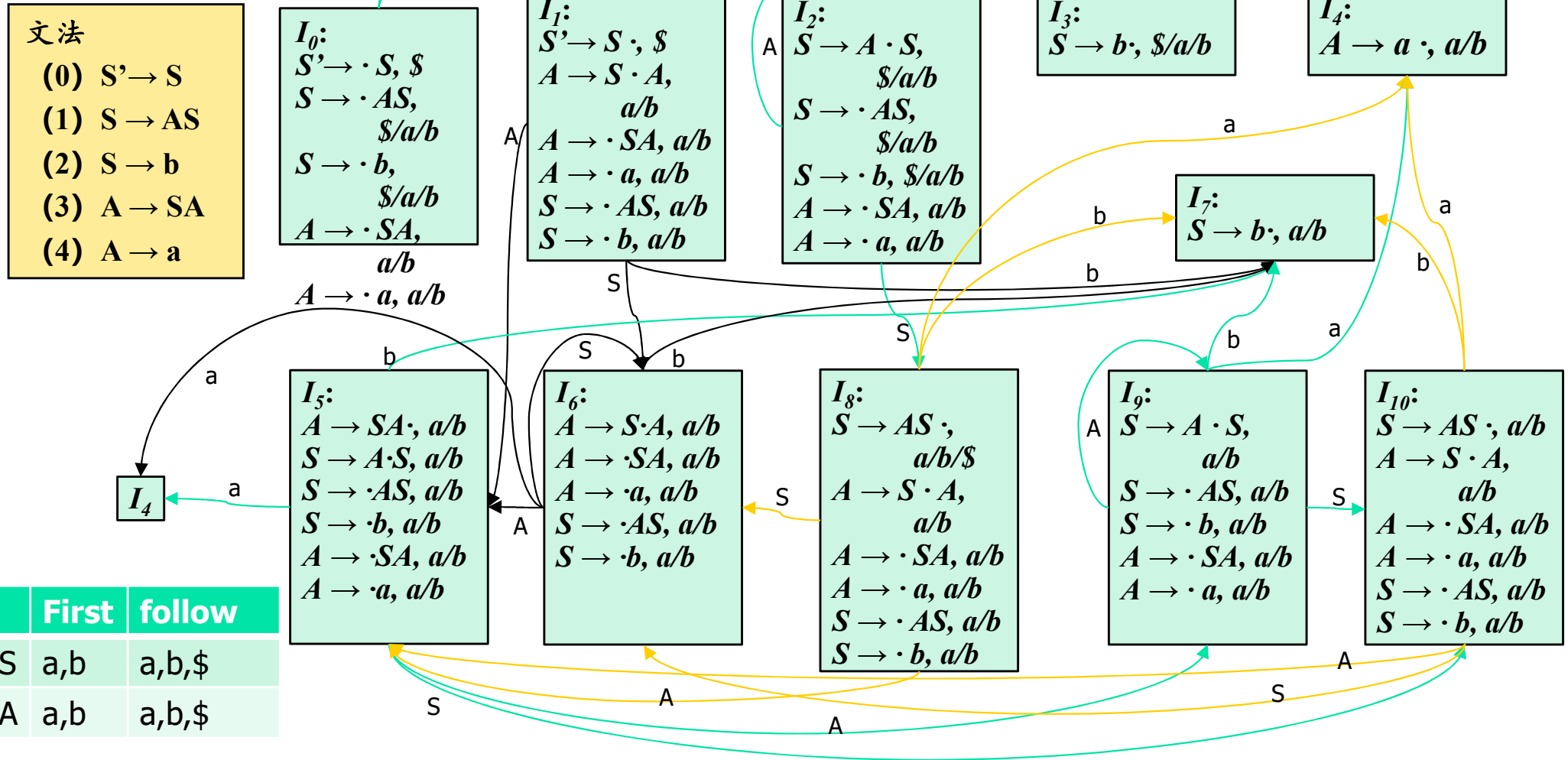
$$S \rightarrow A S \mid b$$

$$A \rightarrow S A \mid a$$

构造出这个文法的规范LR项集族。如果我们试图为这个文法构造出一个LR语法分析表，必然会存在某些冲突动作。都有哪些冲突动作？假设我们使用这个语法分析表，并且在出现冲突时不确定地选择一个可能的动作。给出处理输入**abab**时的做有可能的动作序列。



集族





分析表

文法

(0) $S' \rightarrow S$

(1) $S \rightarrow AS$

(2) $S \rightarrow b$

(3) $A \rightarrow SA$

(4) $A \rightarrow a$

状态	ACTION			GOTO	
	a	b	\$	S	A
0	s4	s3		1	2
1	s4	s7	acc	6	5
2	s4	s3		8	2
3	r2	r2	r2		
4	r4	r4			
5	r3 s4	r3 s7		10	9
6	s4	s7		6	5
7	r2	r2			
8	r1 s4	r1 s7	r1	6	5
9	s4	s7		10	9
10	r1 s4	r1 s7		6	5



abab动作序列1

栈 剩余输入 动作

0

\$ abab\$

04

\$a bab\$

goto2

02

\$A bab\$

023

\$Ab ab\$

goto8

028

\$AS ab\$

(r1)

0

\$S ab\$

01

\$S ab\$

014

\$Sa b\$

goto5

015

\$SA b\$

(r3)

s4

r4

s3

r2

r1|s4

goto1

s4

r4

r3|s7

栈

剩余输入

动作

0

\$A b\$

02

\$A b\$

023

\$Ab \$

goto8

028

\$AS \$

goto1

01

\$S \$

goto2

s3

r2

r1

acc



abab动作序列2

栈 剩余输入 动作

0

\$ abab\$

04

\$a bab\$

goto2

02

\$A bab\$

023

\$Ab ab\$

goto8

028

\$AS ab\$

(r1)

0

\$S ab\$

01

\$S ab\$

014

\$Sa b\$

goto5

015

\$SA b\$

(s7)

s4

栈 剩余输入 动作

0157

\$SAb \$

ERR

r4

s3

r2

r1|s4

goto1

s4

r4

r3|s7



abab动作序列3

栈 剩余输入 动作

0

\$ abab\$

04

\$a bab\$

goto2

02

\$A bab\$

023

\$Ab ab\$

goto8

028

\$AS ab\$

(s4)

0284

\$ASa b\$

goto5

0285

\$ASA b\$

(r3)

02

\$AA b\$

022

\$AA b\$

s4

r4

s3

r2

r1|s4

r4

r3|s7

goto2

s3

栈

剩余输入

动作

0223

\$AAb \$

r2

goto8

0228

\$AAS \$

r1

goto8

028

\$AS \$

r1

goto1

01

\$S \$

acc



abab动作序列4

栈	剩余输入	动作
0		
\$	<i>abab</i> \$	s4
04		
\$a	<i>bab</i> \$	r4
	goto2	
02		
\$A	<i>bab</i> \$	s3
023		
\$Ab	<i>ab</i> \$	r2
	goto8	
028		
\$AS	<i>ab</i> \$	r1 s4
	(s4)	
0284		
\$ASa	<i>b</i> \$	r4
	goto5	
0285		
\$ASA	<i>b</i> \$	r3 s7
	(s7)	
02857		
\$ASAb	\$	ERR



习题7.6

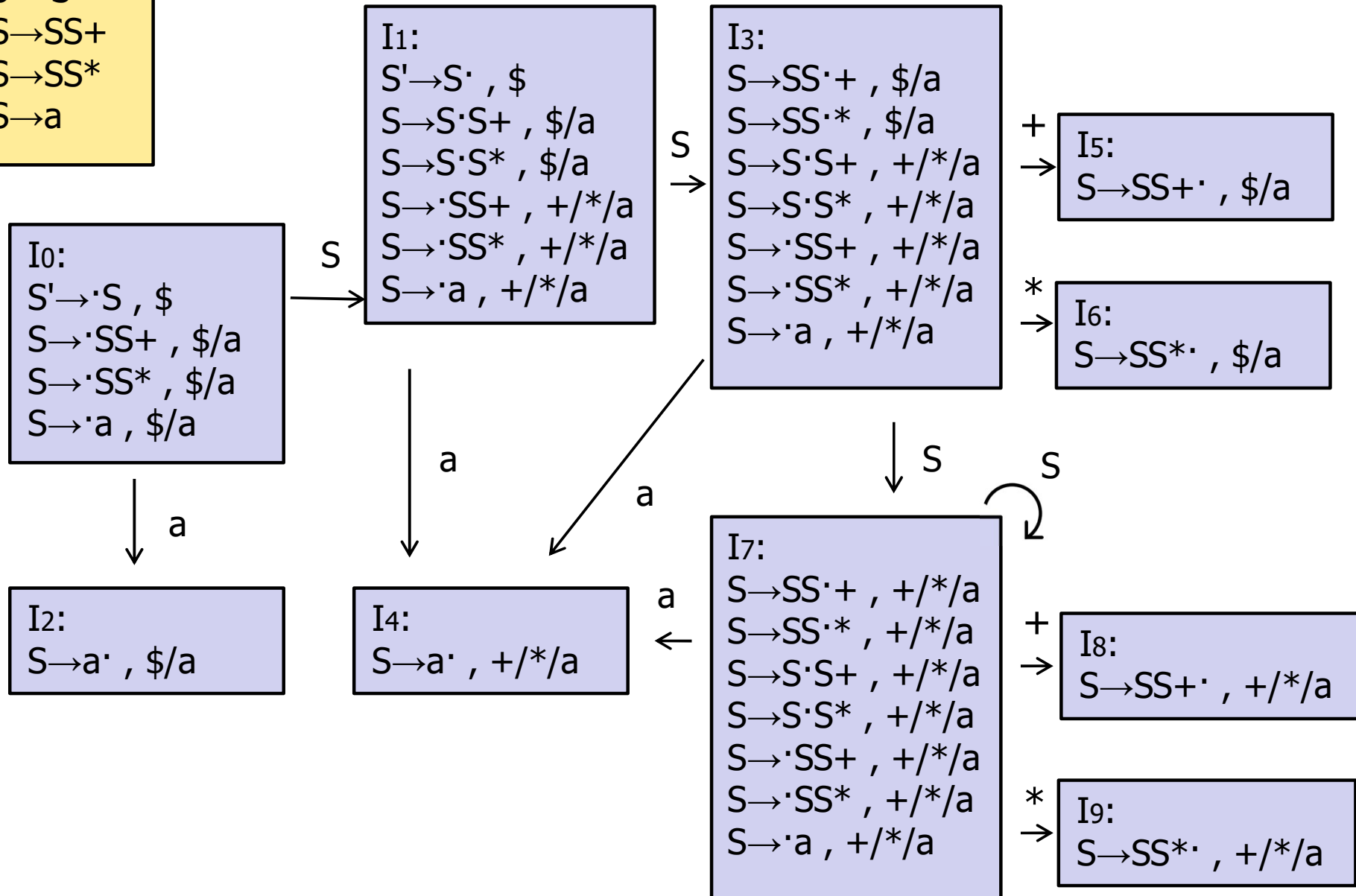
➤ 为文法 $S \rightarrow SS+ | SS* | a$ 构造

① 规范LR项集族。

② LALR项集族。

LR(1)

- 0) $S' \rightarrow S$
- 1) $S \rightarrow SS+$
- 2) $S \rightarrow SS^*$
- 3) $S \rightarrow a$





LALR

- 0) $S' \rightarrow S$
- 1) $S \rightarrow SS+$
- 2) $S \rightarrow SS^*$
- 3) $S \rightarrow a$

I₀:
 $S' \rightarrow \cdot S, \$$
 $S \rightarrow \cdot SS+, \$ / a$
 $S \rightarrow \cdot SS^*, \$ / a$
 $S \rightarrow \cdot a, \$ / a$

S

a

I₁:
 $S' \rightarrow S \cdot, \$$
 $S \rightarrow S \cdot S+, \$ / a$
 $S \rightarrow S \cdot S^*, \$ / a$
 $S \rightarrow \cdot SS+, + / * / a$
 $S \rightarrow \cdot SS^*, + / * / a$
 $S \rightarrow \cdot a, + / * / a$

S

a

a

I₂:
 $S \rightarrow a \cdot, \$ / + / * / a$

I₃:
 $S \rightarrow SS \cdot +, \$ / + / * / a$
 $S \rightarrow SS \cdot *, \$ / + / * / a$
 $S \rightarrow S \cdot S+, + / * / a$
 $S \rightarrow S \cdot S^*, + / * / a$
 $S \rightarrow \cdot SS+, + / * / a$
 $S \rightarrow \cdot SS^*, + / * / a$
 $S \rightarrow \cdot a, + / * / a$

S

+

*

I₄:
 $S \rightarrow SS+ \cdot, \$ / + / * / a$

I₅:
 $S \rightarrow SS^* \cdot, \$ / + / * / a$



习题7.7

➤ 对于下列各（增广）文法，构造

① 规范LR项集族

② LALR项集族

➤ (1) $S \rightarrow 0 S 1 \mid 0 1$

➤ (2) $S \rightarrow + S S \mid * S S \mid a$

➤ (3) $S \rightarrow S (S) S \mid \varepsilon$

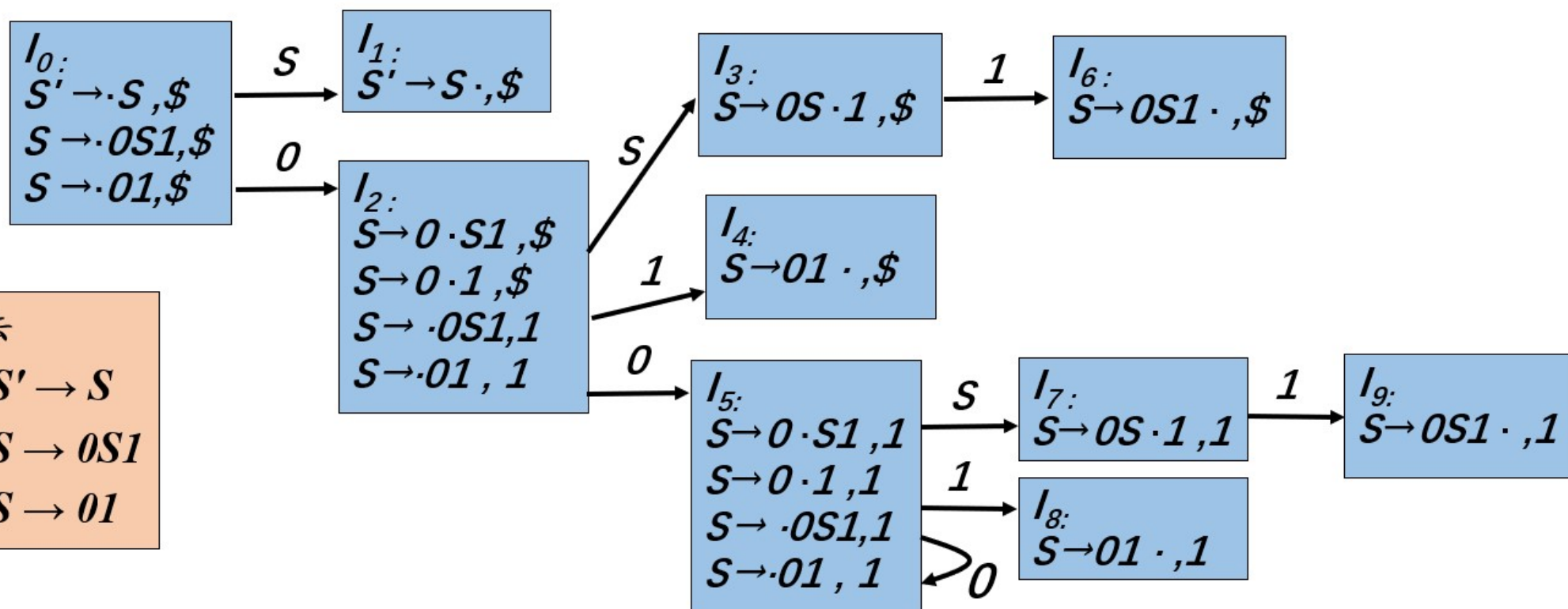
➤ (4) $S \rightarrow (L) \mid a$ 以及 $L \rightarrow L , S \mid S$



(1)



LR(1):



文法

(0) $S' \rightarrow S$

(1) $S \rightarrow 0S1$

(2) $S \rightarrow 01$



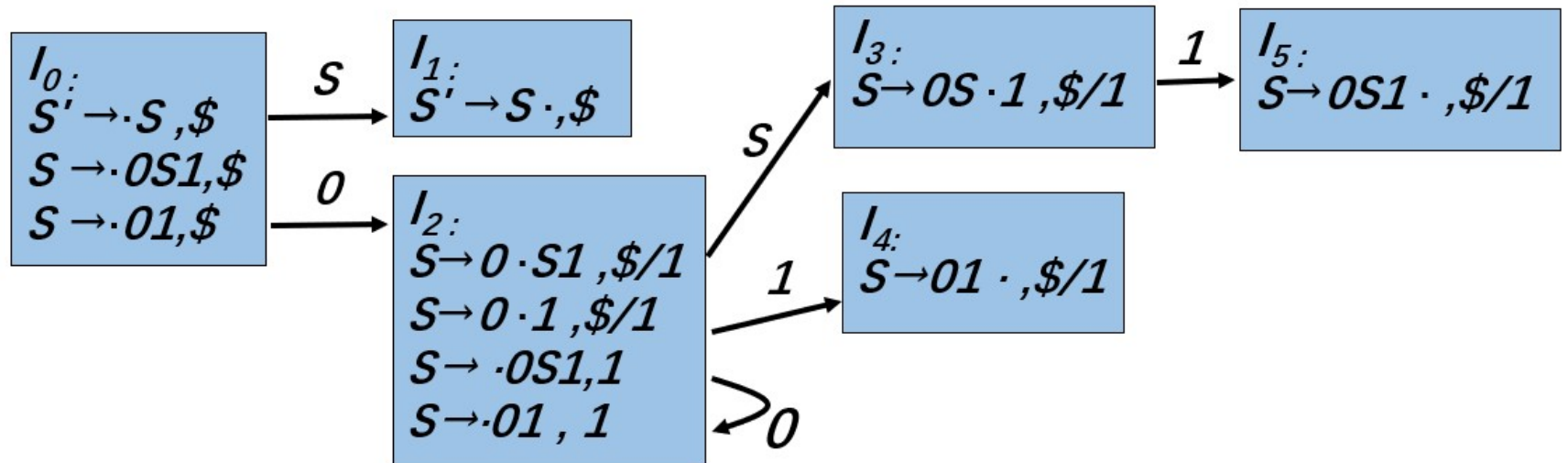
LALR (1) :

文法

(0) $S' \rightarrow S$

(1) $S \rightarrow 0S1$

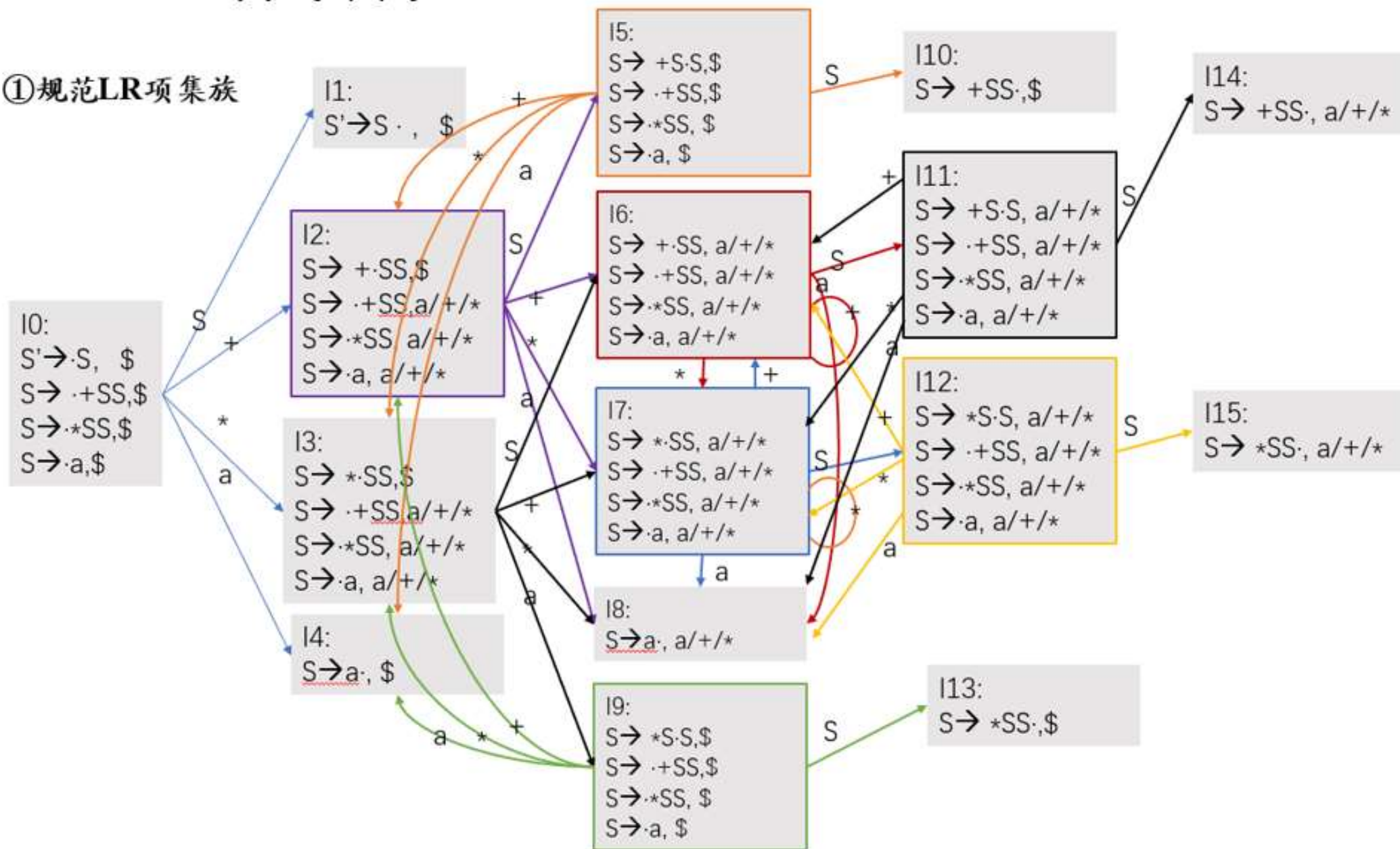
(2) $S \rightarrow 01$



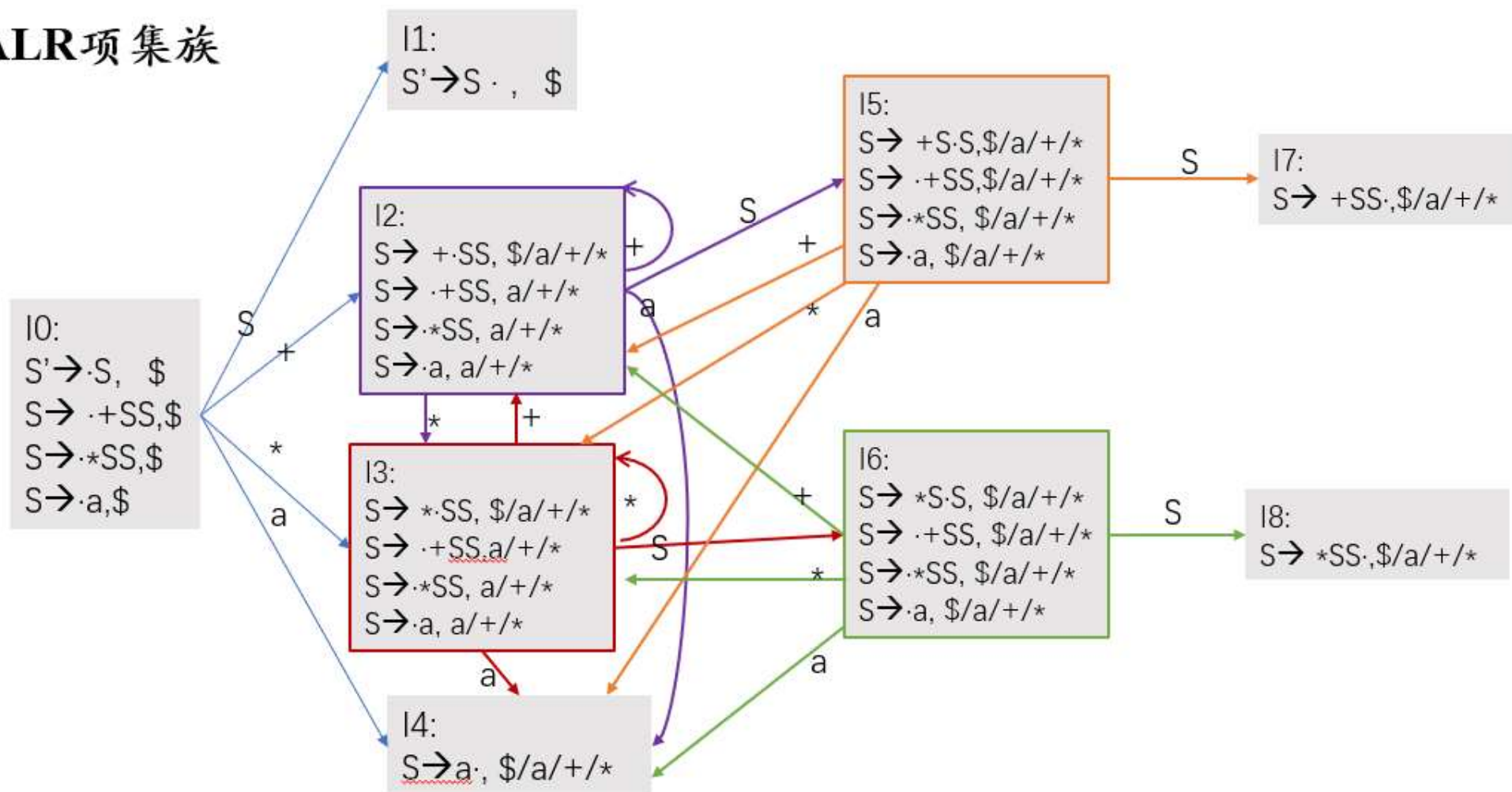
(2)

$\text{FIRST}(S) = \{+, *, a\}$

① 规范LR项集族



②LALR项集族





(3)

I_0 :
 $S' \rightarrow \cdot S, \$$
 $S \rightarrow \cdot S(S)S, \$$
 $S \rightarrow \cdot, \$$
 $S \rightarrow \cdot, ($
 $S \rightarrow \cdot S(S)S, ($

I_1 :
 $S' \rightarrow S \cdot, \$$
 $S \rightarrow S \cdot (S)S, \$/ ($

I_2 :
 $S \rightarrow S (\cdot S)S, \$/ ($
 $S \rightarrow \cdot S(S)S,)$
 $S \rightarrow \cdot,)$

I_3 :
 $S \rightarrow S (S \cdot)S, \$/ ($
 $S \rightarrow S \cdot (S)S, (/)$

I_4 :
 $S \rightarrow S(S) \cdot S, \$/ ($
 $S \rightarrow \cdot S(S)S, \$/ ($
 $S \rightarrow \cdot, \$/ ($

I_6 :
 $S \rightarrow S(S)S \cdot, (/ \$$
 $S \rightarrow S \cdot (S)S, (/ \$$

I_5 :
 $S \rightarrow S (\cdot S)S, (/)$
 $S \rightarrow \cdot, (/)$
 $S \rightarrow S \cdot (S)S, (/)$

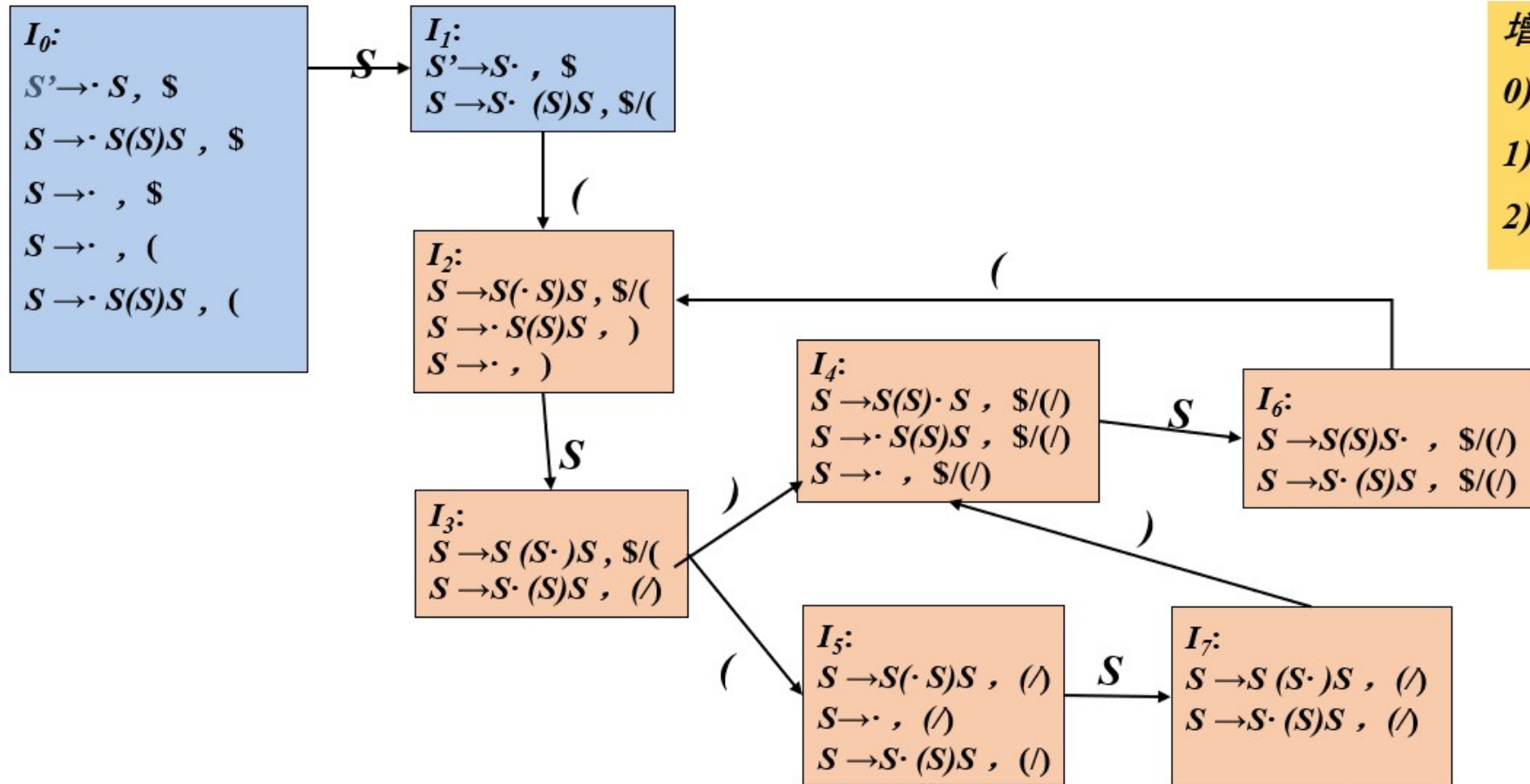
I_7 :
 $S \rightarrow S (S \cdot)S, (/)$
 $S \rightarrow S \cdot (S)S, (/)$

I_8 :
 $S \rightarrow S (S) \cdot S, (/)$
 $S \rightarrow \cdot, (/)$
 $S \rightarrow \cdot S (S)S, (/)$

I_9 :
 $S \rightarrow S (S)S \cdot, (/)$
 $S \rightarrow S \cdot (S)S, (/)$

增广文法:
0) $S' \rightarrow S$
1) $S \rightarrow S(S)S$
2) $S \rightarrow \varepsilon$

LR



增广文法:

0) $S' \rightarrow S$

1) $S \rightarrow S(S)S$

2) $S \rightarrow \varepsilon$

LALR



(4)

增广文法:

$S' \rightarrow S$

$S \rightarrow (L)$

$S \rightarrow a$

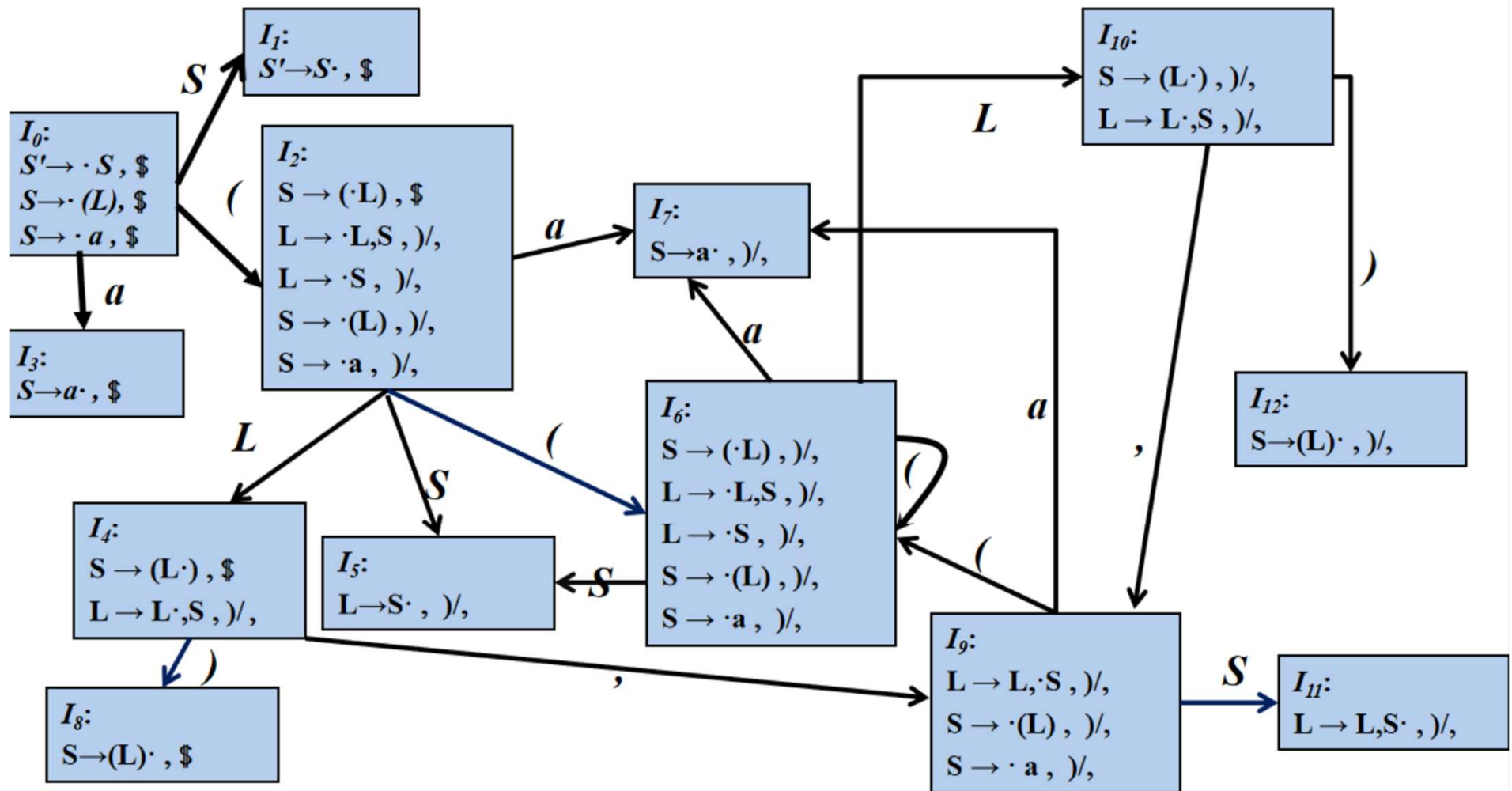
$L \rightarrow L, S$

$L \rightarrow S$

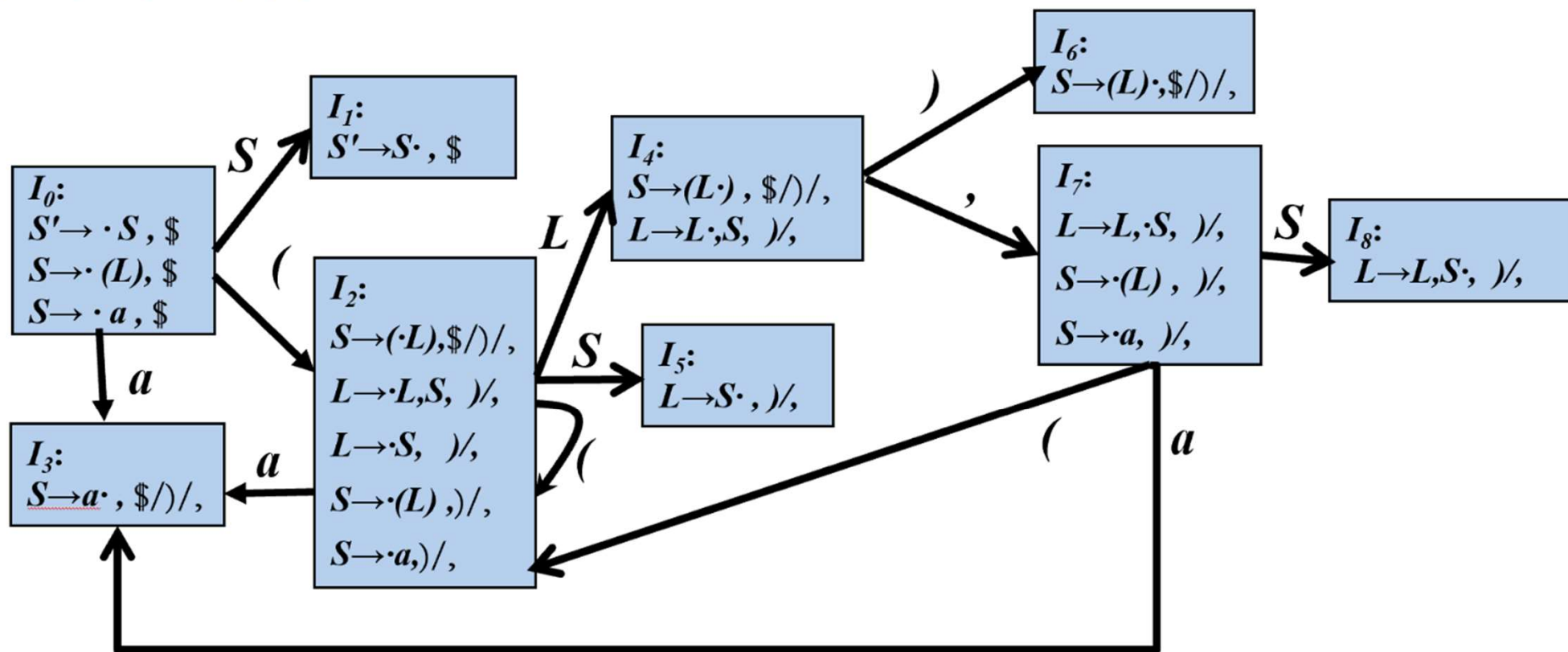
FIRST集:

$\text{FIRST}(S) = \{ (, a \}$

$\text{FIRST}(L) = \{ (, a \}$



* 合并同心项集





习题7.8

➤说明下面的文法

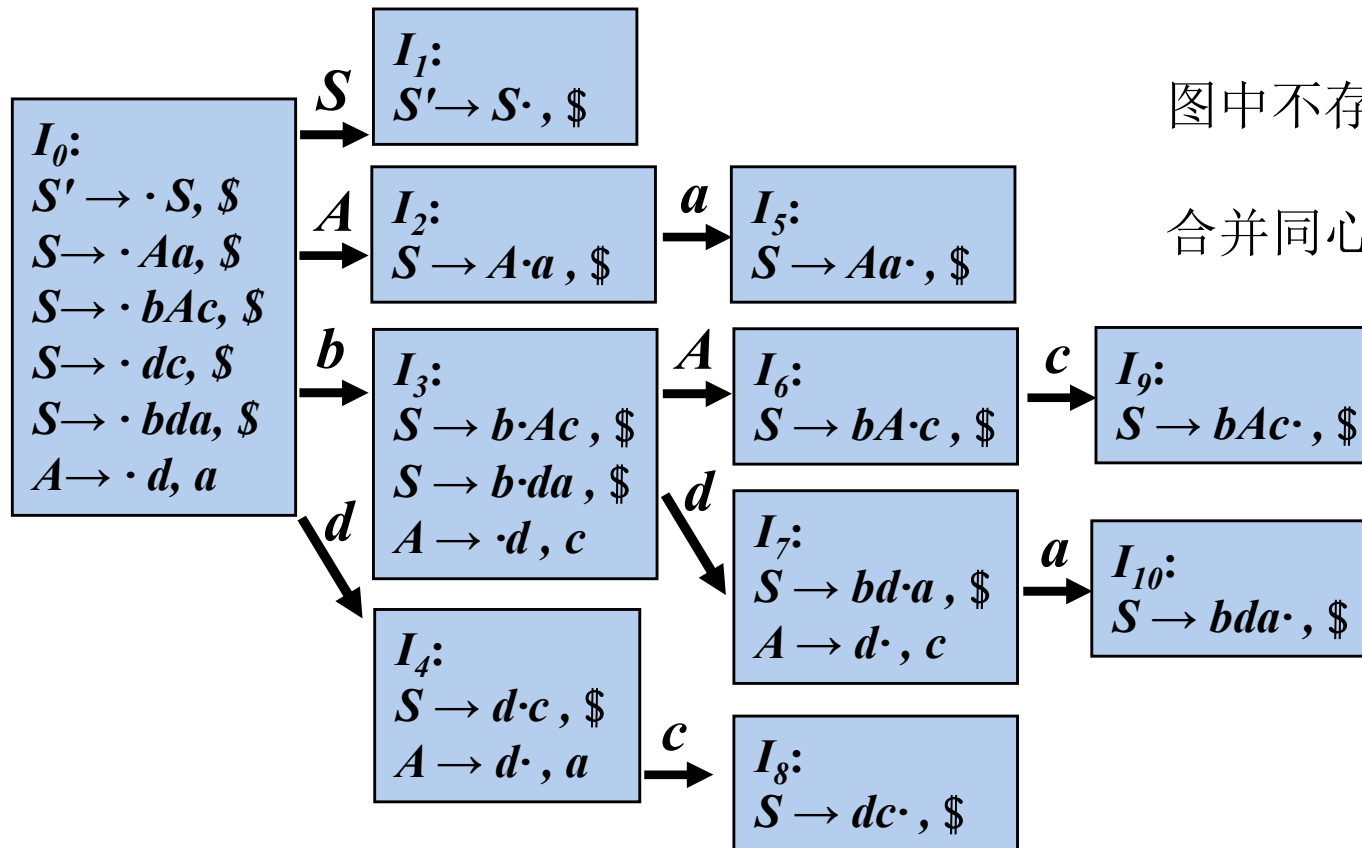
$$S \rightarrow A a \mid b A c \mid d c \mid b d a$$

$$A \rightarrow d$$

是LALR(1)的，但不是SLR(1)的。

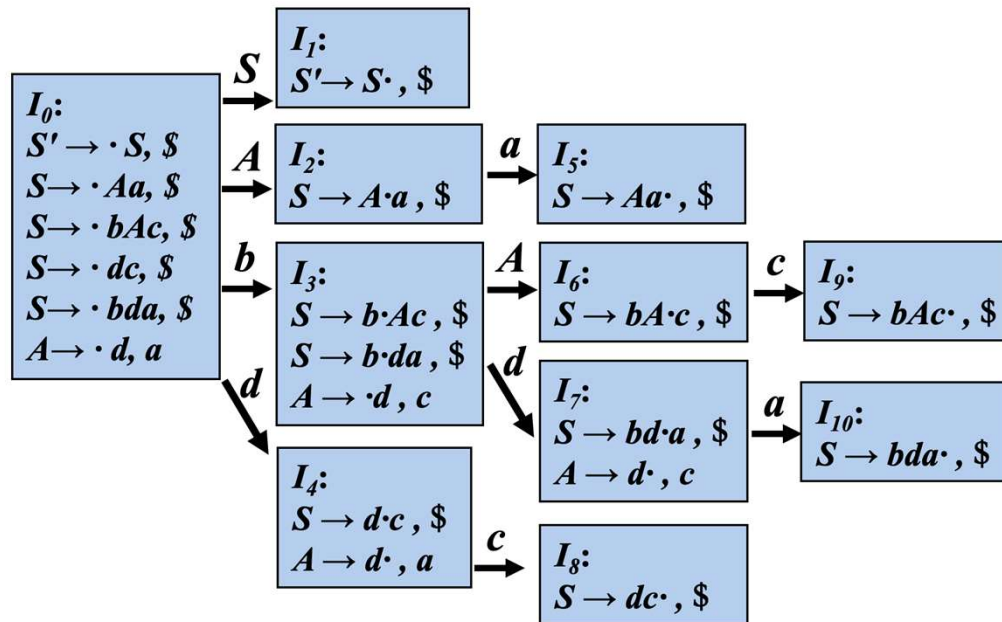


首先构造LR(1)自动机



图中不存在同心项集

合并同心项集后不变



- (0) $S' \rightarrow S$
- (1) $S \rightarrow Aa$
- (2) $S \rightarrow bAc$
- (3) $S \rightarrow dc$
- (4) $S \rightarrow bda$
- (5) $A \rightarrow d$

根据图构造语法分析表

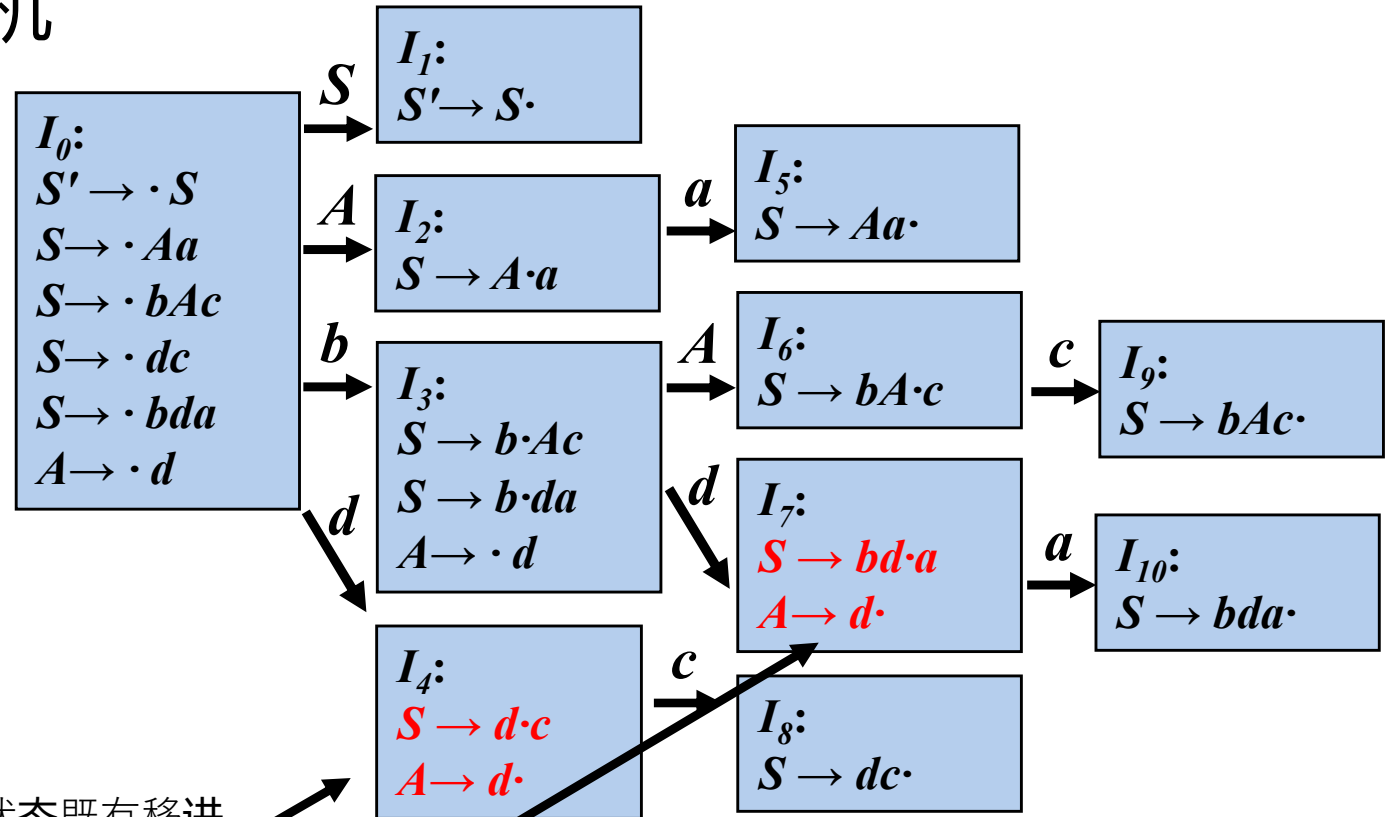
状态	ACTION							GOTO	
	S	A	a	b	c	d	\$	S	A
0	s1	s2		s3		s4		1	2
1							acc		
2			s5						
3		s6				s7			6
4			r5		s8				
5							r1		
6					s9				
7			s10		r5				
8							r3		
9							r2		
1							r4		

表中没有语法分析动作冲突
故此文法为LALR (1) 文法



SLR(1)自动机

<i>X</i>	<i>FOLLOW</i> (<i>X</i>)
<i>S</i>	\$
<i>A</i>	a, c



14和17状态既有移进
又有规约
在建立SLR分析表时冲突
故此文法不是SLR文法



习题7.9

➤说明下面的文法

$$S \rightarrow A a \mid b A c \mid B c \mid b B a$$

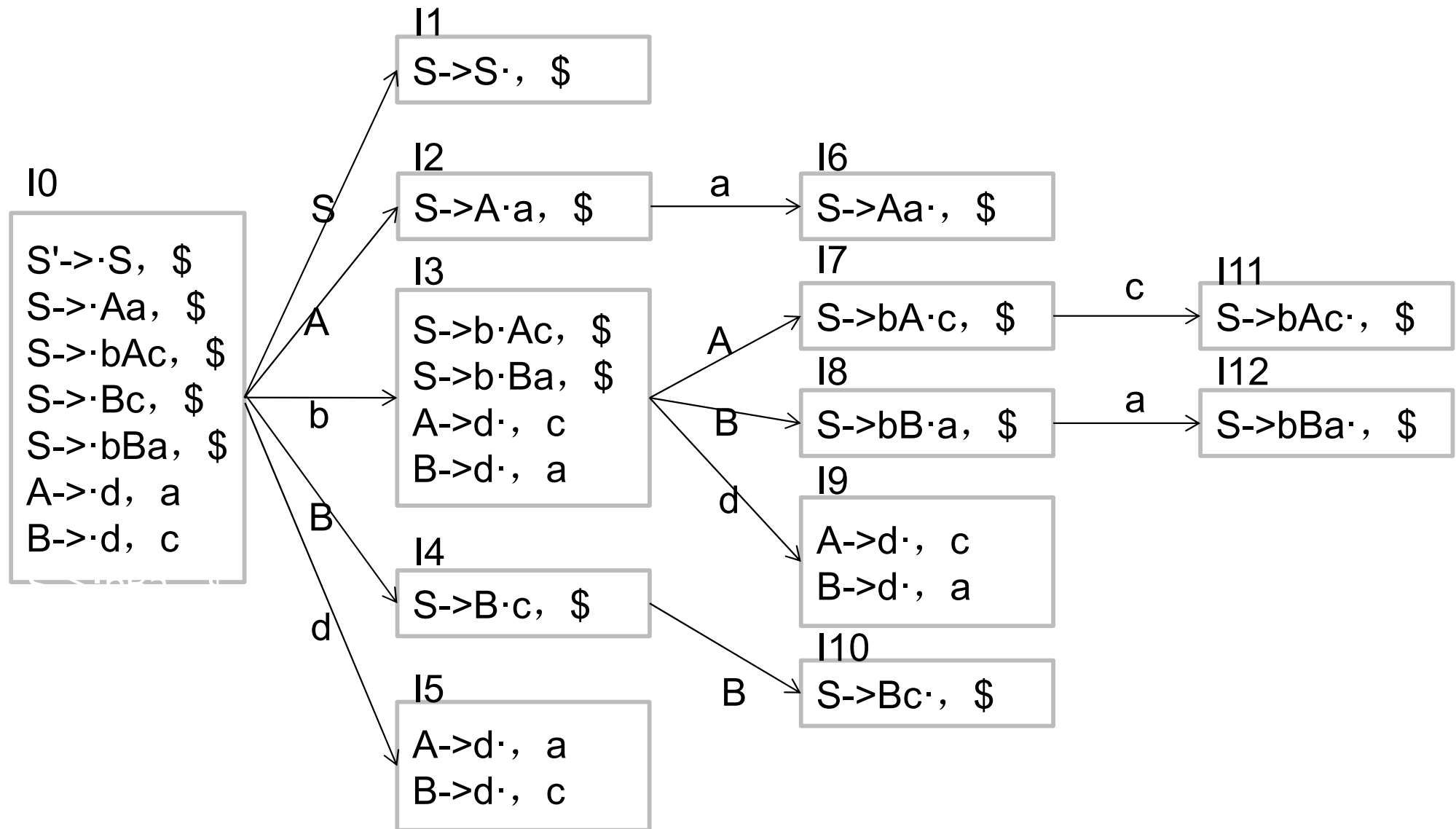
$$A \rightarrow d$$

$$B \rightarrow d$$

是LR(1)的，但不是LALR(1)的。



■ 2. 等价项目集与Go函数

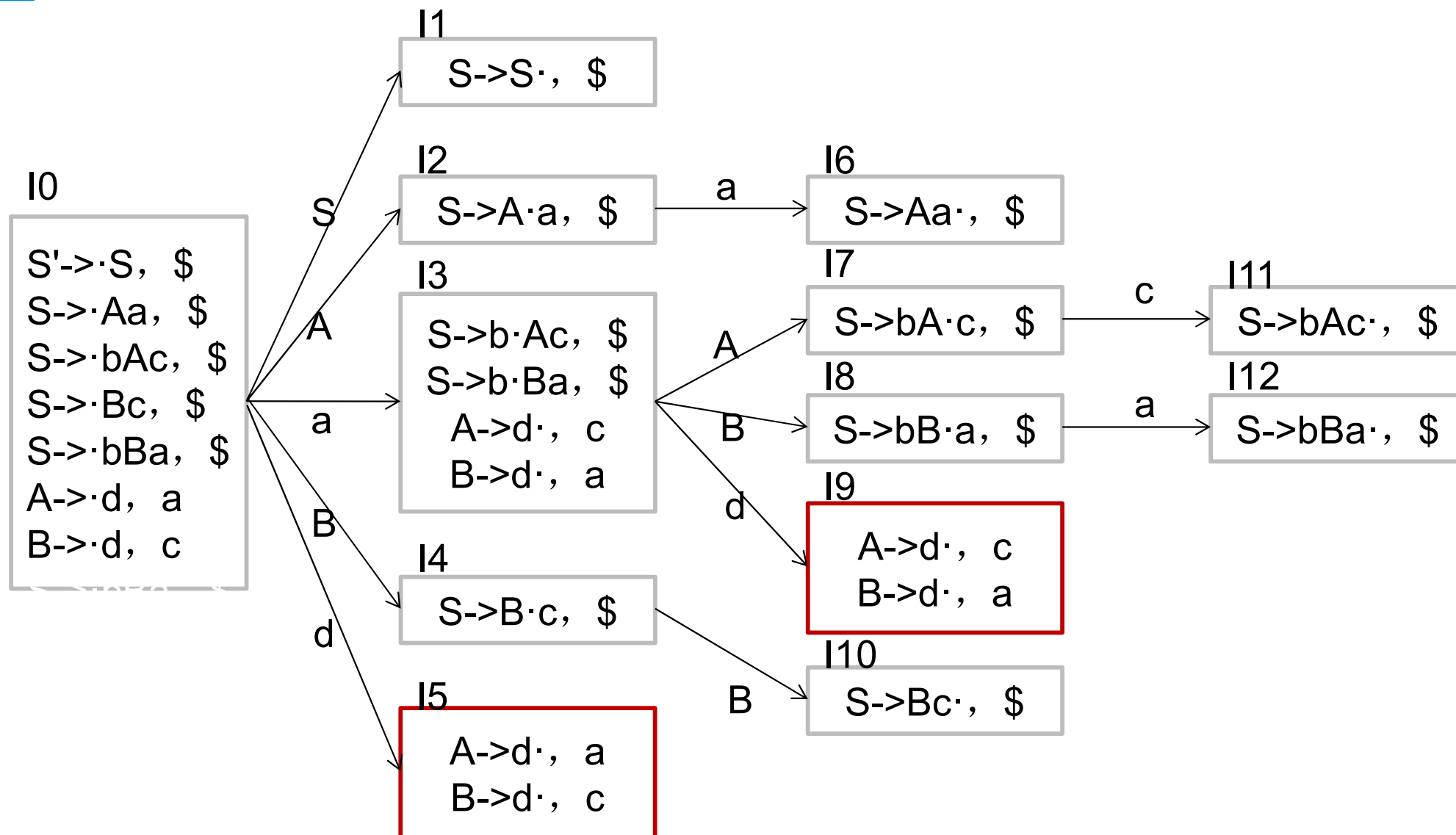




■ 3. 画LR分析表

没有冲突，所以该文法是LR（1）文法

	a	b	c	d	\$	S	A	B
0		s3		s5		1	2	4
1					acc			
2	s6							
3				s9			7	8
4								10
5	r5		r6					
6					r1			
7								
8	s12							
9	r6		r5					
10					r3			
11					r2			
12					r4			



然而，尝试合并同心项目集的时候，会发现
状态I5与I9会产生归约-归约冲突，所以该文法不是LALR文
法



习题8.1

➤ 对于下图所示的**SDD**，给出下列表达式对应的注释语法分析树：

产生式	语义规则
1) $L \rightarrow E \ n$	$L.val = E.val$
2) $E \rightarrow E_1 + T$	$E.val = E_1.val + T.val$
3) $E \rightarrow T$	$E.val = T.val$
4) $T \rightarrow T_1 * F$	$T.val = T_1.val \times F.val$
5) $T \rightarrow F$	$T.val = F.val$
6) $F \rightarrow (E)$	$F.val = E.val$
7) $F \rightarrow \text{digit}$	$F.val = \text{digit.lexval}$

➤ (1) $(3+4)*(5+6)n$

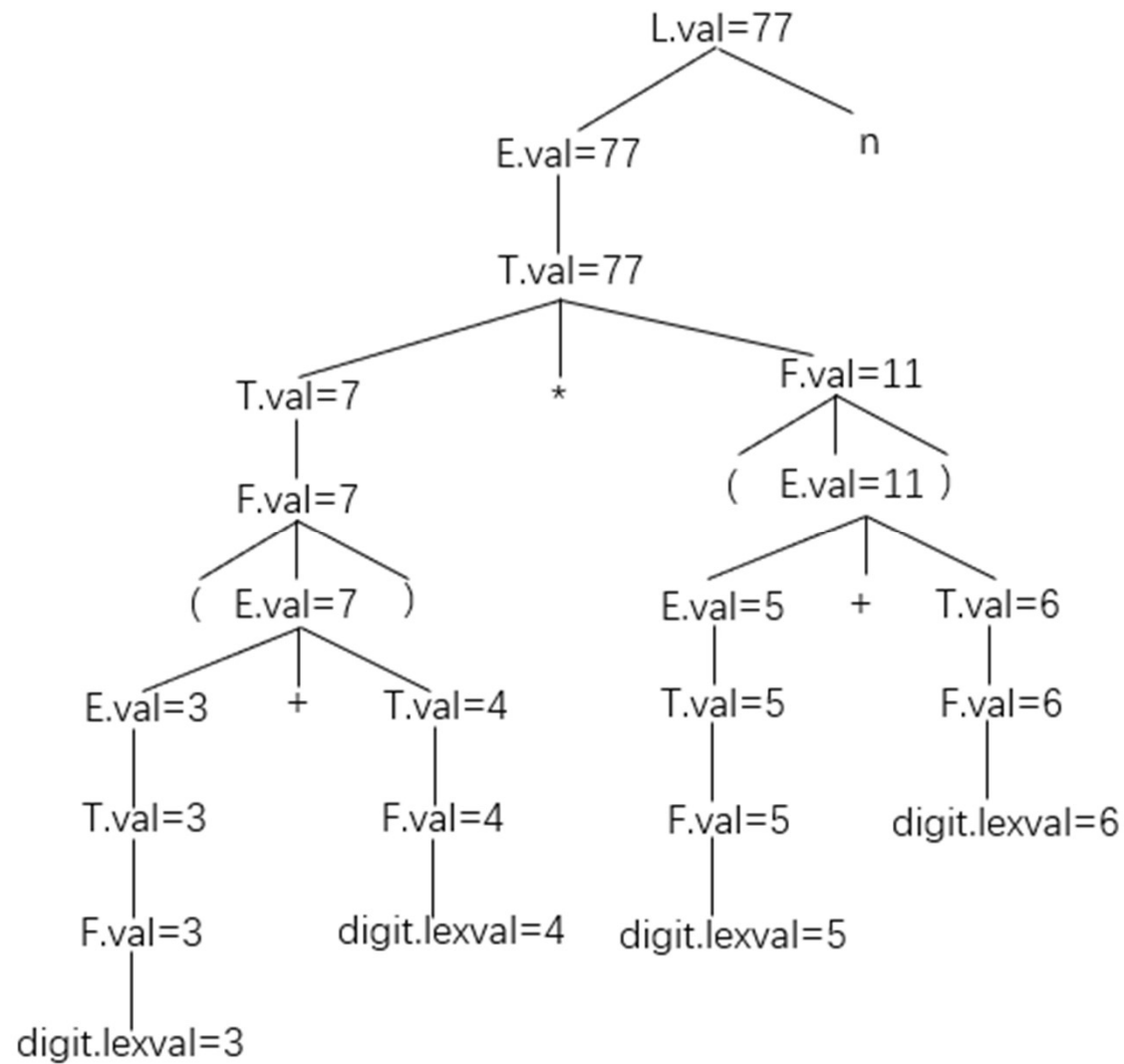
➤ (2) $1*2*3*(4+5)n$

➤ (3) $(9+8*(7+6)+5)*4n$



习题8.1

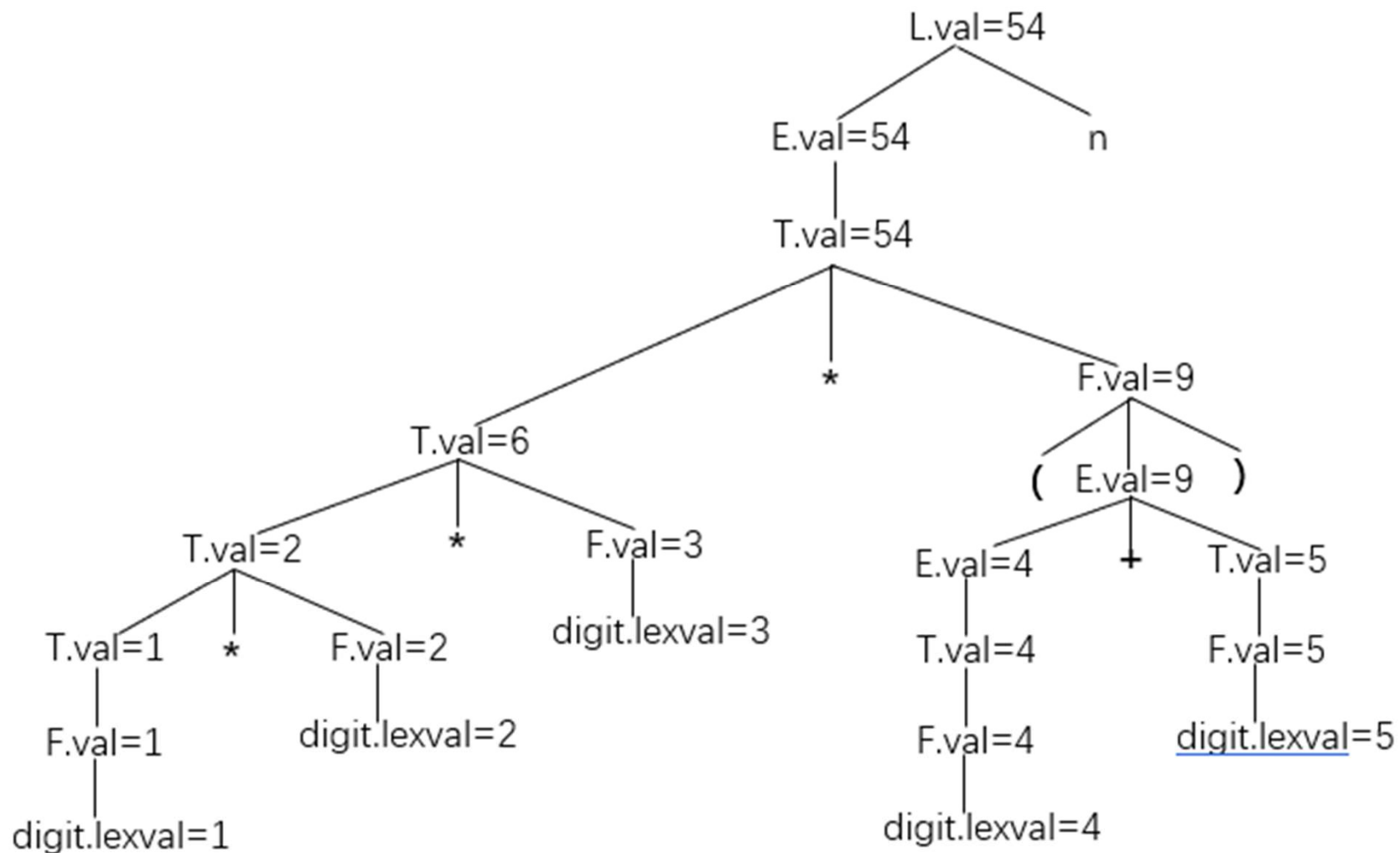
(1) $(3+4)*(5+6)n$





习题8.1

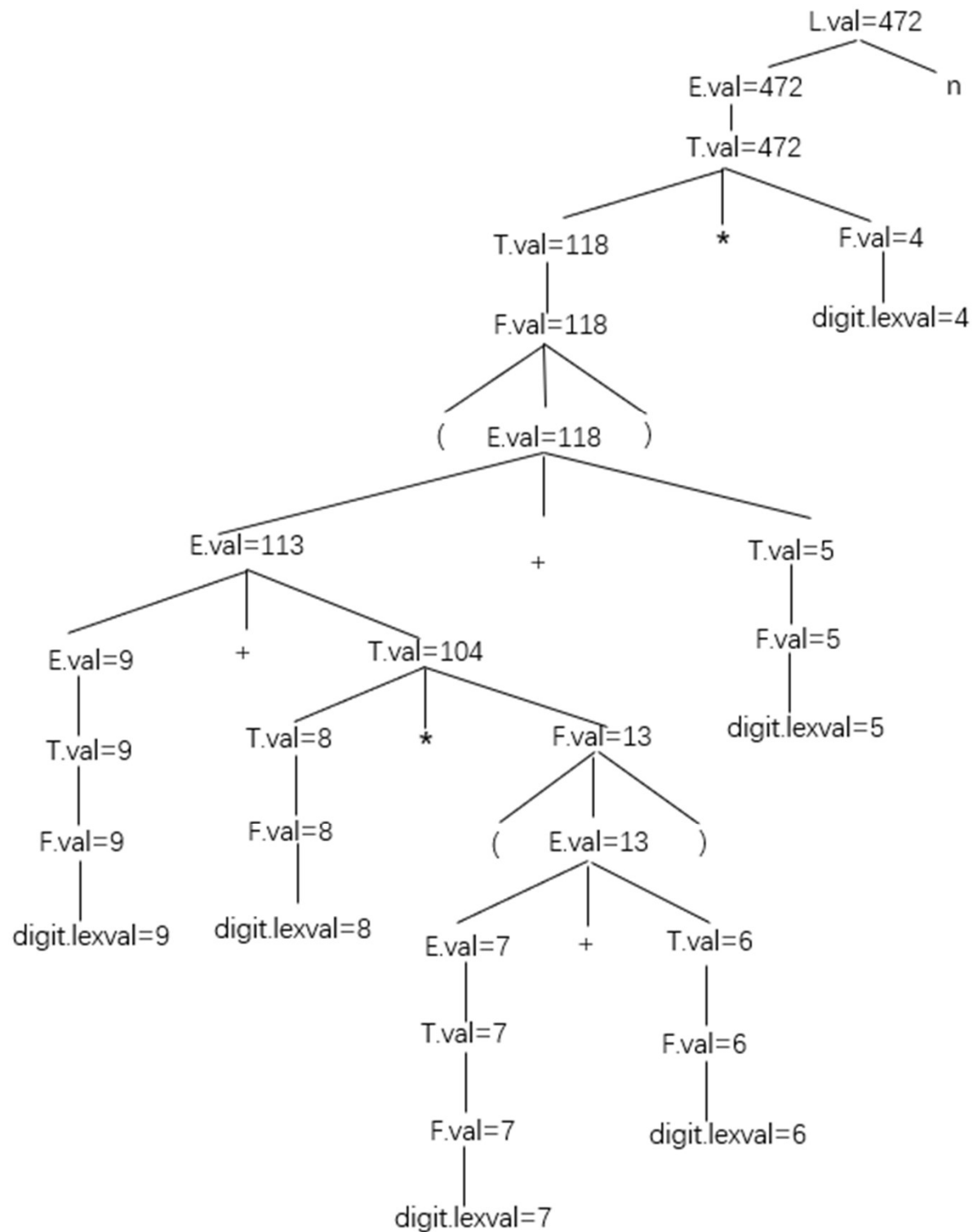
(2) $1*2*3*(4+5)n$





习题8.1

(3) $(9+8*(7+6)+5)*4n$





习题8.2

➤ 给定图1所示的SDD，图2是句子 $3*5$ 的注释分析树的依赖图。图2的全部拓扑顺序有哪些？

产生式	语义规则
1) $T \rightarrow F T'$	$T'.inh = F.val$ $T.val = T'.syn$
2) $T' \rightarrow * F T'_1$	$T'_1.inh = T'.inh \times F.val$ $T'.syn = T'_1.syn$
3) $T' \rightarrow \epsilon$	$T'.syn = T'.inh$
4) $F \rightarrow \text{digit}$	$F.val = \text{digit.lexval}$

图1

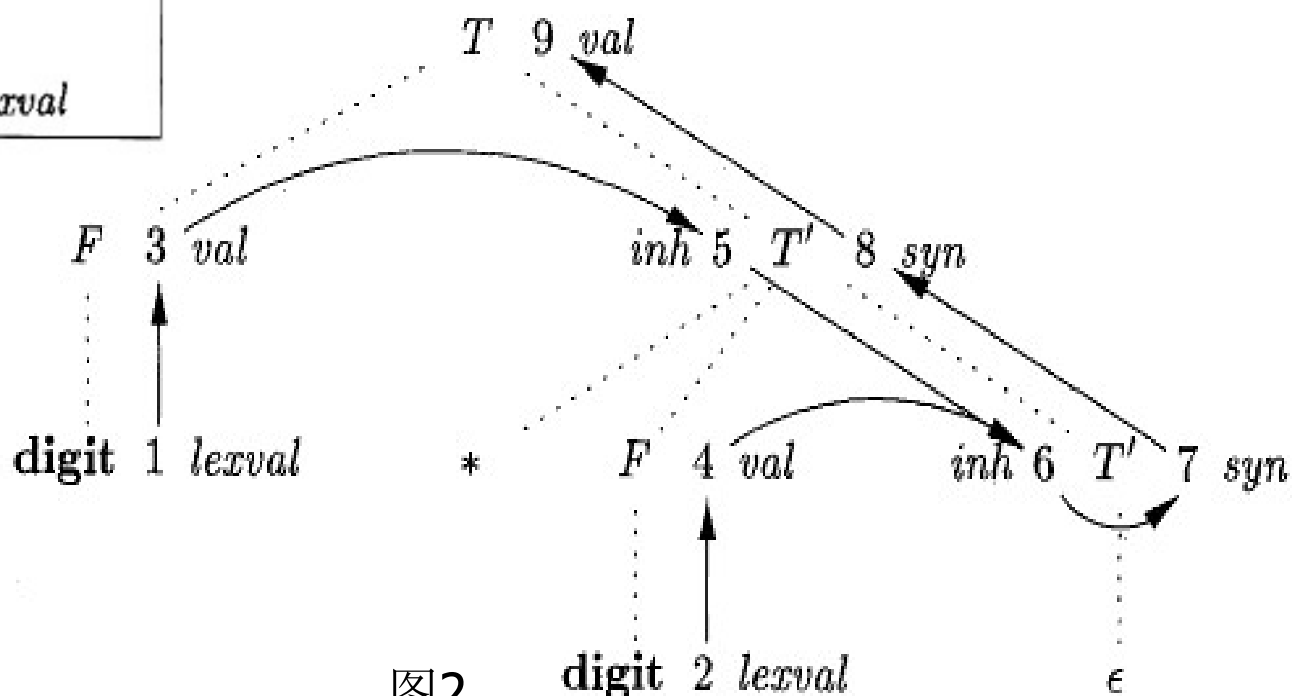


图2



习题8.2

共10种拓扑排序，分别是：

- (1) 24 135 6789
- (2) 1 24 35 6789
- (3) 13 24 5 6789
- (4) 135 24 6789
- (5) 21 435 6789
- (6) 21 345 6789
- (7) 21 354 6789
- (8) 12 345 6789
- (9) 12 354 6789
- (10) 13 254 6789



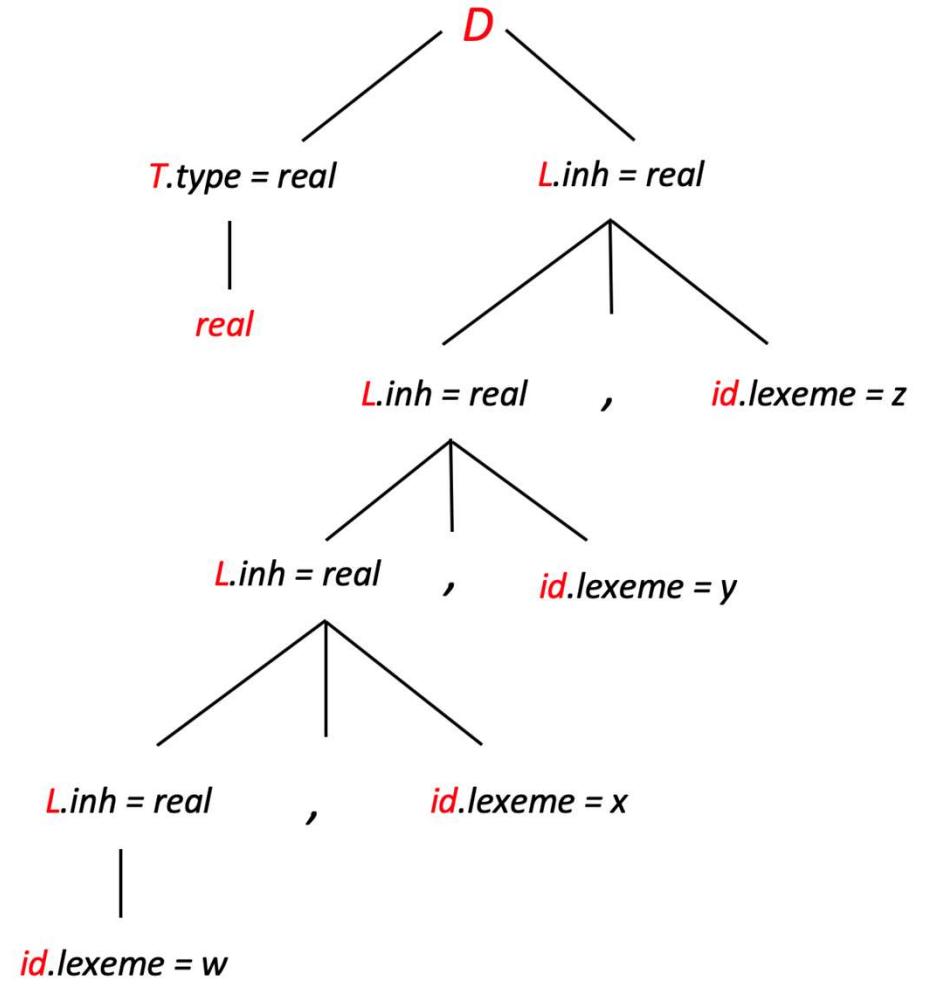
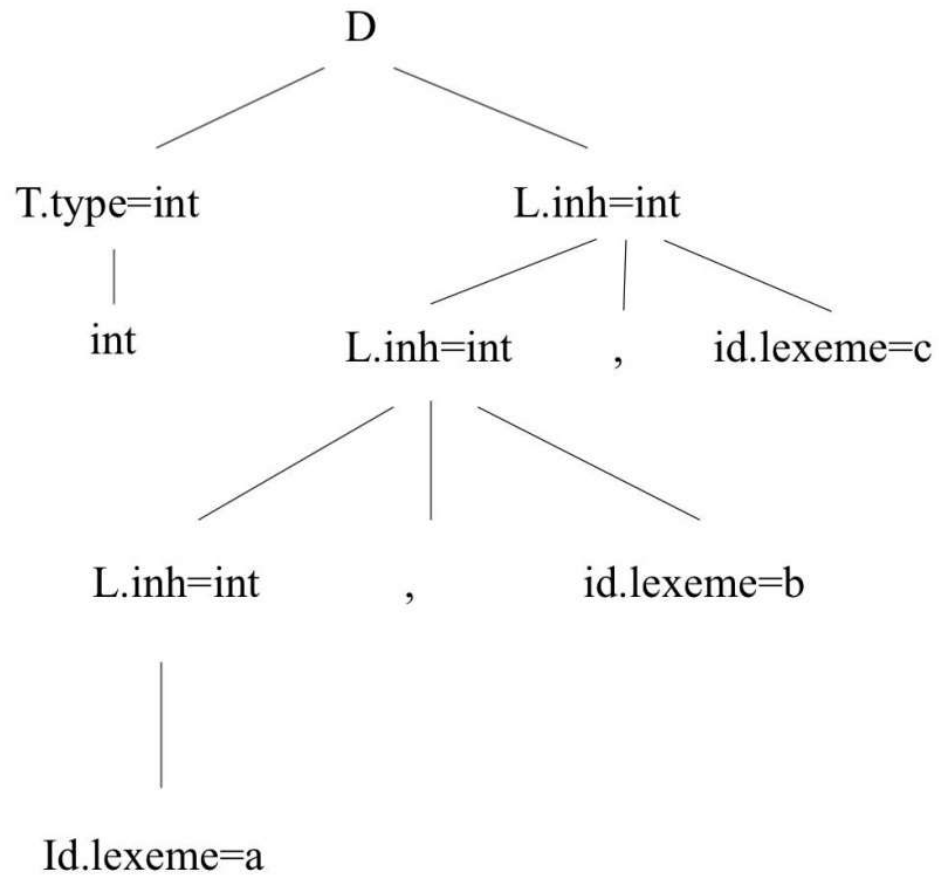
习题8.3

➤ 对于下图中的**SDD**，给出下列表达式对应的注释语法分析树：

➤ (1) **int a,b,c**

➤ (2) **real w,x,y,z**

	产生式	语义规则
(1)	$D \rightarrow T L$	$L.inh = T.type$
(2)	$T \rightarrow \text{int}$	$T.type = \text{int}$
(3)	$T \rightarrow \text{real}$	$T.type = \text{real}$
(4)	$L \rightarrow L_1, \text{id}$	$L_1.inh = L.inh$ $addtype(\text{id.lexeme}, L.inh)$
(5)	$L \rightarrow \text{id}$	$addtype(\text{id.lexeme}, L.inh)$





习题8.4

➤ 假设我们有一个产生式 $A \rightarrow BCD$ 。A、B、C、D 这四个非终结符都有两个属性：s 是一个综合属性，而 i 是一个继承属性。对于下面的每组规则，指出

- ① 这些规则是否满足 S 属性定义的要求。
- ② 这些规则是否满足 L 属性定义的要求。
- ③ 是否存在和这些规则一致的求值过程？

➤ (1) $A.s = B.i + C.s$



习题8.4

- 仅仅使用综合属性的SDD称为S属性的SDD，
或S-属性定义、S-SDD

$$A.s = B.i + C.s$$



继承属性

所以不满足S属性定义的要求

习题8.4

- 一个SDD是**L-属性定义**，当且仅当它的每个属性要么是一个**综合属性**，要么是满足如下条件的继承属性：假设存在一个产生式 $A \rightarrow X_1X_2...X_n$ ，其右部符号 $X_i (1 \leq i \leq n)$ 的**继承属性**仅依赖于下列属性：
 - A 的**继承属性**
 - 产生式中 X_i **左边**的符号 $X_1, X_2, \dots, X_{i-1}$ 的属性
 - X_i 本身的属性，但 X_i 的全部属性不能在依赖图中形成环路

每个S-属性定义都是L-属性定义

$$A.s = B.i + C.s$$

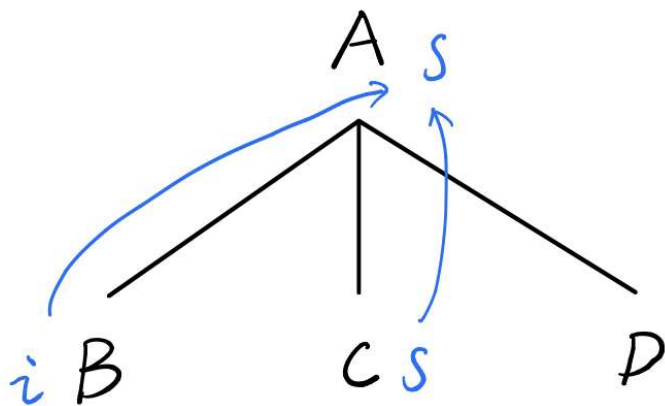
无右部符号的规则，即产生式的右部符号满足定义

所以是L属性定义的



习题8.4

➤ 是否存在和这些规则一致的求值过程？



关键是：是否存在一个求值顺序？

“如果图中没有环 则至少存在一个拓扑排序”很显然，图中并无环；而且，由上问：满足L-属性，满足L-属性定义是没有环存在的。因此存在和规则 $A.s = B.i + C.s$ 一致的求值顺序



习题8.4

➤ 假设我们有一个产生式 $A \rightarrow BCD$ 。A、B、C、D 这四个非终结符都有两个属性：s 是一个综合属性，而 i 是一个继承属性。对于下面的每组规则，指出

- ① 这些规则是否满足 S 属性定义的要求。
- ② 这些规则是否满足 L 属性定义的要求。
- ③ 是否存在和这些规则一致的求值过程？

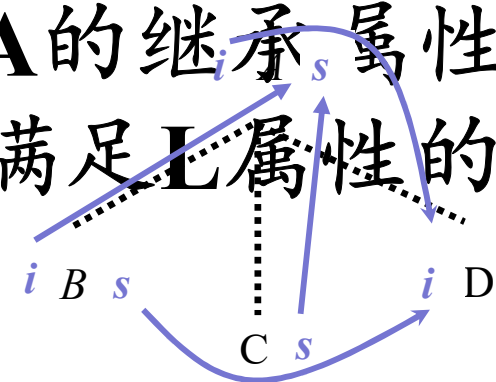
➤ (2) $A.s = B.i + C.s$

$D.i = A.i + B.s$

(2) $A.s = B.i + C.s$

$D.i = A.i + B.s$

- S属性：不满足，因为使用了继承属性
- L属性：满足，因为第一个式子计算综合属性，第二个式子依赖于A的继承属性和他左边符号的综合属性，满足L属性的定义。



- 求值过程：存在

eg:

$B.s \rightarrow A.i \rightarrow D.i \rightarrow B.i \rightarrow C.s \rightarrow A.s$

拓扑序列一共有80种，不一一给出。



习题8.4

➤ 假设我们有一个产生式 $A \rightarrow BCD$ 。A、B、C、D 这四个非终结符都有两个属性：s 是一个综合属性，而 i 是一个继承属性。对于下面的每组规则，指出

① 这些规则是否满足 S 属性定义的要求。

② 这些规则是否满足 L 属性定义的要求。

③ 是否存在和这些规则一致的求值过程？

➤ (3) $A.s = B.s + D.s$



习题8.4 (3)

➤ (3) $A.s = B.s + D.s$

➤ 仅仅使用综合属性

→

满足S属性定义的要求

➤ 每个属性都是综合属性

→

满足L属性定义的要求

➤ S属性性质:

➤ 如果一个SDD是S属性的, 可以按照语法分析树节点的任何
自底向上顺序来计算它的各个属性值

➤ S-属性定义可以在自底向上的语法分析过程中实现

终结符可以具有综合属性。终结符的综合属性值是由词法分析器提供的词法值, 因此在SDD中没有计算终结符属性值的语义规则

■ 存在和此规则一致的求值过程



习题8.4

➤ 假设我们有一个产生式 $A \rightarrow BCD$ 。A、B、C、D 这四个非终结符都有两个属性：s 是一个综合属性，而 i 是一个继承属性。对于下面的每组规则，指出

- ① 这些规则是否满足 S 属性定义的要求。
- ② 这些规则是否满足 L 属性定义的要求。
- ③ 是否存在和这些规则一致的求值过程？

➤ (4) $A.s = D.i$

$$B.i = A.s + C.s$$

$$C.i = B.s$$

$$D.i = B.i + C.i$$



习题8.4

- (1) 语义规则里有继承属性，故**不满足**S属性定义。
- (2) B的继承属性依赖了它右边的属性，故**不满足**L属性定义。
- (3) **不存在**，因为依赖关系成了环（ $A.s \leftarrow B.i \leftarrow D.i \leftarrow A.s$ ）。

- $A.s = D.i$
- $B.i = A.s + C.s$
- $C.i = B.s$
- $D.i = B.i + C.i$

习题9.1

- 下图中的SDD计算诸如 $3*5$ 和 $3*5*7$ 这样的项。扩展下图中的SDD，使它可以像习题8.1图中所示的那样处理表达式。

产生式	语义规则
1) $T \rightarrow F T'$	$T'.inh = F.val$ $T.val = T'.syn$
2) $T' \rightarrow * F T'_1$	$T'_1.inh = T'.inh \times F.val$ $T'.syn = T'_1.syn$
3) $T' \rightarrow \epsilon$	$T'.syn = T'.inh$
4) $F \rightarrow \text{digit}$	$F.val = \text{digit.lexval}$



习题9.1

➤ 8.1的SDD能够处理的表达式：以n结尾，包含+、*、()的表达式，例如 $(3+5)*2*4+1+2+3$ ，是S-SDD

产生式	语义规则
1) $L \rightarrow E n$	$L.val = E.val$
2) $E \rightarrow E_1 + T$	$E.val = E_1.val + T.val$
3) $E \rightarrow T$	$E.val = T.val$
4) $T \rightarrow T_1 * F$	$T.val = T_1.val \times F.val$
5) $T \rightarrow F$	$T.val = F.val$
6) $F \rightarrow (E)$	$F.val = E.val$
7) $F \rightarrow \text{digit}$	$F.val = \text{digit.lexval}$



习题9.1

➤ 9.1的SDD能够处理的表达式：包含*的表达式，是L-SDD

	产生式	语法规则
1	$T \rightarrow FT'$	$T'.inh = F.val$ $T.val = T'.syn$
2	$T' \rightarrow *FT'_1$	$T1'.inh = T'.inh \times F.val$ $T'.syn = T1'.syn$
3	$T' \rightarrow \epsilon$	$T'.syn = T'.inh$
4	$F \rightarrow digit$	$F.val = digit.lexval$



习题9.1

➤ 举一反三：能够处理包含+的表达式的L-SDD

	产生式	语法规则
1	$A \rightarrow BA'$	$A'.inh = B.val$ $A.val = A'.syn$
2	$A' \rightarrow +BA'_1$	$A1'.inh = A'.inh + B.val$ $A'.syn = A1'.syn$
3	$A' \rightarrow \epsilon$	$A'.syn = A'.inh$
4	$B \rightarrow digit$	$B.val = digit.lexval$



习题9.1

- 分析得知，这里的**B**是连加法表达式的每一项，而算术表达式的每一项还可以是连乘表达式，因此添加一个产生式及语法规则：

	产生式	语法规则
1	$A \rightarrow BA'$	$A'.inh = B.val$ $A.val = A'.syn$
2	$A' \rightarrow +BA'_1$	$A1'.inh = A'.inh + B.val$ $A'.syn = A1'.syn$
3	$A' \rightarrow \epsilon$	$A'.syn = A'.inh$
4	$B \rightarrow digit$	$B.val = digit.lexval$
5	$B \rightarrow T$	$B.val = T.val$



习题9.1

- 分析得知，这里的F是连乘法表达式的每一项，而算术表达式的每一项还可以是加上括号的连加表达式，因此添加一个产生式及语法规则：

	产生式	语法规则
1	$T \rightarrow FT'$	$T'.inh = F.val$ $T.val = T'.syn$
2	$T' \rightarrow *FT'_1$	$T1'.inh = T'.inh \times F.val$ $T'.syn = T1'.syn$
3	$T' \rightarrow \epsilon$	$T'.syn = T'.inh$
4	$F \rightarrow digit$	$F.val = digit.lexval$
5	$F \rightarrow (A)$	$F.val = A.val$



习题9.1

➤ 综合在一起

	产生式	语法规则
1	$C \rightarrow An$	$C.val = A.val$
2	$T \rightarrow FT'$	$T'.inh = F.val$ $T.val = T'.syn$
3	$T' \rightarrow *FT'_1$	$T1'.inh = T'.inh * F.val$ $T'.syn = T1'.syn$
4	$T' \rightarrow \epsilon$	$T'.syn = T'.inh$
5	$F \rightarrow digit$	$F.val = digit.lexval$
6	$F \rightarrow (A)$	$F.val = A.val$
7	$A \rightarrow BA'$	$A'.inh = B.val$ $A.val = A'.syn$
8	$A' \rightarrow +BA'_1$	$A1'.inh = A'.inh + B.val$ $A'.syn = A1'.syn$
9	$A' \rightarrow \epsilon$	$A'.syn = A'.inh$
10	$B \rightarrow digit$	$B.val = digit.lexval$
11	$B \rightarrow T$	$B.val = T.val$



习题9.1

➤ 经过讨论，
我们发现10
和11两个产
生式没有用，
用T来代替B
就可以

	产生式	语法规则
1	$C \rightarrow An$	$C.val = A.val$
2	$T \rightarrow FT'$	$T'.inh = F.val$ $T.val = T'.syn$
3	$T' \rightarrow *FT'_1$	$T1'.inh = T'.inh * F.val$ $T'.syn = T1'.syn$
4	$T' \rightarrow \epsilon$	$T'.syn = T'.inh$
5	$F \rightarrow digit$	$F.val = digit.lexval$
6	$F \rightarrow (A)$	$F.val = A.val$
7	$A \rightarrow TA'$	$A'.inh = T.val$ $A.val = A'.syn$
8	$A' \rightarrow +TA'_1$	$A1'.inh = A'.inh + T.val$ $A'.syn = A1'.syn$
9	$A' \rightarrow \epsilon$	$A'.syn = A'.inh$



习题9.2

➤这个文法生成了含“小数点”的二进制数：

$$S \rightarrow L . L \mid L$$

$$L \rightarrow L B \mid B$$

$$B \rightarrow 0 \mid 1$$

设计一个**L属性**的SDD来计算S.val，即输入串的十进制数值。比如，串101.101应该被翻译成十进制数5.625。提示：使用一个继承属性L.side来指明一个二进制位在小数点的那一边。



答案一:

	产生式	语法规则
1)	$S \rightarrow L_1.L_2$	$L_1.isLeft = true$ $L_2.isLeft = false$ $S.val = L_1.val + L_2.val$
2)	$S \rightarrow L$	$L.isLeft = true$ $S.val = L.val$
3)	$L \rightarrow L_1B$	$L_1.isLeft = L.isLeft$ $L.len = L_1.len + 1$ $L.val = L.isLeft ? L_1.val * 2 + B.val : L_1.val + B.val * 2^{(-L.len)}$
4)	$L \rightarrow B$	$L.len = 1$ $L.val = L.isLeft ? B.val : B.val/2$
5)	$B \rightarrow 0$	$B.val = 0$
6)	$B \rightarrow 1$	$B.val = 1$



答案二：

- 消除左递归：
- $S \rightarrow L.L | L \quad L \rightarrow BL' \quad L' \rightarrow BL' | \varepsilon \quad B \rightarrow 0 | 1$
- 同时使用继承属性 **L.side** 表示 **L** 在小数点左边 (**true**) 还是右边 (**false**)，综合属性 **L.val** 表示具体的二进制值，综合属性 **L.dep** 表示 **B** 的幂次



产生式	语义规则
$S \rightarrow LS'$	$S.val = L.val + S'.val$ $L.left = true$
$S' \rightarrow .L$	$S'.val = L.val$ $L.left = false$
$S' \rightarrow \varepsilon$	$S'.val = 0$
$L \rightarrow BL'$	$L'.left = L.left$ $L.val = L.left ? L'.val + B.val * 2^{(L'.dep)} : L'.val / 2 + B.val / 2$
$L' \rightarrow BL1'$	$L1'.left = L'.left$ $L'.dep = L1'.dep + 1$ $L'.val = L'.left ? L1'.val + B.val * 2^{(L1'.dep)} : L1'.val / 2 + B.val / 2$
$L' \rightarrow \varepsilon$	$L'.val = 0$ $L'.dep = 0$
$B \rightarrow 0$	$B.val = 0$
$B \rightarrow 1$	$B.val = 1$



习题9.3

➤这个文法生成了含“小数点”的二进制数：

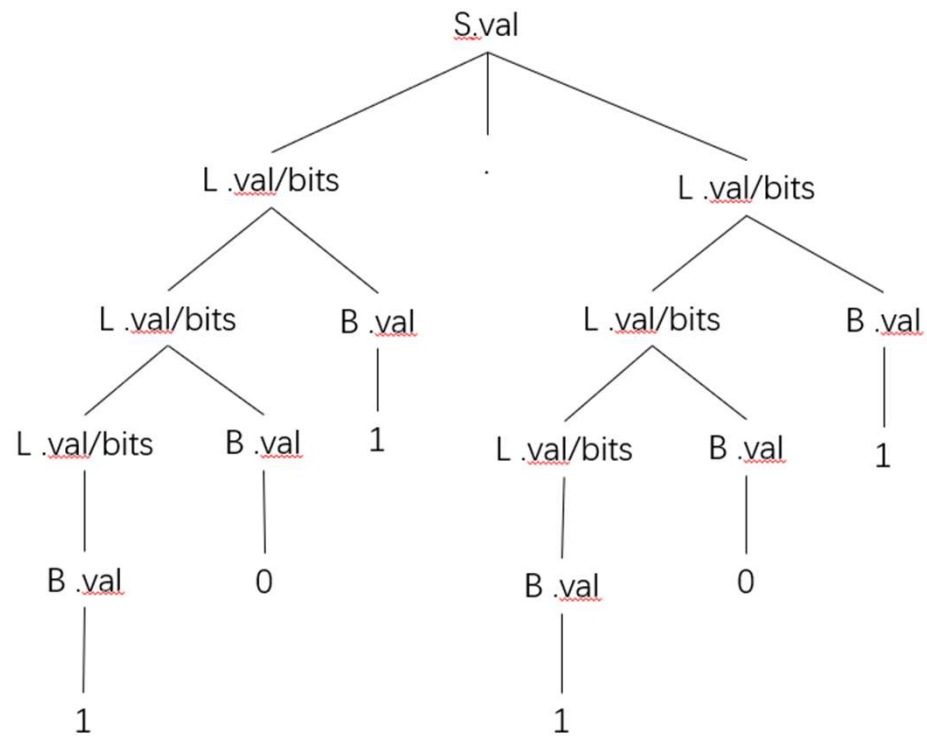
$$S \rightarrow L . L \mid L$$

$$L \rightarrow L B \mid B$$

$$B \rightarrow 0 \mid 1$$

设计一个**S属性**的SDD来计算S.val，即输入串的十进制数值。比如，串101.101应该被翻译成十进制数5.625。

完整的注释分析树



S属性的SDD

产生式	语义规则
$S \rightarrow L_1 \cdot L_2$	$S.val = L_1.val + L_2.val \times (1/2)^{L_2.bits}$
$S \rightarrow L$	$S.val = L.val$
$L \rightarrow L_1 B$	$L.val = L_1.val \times 2 + B.val, L.bits = L_1.bits + 1$
$L \rightarrow B$	$L.val = B.val, L.bits = 1$
$B \rightarrow 0$	$B.val = 0$
$B \rightarrow 1$	$B.val = 1$



习题9.4

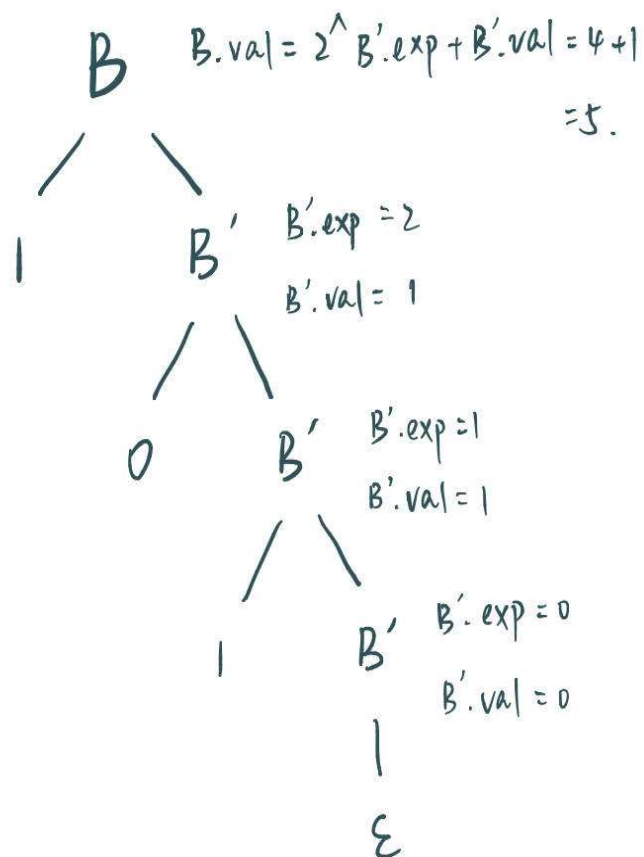
- 下面的SDT计算了一个由0和1组成的串的值。它把输入的符号串当作按照正二进制数来解释。

$B \rightarrow B_1 0 \{B.val = 2 * B_1.val\}$

$| B_1 1 \{B.val = 2 * B_1.val + 1\}$

$| 1 \{B.val = 1\}$

改写这个SDT，使得基础文法不再是左递归的，但仍然可以计算出整个输入串的相同的**B.val**的值。



输入串101时的分析树

$B.val$ 、 $B'.exp$ 、 $B'.val$ 都是综合属性

$B \rightarrow 1B' \{B.val = 2^{B'.exp} + B'.val\}$

$B' \rightarrow 0B'_1 \{B'.exp = B'_1.exp + 1; B'.val = B'_1.val\} |$

$1B'_1 \{B'.exp = B'_1.exp + 1; B'.val = B'_1.val + 2^{B'_1.exp}\} |$

$\epsilon \{B'.exp = 0; B'.val = 0\}$



习题10.1

- 将习题9.2中得到的SDD实现为递归下降的语法分析器。



习题9.2

➤9.2相关解答:

	产生式	语法规则
1)	$S \rightarrow L_1.L_2$	$L_1.isLeft = true$ $L_2.isLeft = false$ $S.val = L_1.val + L_2.val$
2)	$S \rightarrow L$	$L.isLeft = true$ $S.val = L.val$
3)	$L \rightarrow L_1B$	$L_1.isLeft = L.isLeft$ $L.len = L_1.len + 1$ $L.val = L.isLeft ? L_1.val * 2 + B.val : L_1.val + B.val * 2^{(-L.len)}$
4)	$L \rightarrow B$	$L.len = 1$ $L.val = L.isLeft ? B.val : B.val/2$
5)	$B \rightarrow 0$	$B.val = 0$
6)	$B \rightarrow 1$	$B.val = 1$

习题10.1

➤ 为了构造相应的递归下降SDD。

- 如果 X 是一个终结符, 那么 $FIRST(X) = \{X\}$
- 如果 X 是一个非终结符, 且 $X \rightarrow Y_1 \dots Y_k \in P (k \geq 1)$, 那么如果对于某个 i , a 在 $FIRST(Y_i)$ 中且 ε 在所有的 $FIRST(Y_1), \dots, FIRST(Y_{i-1})$ 中(即 $Y_1 \dots Y_{i-1} \Rightarrow^* \varepsilon$), 就把 a 加入到 $FIRST(X)$ 中。如果对于所有的 $j = 1, 2, \dots, k$, ε 在 $FIRST(Y_j)$ 中, 那么将 ε 加入到 $FIRST(X)$
- 如果 $X \rightarrow \varepsilon \in P$, 那么将 ε 加入到 $FIRST(X)$ 中

- 将 S 放入 $FOLLOW(S)$ 中, 其中 S 是开始符号, S 是输入右端的结束标记
- 如果存在一个产生式 $A \rightarrow aB\beta$, 那么 $FIRST(\beta)$ 中除 ε 之外的所有符号都在 $FOLLOW(B)$ 中
- 如果存在一个产生式 $A \rightarrow aB$, 或存在产生式 $A \rightarrow aB\beta$ 且 $FIRST(\beta)$ 包含 ε , 那么 $FOLLOW(A)$ 中的所有符号都在 $FOLLOW(B)$ 中

1. 消除直接左递归
将 $L \rightarrow LB|B$ 改为
 $L \rightarrow BL'$
 $L' \rightarrow BL'|\varepsilon$
 2. 提取左公因子
将 $S \rightarrow LL|L$
 $S \rightarrow LS'$
 $S' \rightarrow .L|\varepsilon$
 3. Select 集
此时的公式
- | | |
|-------------------------------------|-------------------------|
| $FIRST(S) = \{0, 1\}$ | $SELECT(1) = \{0, 1\}$ |
| $FIRST(S') = \{., \varepsilon\}$ | $SELECT(2) = \{.\}$ |
| $FIRST(L) = \{0, 1\}$ | $SELECT(3) = \{\$ \}$ |
| $FIRST(L') = \{0, 1, \varepsilon\}$ | $SELECT(4) = \{0, 1\}$ |
| $FIRST(B) = \{0, 1\}$ | $SELECT(5) = \{0, 1\}$ |
| | $SELECT(6) = \{\$, .\}$ |
| $FOLLOW(S) = \{\$ \}$ | $SELECT(7) = \{0\}$ |
| $FOLLOW(S') = \{\$ \}$ | $SELECT(8) = \{1\}$ |
| $FOLLOW(L) = \{\$, .\}$ | |
| $FOLLOW(L') = \{\$, .\}$ | |
| $FOLLOW(B) = \{\$, 0, 1, .\}$ | |
- (1) $S \rightarrow LS'$
 - (2) $S' \rightarrow .L$
 - (3) $S' \rightarrow \varepsilon$
 - (4) $L \rightarrow BL'$
 - (5) $L' \rightarrow BL'$
 - (6) $L' \rightarrow \varepsilon$
 - (7) $B \rightarrow 0$
 - (8) $B \rightarrow 1$



习题10.1

$S \rightarrow LS'$	$L.isleft = true$ $S.val = L.val + S'.val$
$S' \rightarrow \cdot L$	$L.isleft = false$ $S'.val = L.val$
$S' \rightarrow \epsilon$	$S'.val = 0$
$L \rightarrow BL'$	$L'.isleft = L.isleft$ $L.len = L'.len + 1$ $L.val = L.isLeft ? B.val \times 2^{L'.len} + L'.val : (B.val + L'.val) \times 2^{(-1)}$
$L' \rightarrow BL'_1$	$L_1.isleft = L'.isleft$ $L'.len = L'_1.len + 1$ $L'.val = L'.isLeft ? B.val \times 2^{L'_1.len} + L'_1.val : (B.val + L'_1.val) \times 2^{(-1)}$
$L' \rightarrow \epsilon$	$L'.val = 0$ $L'.len = 0$
$B \rightarrow 0$	$B.val = 0$
$B \rightarrow 1$	$B.val = 1$



习题10.1

➤ 翻译结果如下:

```
DESCENT()
{
    D:Sval;
    GETNEXT(TOKEN);
    Sval=S(TOKEN);
    if TOKEN≠' '$' then ERROR;
    return;
}
```

```
Sval S(TOKEN)
{
    D:Lisleft,Lval ,S'val;
    if TOKEN ='0' or TOKEN = '1' then
    {
        Lisleft = true;
        {Lval,~} =L(TOKEN,Lisleft);
        S'val =S'(TOKEN);
        Sval =Lval + S'val
        Return Sval
    }
    Else ERROR;
}
```

$S \rightarrow \{L . \text{isleft} = \text{true}\} LS' \{S.\text{val} = L.\text{val} + S'.\text{val}\}$



S'val S'(TOKEN)

```
{  
    D:Lisleft,Lval;  
    if TOKEN != ' then  
    {  
        Lisleft = false;  
        GETNEXT(TOKEN);  
        {Lval,~} = L(TOKEN,Lisleft);  
        S'val = Lval;  
        Return S'val;  
    }  
    Else if TOKEN == $ then  
    {  
        S'val=0;  
        Return S'val;  
    }  
    Else ERROR;  
}
```

$S' \rightarrow \cdot \{L.isleft = false\}L \{S'.val = L.val\}$

$S' \rightarrow \varepsilon \{S'.val = 0\}$



```
{Lval, Llen} L(TOKEN, Lisleft)
{
  D:L'isleft, Lisleft, L'len, L'val, Bval;
  if TOKEN = '0' or TOKEN = '1' then
  {
    Bval=B(TOKEN);
    L'isleft = Lisleft;
    { L'val, L'len }=L'(TOKEN, L'isleft);
    Llen = L'len + 1;
    L'val = L'isLeft ? Bval x 2^L'len + L'val : ( Bval +L'val)x2^(-1) ;
    Return {Lval, Llen};
  }
  Else ERROR;
}
```

$L \rightarrow B \{L'.isleft = L.isleft\} L' \{L.len = L'.len + 1, L.val = L.isleft ? B.val * 2^{L'.len} + L'.val : (B.val + L'.val) * 2^{-1}\}$



习题10.1

➤ 翻译结果如下：

```
{L'.val', L'.len'} L'(TOKEN, L'.isleft)
{
  D: L'.isleft, L'.isleft, L'.len, B.val, L'.val;
  if TOKEN = '0' or TOKEN = '1' then
  {
    B.val = B(TOKEN);
    L'.isleft = L'.isleft
    {L'.val, L'.len} = L'.len (TOKEN, L'.isleft);
    L'.len = L'.len + 1;
    L'.val = L'.isleft ? B.val x 2^L'.len + L'.val : (B.val + L'.val)x2^(-1);
    GETNEXT(TOKEN);
    Return {L'.val, L'.len};
  }
  Else if TOKEN = '$' or TOKEN = '.' then
  {
    L'.val = 0;
    L'.len = 0;
    Return {L'.val, L'.len};
  }
  Else ERROR;
}
```

$$L' \rightarrow B \{ L'_1.isleft = L'.isleft \} L'_1 \{ L'.len = L'_1.len + 1, L'.val = L'.isleft ? B.val * 2^{L'_1.len} + L'_1.val : (B.val + L'_1.val) * 2^{-1} \}$$

$$L' \rightarrow \varepsilon \{ L'.val = 0, L'.len = 0 \}$$



```
Bval B(TOKEN)
{
    if TOKEN ='0' then
    {
        Bval = 0;
        GETNEXT(TOKEN);
        Return Bval;
    }
    Else if TOKEN = '1' then
    {
        Bval = 1;
        GETNEXT(TOKEN);
        Return Bval;
    }
    Else ERROR;
}
```

B -> 0 {B.val = 0}

B -> 1 {B.val = 1}-



习题12.1

➤ 将算术表达式 $a + -(b + c)$ 翻译成四元式序列

(1) $(+, b, c, t1)$

(2) $(-, t1, /, t2)$

(3) $(+, a, t2, t3)$



习题12.2

➤ 将下列赋值语句翻译成四元式序列（假设每个数组元素占8个存储单元）。

➤ (1) $a = b[i] + c[j]$

➤ (2) $a[i] = b * c - b * d$

➤ (3) $x = f(y+1) + 2$

➤ (4) $x = *p + \&y$



解12.2

(1) $a = b[i] + c[j]$

三地址码

四元式

$t1 = i * 8$

$(*, i, 8, t1)$

$t2 = b[t1]$

$(=[], b, t1, t3)$

$t3 = j * 8$

$(*, j, 8, t3)$

$t4 = c[t2]$

$(=[], c, t2, t4)$

$t5 = t3 + t4$

$(+, t3, t4, t5)$

$a = t5$

$(=, t5, _, a)$

8	数组引用	$x = y[i]$
9	数组赋值	$x[i] = y$

$\triangleright x = y[i]$ $(_, \bar{y}, \bar{i}, x)$
 $x[i] = y$ $(\bar{i}, y, x, _)$



解12.2

(2) $a[i] = b*c - b*d$

三地址码

四元式

$t1 = i*8$

$(*, i, 8, t1)$

$t3 = b*c$

$(*, b, c, t3)$

$t4 = b*d$

$(*, b, d, t4)$

$t5 = t3 - t4$

$(-, t3, t4, t5)$

$a = t5$

$([] =, t5, a, t1)$



解12.2

(3) $x = f(y+1) + 2$

三地址码

$t1 = y+1$

param t1

$t2 = \text{call } f, 1$

$t3 = t2+2$

$x = t3$

四元式

$(+, y, 1, t1)$

$(\text{param}, _, _, t1)$

$(\text{call}, f, 1, t2)$

$(+, t2, 2, t3)$

$(=, t3, _, x)$

5	参数传递	param x
6	过程调用 函数调用	call p, n $y = \text{call } p, n$

param x
 $y = \text{call } p, n$

(param, _ , _ , x)
(call , p , n , y)



解12.2

(4) $x = *p + \&y$

三地址码

$t1 = *p$

$t2 = \&y$

$t3 = t1 + t2$

$x = t3$

四元式

$(=*, p, _, t1)$

$(\&, y, _, t2)$

$(+, t1, t2, t3)$

$(=, t3, _, x)$

10	地址及 指针操作	$x = \&y$ $x = *y$ $*x = y$
----	-------------	-----------------------------------

$x = \&y$

$x = *y$

$*x = y$

$(\&, y, _, x)$

$(=* , y, _, x)$

$(*= , y, _, x)$

习题12.3

- 使用讲义中的翻译方案（如下图所示）翻译下列赋值语句。
假设a和b的类型表达式都是 $\text{array}(3, \text{array}(5, \text{real}))$ ，数组c中每个元素（real类型）占8个存储单元

$S \rightarrow \text{id} = E;$

$| L = E; \{ \text{gen}(L.\text{array} \text{ '[' } L.\text{offset} \text{ ']' '}' E.\text{addr}); \}$

$E \rightarrow E_1 + E_2 \mid -E_1 \mid (E_1) \mid \text{id}$

$| L \{ E.\text{addr} = \text{newtemp}(); \text{gen}(E.\text{addr} \text{ '=' } L.\text{array} \text{ '[' } L.\text{offset} \text{ ']' '}); \}$

$L \rightarrow \text{id} [E] \{ L.\text{array} = \text{lookup}(\text{id}.\text{lexeme}); \text{if } L.\text{array} == \text{nil then error};$

$L.\text{type} = L.\text{array}.\text{type}.\text{elem};$

$L.\text{offset} = \text{newtemp}();$

$\text{gen}(L.\text{offset} \text{ '=' } E.\text{addr} \text{ '*' } L.\text{type}.\text{width}); \}$

$| L_1[E] \{ L.\text{array} = L_1.\text{array};$

$L.\text{type} = L_1.\text{type}.\text{elem};$

$t = \text{newtemp}();$

$\text{gen}(t \text{ '=' } E.\text{addr} \text{ '*' } L.\text{type}.\text{width});$

$L.\text{offset} = \text{newtemp}();$

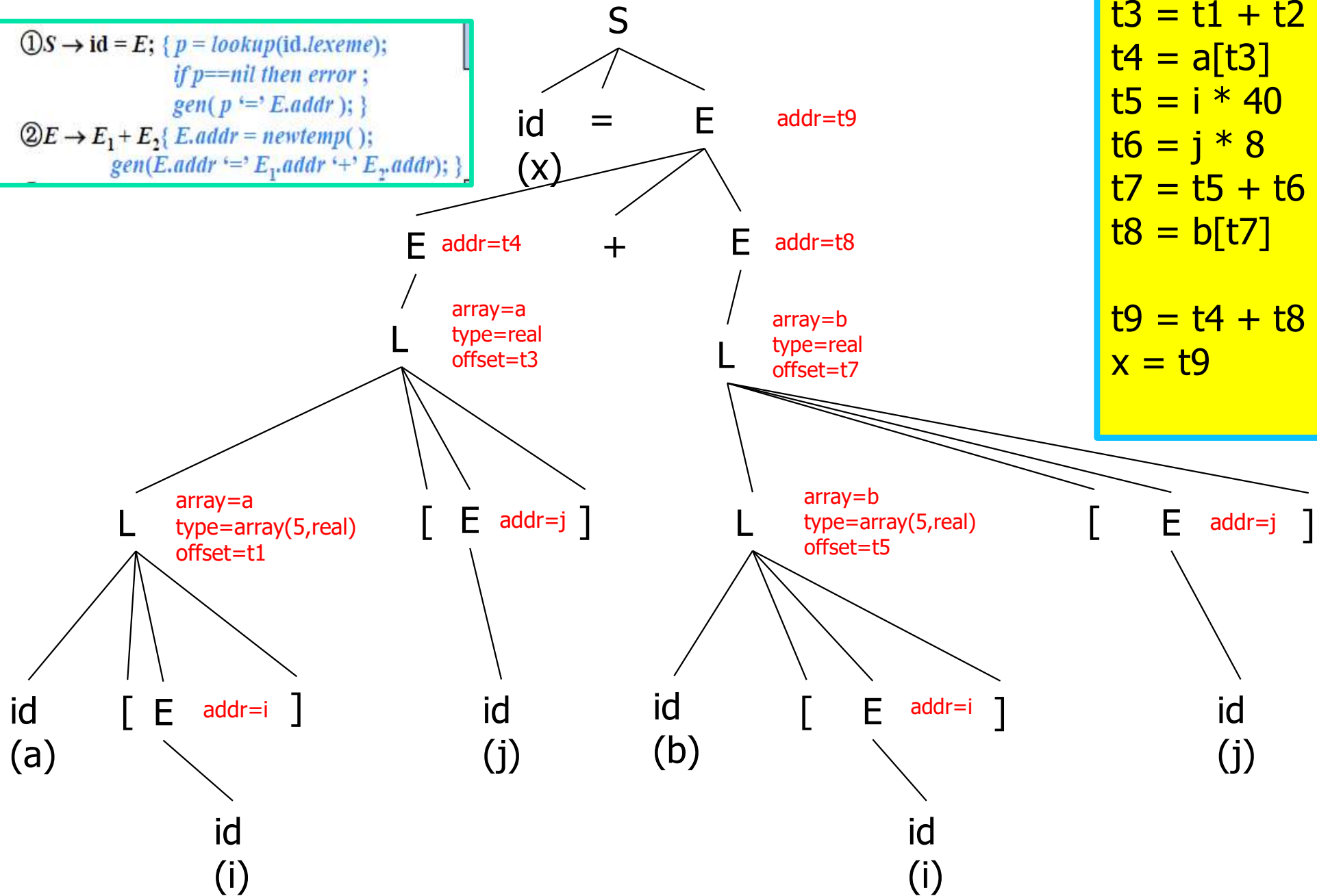
$\text{gen}(L.\text{offset} \text{ '=' } L_1.\text{offset} \text{ '+' } t); \}$

➤ (1) $x = a[i][j] + b[i][j]$

➤ (2) $x = a[b[i][j]][c[k]]$

答案

① $S \rightarrow id = E; \{ p = lookup(id.lexeme);$
if $p == nil$ then error;
gen($p := E.addr$); $\}$
② $E \rightarrow E_1 + E_2 \{ E.addr = newtemp();$
gen($E.addr := E_1.addr '+' E_2.addr$); $\}$



t1 = i * 40
t2 = j * 8
t3 = t1 + t2
t4 = a[t3]
t5 = i * 40
t6 = j * 8
t7 = t5 + t6
t8 = b[t7]

t9 = t4 + t8
x = t9

习题12.3

$type(a)=type(b)=array(3, array(5, real)), type(c) = array(n, real)$,
翻译语句片段 “ $x = a[b[i][j]][c[k]]$;

① $S \rightarrow id = E$; { $p = lookup(id.lexeme)$;
if $p == nil$ then error;
 $gen(p '=' E.addr)$; }

② $S \rightarrow L = E$;
{ $gen(L.array '[' L.offset ']' '=' E.addr)$; }

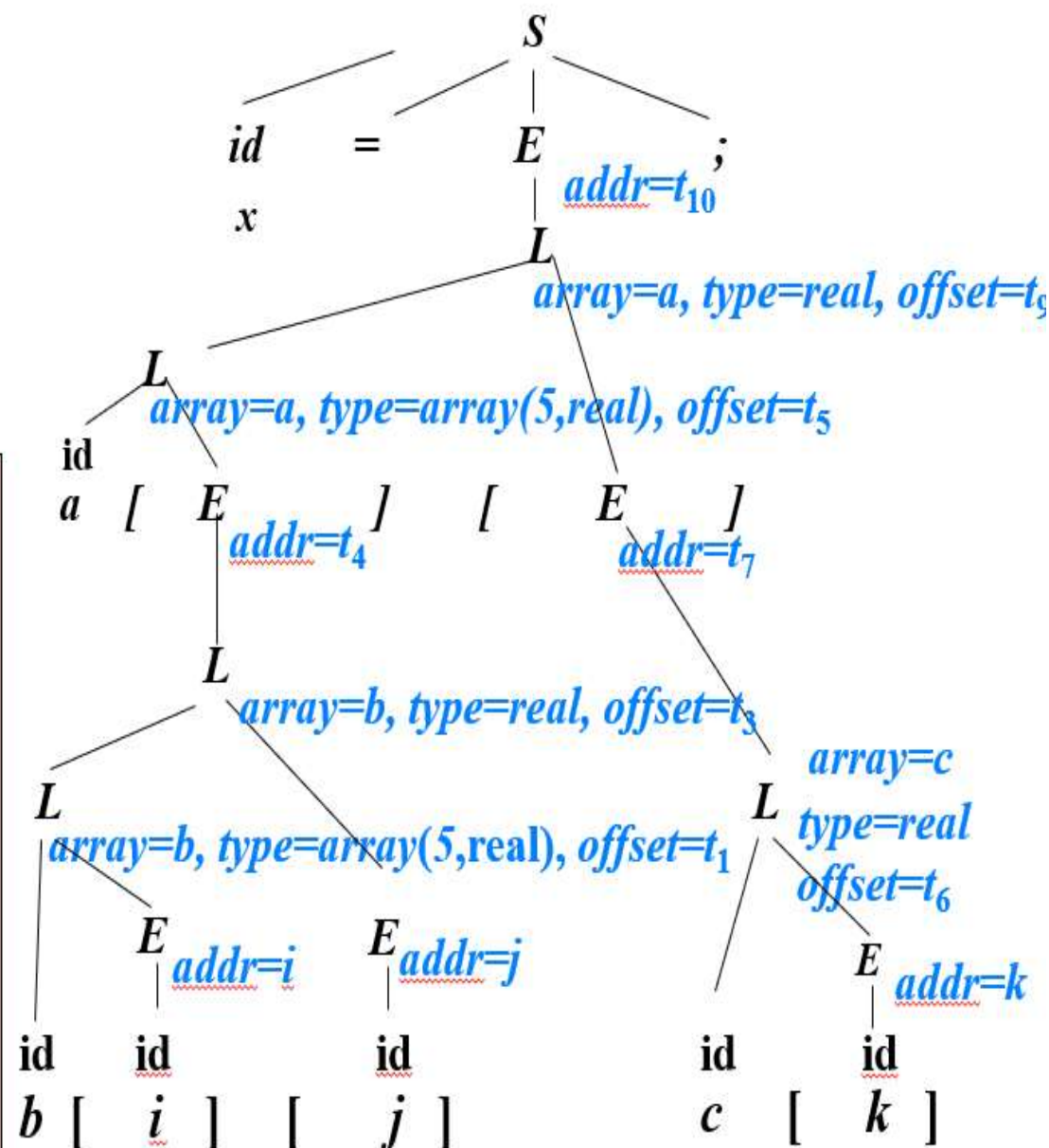
③ $E \rightarrow id$ { $E.addr = lookup(id.lexeme)$;
if $E.addr == nil$ then error; }

④ $E \rightarrow L$ { $E.addr = newtemp()$;
 $gen(E.addr '=' L.array '[' L.offset '])$; }

⑤ $L \rightarrow id [E]$ { $L.array = lookup(id.lexeme)$;
if $L.array == nil$ then error;
 $L.type = L.array.type.elem$;
 $L.offset = newtemp()$;
 $gen(L.offset '=' E.addr '*' L.type.width)$; }

⑥ $L \rightarrow L_1 [E]$ { $L.array = L_1.array$;
 $L.type = L_1.type.elem$;
 $t = newtemp()$;
 $gen(t '=' E.addr '*' L.type.width)$;
 $L.offset = newtemp()$;
 $gen(L.offset '=' L_1.offset '+' t)$; }

$t_1 = i * 40$
 $t_2 = j * 8$
 $t_3 = t_1 + t_2$
 $t_4 = b[t_3]$
 $t_5 = t_4 * 40$
 $t_6 = k * 8$
 $t_7 = c[t_6]$
 $t_8 = t_7 * 8$
 $t_9 = t_5 + t_8$
 $t_{10} = a[t_9]$
 $x = t_{10}$





习题12.4

- 修改下图中的翻译方案，使之适合Fortran风格的数组引用，也就是说， n 维数组的引用为 $\text{id}[E_1, E_2, \dots, E_n]$ 。

```
 $S \rightarrow \text{id} = E;$   
  |  $L = E;$  {  $\text{gen}(L.\text{array} \text{ '[' } L.\text{offset} \text{ ']' '}' E.\text{addr});$  }  
 $E \rightarrow E_1 + E_2 \mid -E_1 \mid (E_1) \mid \text{id}$   
  |  $L$  {  $E.\text{addr} = \text{newtemp}(); \text{gen}(E.\text{addr} \text{ '=' } L.\text{array} \text{ '[' } L.\text{offset} \text{ ']' });$  }  
 $L \rightarrow \text{id} [E]$  {  $L.\text{array} = \text{lookup}(\text{id.lexeme});$  if  $L.\text{array} == \text{nil}$  then error ;  
                   $L.\text{type} = L.\text{array.type.elem};$   
                   $L.\text{offset} = \text{newtemp}();$   
                   $\text{gen}(L.\text{offset} \text{ '=' } E.\text{addr} \text{ '*' } L.\text{type.width});$  }  
  |  $L_1[E]$  {  $L.\text{array} = L_1.\text{array};$   
               $L.\text{type} = L_1.\text{type.elem};$   
               $t = \text{newtemp}();$   
               $\text{gen}(t \text{ '=' } E.\text{addr} \text{ '*' } L.\text{type.width});$   
               $L.\text{offset} = \text{newtemp}();$   
               $\text{gen}(L.\text{offset} \text{ '=' } L_1.\text{offset} \text{ '+' } t);$  }
```

$S \rightarrow id = E;$

$| L = E; \{ gen(L.array \text{ '[' } L.offset \text{ ']' } '=' E.addr); \}$

$E \rightarrow E_1 + E_2 \mid -E_1 \mid (E_1) \mid id$

$| L \{ E.addr = newtemp(); gen(E.addr '=' L.array \text{ '[' } L.offset \text{ ']' }); \}$

$L \rightarrow FE] \{ L.array = F.array;$

$L.type = F.type.elem ;$

$t = newtemp();$

$gen(t '=' E.addr '*' L.type.width);$

$L.offset = newtemp();$

$gen(L.offset '=' F.offset '+' t); \}$

$F \rightarrow F_1 E, \{ F.array = F_1.array;$

$F.type = F_1.type.elem ;$

$t = newtemp();$

$gen(t '=' E.addr '*' F.type.width);$

$F.offset = newtemp();$

$gen(F.offset '=' F_1.offset '+' t); \}$

$id [\{ F.array = lookup(id.lexeme); if F.array == nil then error ;$

$F.type = F.array.type;$

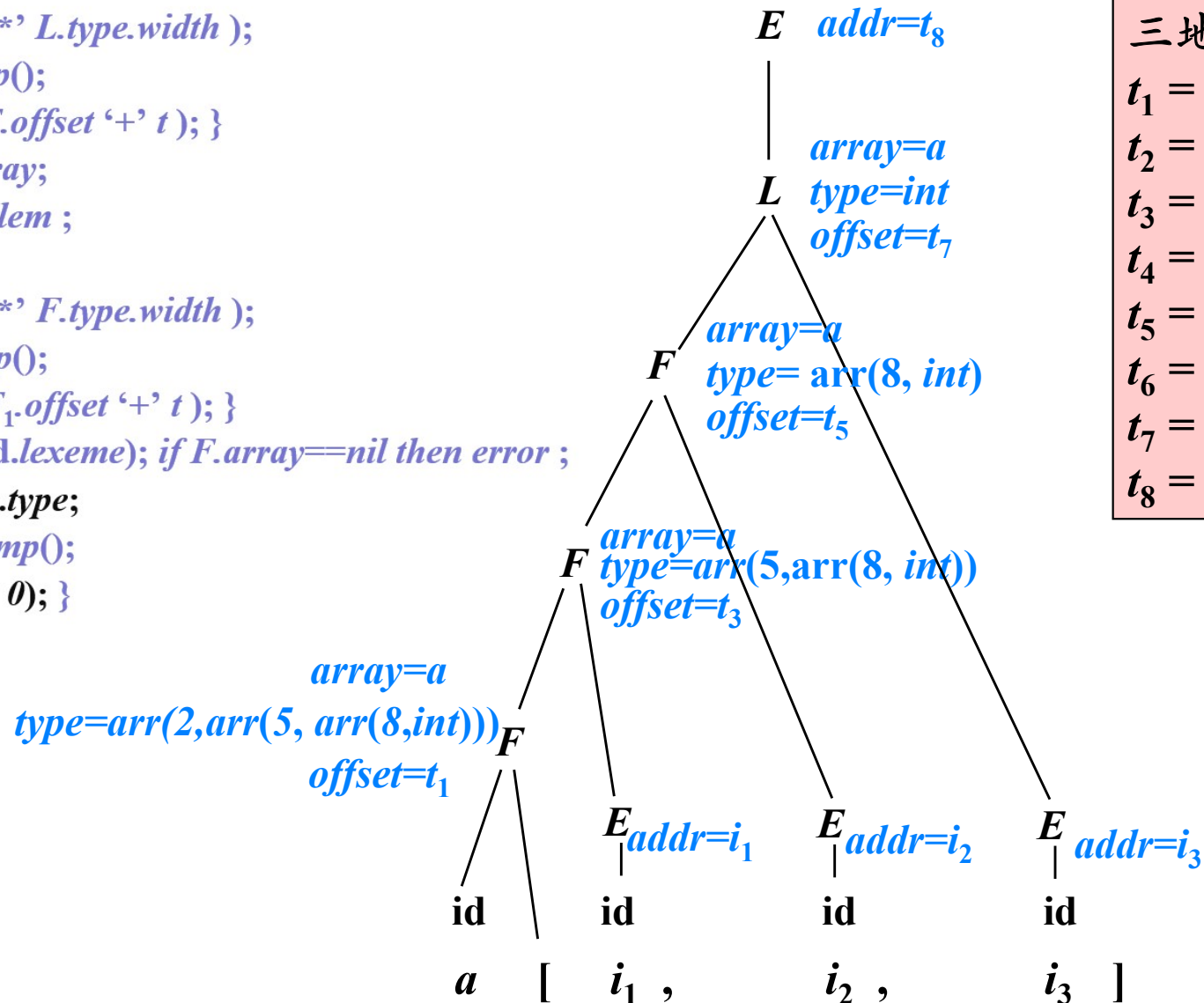
$F.offset = newtemp();$

$gen(F.offset '=' 0); \}$

样例分析:

假设a类型为:

$arr(2, arr(5, arr(8, int)))$



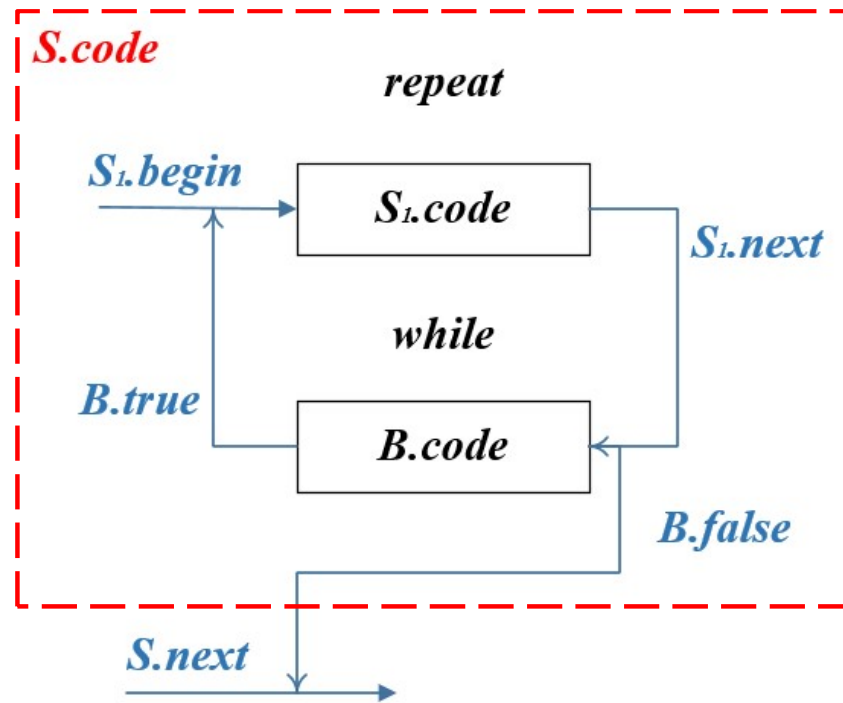


习题13.1

- 在SPOC讲义6.3节（控制语句的翻译）所示的SDT中添加处理下列控制流构造的翻译方案
 - (1) repeat语句： $S \rightarrow \textit{repeat } S_1 \textit{ while } B$
 - (2) for循环语句： $S \rightarrow \textit{for } (S_1 ; B ; S_2) S_3$



答案



$S \rightarrow \text{repeat } S_1 \text{ while } B$

$S \rightarrow \text{repeat } \{S_1.begin = \text{newlabel}();$

$\text{label}(S_1.begin); S_1.next = \text{newlabel}();\} S_1$

$\text{while } \{ \text{label}(S_1.next); B.true = S_1.begin;$

$B.false = S.next\} B$

翻译方案

S ->

for(

{S1.next=newlabel()}S1;

{B.begin = newlabel(); label(B.begin); label(S1.next);

B.true = newlabel(); B.false = S.next;} B;

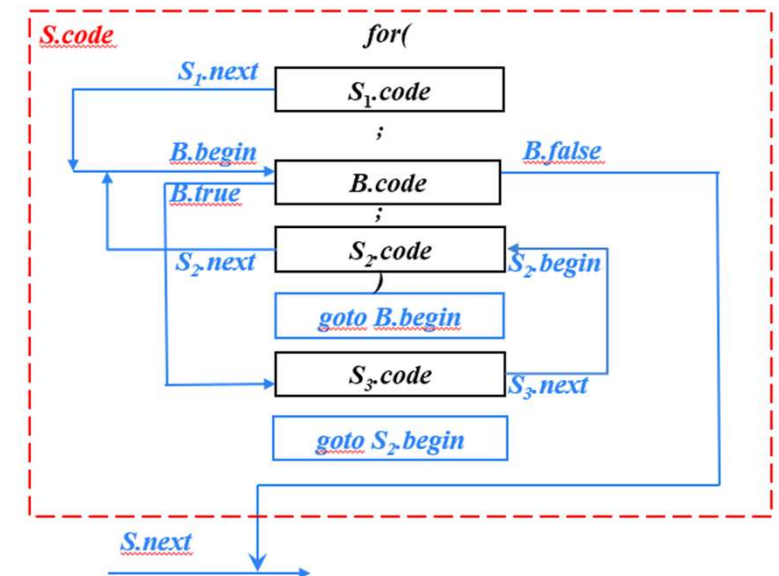
{S2.begin= newlabel(); label(S2.begin);

S2.next = B.begin;} S2)

{gen('goto' B.begin);

label(B.true); S3.next = S2.begin;} S3

{gen('goto' S2.begin)}





习题13.2

- 为下面的产生式写出一个和SPOC讲义6.3节（控制语句的翻译）中SDT类似的SDT。该产生式表示一个常见的C语言中的控制流结构。

$$S \rightarrow \{ L \}$$

$$L \rightarrow LS \mid \varepsilon$$



得到新的产生式

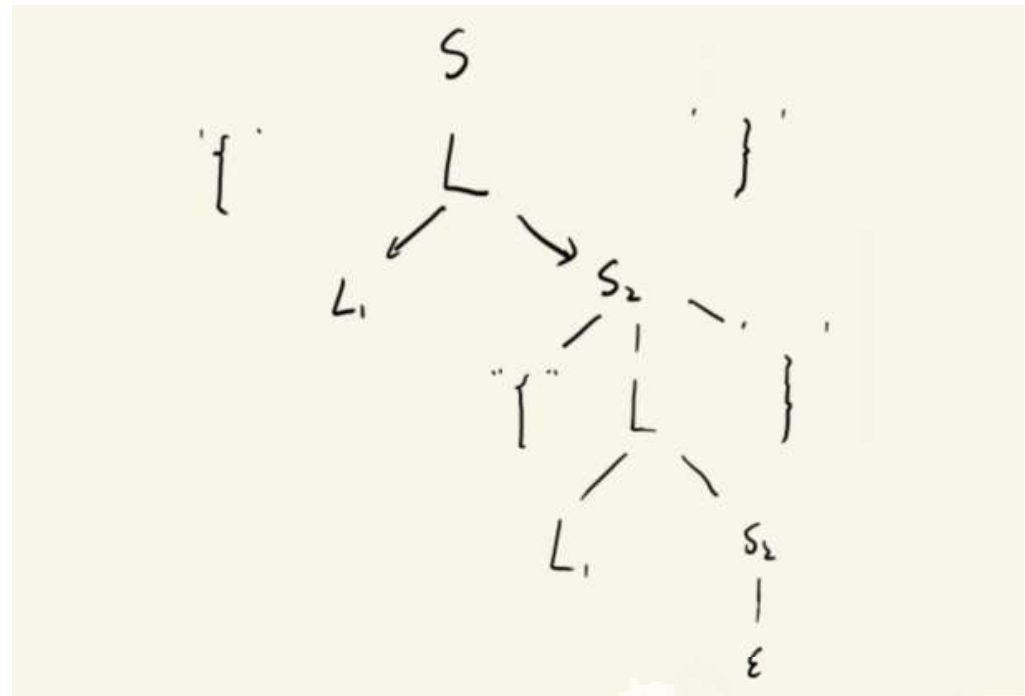
$S \rightarrow \{ L \}$

$L \rightarrow L S$

$L \rightarrow \epsilon$

$S \rightarrow \{ \{ L.next = S.next \} L(label(L.next)) \}$

$L \rightarrow \{ L1.next = newlabel \} L1 \{ label(L1.next); S.next = L1.next \} S$



$S \rightarrow \{ \{ L.next = newlabel() \} L(label(L.next)) \}$

$L \rightarrow \{ L1.next = newlabel \} L1 \{ label(L1.next); S.next = L1.next \} S$



习题13.3(1)

➤ 试将下面的语句翻译成四元式序列

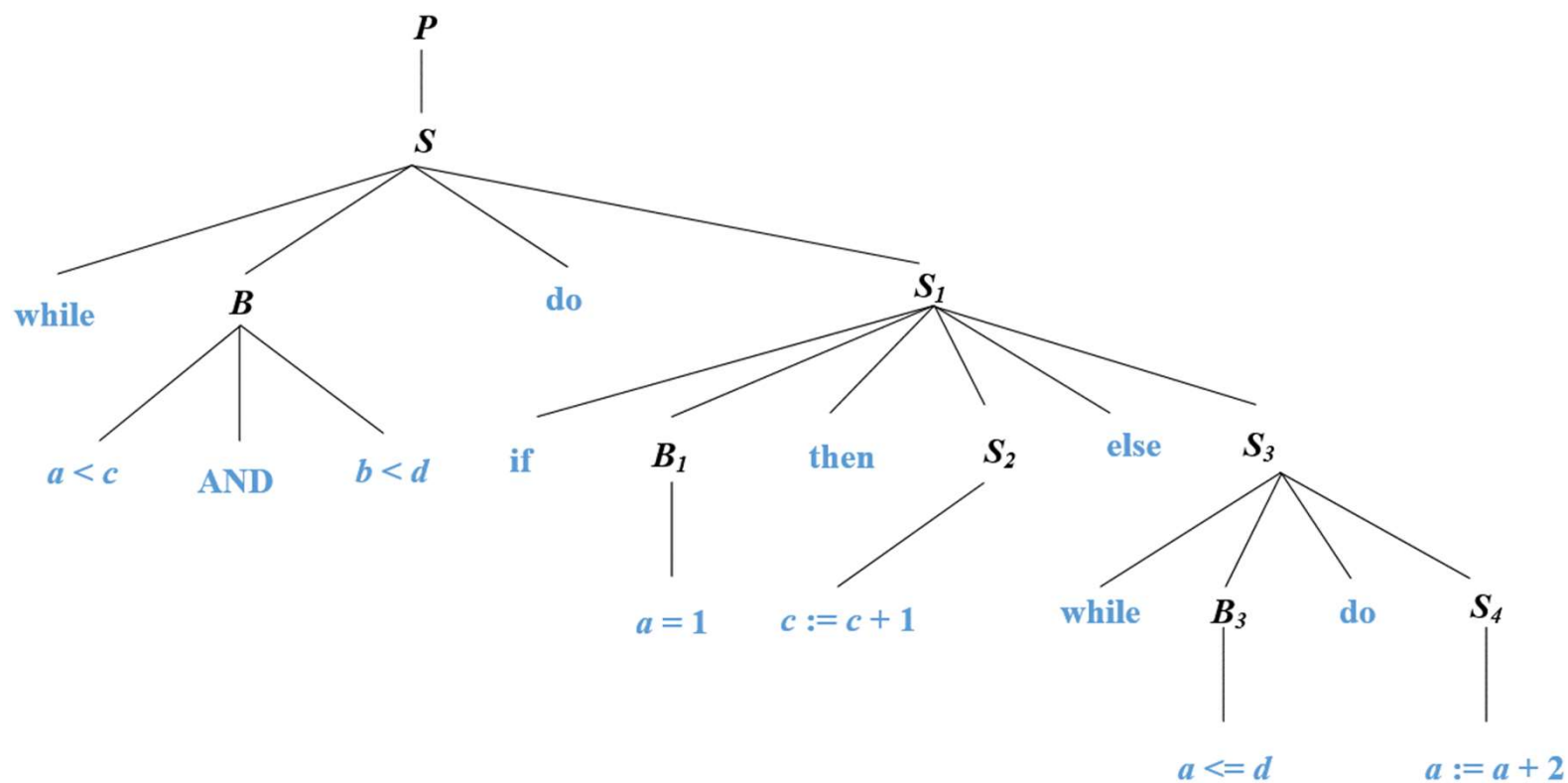
(1) while $a < c \wedge b < d$ do

if $a = 1$ then $c := c + 1$

else while $a \leq d$ do

$a := a + 2$

解答-语法分析树



```
while a < c ^ b < d do
  if a = 1
    then c := c + 1
  else
    while a <= d do
      a := a + 2
```

可以看出，有两个`while-do`的`SDT`，一个`if-then`的`SDT`，一个布尔表达式`AND`的`SDT`

解答-四元式

```
1: if  $a < c$  goto 3
2: goto 16
3: if  $b < d$  goto 5
4: goto 16
5: if  $a = 1$  goto 7
6: goto 10
7:  $t_1 = c + 1$ 
8:  $c := t_1$ 
9: goto 1
10: if  $a \leq d$  goto 12
11: goto 1
12:  $t_2 = a + 2$ 
13:  $a := t_2$ 
14: goto 10
15: goto 1
16:
```

```
1: ( $j <$ ,  $a$ ,  $c$ , 3)
2: ( $j$ , -, -, 16)
3: ( $j <$ ,  $b$ ,  $d$ , 5)
4: ( $j$ , -, -, 16)
5: ( $j =$ ,  $a$ , 1, 7)
6: ( $j$ , -, -, 10)
7: (+,  $c$ , 1,  $t_1$ )
8: ( $:=$ ,  $t_1$ , -,  $c$ )
9: ( $j$ , -, -, 1)
10: ( $j \leq$ ,  $a$ ,  $d$ , 12)
11: ( $j$ , -, -, 1)
12: (+,  $a$ , 2,  $t_2$ )
13: ( $:=$ ,  $t_2$ , -,  $a$ )
14: ( $j$ , -, -, 10)
15: ( $j$ , -, -, 1)
16:
```

```
while  $a < c \wedge b < d$  do
    if  $a = 1$ 
        then  $c := c + 1$ 
    else
        while  $a \leq d$  do
             $a := a + 2$ 
```



习题13.3(2)

➤ 试将下面的语句翻译成四元式序列

(2) for i:=m step 2 until n do

if a<b then x:=x+1



习题13.3(2)

**for i:=m step 2 until n do
if a<b then x:=x+1**

三地址指令

```
100: i=m
101: goto 103
102: i=i+2
103: if i<n, goto 105
104: goto __
105: if a<b, goto 107
106: goto 109
107: t=x+1
108: x=t
109: goto 102
110:
```

四元式

```
100: (=, m, _, i)
101: (j, _, _, 103)
102: (+, i, 2, i)
103: (j<, i, n, 105)
104: (j, _, _, -)
105: (j<, a, b, 107)
106: (j, _, _, 109)
107: (+, x, 1, t)
108: (=, t, _, x)
109: (j, _, _, 102)
110:
```



习题13.3(3)

➤ 试将下面的语句翻译成四元式序列

(3) for i:=1 step 1 until n do

while a<b do

if c>d then x:=-b+c else x:=a*b+c



for until语句的SDT

$:= a * b + c$

$S \rightarrow \text{for id} = \text{id1 step id2 until id3 do } S_1$

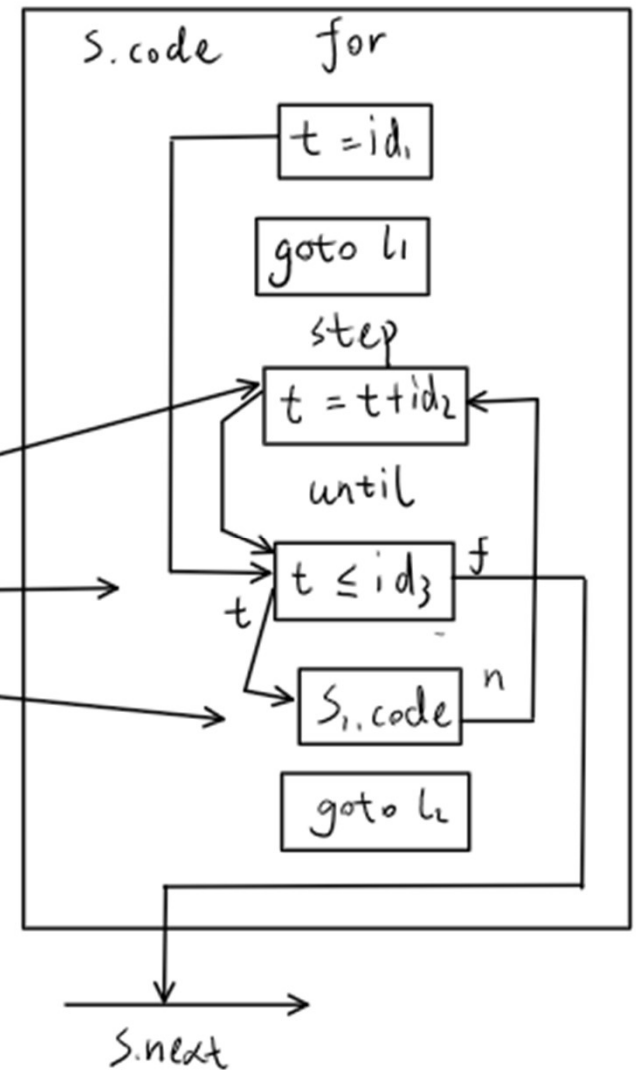
```
S -> for id := id1
{t = lookup(id.lexeme); if p == nil then error; gen(p '=' id1)
  l_1 = newlabel(); gen('goto', l_1);} step
{l_2 = newlabel(); label(l_2);} id2
{t1 = newtemp(); gen(t1 '=' t + id2);
  t = lookup(id.lexeme); gen(t '=' t1);} until
{l_3 = newlabel(); label(l_1);} id3
{gen('if' t '<=' id3 'goto' l_3);
  gen('goto', S.next); label(l_3); S_1.next = l_2} S_1
{gen('goto' l_2);}
```

$t = \text{循环变量}$

l_2

l_1

l_3





开始干啊

$x := a * b + c$

先画树

for until do

$S \rightarrow \text{for id} := \text{id1}$

{ $t = \text{lookup}(\text{id.lexeme}); \text{if } p == \text{nil} \text{ then error; gen}(p := \text{id1})$

$l_1 = \text{newlabel}(); \text{gen}(\text{'goto', } l_1); \}$ step

{ $l_2 = \text{newlabel}(); \text{label}(l_2); \}$ id2

{ $t_1 = \text{newtemp}(); \text{gen}(t_1 := t + \text{id2});$

$t = \text{lookup}(\text{id.lexeme}); \text{gen}(t := t_1); \}$ until

{ $l_3 = \text{newlabel}(); \text{label}(l_1); \}$ id3

{ $\text{gen}(\text{'if' } t <= \text{id3 'goto' } l_3);$

$\text{gen}(\text{'goto', } S.\text{next}); \text{label}(l_3); S_1.\text{next} = l_2; \}$ S_1

{ $\text{gen}(\text{'goto' } l_2); \}$

while do

$S \rightarrow \text{while } \{S.\text{begin} = \text{newlabel}(); \}$ $S_1.\text{next}$

$\text{label}(S.\text{begin});$

$B.\text{true} = \text{newlabel}();$

$B.\text{false} = S.\text{next}; \}$ B

$\text{do } \{ \text{label}(B.\text{true}); S_1.\text{next} = S.\text{begin}; \}$ S_1

$\{ \text{gen}(\text{'goto' } S.\text{begin}); \}$

if then else

$S \rightarrow \text{if } \{ B.\text{true} = \text{newlabel}(); B.\text{false} = \text{newlabel}(); \} B$

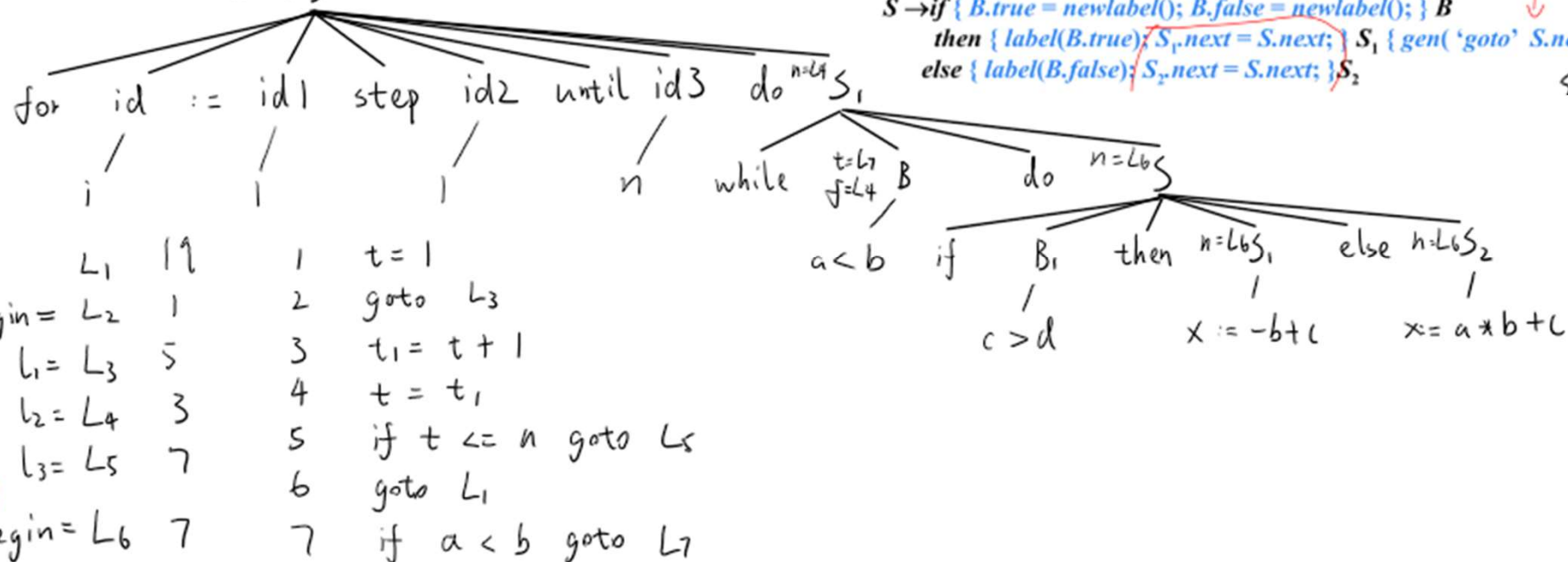
$\text{then } \{ \text{label}(B.\text{true}); S_1.\text{next} = S.\text{next}; \} S_1 \{ \text{gen}(\text{'goto' } S.\text{next}); \}$

$\text{else } \{ \text{label}(B.\text{false}); S_2.\text{next} = S.\text{next}; \} S_2$

$S_1.\text{he}$

until

1



$S.\text{next}$

for i:=1 step 1 until n do

while a<b do

if c>d then x:=-b+c else x:=a*b+c

	L ₁	1	t = 1
S.begin =	L ₂	1	goto L ₃
L ₁ =	L ₃	5	t ₁ = t + 1
L ₂ =	L ₄	3	t = t ₁
↓ S.next)	L ₃ =	7	if t ≤ n goto L ₅
S ₁ .begin =	L ₆	7	goto L ₁
B ₁ .true =	L ₇	9	if a < b goto L ₇
B ₁ .true =	L ₈	11	goto L ₄
B ₁ .false =	L ₉	14	if c > d goto L ₈
		10	goto L ₉
		11	t ₂ = -b + c
		12	x = t ₂
		13	goto L ₆
		14	t ₃ = a * b
		15	t ₄ = t ₃ + c
		16	x = t ₄
		17	goto L ₆
		18	goto L ₄
		19	
		20	

1	t = 1
2	goto 5
3	t1 = t + 1
4	t = t1
5	if t ≤ n goto 7
6	goto _
7	if a < b goto 9
8	goto 3
9	if c > d goto 11
10	goto 15
11	t2 = -b
12	t3 = t2 + c
13	x = t3
14	goto 7
15	t3 = a * b
16	t4 = t3 + c
17	x = t4
18	goto 7
19	goto 3
20	

1	=	1	-	t
2	goto	-	-	5
3	+	t	1	t1
4	=	t1	-	t
5	≤	t	n	7
6	goto	-	-	_
7	<	a	b	9
8	goto	-	-	3
9	>	c	d	11
10	goto	-	-	15
11	-	b	-	t2
12	+	t2	c	t3
13	=	t3	-	x
14	goto	-	-	7
15	*	a	b	t4
16	+	t4	c	t5
17	=	t5	-	x
18	goto	-	-	7
19	goto	-	-	3
20				



习题13.3(4)

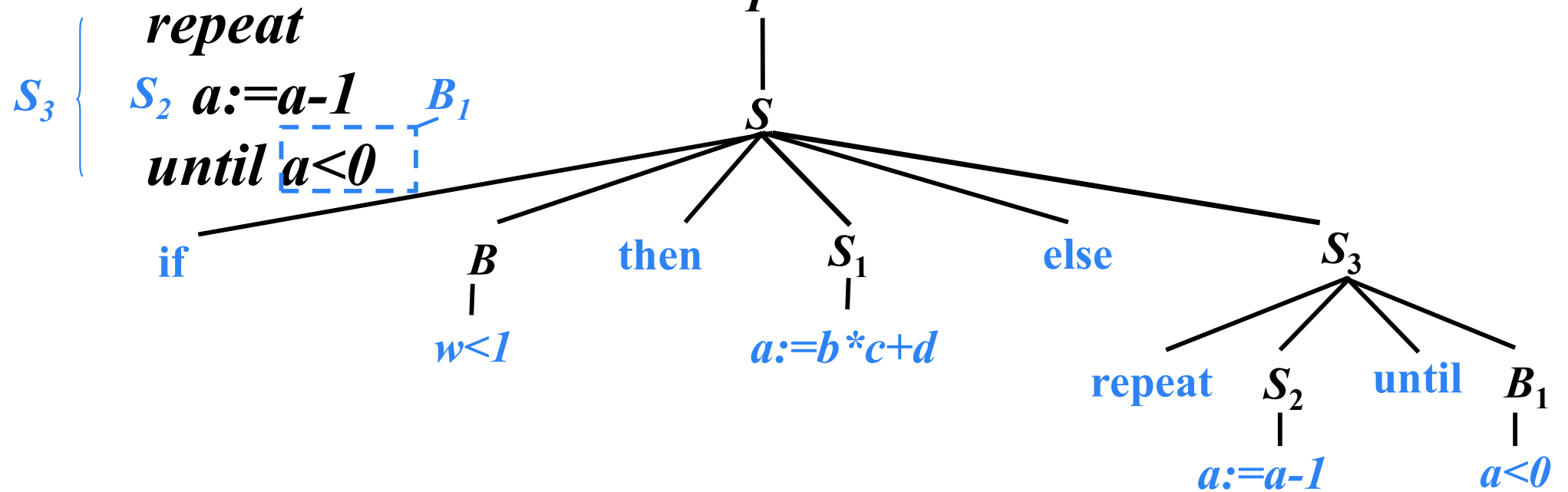
➤ 试将下面的语句翻译成四元式序列

(4) **if $w < 1$ then $a := b * c + d$ else repeat $a := a - 1$ until $a < 0$**



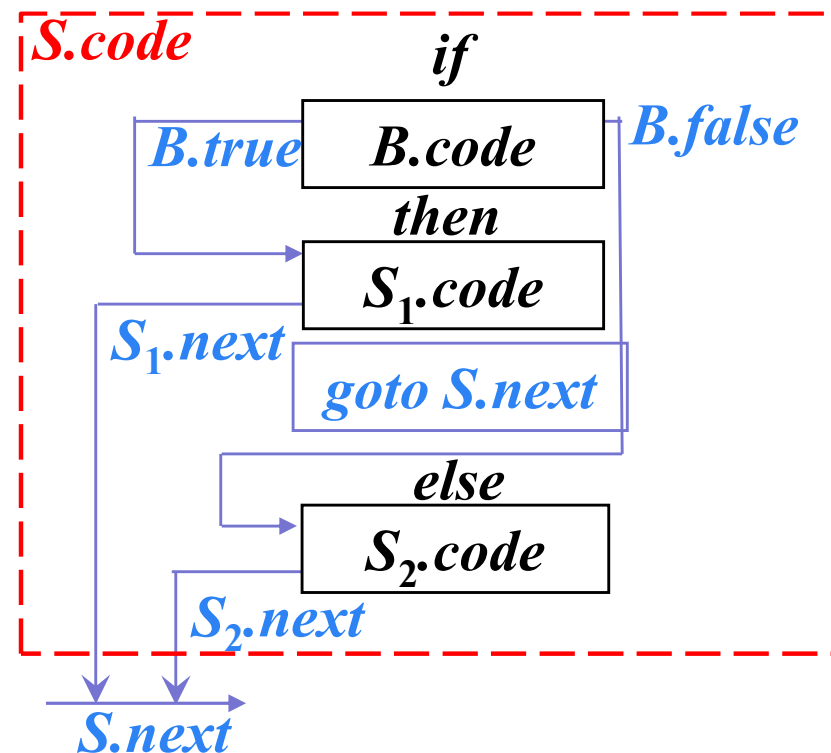
语法分析树

if $w < 1$ *then* S_1 $a := b * c + d$
else



*if-then-else*语句的SDT

$S \rightarrow \text{if } B \text{ then } S_1 \text{ else } S_2$

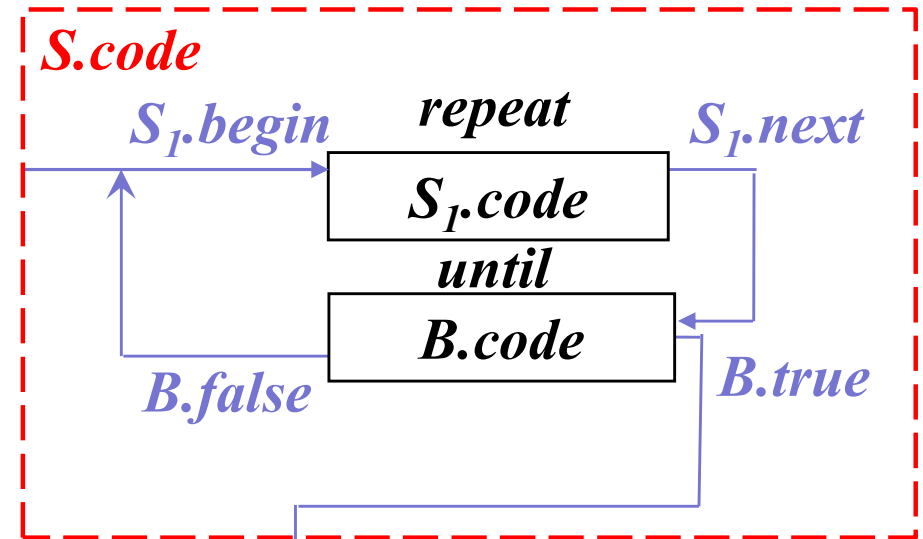


$S \rightarrow \text{if } \{ B.true = \text{newlabel}(); B.false = \text{newlabel}(); \} B$
 $\quad \text{then } \{ \text{label}(B.true); S_1.next = S.next; \} S_1 \{ \text{gen}('goto' S.next) \}$
 $\quad \text{else } \{ \text{label}(B.false); S_2.next = S.next; \} S_2$



*repeat-until*语句的SDT

$S \rightarrow \text{repeat } S_1 \text{ until } B$

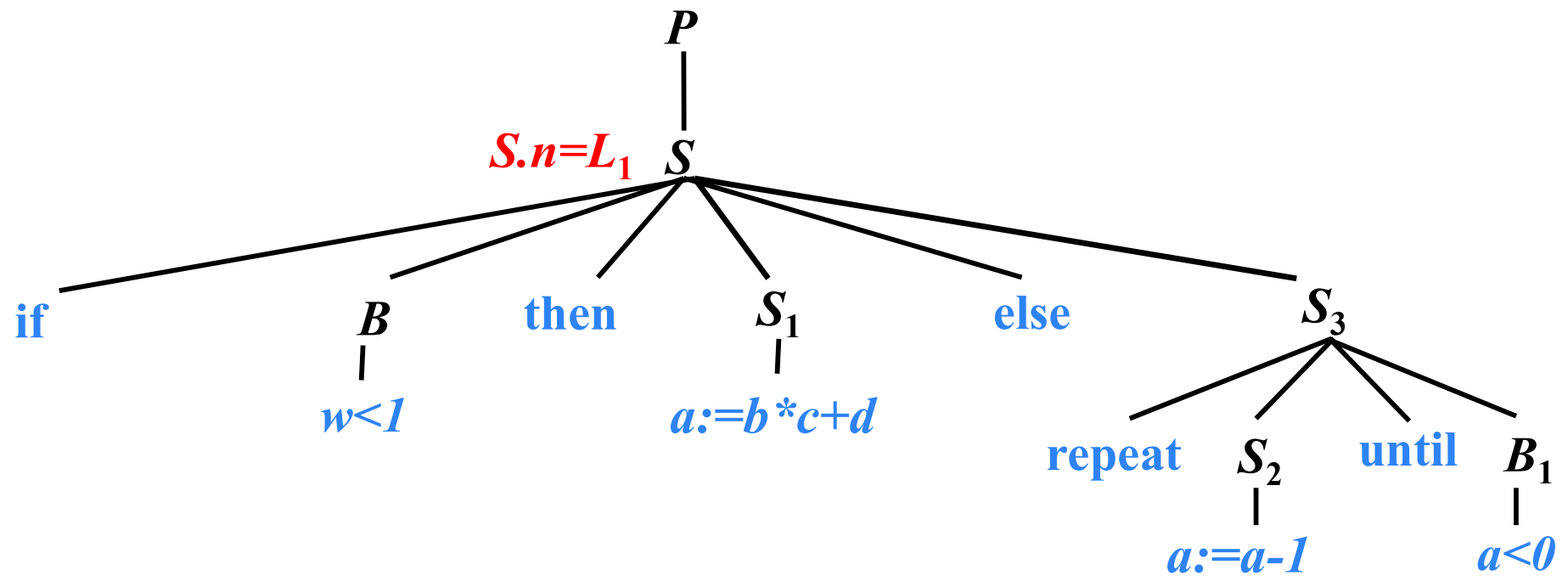


$S \rightarrow \text{repeat } \{ S_1.\text{begin} = \text{newlabel}();$
 $\text{label}(S_1.\text{begin});$
 $S_1.\text{next} = \text{newlabel}(); \} S_1$
 $\text{until } \{ \text{label}(S_1.\text{next});$
 $B.\text{true} = S.\text{next}; B.\text{false} = S_1.\text{begin}; \} B$



动作执行

$P \rightarrow \{ S.next = newlabel(); \} S \{ label(S.next); \}$



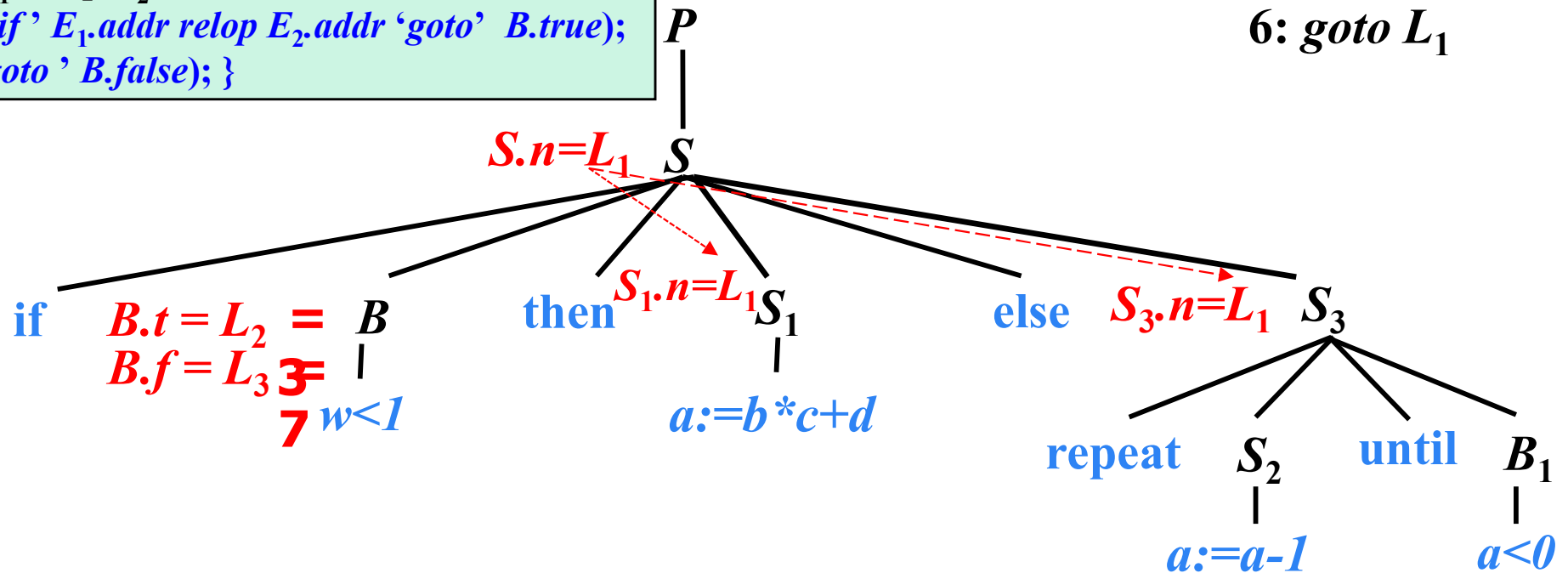


动作执行

$S \rightarrow \text{if } \{ B.\text{true} = \text{newlabel}(); B.\text{false} = \text{newlabel}(); \} B$
 then $\{ \text{label}(B.\text{true}); S_1.\text{next} = S.\text{next}; \} S_1 \{ \text{gen}(\text{'goto' } S.\text{next}) \}$
 else $\{ \text{label}(B.\text{false}); S_2.\text{next} = S.\text{next}; \} S_2$

$B \rightarrow E_1 \text{ relop } E_2$
 $\{ \text{gen}(\text{'if' } E_1.\text{addr relop } E_2.\text{addr 'goto' } B.\text{true});$
 $\text{gen}(\text{'goto' } B.\text{false}); \}$

1: *if* $w < 1$ *goto* L_2
2: *goto* L_3
3: $t_1 = b * c$
4: $t_2 = t_1 + d$
5: $a = t_2$
6: *goto* L_1





动作执行

```

S  $\rightarrow$  repeat { S1.begin = new label() ; label(S1.begin) ;
               S1.next = newlabel() ; } S1
until { label(S1.next) ;
       B.true = S.next ; B.false = S1.begin ; } B

```

1: *if* $w < 1$ *goto* L_2

2: goto L_3 7 3

3: $t_1 = b * c$

4: $t_2 = t_1 + d$

5: $a = t_2$

6: *goto* L_1

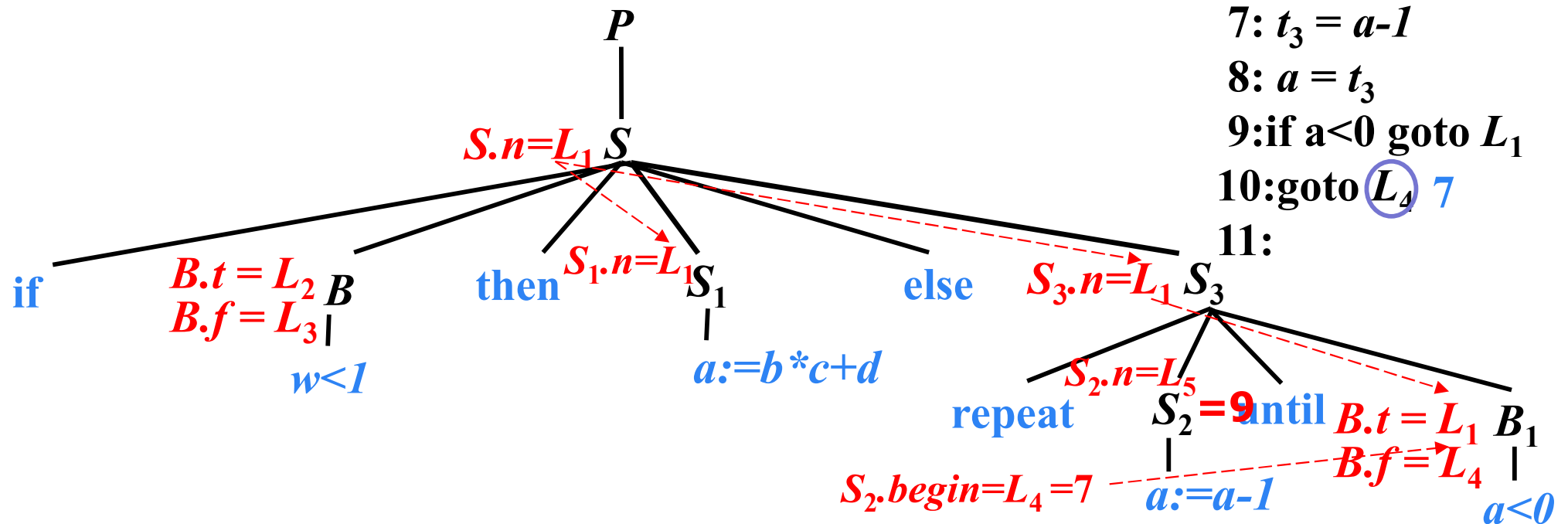
7: $t_3 = a-1$

8: $a = t_3$

9:if a<0 goto L_1

10:goto L_4 7

11:





语句 “if $w < 1$ then $a := b * c + d$ else repeat $a := a - 1$ until $a < 0$ ” 的四元式序列

1: <i>if</i> $w < 1$ <i>goto</i> 3	1: (<i>j</i> <, w, 1, 3)
2: <i>goto</i> 7	2: (<i>j</i>, -, -, 7)
3: $t_1 = b * c$	3: (*, b, c, t_1)
4: $t_2 = t_1 + d$	4: (+, t_1, d, t_2)
5: $a = t_2$	5: (=, t_2, -, a)
6: <i>goto</i> 11	6: (<i>j</i>, -, -, 11)
7: $t_3 = a - 1$	7: (-, a, 1, t_3)
8: $a = t_3$	8: (=, t_3, -, a)
9: <i>if</i> $a < 0$ <i>goto</i> 11	9: (<i>j</i> <, a, 0, 11)
10: <i>goto</i> 7	10: (<i>j</i>, -, -, 7)
11:	11:



习题13.4

➤ Pascal语言的标准将for语句

for v := E1 to E2 do S1

定义成和下面的代码序列有同样的含义：

begin

v := E1;

while v <= E2 do begin

S1;

v := succ(v);

end

end

请为for语句设计一种合理的中间代码结构，并写出产生中间代码的翻译方案

习题13.4

➤ Pascal语言的标准将for语句

for $v := E1$ to $E2$ do $S1$

定义成和下面的代码序列有同样的含义:

begin

$v := E1$;

while $v \leq E2$ do begin

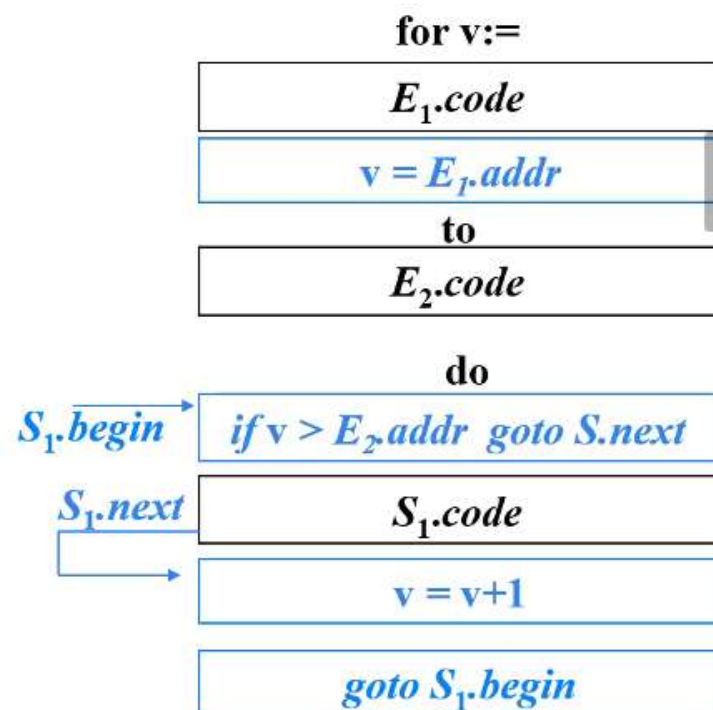
$S1$;

$v := \text{succ}(v)$;

end

end

请为for语句设计一种合理的中间代码结构,
并写出产生中间代码的翻译方案





习题13.4

翻译方案:

```
S → for
    V :=
    E1 { p = lookup('v'.lexeme); if p == nil then
        error; gen(p '=' E1.addr); }
    to
    E2 { S1.begin = newlabel(); label(S1.begin)
        gen('if' p '>' E2.addr 'goto' S.next);
        S1.next = newlabel(); }
Do S1 { label(S1.next); gen(p '=' p '+' 1);
        gen('goto' S1.begin); }
```

begin

v := E1;

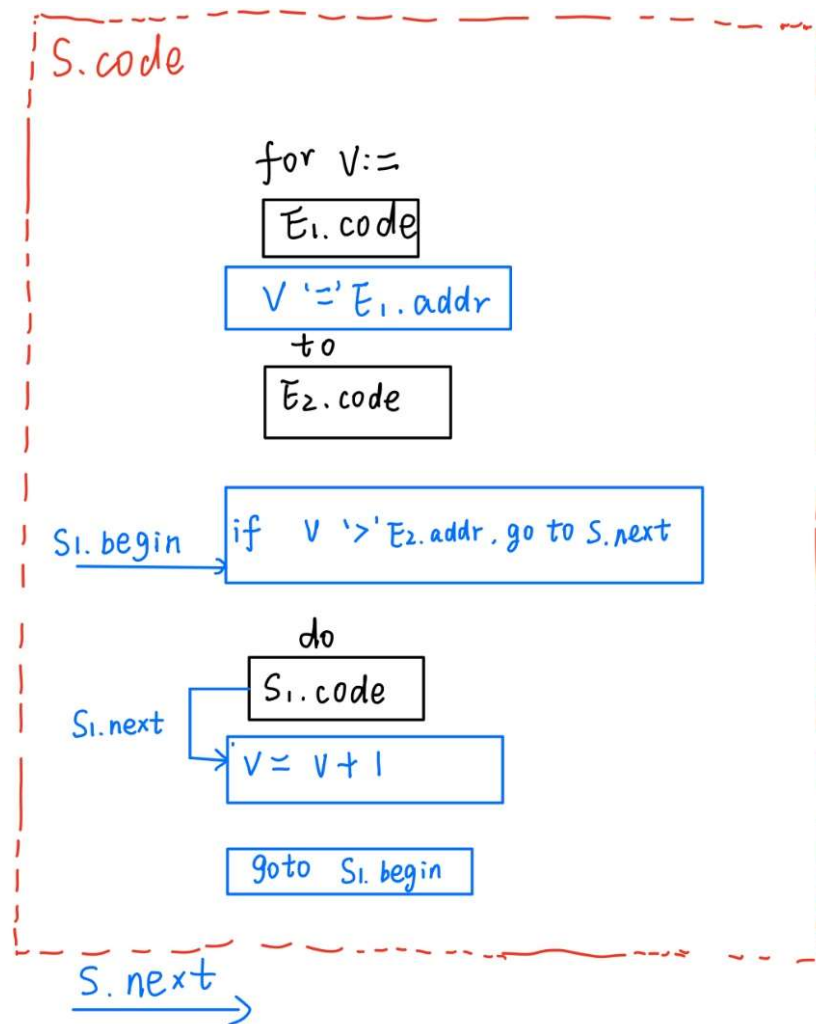
while v <= E2 do begin

S1;

v := succ(v);

end

end





习题13.5

➤ 设有如下的文法G:

$S \rightarrow \text{id} := E$
| **if** B **then** S
| **while** B **do** S
| **begin** S ; S **end**
| **break**

试给出完成如下翻译所要求的翻译模式：如果发现**break**未出现在循环语句中，则报告错误。

习题13.5

继承 属性 $S.inwhile$: S 是否处于 while 语句中

继承 属性 $S.bnext$: $break$ 语句的目标跳转指令标号

- $P \rightarrow \{S.inwhile = 0; S.next = newlabel();\} S \{ label(S.next); \}$
- $S \rightarrow \text{if } B \text{ then } \{S_1.inwhile = S.inwhile; \text{if } S_1.inwhile \text{ then } S_1.bnext = S.bnext;\} S_1$
- $S \rightarrow \text{while } B \text{ do } \{S_1.inwhile = 1; S_1.bnext = S.next;\} S_1$
- $S \rightarrow \text{begin } \{S_1.inwhile = S.inwhile; \text{if } S_1.inwhile \text{ then } S_1.bnext = S.bnext;\} S_1 ;$
 $\{S_2.inwhile = S.inwhile; \text{if } S_2.inwhile \text{ then } S_2.bnext = S.bnext;\} S_2 \text{ end}$
- $S \rightarrow \text{break} \{ \text{if } S.inwhile == 0 \text{ then error; gen('goto' } S.bnext); \}$



解题思路

- 增加继承属性**S.loop**，表示当前是否在**while**循环中（**true/false**）。
- 增加继承属性**S.break**记录在**while**循环中遇到**break**需要跳转到的位置（最内层**while**循环的下一条指令地址）。
- **S.loop**属性一定要从父节点继承，然后通过**S.loop**来判断当前是否处于**while**循环中，如果在**while**循环中，则继承**S.break**记录的**while**循环中遇到**break**需要跳转到的位置。
- 遇到**break**时，判断**S.loop**，如果为真，则正常跳转到**S.break**；否则为在**while**循环外出现**break**的情况，报错。

- 
- $S' \rightarrow \{S.next = newlabel(); S.loop = false; \} S \{ label(S.next); \}$

初始情况下，肯定不在循环里，设S.loop为false。

- $S \rightarrow id := E \{ p = lookup(id.lexeme); if p == nil then error ; S.code = E.code || gen(p ':=' E.addr); \}$

简单的赋值语句SDT。

- $S \rightarrow if \{ B.true = newlabel(); B.false = S.next; \} B then \{ label(B.true); S_1.next = S.next; S_1.loop = S.loop; if S.loop then S_1.break = S.break; \} S_1$

继承父节点的loop，并且如果S已经在循环中，S₁必定在循环中，且目前最内层的while循环为S对应的while循环，需要继承S的break，来知道对应的break跳转的位置。

- 
- $S \rightarrow \text{while } \{B.begin = \text{newlabel}(); \text{label}(B.begin); B.true = \text{newlabel}(); B.false = S.next;\} B \text{ do } \{ \text{label}(B.true); S_1.next = B.begin; S_1.loop = \text{true}; S_1.break = S.next; \} S_1 \{ \text{gen('goto' } B.begin); \}$

遇到while循环时，将 S_1 的loop值设为true，并且其break值设为父节点的next。这样即使有嵌套循环，break永远只会跳到当前的父节点的next，即跳出最内层循环。

- $S \rightarrow \text{begin } \{S_1.next = \text{newlabel}(); S_1.loop = S.loop; \text{if } S.loop \text{ then } S_1.break = S.break;\} S_1; \{ \text{label}(S_1.next); S_2.next = S.next; S_2.loop = S.loop; \text{if } S.loop \text{ then } S_1.break = S.break;\} S_2 \text{ end}$

继承S的loop，并且如果S在循环中，还需要继承其break值，以便遇到break语句时进行跳转。

$S \rightarrow \text{break } \{ \text{if not } S.loop \text{ then error; gen('goto' } S.break); \}$

遇到break时，判断是否在循环中，如果不在则报错，否则正常跳转。



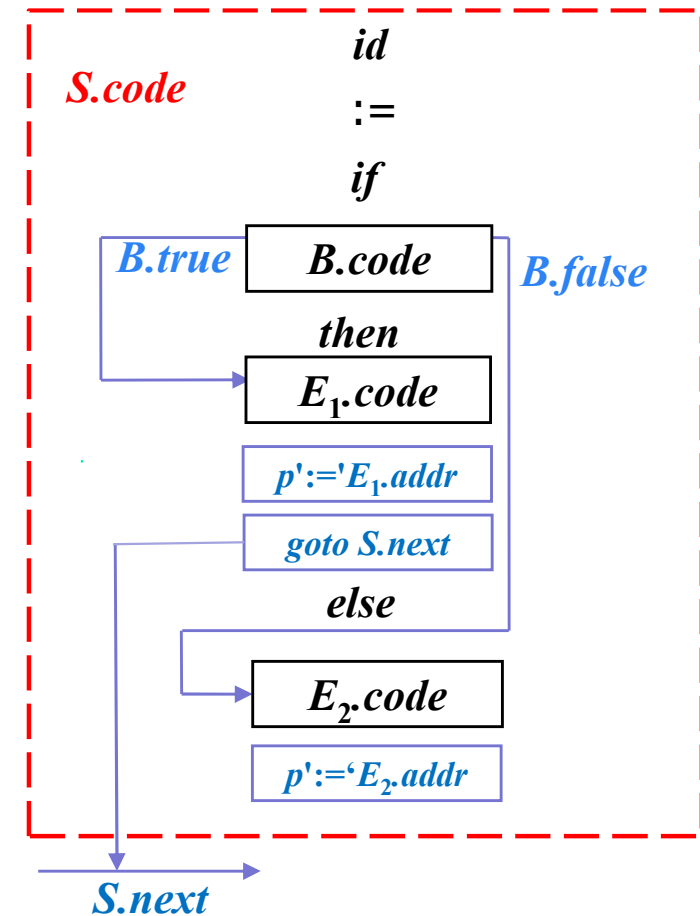
习题13.6

- 写出条件赋值语句 **$\text{id} := \text{if } B \text{ then } E_1 \text{ else } E_2$** 的语义子程序。其中， **$B$** 为布尔表达式， **$E_1$** 和 **$E_2$** 是算术表达式， **$i$** 代表与 **$E_1$** 和 **$E_2$** 同类的简单变量。按照写出的语义子程序，生成条件赋值语句 **$m := \text{if } a > c \text{ then } x + y \text{ else } x - y + 0.5$** 的四元式序列



➤ $S \rightarrow id := \text{if } \{B.true = \text{newlabel}();$
 $B.false = \text{newlabel}();\}$
 $B \text{ then } \{label(B.true);\}$
 $E1 \{p = \text{lookup}(id.lexeme); \text{if } p == nil \text{ then}$
error;
 $\text{gen}(p = 'E1.addr);$
 $\text{gen}(\text{goto } S.next);\}$
 $\text{else } \{label(B.false);\}$
 $E2 \{p = \text{lookup}(id.lexeme); \text{if } p == nil \text{ then error;}$
 $\text{gen}(p = 'E2.addr);\}$

$S \rightarrow id := \text{if } B \text{ then } E_1 \text{ else } E_2$

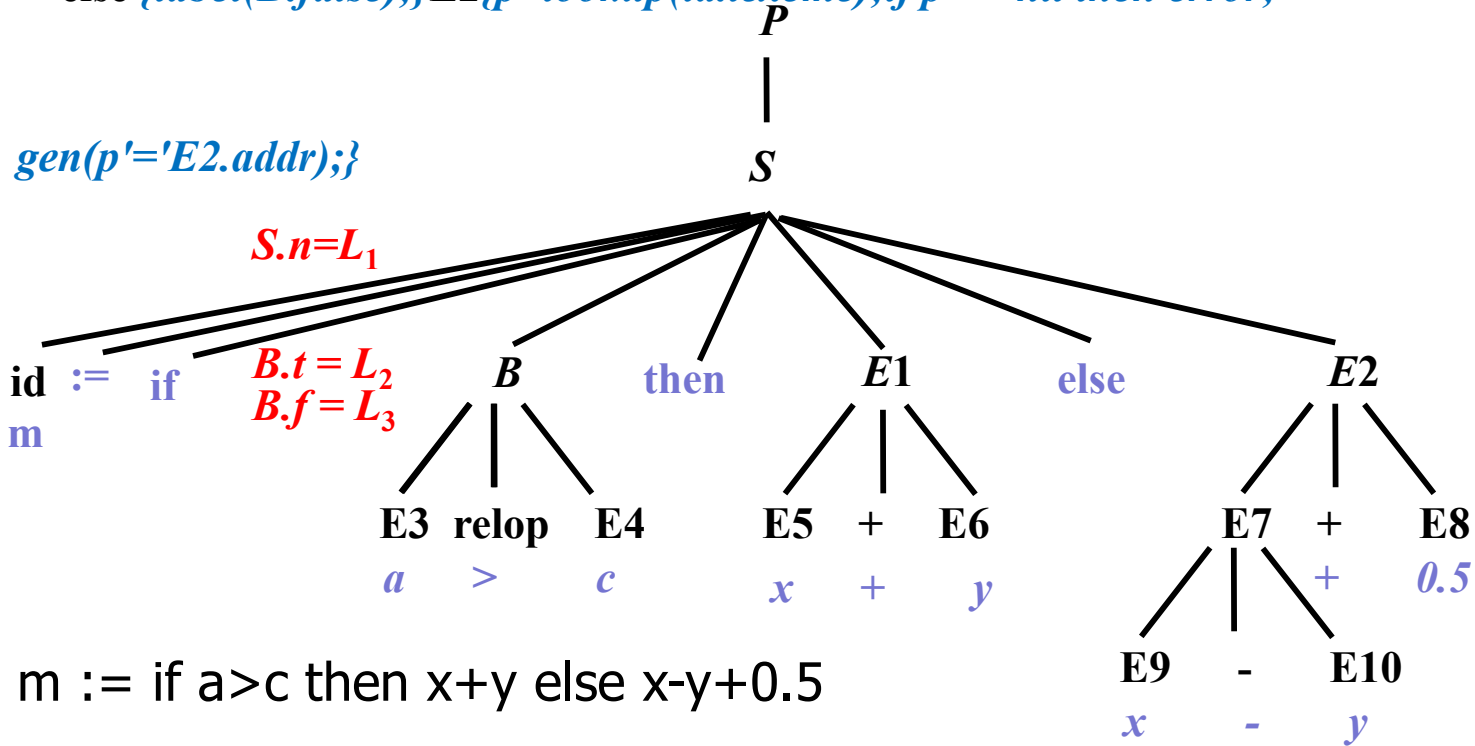




$S \rightarrow id := if \{ B.true = new\ label(); B.false = newlabel(); \} B$
 $P \rightarrow \{ S.next = newlabel(); \} S \{ label(S.next); \}$
 $then \{ label(B.true); \} E1 \{ p = lookup(id.lexeme); if\ p == nil\ then\ error;$

$gen(p' = 'E1.addr); gen(goto\ S.next); \}$

$else \{ label(B.false); \} E2 \{ p = lookup(id.lexeme); if\ p == nil\ then\ error;$



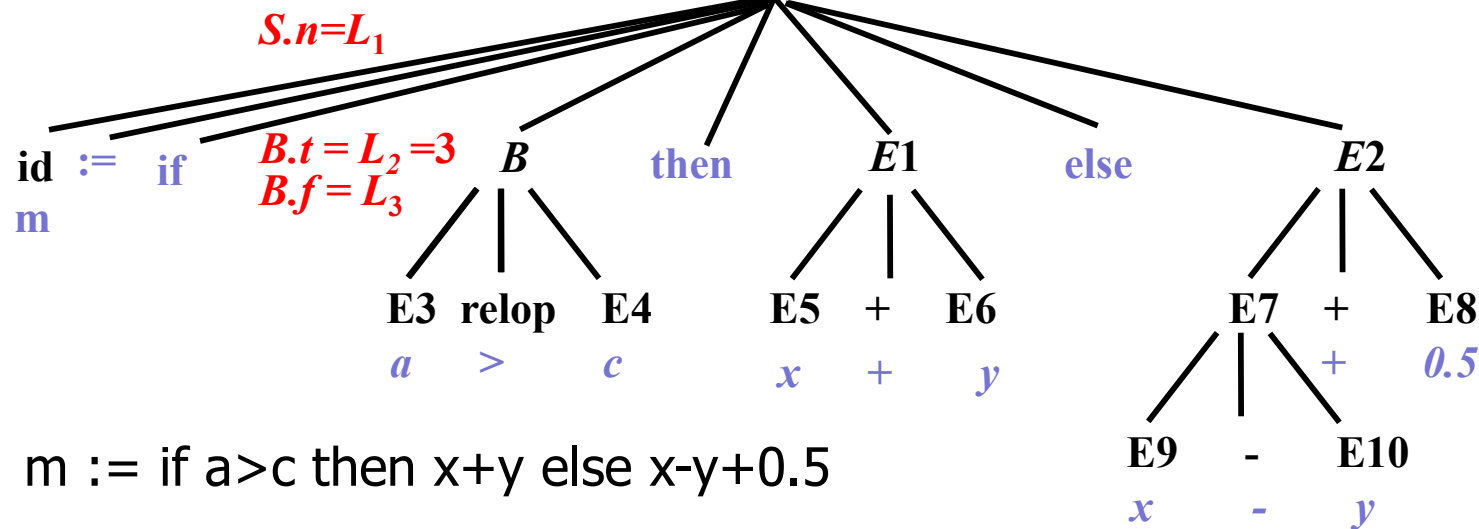


$S \rightarrow id := if \{B.true = new\ label(); B.false = newlabel();\} B$
 $B \rightarrow E_1\ relop\ E_2 \{ gen('if'\ E_1.addr\ relop\ E_2.addr\ 'goto'\ B.true)$
 $then \{ label(B.true); \} E1 \{ p=lookup(id.lexeme); if\ p == nil\ then\ error;$
 $gen(goto\ B.false); \}$
 $gen(p='E1.addr); gen(goto\ S.next); \}$

1: if $a > c$ goto L_2

2: goto L_3

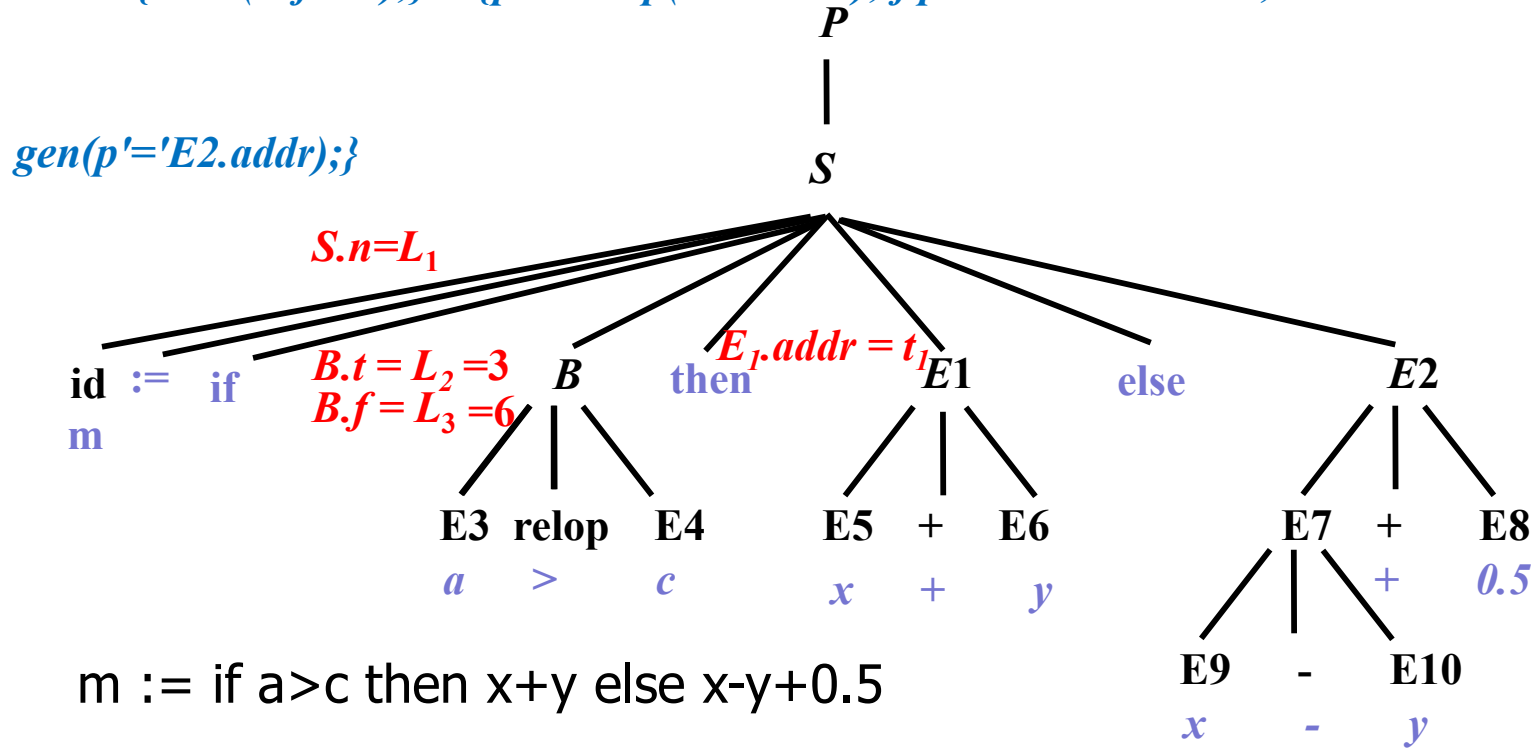
$else \{ label(B.false); \} E2 \{ p=lookup(id.lexeme); if\ p == nil\ then\ error;$
 $gen(p='E2.addr); \}$





$S \rightarrow id := if \{ B, true = newlabel(); B.false = newlabel(); \} B$
 $E \rightarrow E_1 + E_2 \{ E.addr = newtemp();$
then $\{ label(B.true); E1 \{ p = lookup(id.lexeme); if p == nil then error;$
 $gen(p='E1.addr); gen(goto S.next); \}$
else $\{ label(B.false); \} E2 \{ p = lookup(id.lexeme); if p == nil then error;$
 $gen(p='E2.addr); \}$

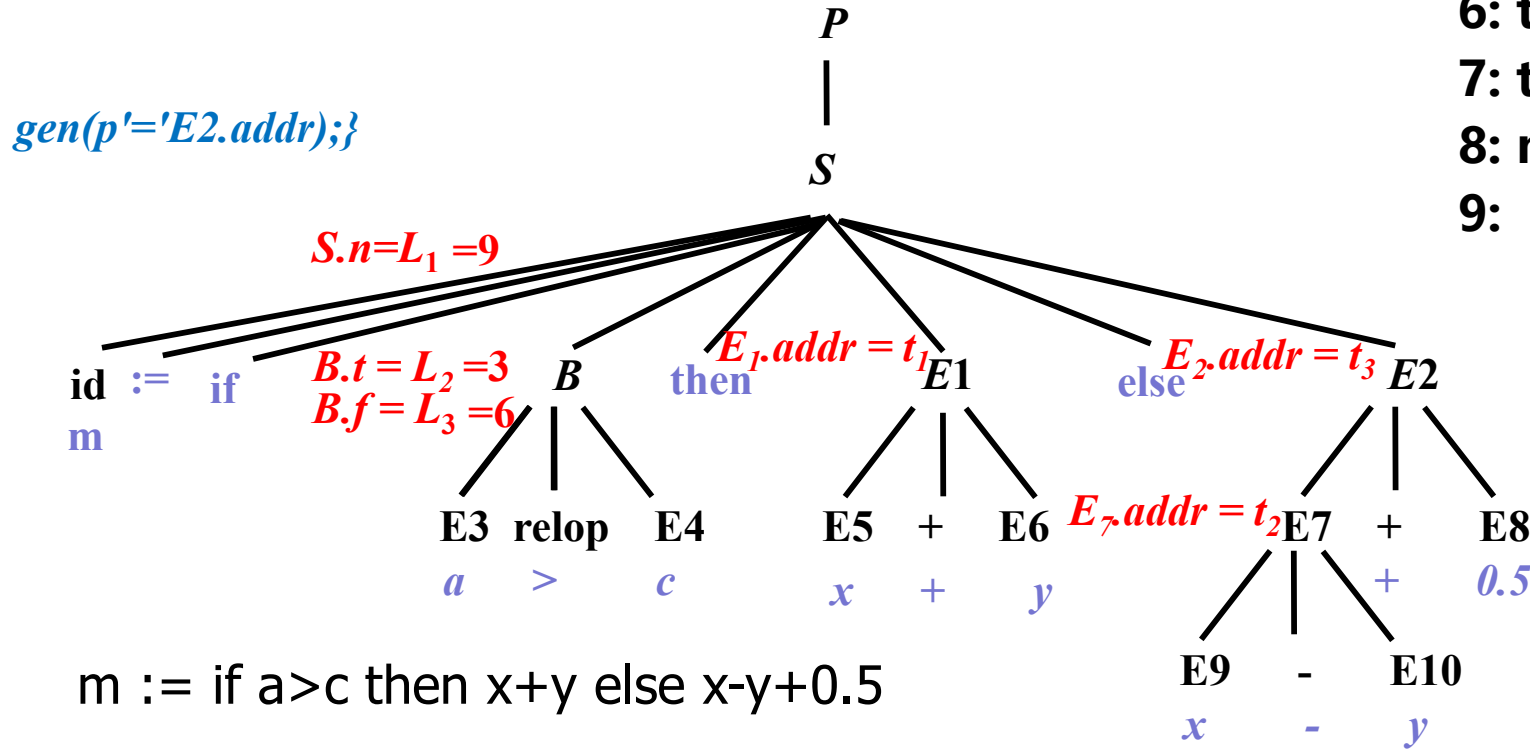
1: if a > c goto L₂
2: goto L₃
3: t₁ = x + y
4: m = t₁
5: goto L₁





$S \rightarrow id := \text{if} \{ B.true = \text{new label}(); B.false = \text{newlabel}(); \} B$
 $\text{then } \{ \text{label}(B.true); E1 \{ p = \text{lookup}(id.lexeme); \text{if } p == \text{nil then error};$
 $\text{gen}(E1.addr = E1.addr + E2.addr); \}$
 $\text{gen}(p = 'E1.addr'); \text{gen}(\text{goto } S.next); \}$
 $\text{else } \{ \text{label}(B.false); \} E2 \{ p = \text{lookup}(id.lexeme); \text{if } p == \text{nil then error};$
 $\text{gen}(p = 'E2.addr'); \}$

1: if a > c goto L₂
2: goto L₃
3: t₁ = x + y
4: m = t₁
5: goto L₁
6: t₂ = x - y
7: t₃ = t₂ + 0.5
8: m = t₃
9:





1: if a > c goto L₂
2: goto L₃
3: t₁ = x + y
4: m = t₁
5: goto L₁
6: t₂ = x - y
7: t₃ = t₂ + 0.5
8: m = t₃
9:

L1 = 9
L2 = 3
L3 = 6

1: if a > c goto 3
2: goto 6
3: t₁ = x + y
4: m = t₁
5: goto 9
6: t₂ = x - y
7: t₃ = t₂ + 0.5
8: m = t₃
9:

四元
序列式

1: (j>,a,c,3)
2: (j,-,-,6)
3: (+,x,y,t₁)
4: (=,t₁,-,m)
5: (j,-,-,9)
6: (-,x,y,t₂)
7: (+,t₂,0.5,t₃)
8: (=,t₃,-,m)
9:



习题14.1

➤使用SPOC讲义6.4节（回填）中的翻译方案翻译下列表达式。给出每个子表达式的**truelist**和**falselist**。你可以假设第一条被生成的指令的地址是100。

➤(1) $a == b \ \&\& \ (c == d \ || \ e == f)$

➤(2) $(a == b \ || \ c == d) \ || \ e == f$

➤(3) $(a == b \ \&\& \ c == d) \ \&\& \ e == f$



解题过程

$B \rightarrow B_1 \text{ or } \mathbf{M} B_2$

$\{ \text{backpatch}(B_1.\text{falselist}, M.\text{quad});$

$B.\text{truelist} = \text{merge}(B_1.\text{truelist}, B_2.\text{truelist});$

$B.\text{falselist} = B_2.\text{falselist} ; \}$

$M \rightarrow \epsilon \{ M.\text{quad} = \text{nextquad} ; \}$

100: if $a==b$ goto _

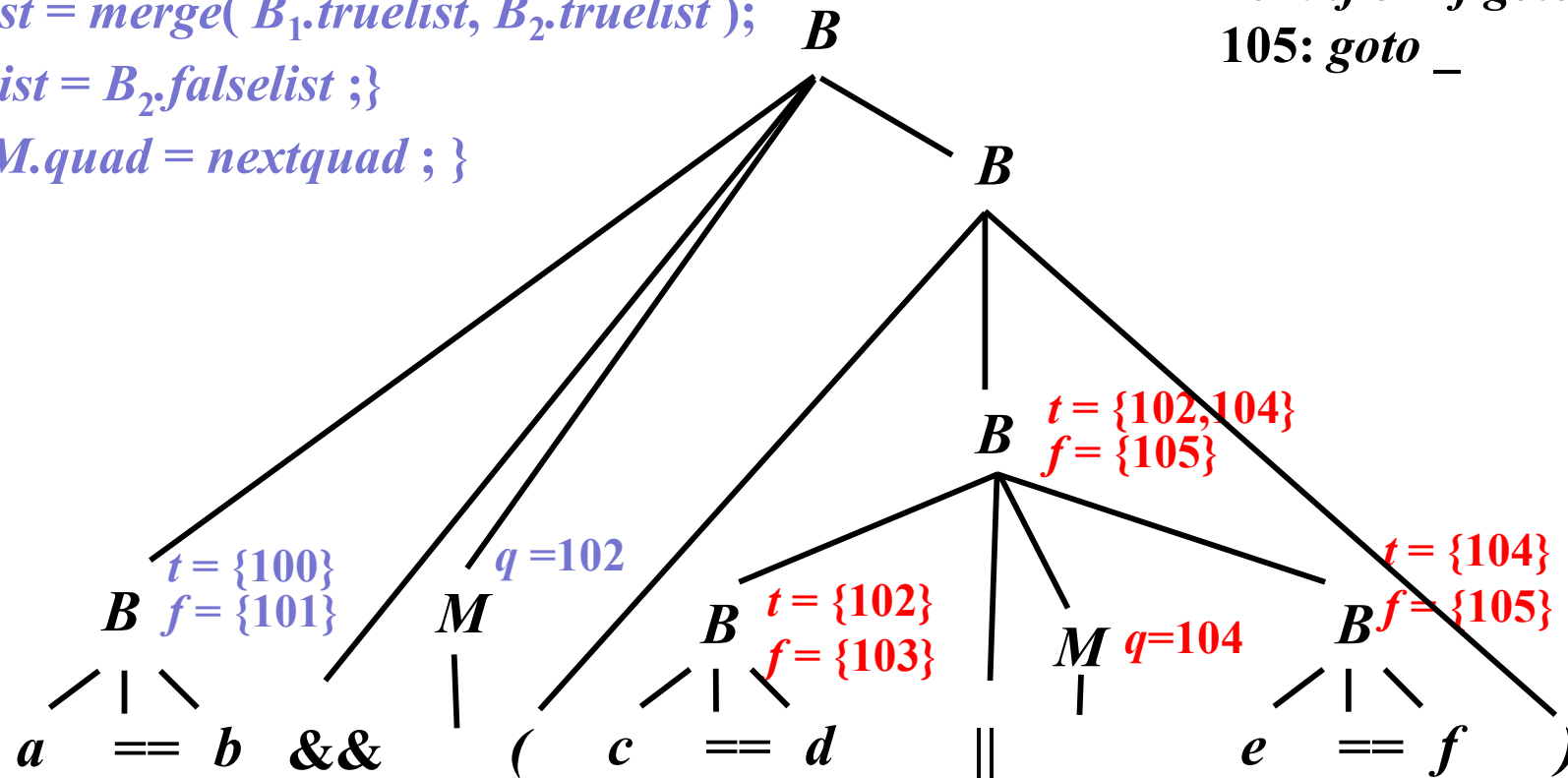
101: goto _

102: if $c==d$ goto _

103: goto **104**

104: if $e==f$ goto _

105: goto _





解题过程

➤ $B \rightarrow (B_1)$

{

$B.truelist = B_1.truelist ;$

$B.falselist = B_1.falselist ;$

}

100: if $a==b$ goto _

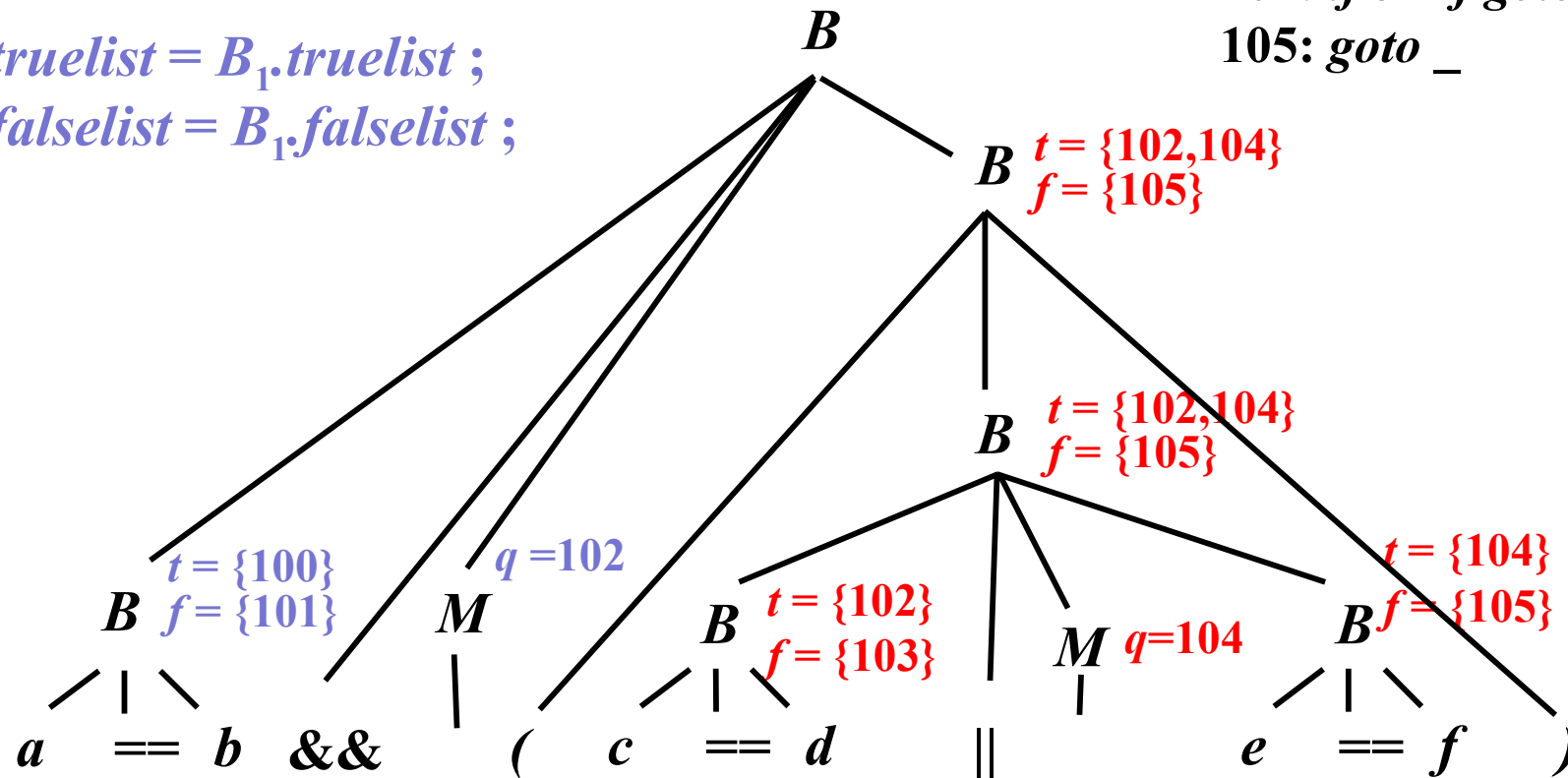
101: goto _

102: if $c==d$ goto _

103: goto **104**

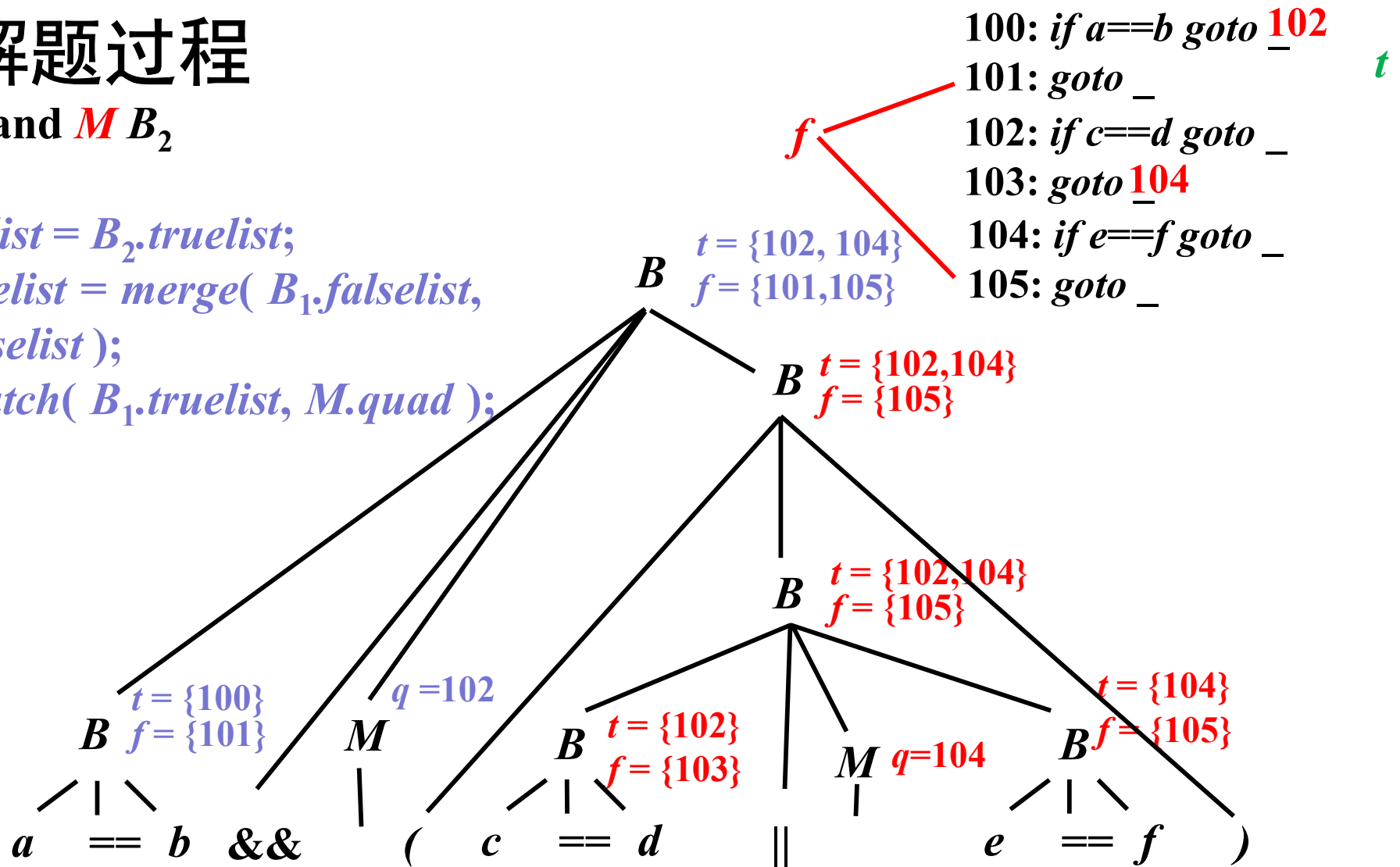
104: if $e==f$ goto _

105: goto _




$$B \rightarrow B_1 \text{ and } M B_2$$

```
B.truelist = B2.truelist;  
B.falselist = merge( B1.falselist,  
B2.falselist );  
backpatch( B1.truelist, M.quad );
```





$(a == b \parallel c == d) \parallel e == f$

$B \rightarrow B_1 \text{ or } M B_2$

{

backpatch(B1.falselist, M.quad);

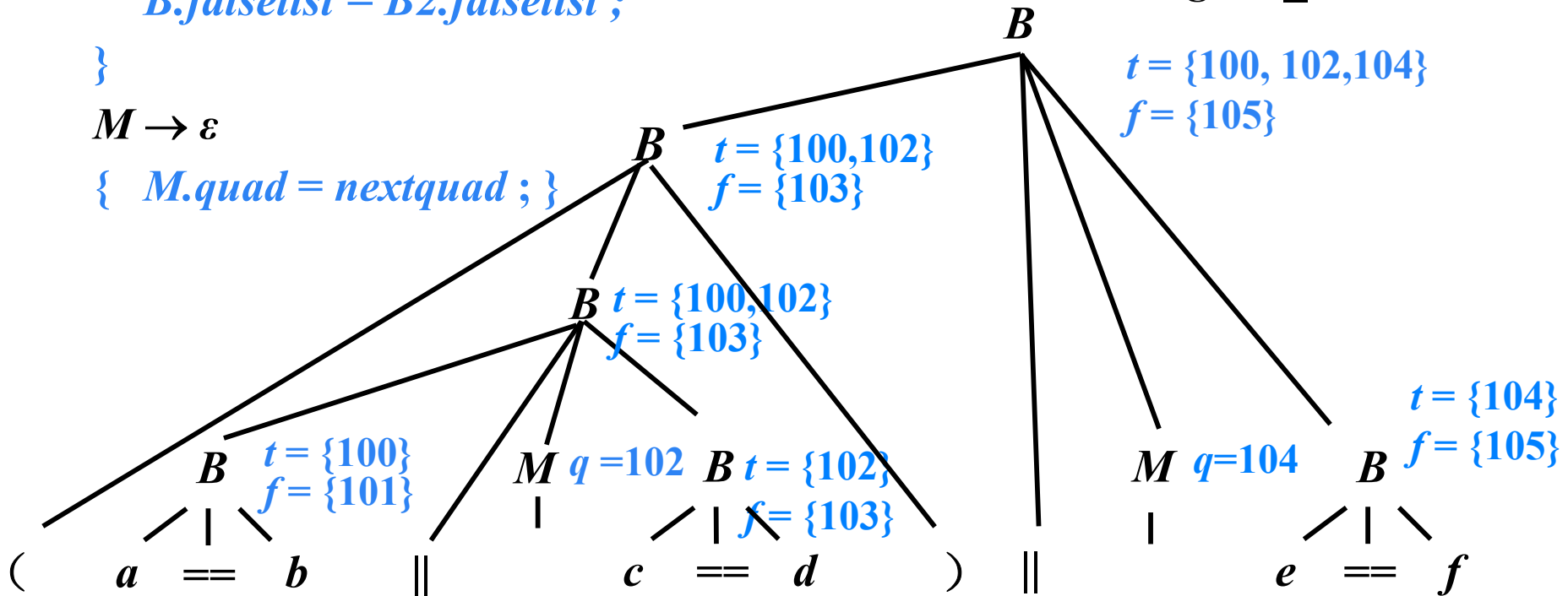
B.truelist = merge(B1.truelist, B2.truelist);

B.falselist = B2.falselist;

}

$M \rightarrow \varepsilon$

{ *M.quad = nextquad;* }



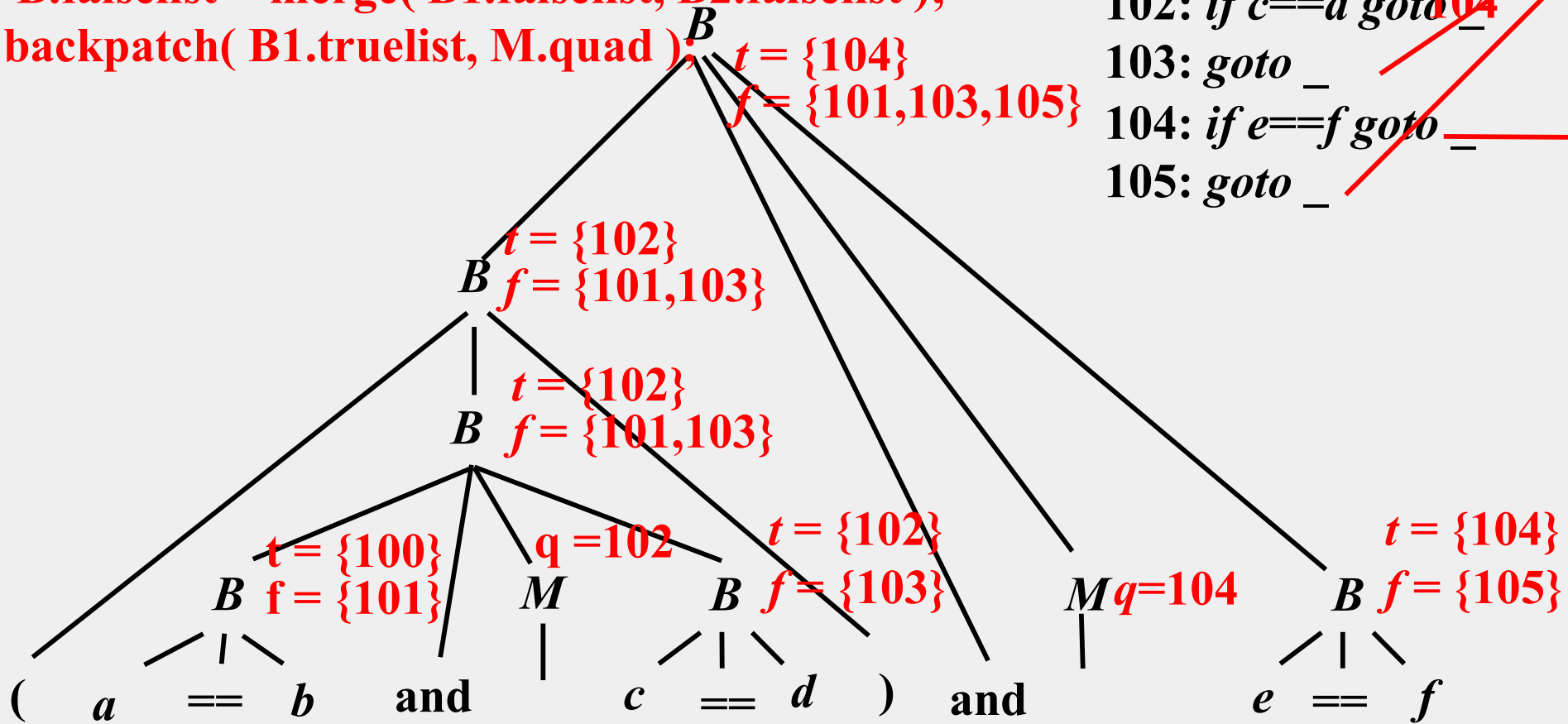
100: *if a==b goto* _ → t
101: *goto* 102 → t
102: *if c==d goto* _ → t
103: *goto* 104 → t
104: *if e==f goto* _ → t
105: *goto* _ → f



$B \rightarrow B_1 \text{ and } M B_2$

```
{
  B.truelist = B2.truelist;
  B.falselist = merge( B1.falselist, B2.falselist );
  backpatch( B1.truelist, M.quad );
}
```

```
100: if a==b goto 102
101: goto         
102: if c==d goto 104
103: goto         
104: if e==f goto         
105: goto         
```





习题14.2

➤ 下图a中给出了一个程序的摘要。b概述了使用SPOC讲义6.4节（回填）中的回填翻译方案生成的三地址代码的结构。这里， $i_1 \sim i_8$ 是每个code区域的第一条被生成的指令的标号。请给出最终回填到下列列表中的标号（即 $i_1 \sim i_8$ 中的某个标号）。

- (1) E_3 .falselist
- (2) S_2 .nextlist
- (3) E_4 .falselist
- (4) S_1 .nextlist
- (5) E_2 .truelist

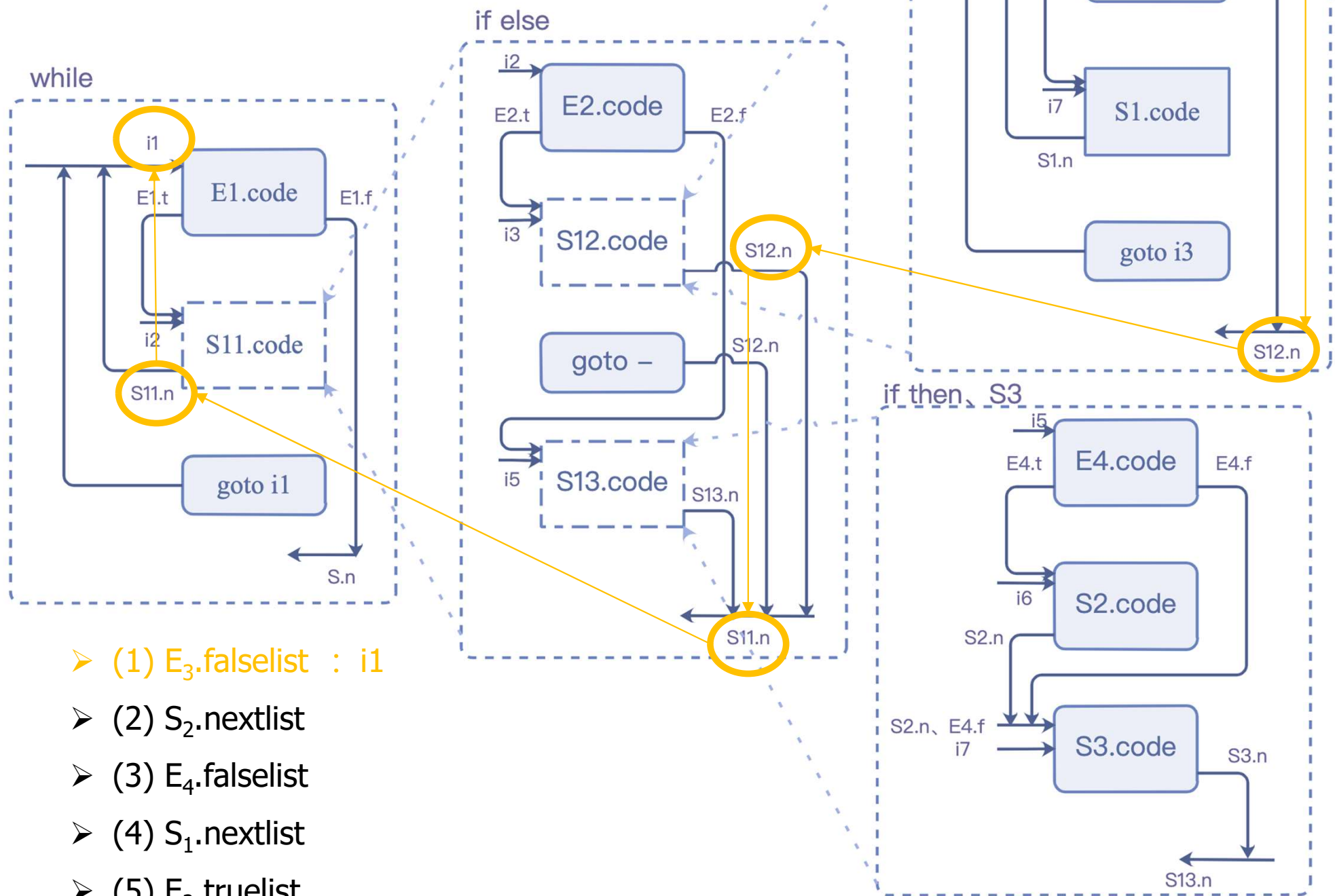
```
while ( $E_1$ ) {  
    if ( $E_2$ )  
        while ( $E_3$ )  
             $S_1$ ;  
    else {  
        if ( $E_4$ )  
             $S_2$ ;  
         $S_3$   
    }  
}
```

(a)

i_1 : Code for E_1
 i_2 : Code for E_2
 i_3 : Code for E_3
 i_4 : Code for S_1
 i_5 : Code for E_4
 i_6 : Code for S_2
 i_7 : Code for S_3
 i_8 : ...

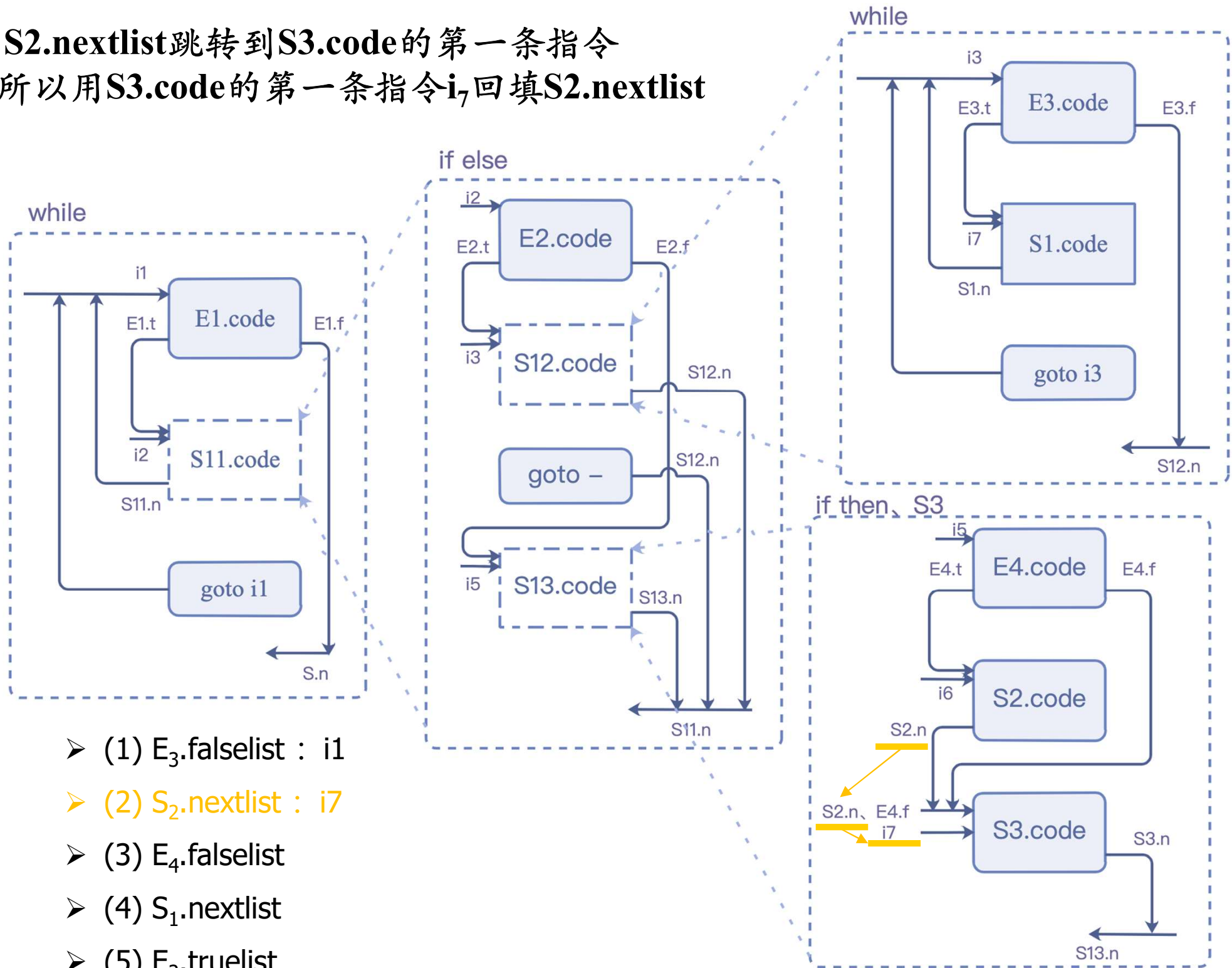
(b)

E3.falselist跳转到最外层while循环的第一条指令
 所以用E1的第一条指令 i_1 回填E3.falselist



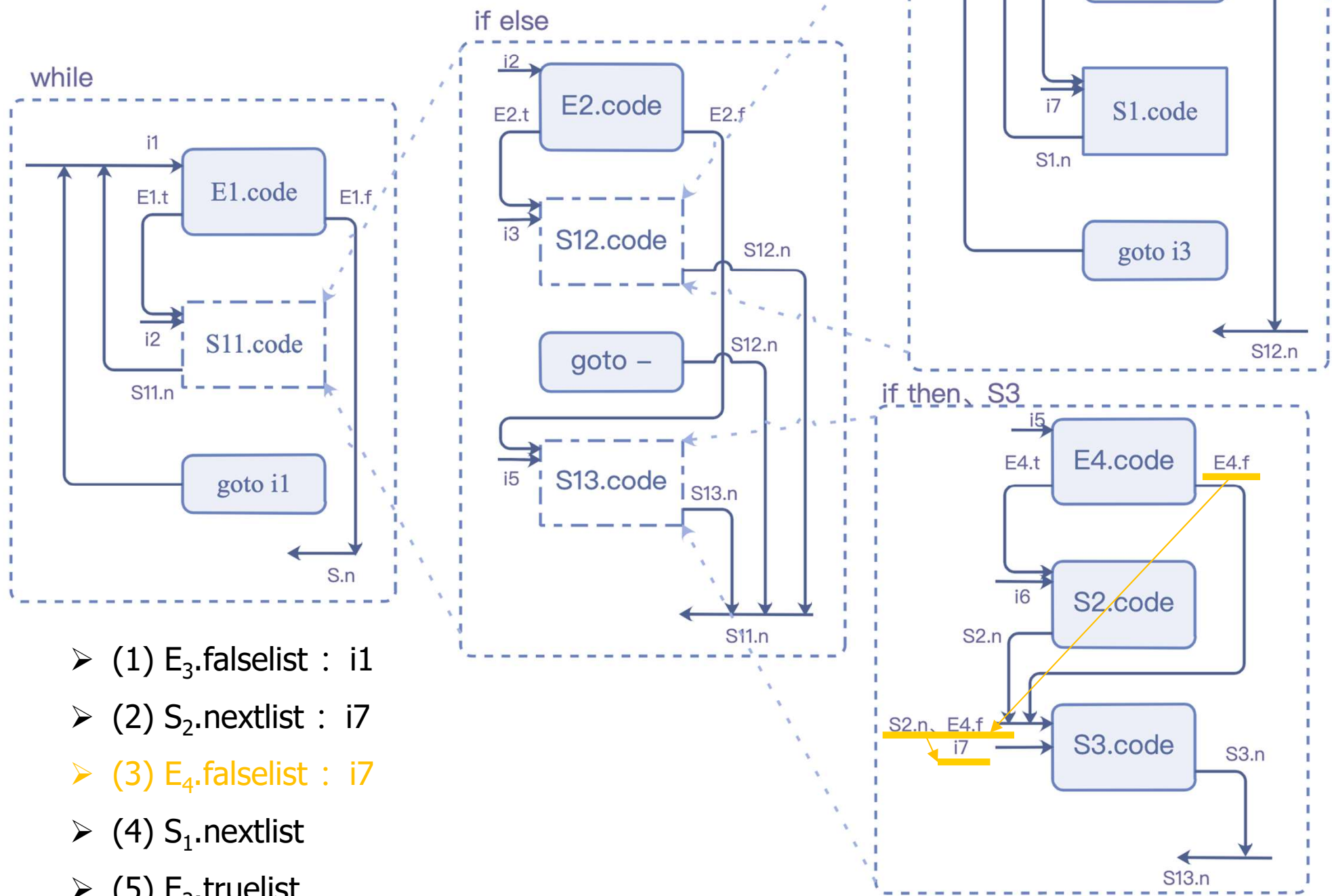
- (1) $E_3.falselist$: i_1
- (2) $S_2.nextlist$
- (3) $E_4.falselist$
- (4) $S_1.nextlist$
- (5) $E_2.truelist$

S2.nextlist跳转到**S3.code**的第一条指令
 所以用**S3.code**的第一条指令*i₇*回填**S2.nextlist**



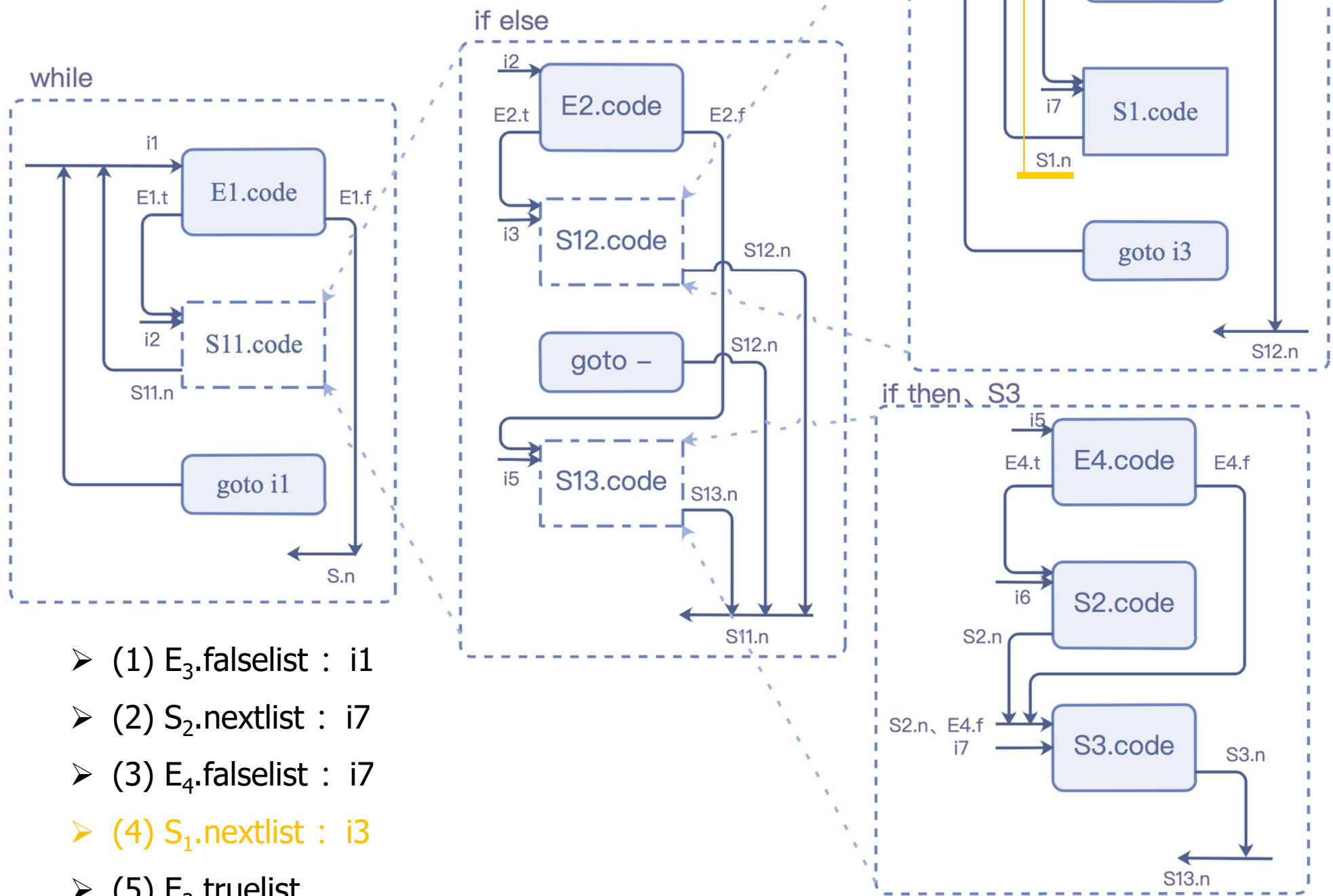
- (1) E_3 .falselist : $i1$
- (2) S_2 .nextlist : $i7$
- (3) E_4 .falselist
- (4) S_1 .nextlist
- (5) E_2 .truelist

E4.falselist跳转到**S3.code**的第一条指令
 所以用**S3.code**的第一条指令*i₇*回填**E4.falselist**



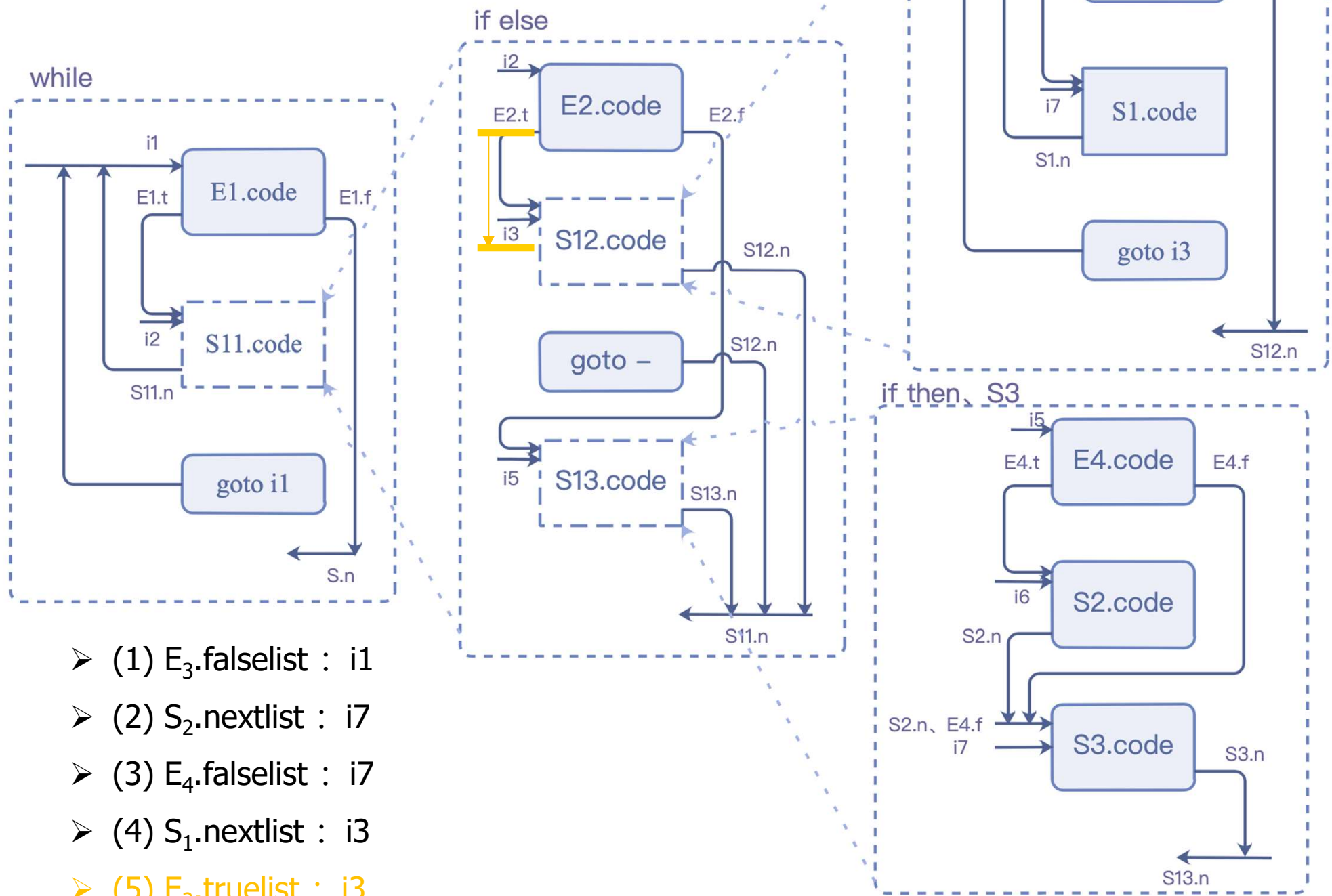
- (1) $E_3.falselist$: $i1$
- (2) $S_2.nextlist$: $i7$
- (3) $E_4.falselist$: $i7$
- (4) $S_1.nextlist$
- (5) $E_2.truelist$

S1.nextlist跳转到内层while循环的第一条指令
 所以用E3的第一条指令 i_3 回填S1.nextlist



- (1) E_3 .falselist : i_1
- (2) S_2 .nextlist : i_7
- (3) E_4 .falselist : i_7
- (4) S_1 .nextlist : i_3
- (5) E_2 .truelist

E2.truelist跳转到内层while循环的第一条指令
 所以用E3的第一条指令*i*₃回填E2.truelist



- (1) *E*₃.falselist : *i1*
- (2) *S*₂.nextlist : *i7*
- (3) *E*₄.falselist : *i7*
- (4) *S*₁.nextlist : *i3*
- (5) *E*₂.truelist : *i3*



习题14.3

- 使用SPOC讲义6.4节（回填）中的翻译方案对下图进行翻译时，我们为每条语句创建S.nextlist列表。一开始是赋值语句 S_1 、 S_2 、 S_3 ，然后逐步处理越来越大的if语句、if-else语句、while语句和语句块。在下图中有5个这种类型的结构语句：

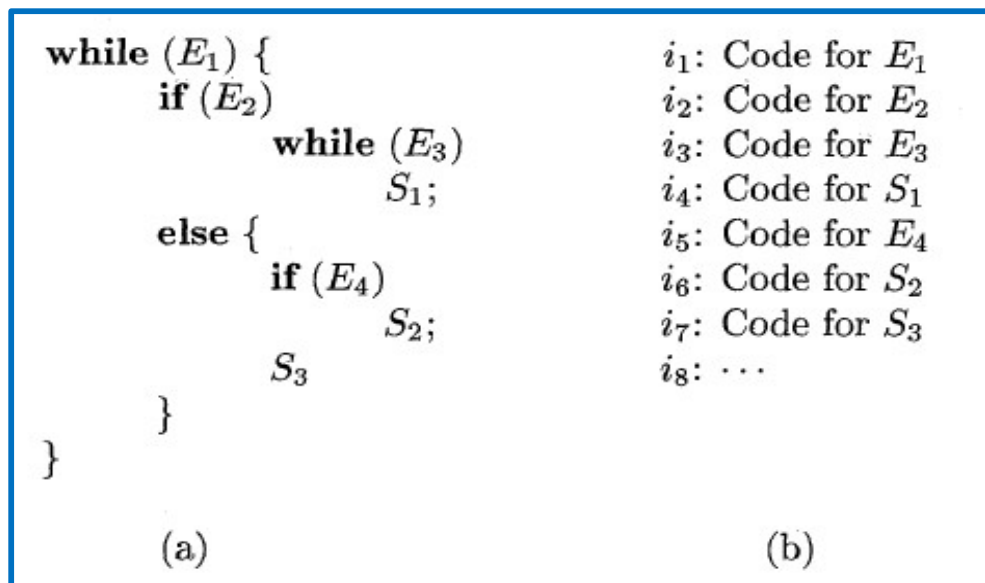
S_4 : while (E_3) S_1

S_5 : if (E_4) S_2

S_6 : 包含 S_5 和 S_3 的语句块

S_7 : 语句if (E_2) S_4 else S_6

S_8 : 整个程序



对于这些结构语句，我们可以通过一个规则用其他的 S_j .nextlist列表以及程序中的表达式的列表 E_k .truelist和 E_k .falselist构造出 S_i .nextlist。给出计算下列nextlist列表的规则：

- (1) S_4 .nextlist
- (2) S_5 .nextlist
- (3) S_6 .nextlist
- (4) S_7 .nextlist
- (5) S_8 .nextlist



SDT

$S_4 \rightarrow \text{while } E_3 \text{ do } S_1$

$S_5 \rightarrow \text{if } E_4 \text{ then } S_3$

$S_6 \rightarrow S_5 \ S_3$

$S_7 \rightarrow \text{if } E_2 \text{ then } S_4 \text{ else } S_6$

$S_8 \rightarrow \text{while } E_1 \text{ do } S_7$

$S_4 \rightarrow \text{while } M_1 \ E_3 \text{ do } M_2 \ S_1 \{$
 $\text{backpatch}(S_1.\text{nextlist}, M_1.\text{quad});$
 $\text{backpatch}(E_3.\text{truelist}, M_2.\text{quad});$
 $S_4.\text{nextlist} = E_3.\text{falselist};$
 $\text{gen}(\text{'goto' } M_1.\text{quad});\}$

$S_5 \rightarrow \text{if } E_4 \text{ then } M \ S_3 \{$
 $\text{backpatch}(E_4.\text{truelist}, M.\text{quad});$
 $S_5.\text{nextlist} = \text{merge}(E_4.\text{falselist}, S_3.\text{nextlist});\}$

$S_6 \rightarrow S_5 \ M \ S_3 \{$
 $\text{backpatch}(S_5.\text{nextlist}, M.\text{quad});$
 $S_6.\text{nextlist} = S_3.\text{nextlist};\}$

$S_7 \rightarrow \text{if } E_2 \text{ then } M_1 \ S_4 \ N \text{ else } M_2 \ S_6 \{$
 $\text{backpatch}(E_2.\text{truelist}, M_1.\text{quad});$
 $\text{backpatch}(E_2.\text{falselist}, M_2.\text{quad});$
 $S_7.\text{nextlist} = \text{merge}(\text{merge}(S_4.\text{nextlist}$
 $(E_3.\text{falselist}), N.\text{nextlist}, S_6.\text{nextlist}$
 $(S_3.\text{nextlist}));\}$

$N \rightarrow \epsilon \{$
 $N.\text{nextlist} = \text{makelist}(\text{nextquad});$
 $\text{gen}(\text{'goto' } _);\}$

$S_8 \rightarrow \text{while } M_1 \ E_1 \text{ do } M_2 \ S_7 \{$
 $\text{backpatch}(S_7.\text{nextlist}, M_1.\text{quad});$
 $\text{backpatch}(E_1.\text{truelist}, M_2.\text{quad});$
 $S_8.\text{nextlist} = E_1.\text{falselist};$
 $\text{gen}(\text{'goto' } M_1.\text{quad});\}$



习题14.4

- 在SPOC讲义6.4节（回填）所示的SDT中添加处理如下for循环语句的翻译方案

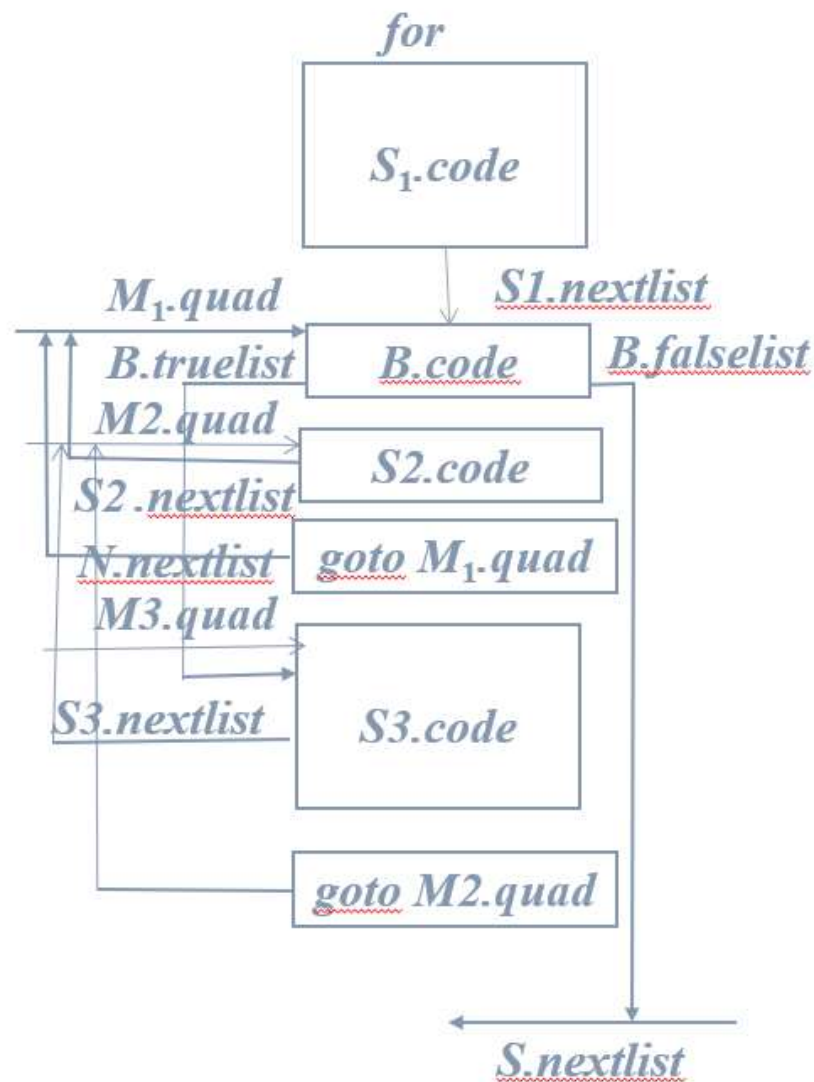
$$S \rightarrow \textit{for} (S_1 ; B ; S_2) S_3$$

$S \rightarrow \text{for } (S1; B; S2) S3$

$S \rightarrow \text{for}(S1; \mathbf{M1} \ B ; \mathbf{M2} \ S2) \mathbf{N} \ \mathbf{M3} S3$

```

{
   $S.nextlist = B.falselist$ ;
   $backpatch(S1.nextlist, M_1.quad)$ ;
   $backpatch(S2.nextlist, M_1.quad)$ ;
   $backpatch(B.truelist, M3.quad)$ ;
   $backpatch(S3.nextlist, M2.quad)$ ;
   $gen('goto' M2.quad)$ ;
}
M  $\rightarrow \epsilon$  {
   $M.quad = nextquad$ ;
}
N  $\rightarrow \epsilon$  {
   $gen('goto' M1.quad)$ ;
}
  
```



翻译方案示意图



习题14.5

- 设有形如 $\text{call}(E) P_1, P_2, P_3$ 的过程调用语句，其中， E 为算术表达式， P_1 、 P_2 和 P_3 是 3 个不带参数的过程的名字。该调用语句执行的流程为：首先计算表达式 E 的值， $E=0$ 时调用 P_1 ， $E>0$ 时调用 P_2 ， $E<0$ 时调用 P_3 。该语句执行时， P_1 、 P_2 和 P_3 中有且仅有一个被调用。
 - (1) 试给出该调用语句的三地址码。
 - (2) 给出该调用语句适于自底向上分析的 SDT



```
E=t1
If E==0 goto L1
goto L2
L2: if E>0 goto L3
    goto L4
L4: if E<0 goto L5
    goto L0
L1: call P1,0
L3: call P2,0
L5: call P3,0
L0:
```

```

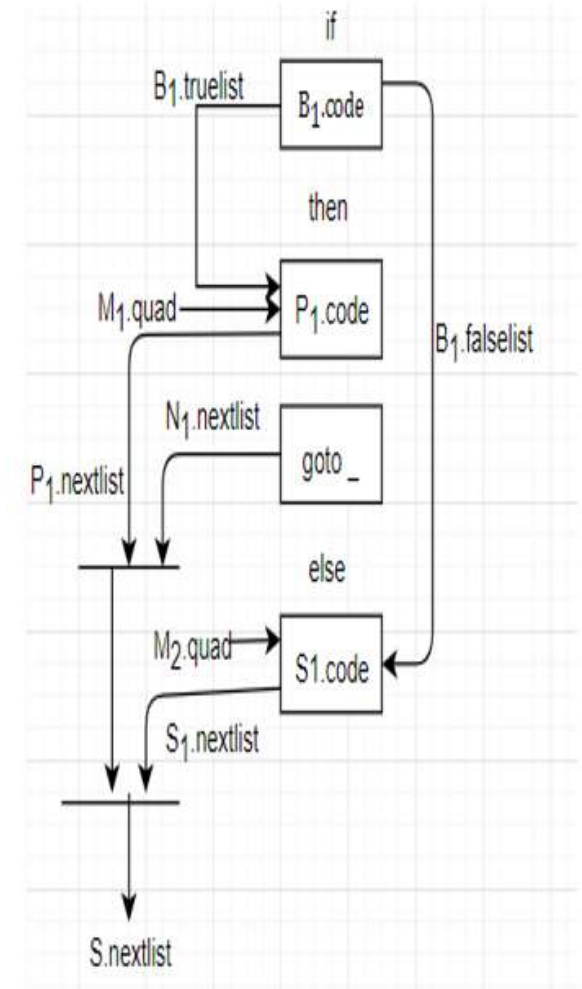
call (E) P1, P2, P3
{
  if(E==0)
    P1
  else
  {
    if(E>0)
      P2
    else
      P3
  }
}

```

```

call (E) P1, P2, P3
{
  gen('param' E.addr);
  gen('call' id.addr, 1);
  S->if B1 then M1P1N1 else M2S2
  {
    backpatch(B1.truelist, M1.quad);
    backpatch(B1.falselist, M2.quad);
    S.nextlist = merge(merge(P1.nextlist, N1.nextlist), S1.nextlist);
  }
  B1->(E==0)
  {
    B1.truelist = makelist(nextquad);
    B1.falselist = makelist(nextquad + 1);
    gen('ifE.addr==0'goto_);
    gen('goto_');
    M1->ε
    { M1.quad = nextquad+2; }
    N1->ε
    { N1.nextlist = makelist(nextquad+3);
      gen('goto_'); }
    S1->ε
    { M2.quad = nextquad+4; }
    S1->if B2 then M2P2N2 else M4P3
    {
      backpatch(B2.truelist, M3.quad);
      backpatch(B2.falselist, M4.quad);
      S1.nextlist = merge(merge(P2.nextlist, N2.nextlist), P3.nextlist);
    }
  }
  B2->(E>0)
  {
    B2.truelist = makelist(nextquad+5);
    B2.falselist = makelist(nextquad + 6);
    gen('ifE.addr>0'goto_);
    gen('goto_');
    M3->ε
    { M3.quad = nextquad+7; }
    N2->ε
    { N2.nextlist = makelist(nextquad+8);
      gen('goto_'); }
    M4->ε
    { M4.quad = nextquad+9; }
  }
}

```





习题14.6

➤ 给定如下文法：

$$S \rightarrow (L) \mid a$$

$$L \rightarrow L, S \mid S$$

- (1) 试设计一个SDD，输出S所生成的串中的配对括号的个数。
- (2) 试设计一个SDT，输出S所生成的串中每个a的嵌套深度。例如，串(a,(a,a))的输出结果是1、2、2。



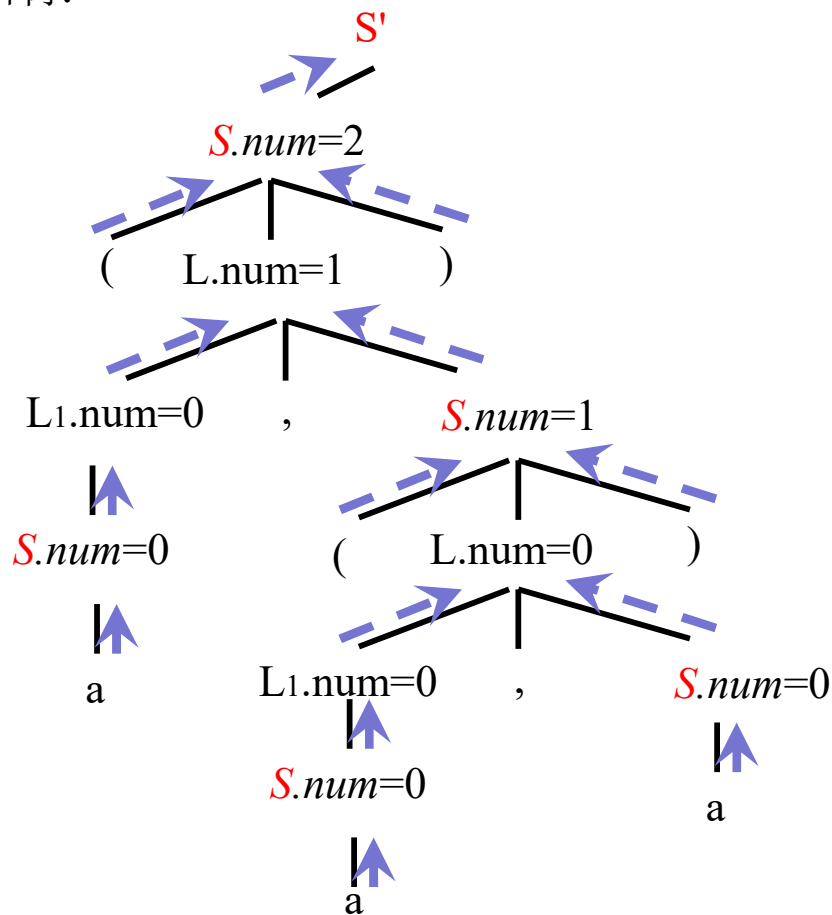
习题14.6

举例：句子是(a, (a, a))

输出结果为2。

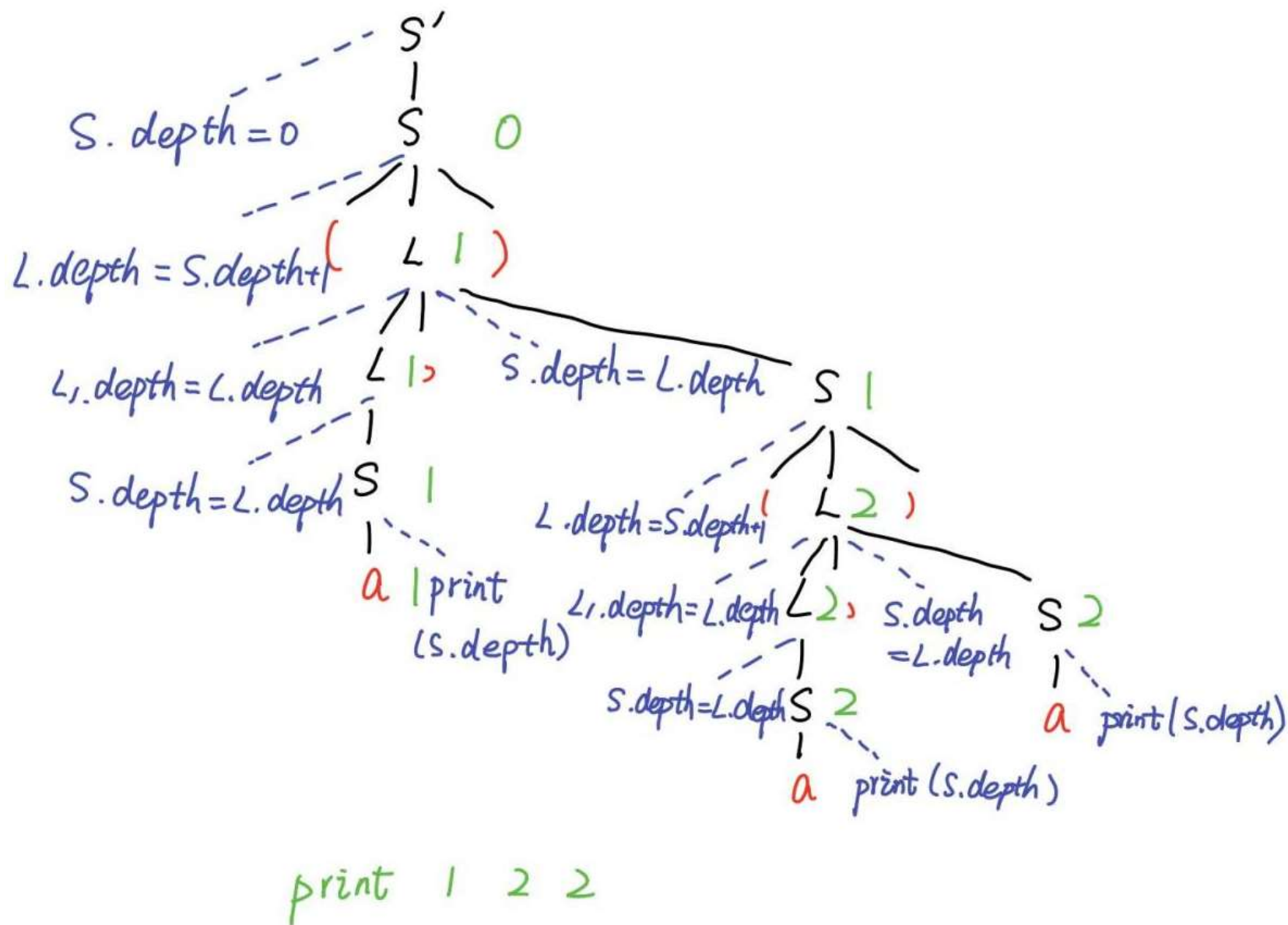
注释分析树：

产生式	语义规则
(1) $S \rightarrow S'$	print(S.num)
(2) $S \rightarrow (L)$	$S.num = L.num + 1$
(3) $S \rightarrow a$	$S.num = 0$
(4) $L \rightarrow L1 , S$	$L.num = L1.num + S.num$
(5) $L \rightarrow S$	$L.num = S.num$





检验



- (1) $S' \rightarrow S$
- (2) $S \rightarrow (L)$
- (3) $S \rightarrow a$
- (4) $L \rightarrow L, S$
- (5) $L \rightarrow S$



SDT:

$S' \rightarrow \{S.depth = 0\} S$
 $S \rightarrow \{L.depth = S.depth + 1\} (L)$
 $S \rightarrow a \{print(S.depth)\}$
 $L \rightarrow \{L_1.depth = L.depth\} L_1,$
 $\quad \{S.depth = L.depth\} S$
 $L \rightarrow \{S.depth = L.depth\} S$



习题15.1

- 假设SPOC讲义7.1节（栈式存储分配）中的程序（如下图所示）使用如下的**partition**函数：该函数总是将 $a[m]$ 作为分割值 v 。同时假设在对数组 $a[m], \dots, a[n]$ 重新排序时总是尽量保存原来的顺序。也就是说，首先是以原顺序保持所有小于 v 的元素，然后保存所有等于 v 的元素，最后按原来顺序保存所有大于 v 的元素。
- (1) 画出对数字9、8、7、6、5、4、3、2、1进行排序时的活动树。
- (2) 同时在栈中出现的活动记录最多有多少个？

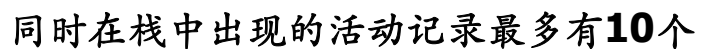


习题15.1 (con.)



例：一个快速排序程序的概要

```
int a[11];
void readArray() /*将9个整数读入到a[1],...,a[9]中*/
{
    int i;
    ...
}
int partition(int m, int n)
{
    /*选择一个分割值v, 划分a[m...n], 使得a[m...p-1]小于v, a[p]=v,
    a[p+1...n]大于等于v。返回p*/
    ...
}
void quicksort(int m, int n)
{
    int i;
    if (n > m) {
        i = partition(m, n);
        quicksort(m, i-1);
        quicksort(i+1, n);
    }
}
main()
{
    readArray();
    a[0] = -9999;
    a[10] = 9999;
    quicksort(1, 9);
}
```



控制栈

<i>main</i>
<code>int a[11]</code>
<i>q(1,9)</i>
<code>int i</code>
<i>q(1,9)</i>
<code>int i</code>
<i>q(1,8)</i>
<code>int i</code>
<i>q(1,6)</i>
<code>int i</code>
■ ■ ■
<i>q(1,2)</i>
<code>int i</code>



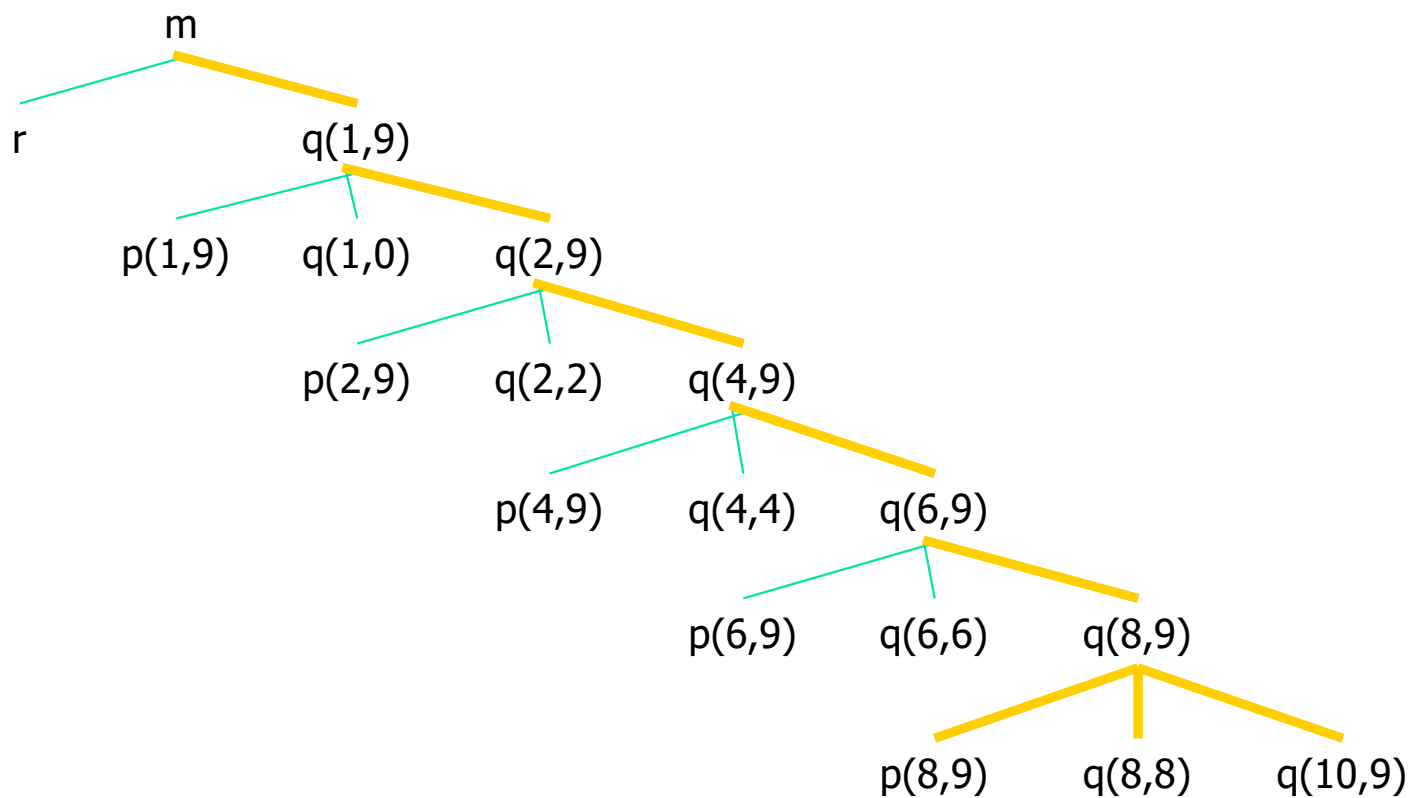
习题15.2

➤ 当初始顺序为1、3、5、7、9、2、4、6、8时，重
复习题15.1

解题思路：

	$a[m]-a[n]$	分割值	排序后的 $a[m]-a[n]$	返回值	排序后的数组
$p(1,9)$	1,3,5,7,9,2,4,6,8	$v = 1$	1,3,5,7,9,2,4,6,8	$p(1,9) = 1$	1,3,5,7,9,2,4,6,8
$p(2,9)$	3,5,7,9,2,4,6,8	$v = 3$	2,3,5,7,9,4,6,8	$p(2,9) = 3$	1,2,3,5,7,9,4,6,8
$p(4,9)$	5,7,9,4,6,8	$v = 5$	4,5,7,9,6,8	$p(4,9) = 5$	1,2,3,4,5,7,9,6,8
$p(6,9)$	7,9,6,8	$v = 7$	6,7,9,8	$p(6,9) = 7$	1,2,3,4,5,6,7,9,8
$p(8,9)$	9,8	$v = 9$	8,9	$p(8,9) = 9$	1,2,3,4,5,6,7,8,9

解答：（1）活动树



（2）同时在栈中出现的活动记录最多有7个



习题15.3

➤ 下图是递归计算Fibonacci数列的C语言代码。假设f的活动记录按顺序包含下列元素：（返回值，参数n，局部变量s，局部变量t）。通常在活动记录中还会有其他元素。下面的问题假设初始调用是f(5)。

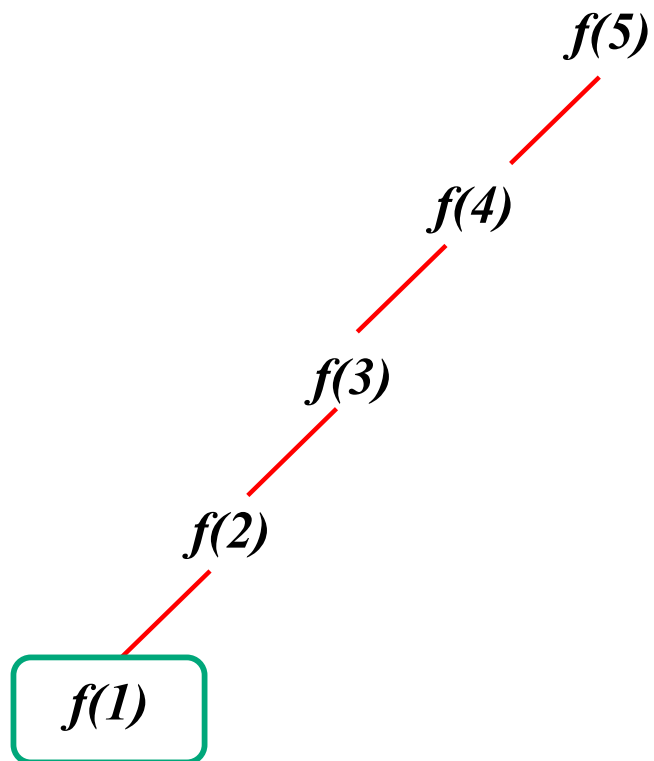
- (1) 给出完整的活动树。
- (2) 当第1个f(1)调用即将返回时，运行时刻栈和其中的活动记录是什么样子的？
- (3) 当第5个f(1)调用即将返回时，运行时刻栈和其中的活动记录是什么样子的？

```
int f(int n) {  
    int t, s;  
    if (n < 2) return 1;  
    s = f(n-1);  
    t = f(n-2);  
    return s+t;  
}
```



第一个 $f(1)$

活动树



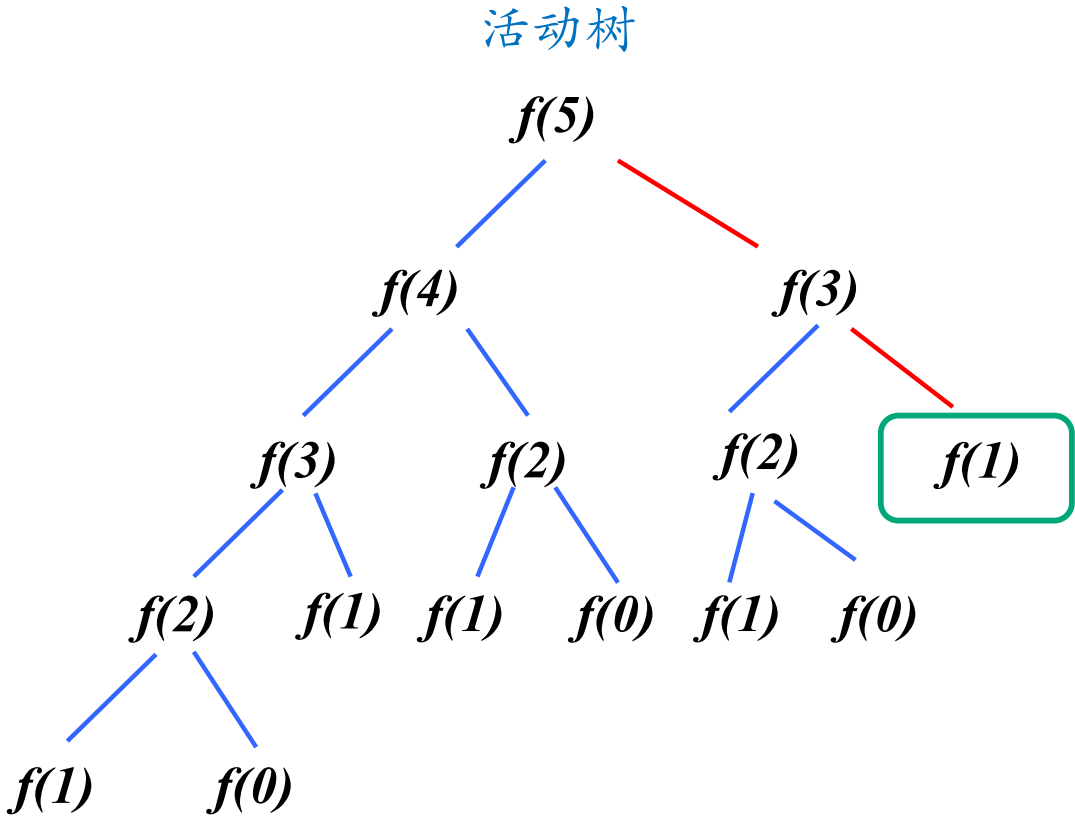
控制栈

$f(5)$
return
int n=5
int s=
int t=
$f(4)$
return
int n=4
int s=
int t=
$f(3)$
return
int n=3
int s=
int t=

$f(2)$
return
int n=2
int s=
int t=
$f(1)$
return 1
int n=1
int s
int t

第五个 $f(1)$

活动树



控制栈

$f(5)$
return
int n=5
int s=5
int t=
$f(3)$
return
int n=3
int s=2
int t=
$f(1)$
return 1
int n=1
int s
int t



习题15.4

➤ 下面是两个C语言函数f和g的概述：

```
int f ( int x) { int i ; ...return i+1; ...}
```

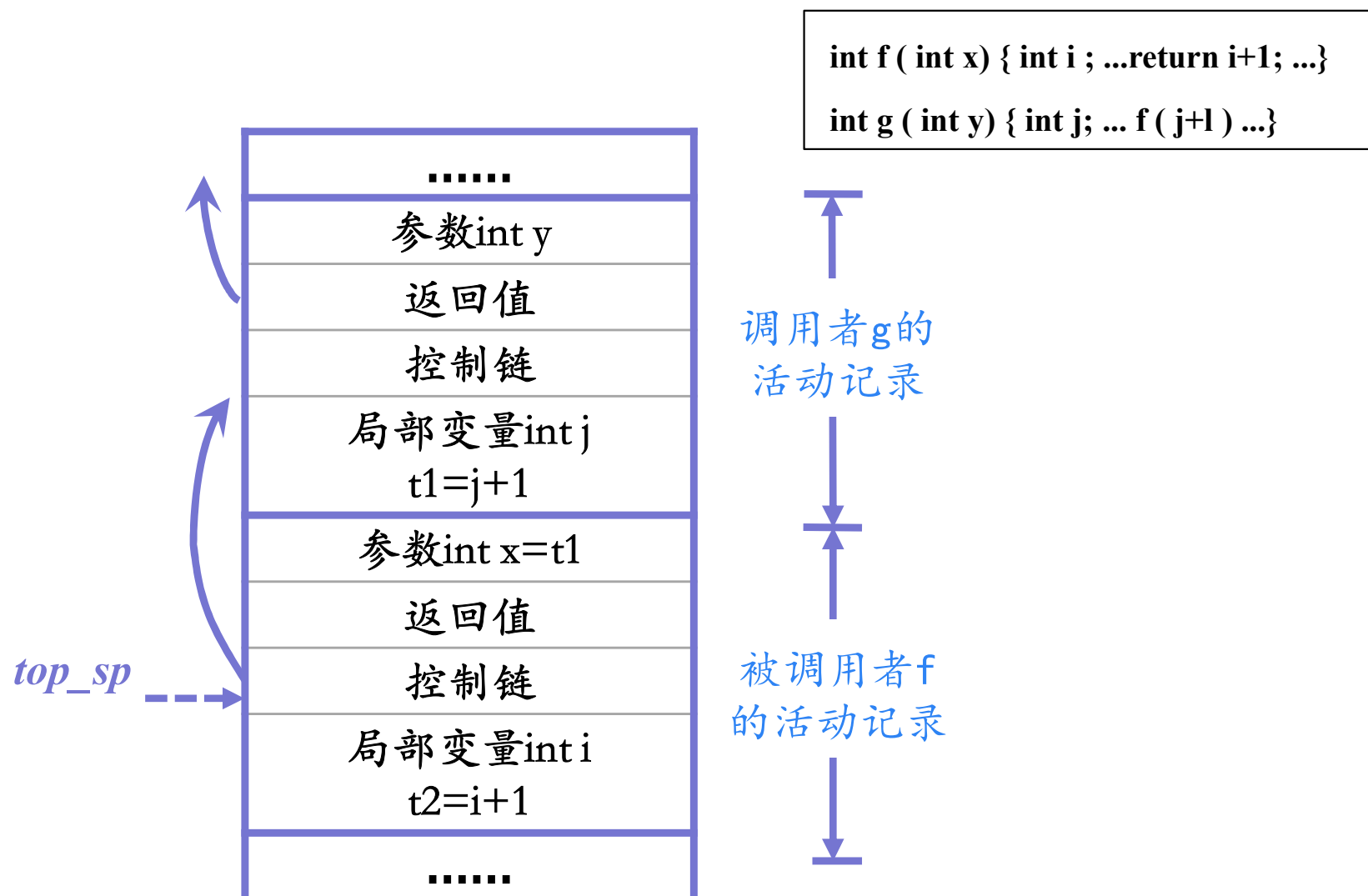
```
int g ( int y) { int j; ... f ( j+1 ) ...}
```

也就是说，函数g调用函数f。画出在g调用f而f即将返回时，运行时刻栈中从g的活动记录开始的顶端部分。你可以只考虑返回值、参数、控制链以及存放局部数据的空间。你不用考虑存放的机器状态，也不用考虑没有在代码中显示的局部值和临时值。但是你应该指出：

- (1) 哪个函数在栈中为各个元素创建了所使用的空间？
- (2) 哪个函数写入了各个元素的值？
- (3) 这些元素属于哪个活动记录？

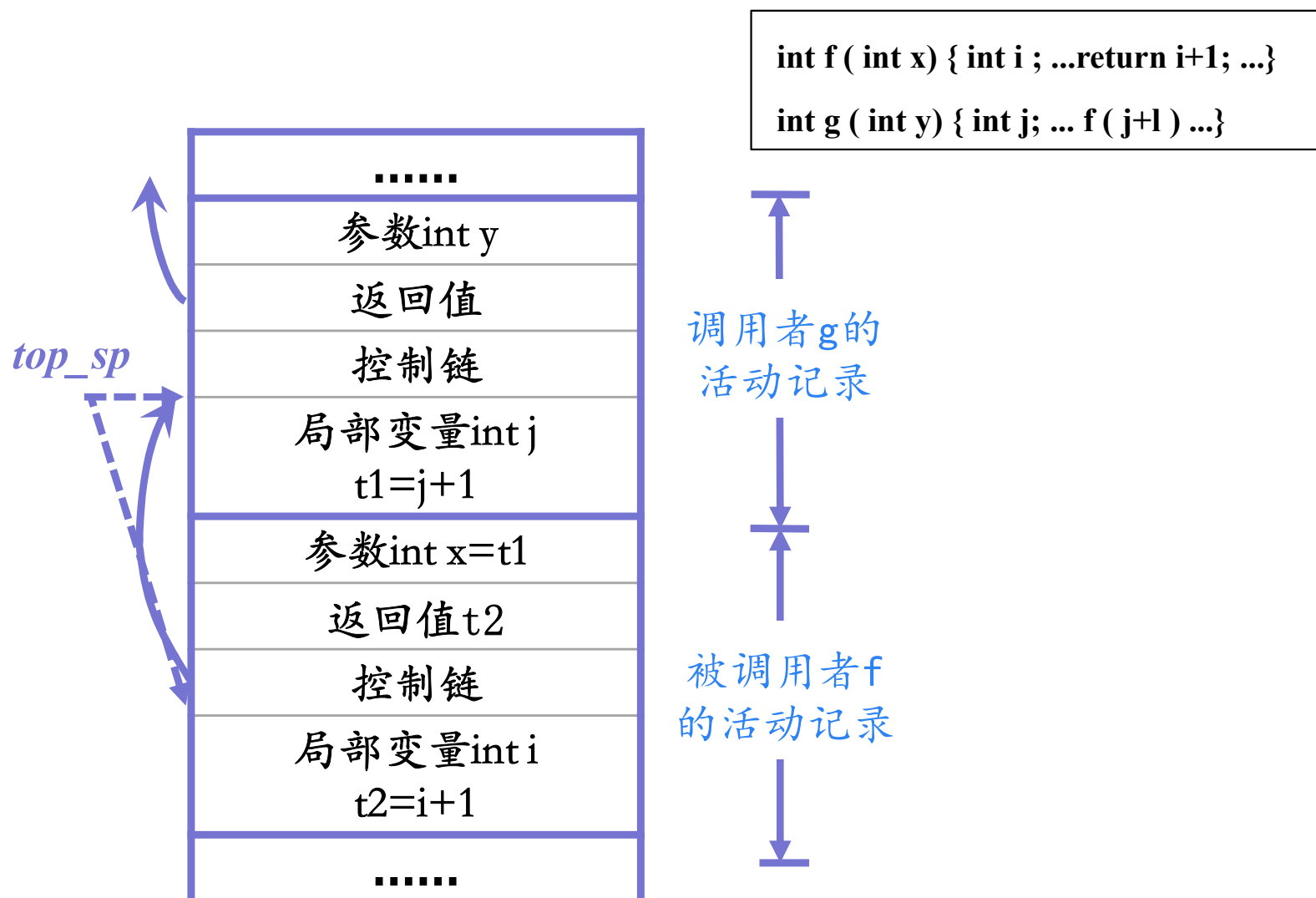


运行时刻栈（即将返回时）



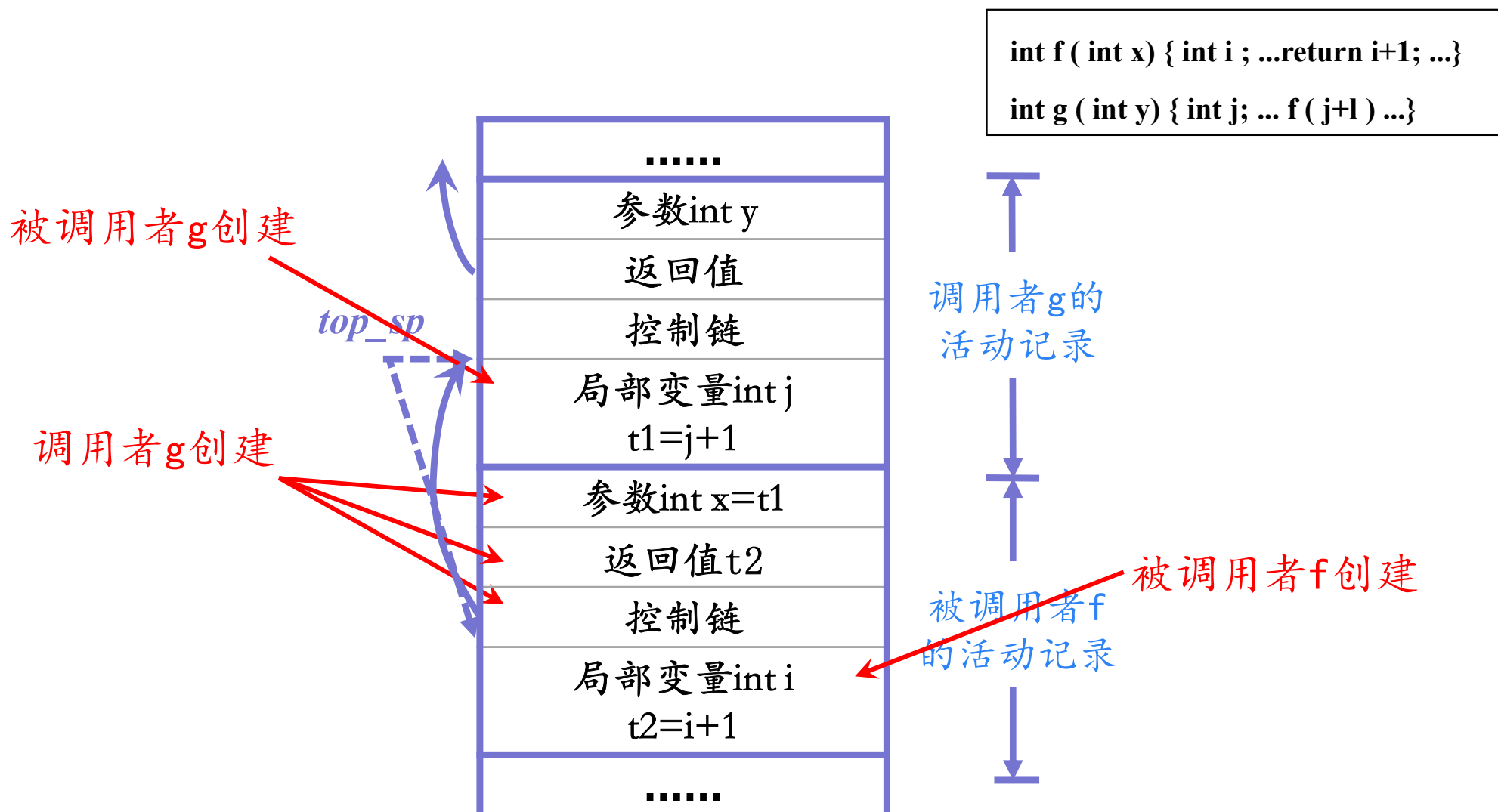


运行时刻栈元素隶属



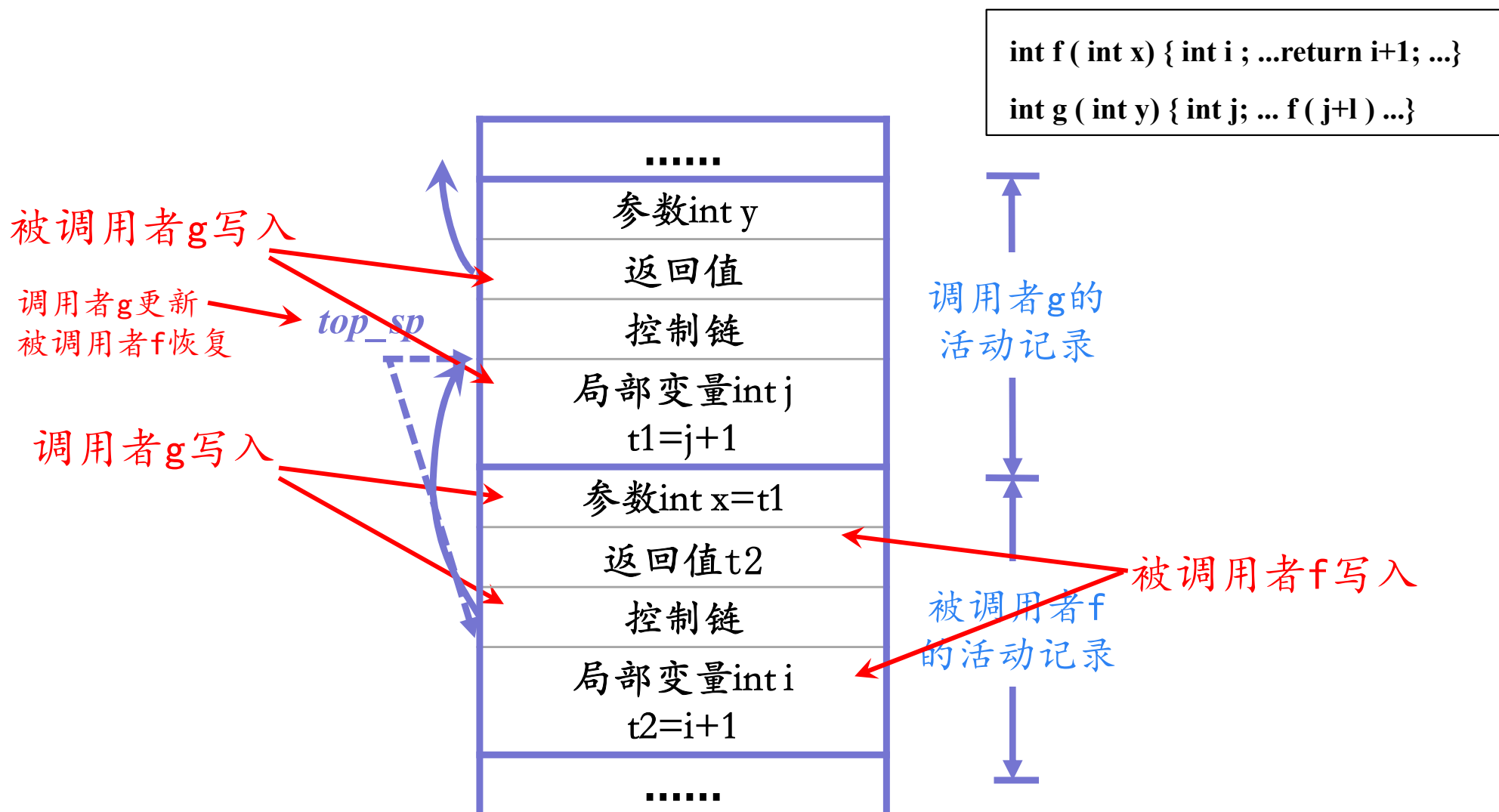


运行时刻栈元素创建





运行时刻栈元素写入





习题15.5

- 下图中给出了一个按照非标准方式计算Fibonacci数的ML语言的函数main。函数fib0将计算第n个Fibonacci数 ($n \geq 0$)。嵌套在fib0中的是fib1，它假设 $n \geq 2$ 并计算第n个Fibonacci。嵌套在fib1中的是fib2，它假设 $n \geq 4$ 。请注意，fib1和fib2都不需要检查基本情况。我们考虑从对main的调用开始，直到（对fib0(1)的）第一次调用即将返回的阶段。请描述出当时的活动记录栈，并给出栈中的各个活动记录的访问链。

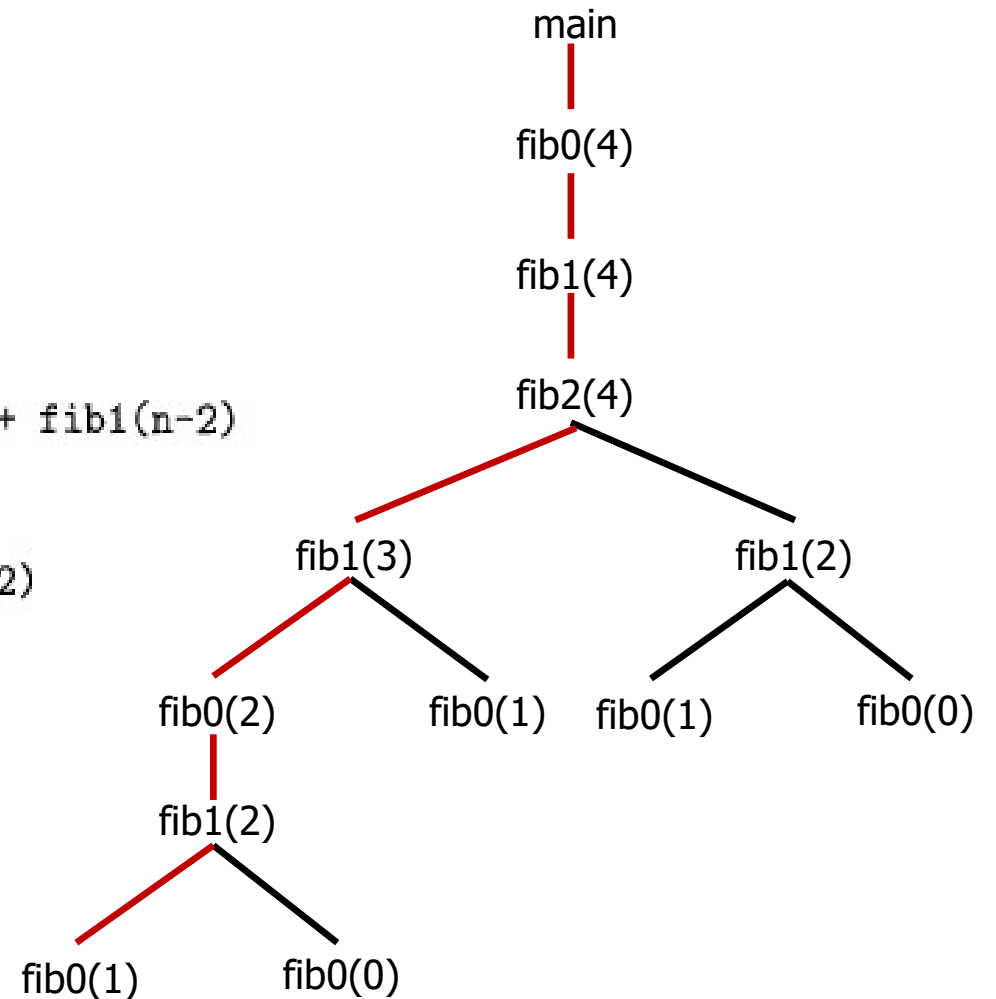


习题15.5 (con.)

```
fun main () {  
  let  
    fun fib0(n) =  
      let  
        fun fib1(n) =  
          let  
            fun fib2(n) = fib1(n-1) + fib1(n-2)  
          in  
            if n >= 4 then fib2(n)  
            else fib0(n-1) + fib0(n-2)  
          end  
        in  
          if n >= 2 then fib1(n)  
          else 1  
        end  
      in  
        fib0(4)  
      end;  
end;
```

活动树

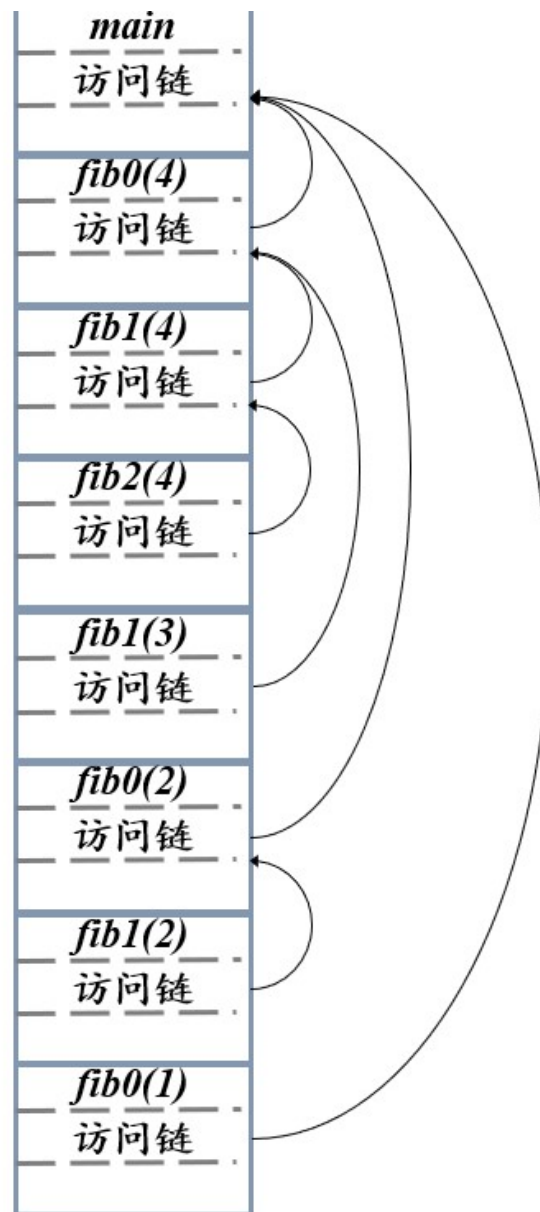
```
fun main () {  
  let  
    fun fib0(n) =  
      let  
        fun fib1(n) =  
          let  
            fun fib2(n) = fib1(n-1) + fib1(n-2)  
          in  
            if n >= 4 then fib2(n)  
            else fib0(n-1) + fib0(n-2)  
          end  
        in  
          if n >= 2 then fib1(n)  
          else 1  
        end  
      in  
        fib0(4)  
      end;  
end;
```





活动记录栈&访问链

过程	嵌套深度
<i>main</i>	1
<i>fib0</i>	2
<i>fib1</i>	3
<i>fib2</i>	4





习题15.6

- 在本讲中，我们提到，编译器通常为每个作用域（程序块）建立一个独立的符号表（SPOC讲义p49）。有关作用域和块结构的概念参见教材1.6.1节（静态和动态的区别）和1.6.3节（静态作用域和块结构）。对于下图所示的块结构代码（为便于引用各语句，增加了行号），假设使用常见的声明的静态作用域规则，请完成以下习题。

```
(1)  {    int w, x, y, z;      /* Block B1 */
(2)      {    int x, z;        /* Block B2 */
(3)          {    int w, x;    /* Block B3 */ }
(4)      }
(5)  {    int w, x;          /* Block B4 */
(6)      {    int y, z;      /* Block B5 */ }
(7)      }
(8)  }
```



习题15.6 (con.)

- (1) 判断下列说法的对错
 - B1声明的w的作用域是1-8行
 - B1声明的y的作用域是1-5行
- (2) 给出该代码片段中12个声明中的每一个的作用域
- (3) 画出该代码片段的符号表



```
(1) {   int w, x, y, z;   /* Block B1 */  
(2)     { int x, z;           /* Block B2 */  
(3)       {   int w, x; /* Block B3 */ }  
(4)     }  
(5)   { int w, x;           /* Block B4 */  
(6)     {   int y, z; /* Block B5 */ }  
(7)   }  
(8) }
```

(1) B1声明的w的作用域是1-8行, 错误B1声明的y的作用域也是1-5,7, 8行, 错误

(2) Block B1: w (1248) , x (18) , y (1-578) , z (1578)

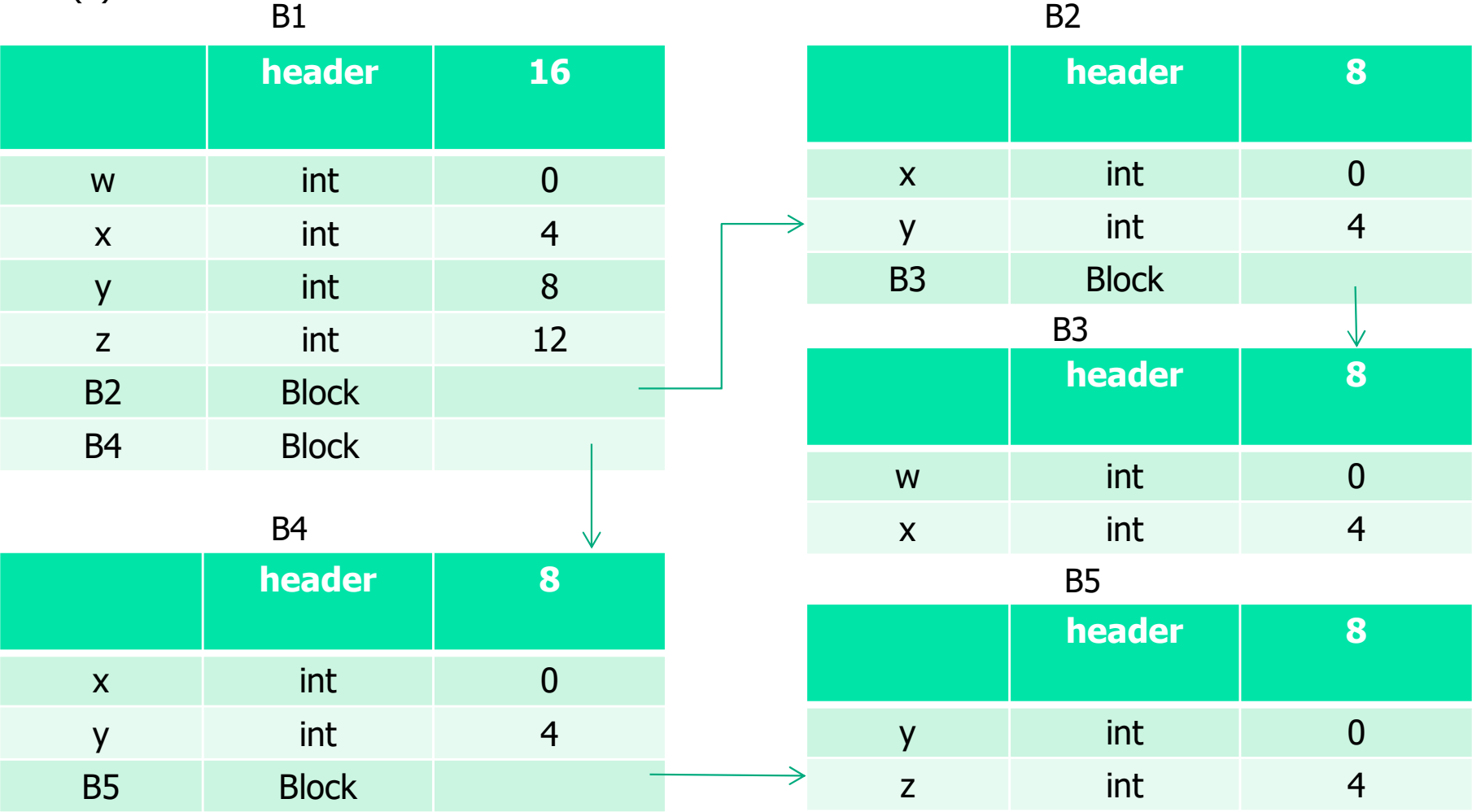
Block B2: x (24) , z作用域是2-4行

Block B3: w, x;作用域是第3行

Block B4: w, x;作用域是5-7行

Block B5: y, z;作用域是第6行

(3)符号表





习题16.1

➤ 下图是一个简单的矩阵乘法程序。

```
for (i=0; i<n; i++)  
    for (j=0; j<n; j++)  
        c[i][j] = 0.0;  
for (i=0; i<n; i++)  
    for (j=0; j<n; j++)  
        for (k=0; k<n; k++)  
            c[i][j] = c[i][j] + a[i][k]*b[k][j];
```

- (1) 假设矩阵的元素是需要8个字节的数值，而且矩阵按行存放。把程序翻译成三地址语句。
- (2) 为(1)中得到的代码构造流图。
- (3) 找出在(2)中得到的流图的循环。



untitled — Atom

File Edit View Selection Find Packages Help

```
untitled
1 B1 1) i = 0
2 B2 2) if i >= n goto(13)
3 B3 3) j = 0
4 B4 4) if j >= n goto(11)
5 B5 5) t1 = n * i
6 6) t2 = t1 + j
7 7) t3 = t2 * 8
8 8) c[t3] = 0.0
9 9) j = j + 1
10 10) goto(4)
11 B6 11) i = i + 1
12 12) goto(2)
13 B7 13) i = 0
14 B8 14) if i >= n goto(40)
15 B9 15) j = 0
16 B10 16) if j >= n goto(38)
17 B11 17) k = 0
18 B12 18) if k >= n goto(36)
19

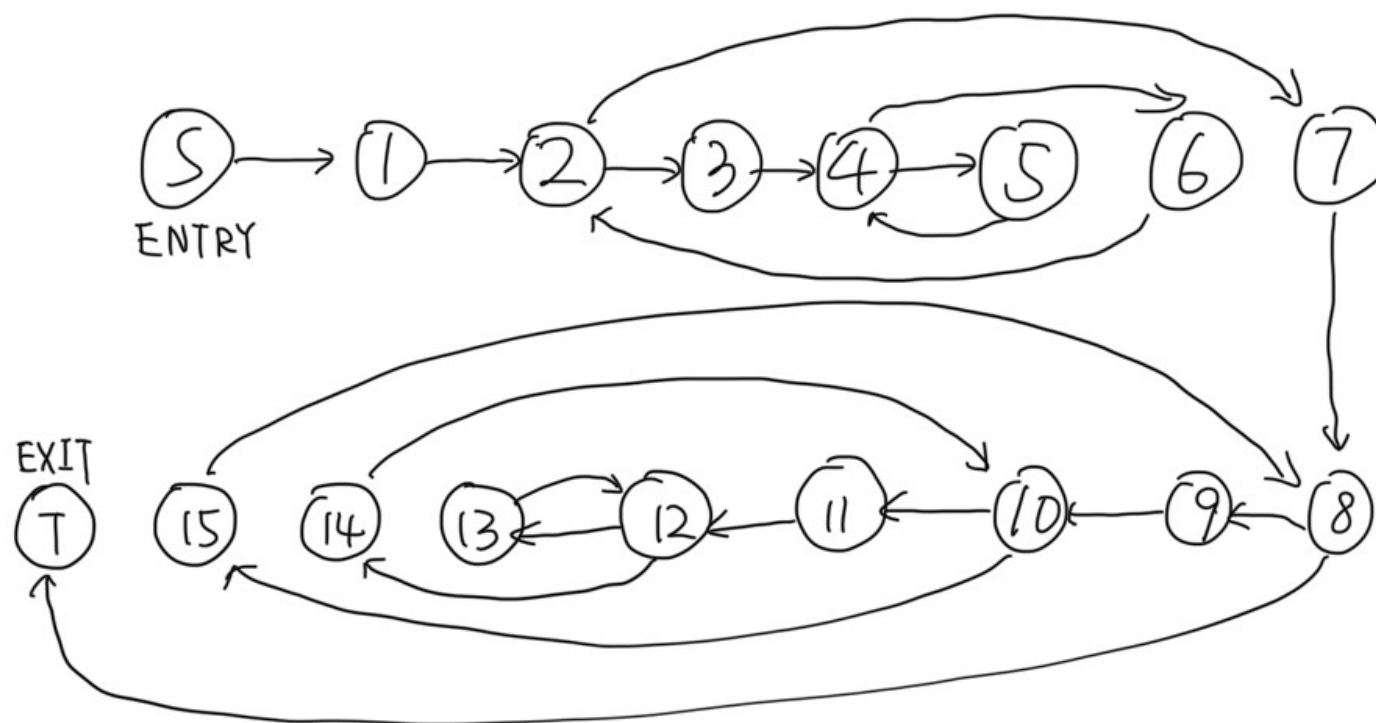
untitled
1 B13 19) t4 = n * i
2 20) t5 = t4 + j
3 21) t6 = t5 * 8
4 22) t7 = c[t6]
5 23) t8 = n * i
6 24) t9 = t8 + k
7 25) t10 = t9 * 8
8 26) t11 = a[t10]
9 27) t12 = n * k
10 28) t13 = t12 + j
11 29) t14 = t13 * 8
12 30) t15 = b[t14]
13 31) t16 = t11 * t15
14 32) t17 = t7 + t16
15 33) c[t6] = t17
16 34) k = k + 1
17 35) goto(18)
18 B14 36) j = j + 1
19 37) goto(16)
20 B15 38) i = i + 1
21 39) goto(14)
```

untitled* 21:22 CRLF UTF-8 C++ GitHub Git (0)



流图循环

- {B2, B3, B4, B6}
- {B4, B5}
- {B8, B9, B10, B15}
- {B10, B11, B12, B14}
- {B12, B13}





习题16.2

➤ 下图中是计算从2~n之间素数个数的代码。

```
for (i=2; i<=n; i++)  
    a[i] = TRUE;  
count = 0;  
s = sqrt(n);  
for (i=2; i<=s; i++)  
    if (a[i]) /* 已知i是一个素数 */ {  
        count++;  
        for (j=2*i; j<=n; j = j+i)  
            a[j] = FALSE; /* i的倍数都不是素数 */  
    }  
}
```

- (1) 假设数组的元素是需要4个字节存放的的整数，把程序翻译成三地址语句。
- (2) 为(1)中得到的代码构造流图。
- (3) 找出在(2)中得到的流图的循环。



习题16.2

三地址码分段

B1

1: i = 2

B2

2: if i <= n goto 4

B3

3: goto 9

B4

4: t1 = i*4

5: a[t1] = TRUE

6: t2 = i+1

7: i = t2

8: goto 2

B5

9: count = 0

10: param n

11: s = call sqrt,1

12: i = 2

B6

13: if i <= s goto 15

B7

14: goto 33

B8

15: t3 = i*4

16: t4 = a[t3]

17: if t4 goto 19

B9

18: goto 30

B10

19: t5 = count + 1

20: count = t5

21: t6 = i*2

22: j = t6

B11

23: if j <= n goto 25

B12

24: goto 30

B13

25: t7 = j*4

26: a[t7] = FALSE

27: t8 = j+i

28: j = t8

29: goto 23

B14

30: t9 = i+1

31: i = t9

32: goto 13

B15

33:



习题16.2 循环的定义

像while语句、do-while语句和for语句这样的程序设计语言构造自然地把循环引入到程序中。因为事实上每个程序会花很多时间执行循环，所以对于一个编译器来说，为循环生成优良的代码就变得非常重要。很多代码转换依赖于对流图中“循环”的识别。如果下列条件成立,我们就说流图中的一个结点集合L是一个循环。

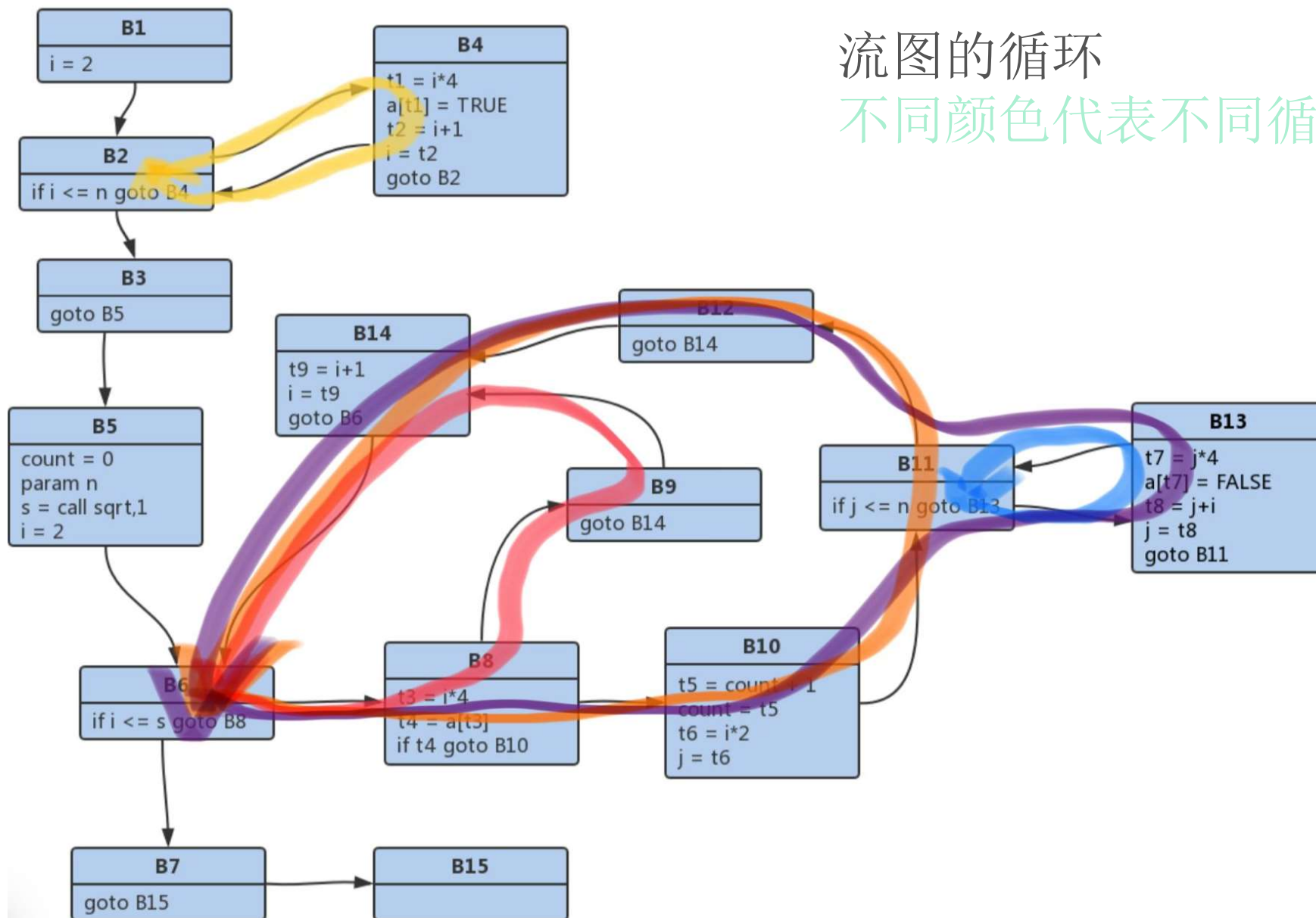
1)在L中有一个被称为循环入口(loopentry)的结点，它是唯一的其前驱可能在L之外的结点。也就是说,从整个流图的人口结点开始到L中的任何结点的路径都必然经过循环入口结点，并且这个循环入口结点不是整个流图的人口结点本身。

2) L中的每个结点都有一个到达L的人口结点的非空路径,并且该路径全部在L中。



习题16.2

流图循环





习题16.3

➤ 为下面的表达式构造DAG

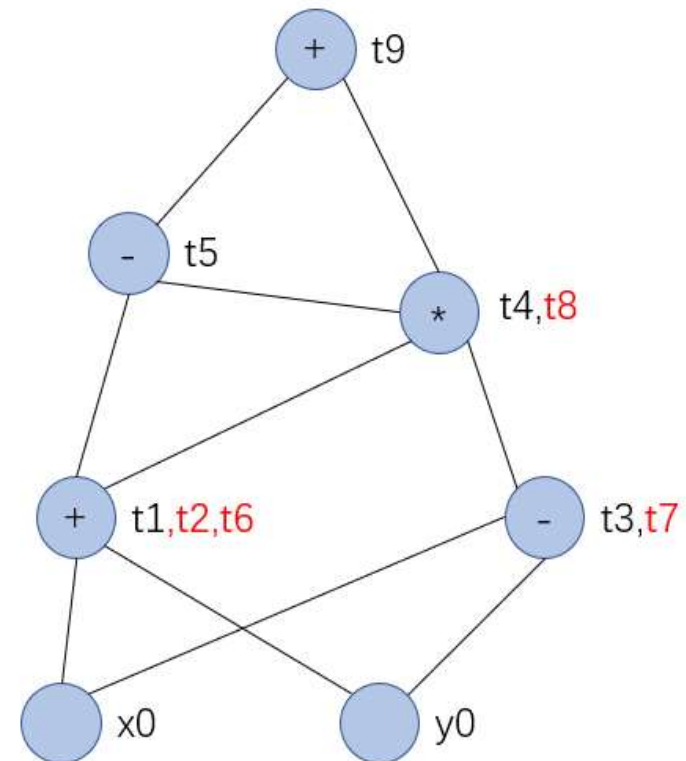
$$((x + y) - ((x + y) * (x - y))) + ((x + y) * (x - y))$$

2. DAG图

- ① $t1 = x + y$
- ② $t2 = x + y$
- ③ $t3 = x - y$
- ④ $t4 = t2 * t3$
- ⑤ $t5 = t1 - t4$
- ⑥ $t6 = x + y$
- ⑦ $t7 = x - y$
- ⑧ $t8 = t6 * t7$
- ⑨ $t9 = t5 + t8$

$t1 = x + y$
 $t3 = x - y$
 $t4 = t1 * t3$
 $t5 = t1 - t4$
 $t9 = t5 + t4$

$x + y, x - y$ 为公共子表达式





习题16.4

➤ 为下列表达式构造**DAG**。假定+是左结合的。

➤ (1) $a + b + (a + b)$

➤ (2) $a + b + a + b$

➤ (3) $a + a + ((a + a + a + (a + a + a + a)))$



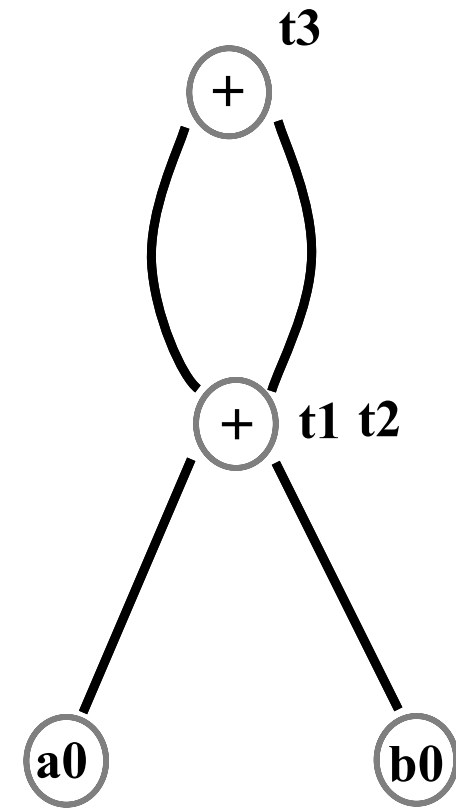
习题16.4

➤(1) $a + b + (a + b)$

$$t1 = a + b$$

$$t2 = a + b$$

$$t3 = t1 + t2$$





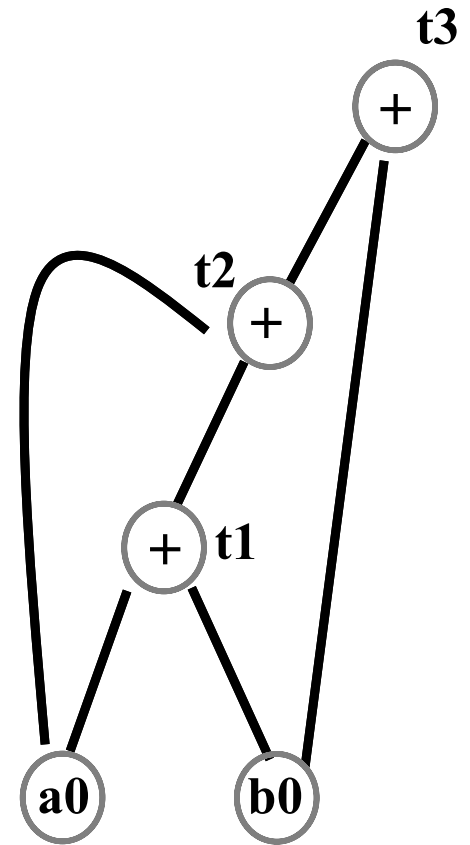
习题16.4

➤(2) $a + b + a + b$

$$t1 = a + b$$

$$t2 = t1 + a$$

$$t3 = t2 + b$$





习题16.4

➤ (3) $a + a + (a + a + a + (a + a + a + a))$

$$t1 = a + a$$

$$t2 = a + a$$

$$t3 = t2 + a$$

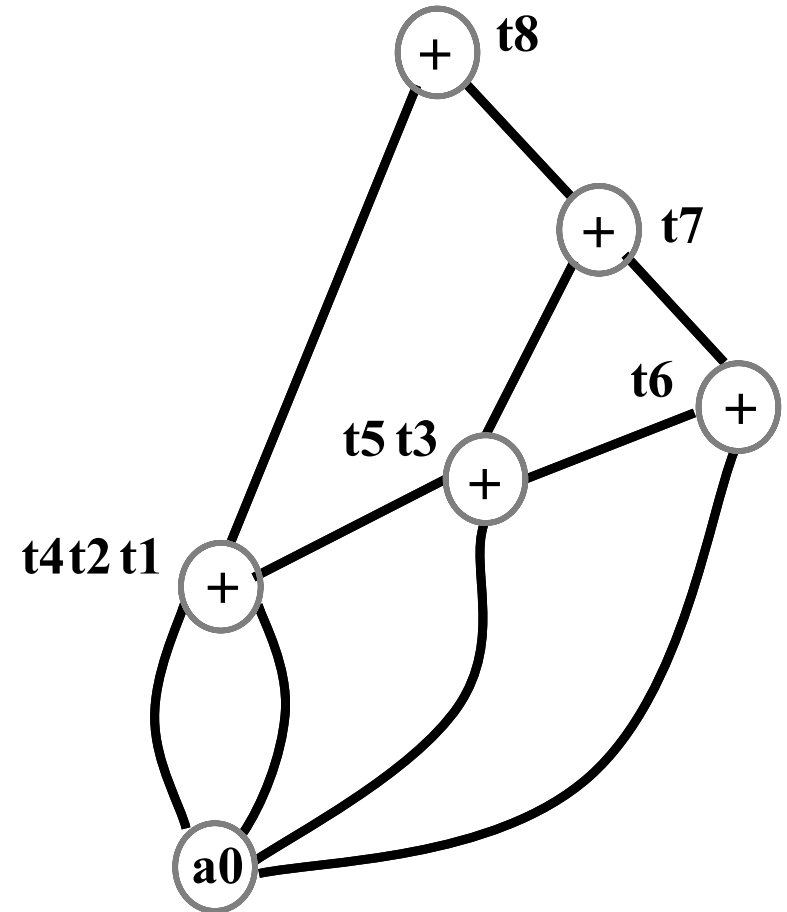
$$t4 = a + a$$

$$t5 = t4 + a$$

$$t6 = t5 + a$$

$$t7 = t3 + t6$$

$$t8 = t1 + t7$$





习题16.5

➤ 对于下面的基本块

$$d = b * c$$

$$e = a + b$$

$$b = b * c$$

$$a = e - d$$

➤ (1) 为该基本块构造DAG

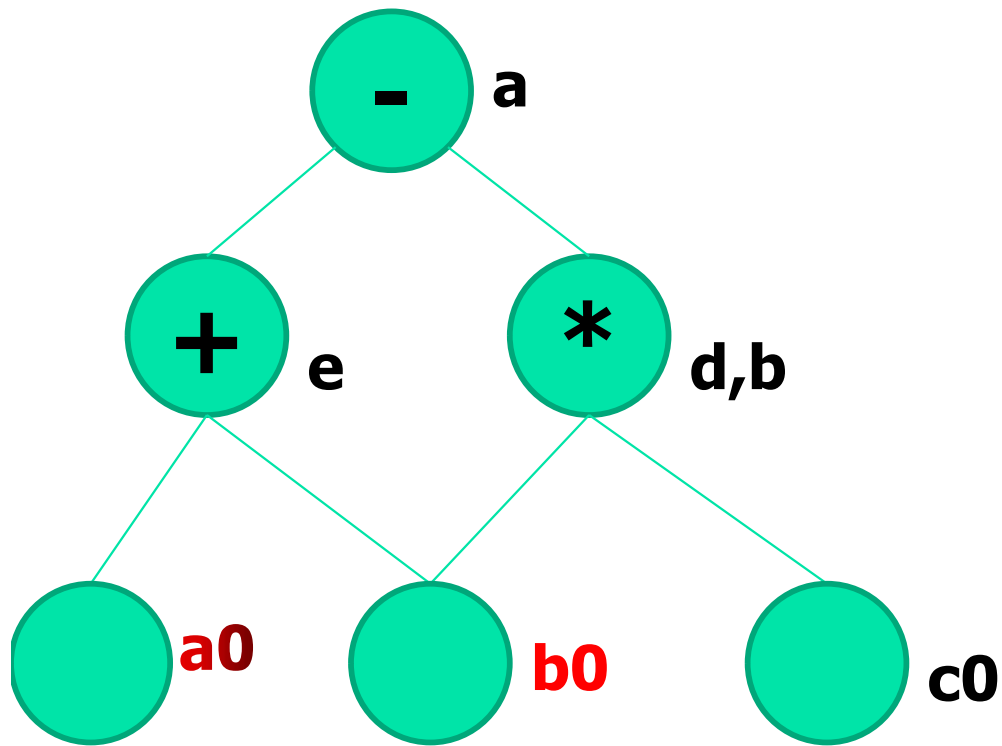
➤ (2) 分别按照下列两种假设简化上述三地址代码

➤ 只有a在基本块的出口活跃

➤ a、b、c在基本块的出口活跃



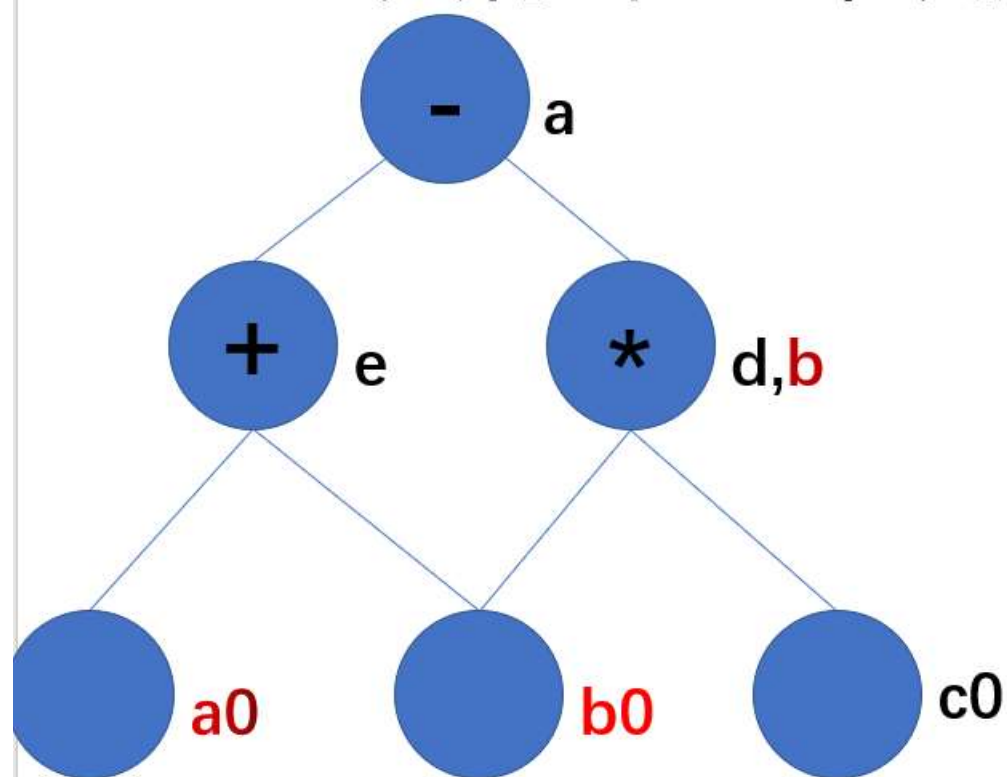
(1) 为该基本块构造DAG



$d = b * c$
 $e = a + b$
 $b = b * c$
 $a = e - d$

(2) 分别按照下列两种假设简化上述三地址代码

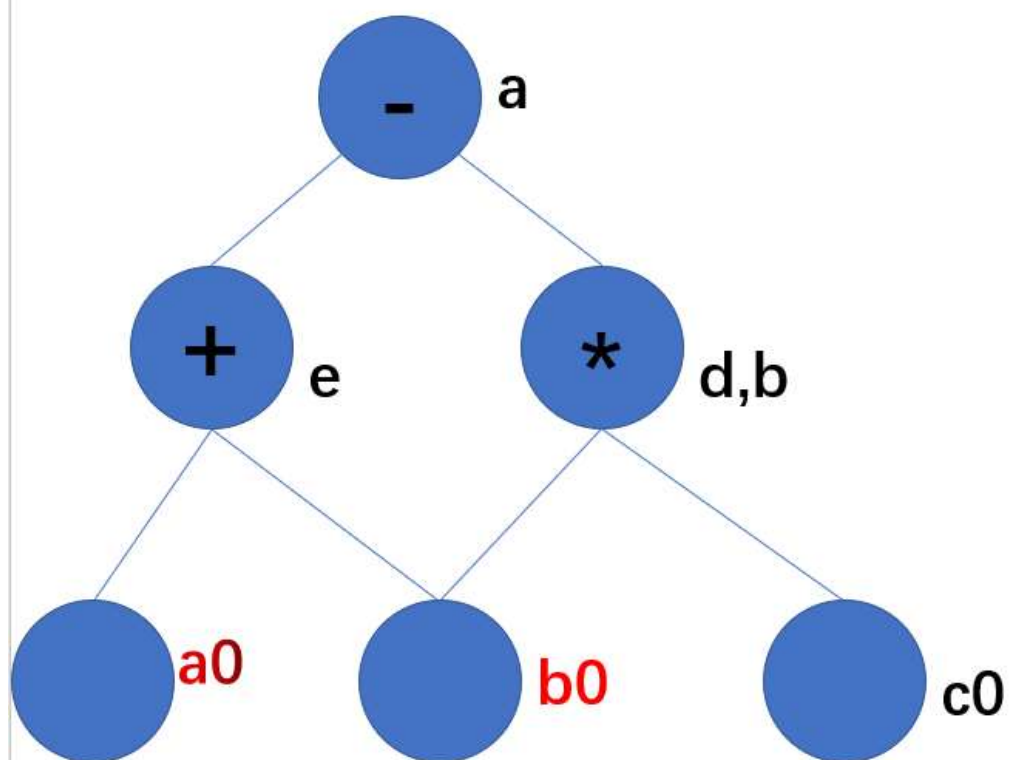
1: 只有a在基本块的出口活跃



$d = b * c$
 $e = a + b$
 $a = e - d$



2: a、b、c在基本块的出口活跃


$$e = a + b$$

b=b*c

$$a = e - b$$



习题16.6

➤ 为下面的基本块构造**DAG**。请不要忘记包含比较指令 $i \leq 10$

$t5 = i - 1$

$t6 = 88 * t5$

$a[t6] = 1.0$

$i = i + 1$

if $i \leq 10$ goto B6



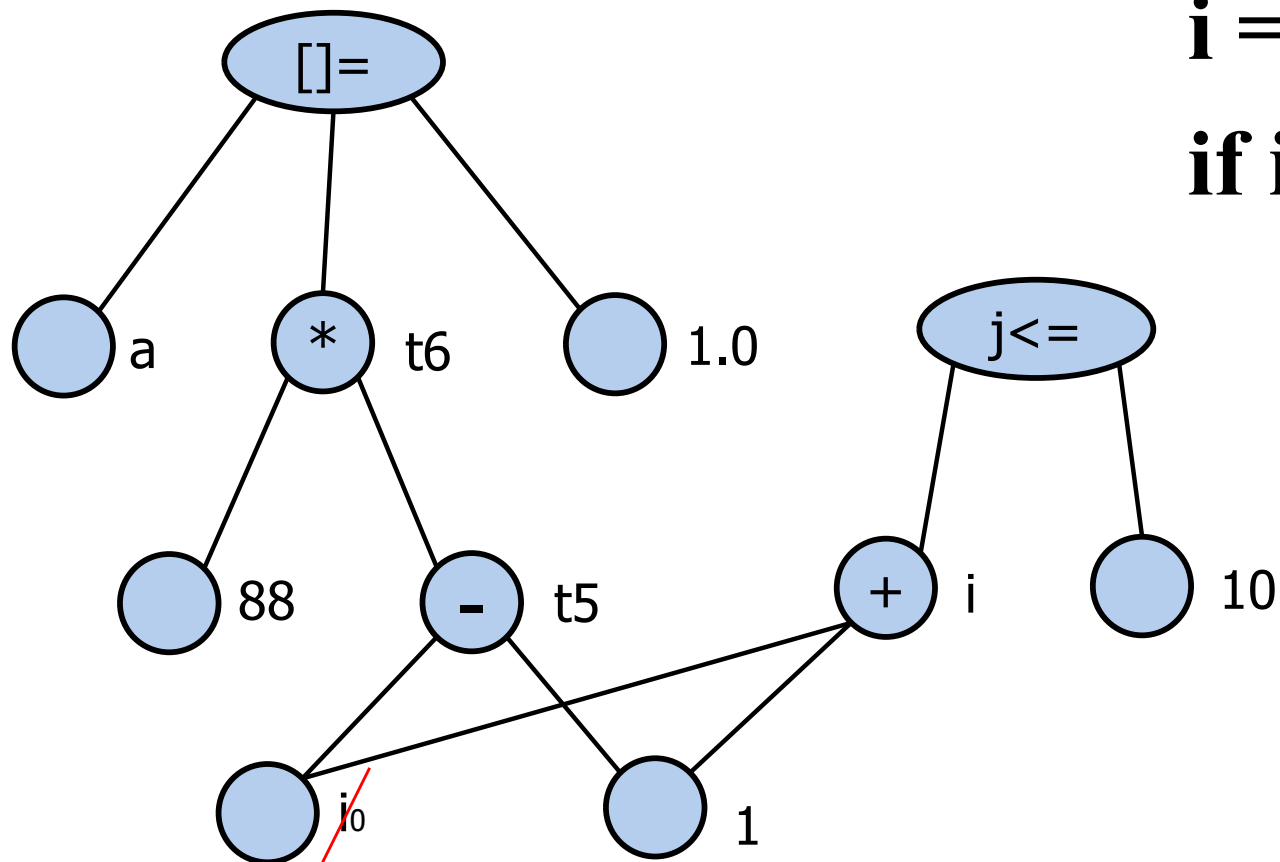
$t5 = i - 1$

$t6 = 88 * t5$

$a[t6] = 1.0$

$i = i + 1$

if $i \leq 10$ goto B6





习题16.7

➤ 为下面的基本块构造DAG。

$t1 = 10 * i$

$t2 = t1 + j$

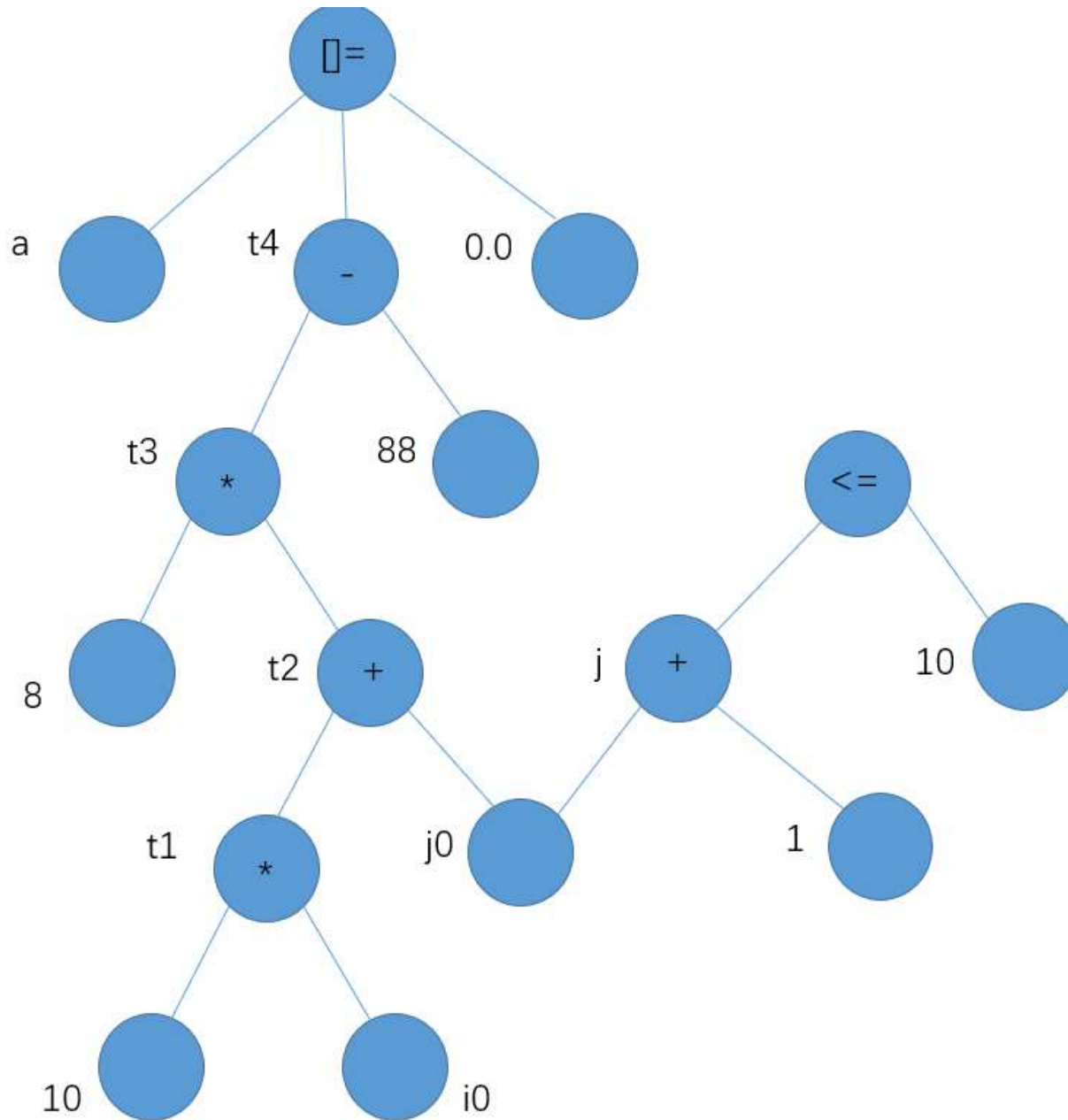
$t3 = 8 * t2$

$t4 = t3 - 88$

$a[t4] = 0.0$

$j = j + 1$

if $j \leq 10$ goto B3



$t1 = 10 * i$

$t2 = t1 + j$

$t3 = 8 * t2$

$t4 = t3 - 88$

$a[t4] = 0.0$

$j = j + 1$

if $j \leq 10$ goto B3



习题16.8

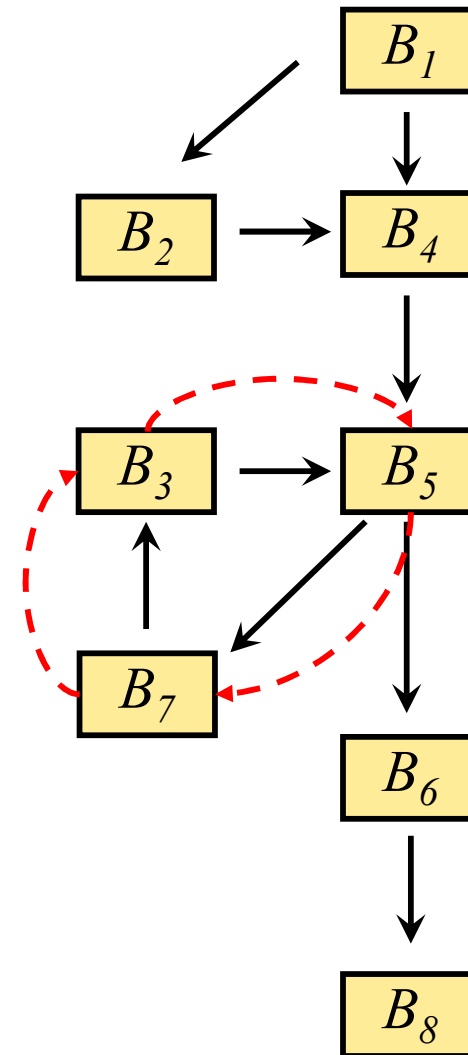
➤ 考虑下面的三地址语句序列：

```
    b = 1
    b = 2
    if w ≤ x goto L2
    e = b
    goto L2
L1:  goto L3
L2:  c = 3
     b = 4
     c = 6
L3:  if y ≤ z goto L4
     goto L5
L4:  g = g + 1
     h = 8
     goto L1
L5:  h = 9
```

- (1) 在该代码中用横线将代码分成基本块，并给每个基本块赋一个序号
- (2) 画出该代码的流图，每个基本块就用(1)中的序号表示
- (3) 若有循环，则列出构成每个循环的结点

16.8 (2)

	b = 1	<i>B</i>₁
	b = 2	
	if w ≤ x goto L2	
	e = b	<i>B</i>₂
	goto L2	
L1:	goto L3	<i>B</i>₃
L2:	c = 3	
	b = 4	<i>B</i>₄
	c = 6	
L3:	if y ≤ z goto L4	<i>B</i>₅
	goto L5	<i>B</i>₆
L4:	g = g + 1	
	h = 8	<i>B</i>₇
	goto L1	
L5:	h = 9	<i>B</i>₈



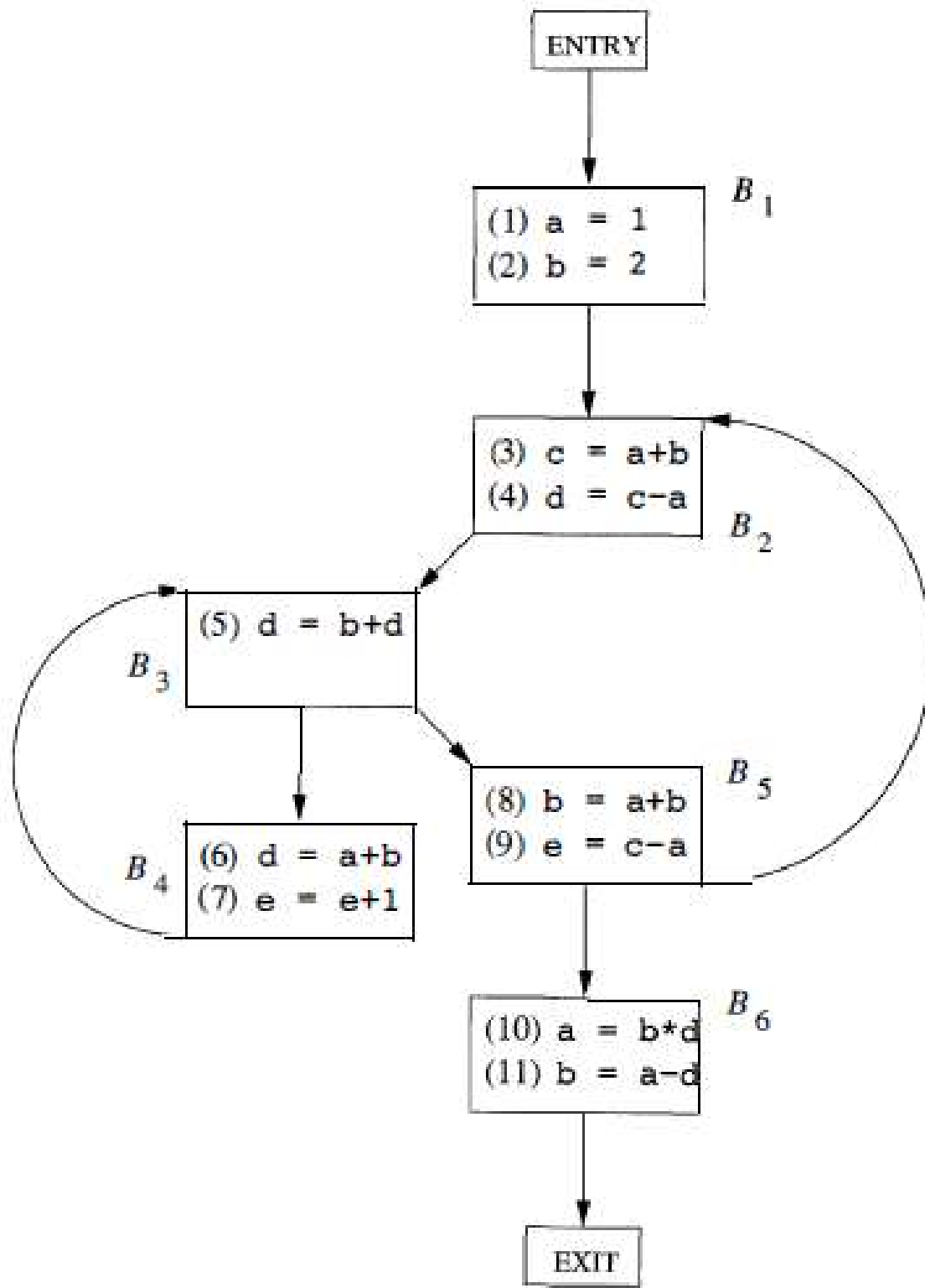
循环结点为：
3、5、7



习题17.1

➤ 对下图中的流图，
计算下列值：

- (1) 每个基本块的
gen和kill集合。
- (2) 每个基本块的IN
和OUT集合。



(1)

genB1={{(1),(2)}}
killB1={{(8),(10),(11)}}

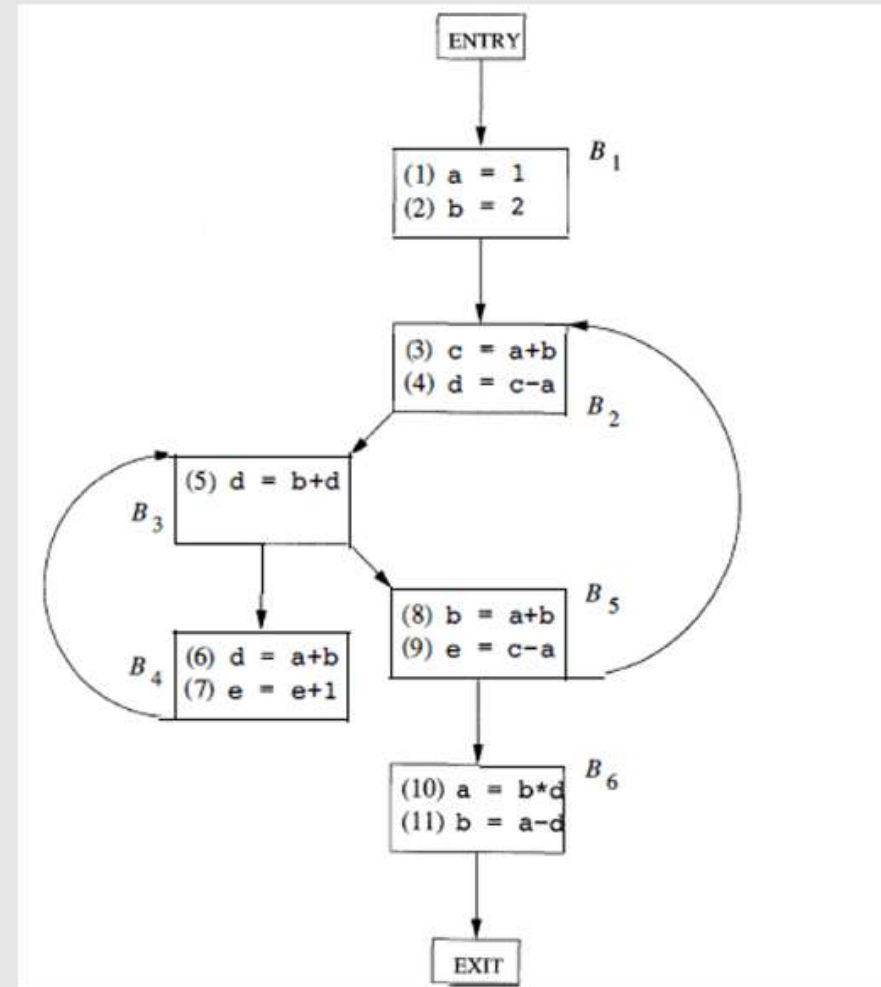
genB2={{(3),(4)}}
killB2={{(5),(6)}}

genB3={{(5)}}
killB3={{(4),(6)}}

genB4={{(6),(7)}}
killB4={{(4),(5),(9)}}

genB5={{(8),(9)}}
killB5={{(2),(7),(11)}}

genB6={{(10),(11)}}
killB6={{(1),(2),(8)}}





(2)

OUT[ENTRY]=null
OUT[B1]=null
OUT[B2]=null
OUT[B3]=null
OUT[B4]=null
OUT[B5]=null
OUT[B6]=null

第一轮

IN[B1] = OUT[ENTRY]=null
OUT[B1] = genB1 U (IN[B1]-killB1)=[(1),(2)]
IN[B2]= OUT[B1]U OUT[B5]=[(1),(2)]
OUT[B2]= genB2 U (IN[B2]-killB2)=[(1),(2),(3),(4)]
IN[B3]= OUT[B2]U OUT[B4]=[(1),(2) ,(3),(4)]
OUT[B3]= genB3 U (IN[B3]-killB3)=[(1),(2),(3),(5)]
IN[B4]= OUT[B3] =[(1),(2) ,(3),(5)]
OUT[B4]= genB4 U (IN[B4]-killB4)=[(1),(2),(3),(6),(7)]
IN[B5]= OUT[B3] =[(1),(2),(3),(5)]
OUT[B5]= genB5 U (IN[B5]-killB5)=[(1),(3),(5),(8),(9)]
IN[B6]= OUT[B5] =[(1),(3),(5),(8),(9)]
OUT[B6]= genB6 U (IN[B6]-killB6)=[(3),(5),(9),(10),(11)]
IN[EXIT]=OUT[B6]= [(3),(5),(9),(10),(11)]
OUT[EXIT]=IN[EXIT]= [(3),(5),(9),(10),(11)]

第二轮

```
IN[B1] = OUT[ENTRY]=null
OUT[B1] =genB1 U (IN[B1]-killB1)=[(1),(2)]
IN[B2]= OUT[B1]U OUT[B5]=[ (1),(2),(3),(5),(8),(9)]
OUT[B2]= genB2 U (IN[B2]-
killB2)=[ (1),(2),(3),(4),(8),(9)]
IN[B3]= OUT[B2]U
OUT[B4]=[ (1),(2),(3),(4),(6),(7),(8),(9)]
OUT[B3]=genB3 U(IN[B3]-
killB3)=[(1),(2),(3),(5),(7),(8),(9)]
IN[B4]= OUT[B3] =[ (1),(2),(3),(5),(7),(8),(9)]
OUT[B4]= genB4 U (IN[B4]-
killB4)=[ (1),(2),(3),(6),(7),(8)]
IN[B5]= OUT[B3] =[ (1),(2),(3),(5),(7),(8),(9)]
OUT[B5]= genB5 U (IN[B5]-killB5)=[ (1),(3),(5),(8),(9)]
IN[B6]= OUT[B5] =[ (1),(3),(5),(8),(9)]
OUT[B6]= genB6 U (IN[B6]-
killB6)=[(3),(5),(9),(10),(11)]
IN[EXIT]=OUT[B6]= [(3),(5),(9),(10),(11)]
OUT[EXIT]=IN[EXIT]= [(3),(5),(9),(10),(11)]
```

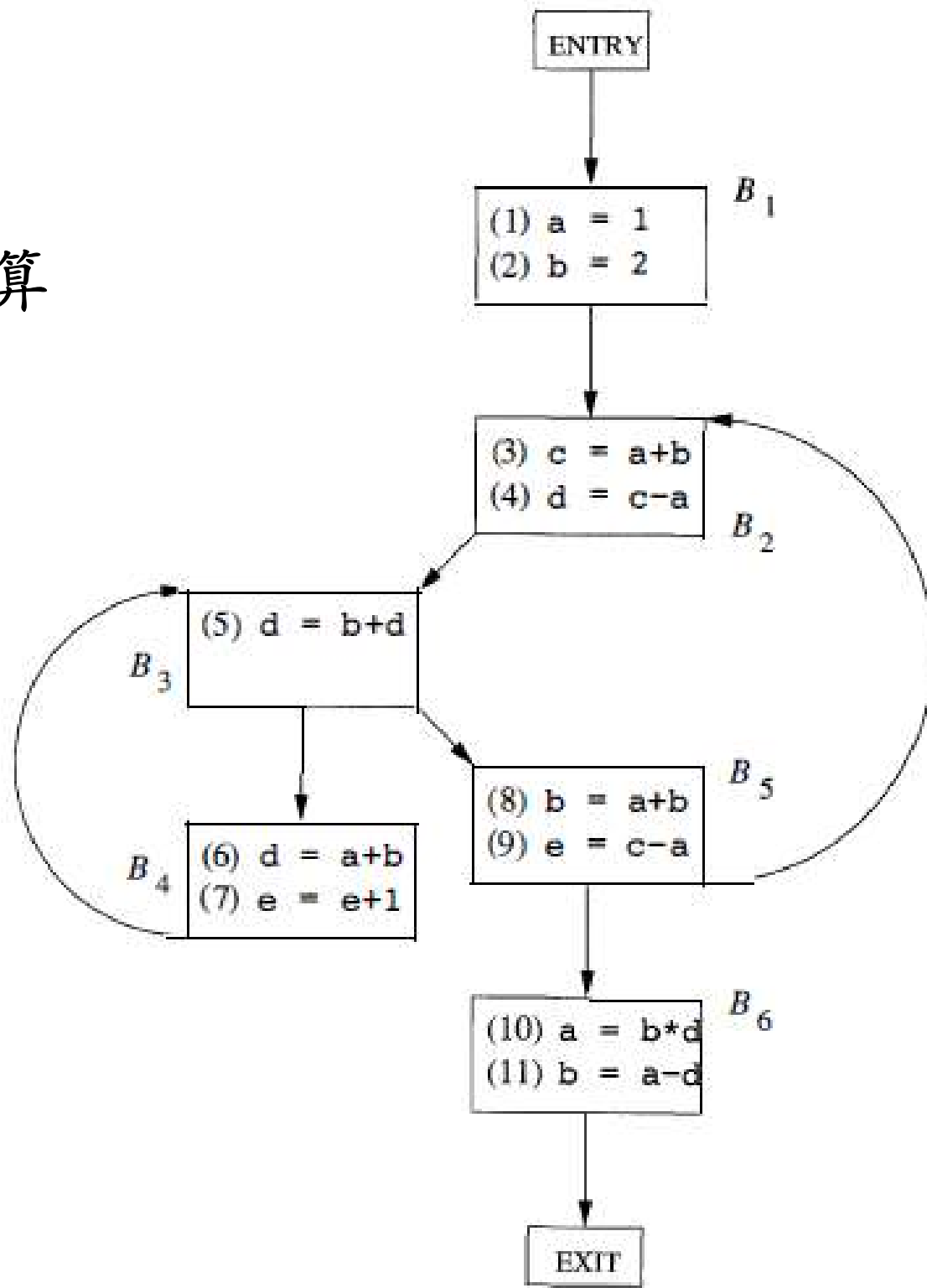
第三轮

```
IN[B1] = OUT[ENTRY]=null
OUT[B1] =genB1 U (IN[B1]-killB1)=[(1),(2)]
IN[B2]= OUT[B1]U OUT[B5]=[ (1),(2),(3),(5),(8),(9)]
OUT[B2]= genB2 U (IN[B2]-
killB2)=[ (1),(2),(3),(4),(8),(9)]
IN[B3]= OUT[B2]U
OUT[B4]=[ (1),(2),(3),(4),(6),(7),(8),(9)]
OUT[B3]=genB3 U(IN[B3]-
killB3)=[(1),(2),(3),(5),(7),(8),(9)]
IN[B4]= OUT[B3] =[ (1),(2),(3),(5),(7),(8),(9)]
OUT[B4]= genB4 U (IN[B4]-
killB4)=[ (1),(2),(3),(6),(7),(8)]
IN[B5]= OUT[B3] =[ (1),(2),(3),(5),(7),(8),(9)]
OUT[B5]= genB5 U (IN[B5]-
killB5)=[ (1),(3),(5),(8),(9)]
IN[B6]= OUT[B5] =[ (1),(3),(5),(8),(9)]
OUT[B6]= genB6 U (IN[B6]-
killB6)=[(3),(5),(9),(10),(11)]
IN[EXIT]=OUT[B6]= [(3),(5),(9),(10),(11)]
OUT[EXIT]=IN[EXIT]= [(3),(5),(9),(10),(11)]
与上一轮对比无改变 输出结果
```

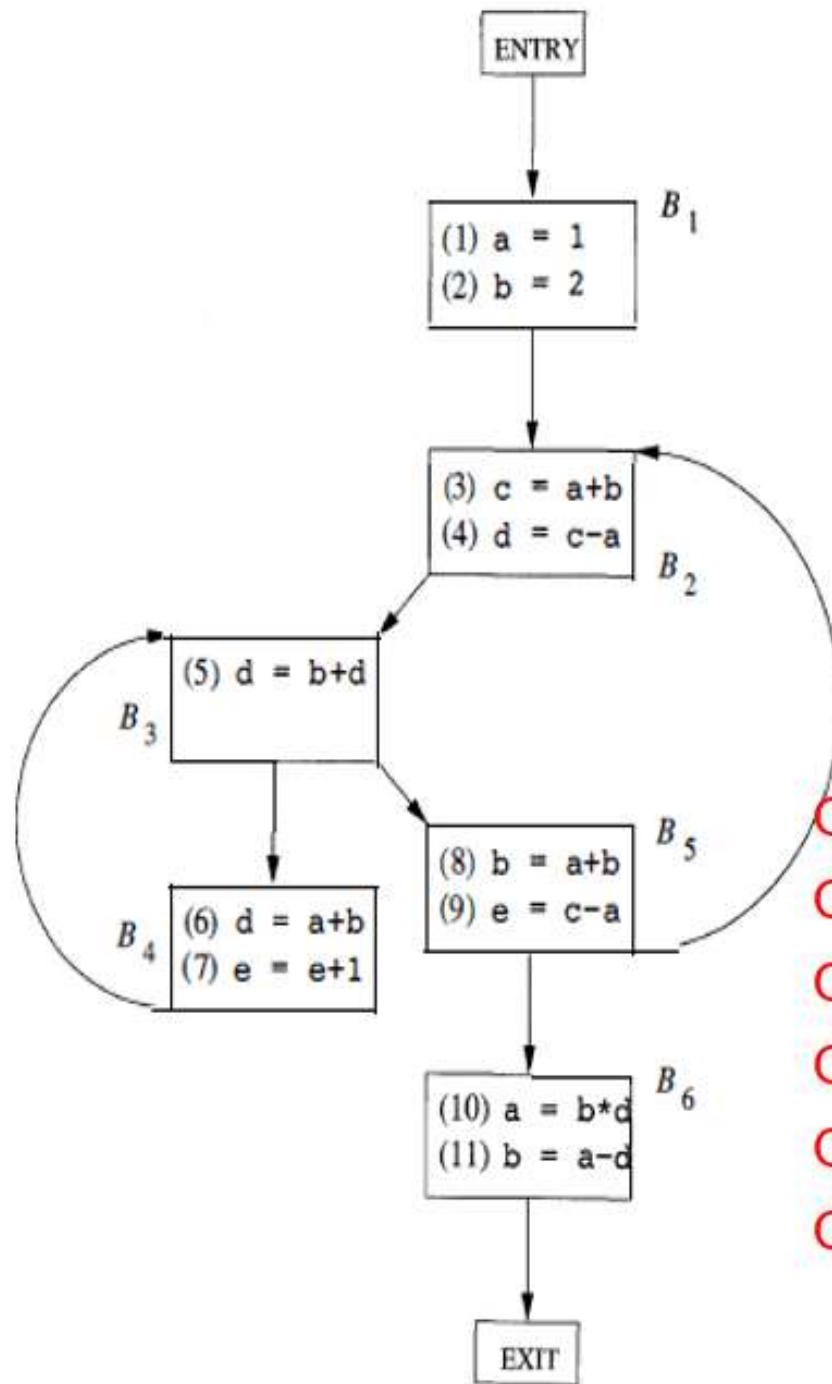


习题18.1

➤ 对下图中的流图，计算可用表达式问题中的 e_gen 、 e_kill 、IN 和 OUT 集合。



第一次迭代



$IN[B1] = \emptyset$

$IN[B2] = \emptyset$

$IN[B3] = \{a+b, c-a\}$

$IN[B4] = \{a+b, c-a\}$

$IN[B5] = \{a+b, c-a\}$

$IN[B6] = \{c-a\}$

$e_genB1 = \emptyset$

$e_genB2 = \{a+b, c-a\}$

$e_genB3 = \emptyset$

$e_genB4 = \{a+b\}$

$e_genB5 = \{c-a\}$

$e_genB6 = \{a-d\}$

$OUT[B1] = \emptyset$

$OUT[B2] = \{a+b, c-a\}$

$OUT[B3] = \{a+b, c-a\}$

$OUT[B4] = \{a+b, c-a\}$

$OUT[B5] = \{c-a\}$

$OUT[B6] = \{c-a, a-d\}$

$e_killB1 = \{a+b, c-a, b+d, b*d, a-d\}$

$e_killB2 = \{b+d, b*d, a-d\}$

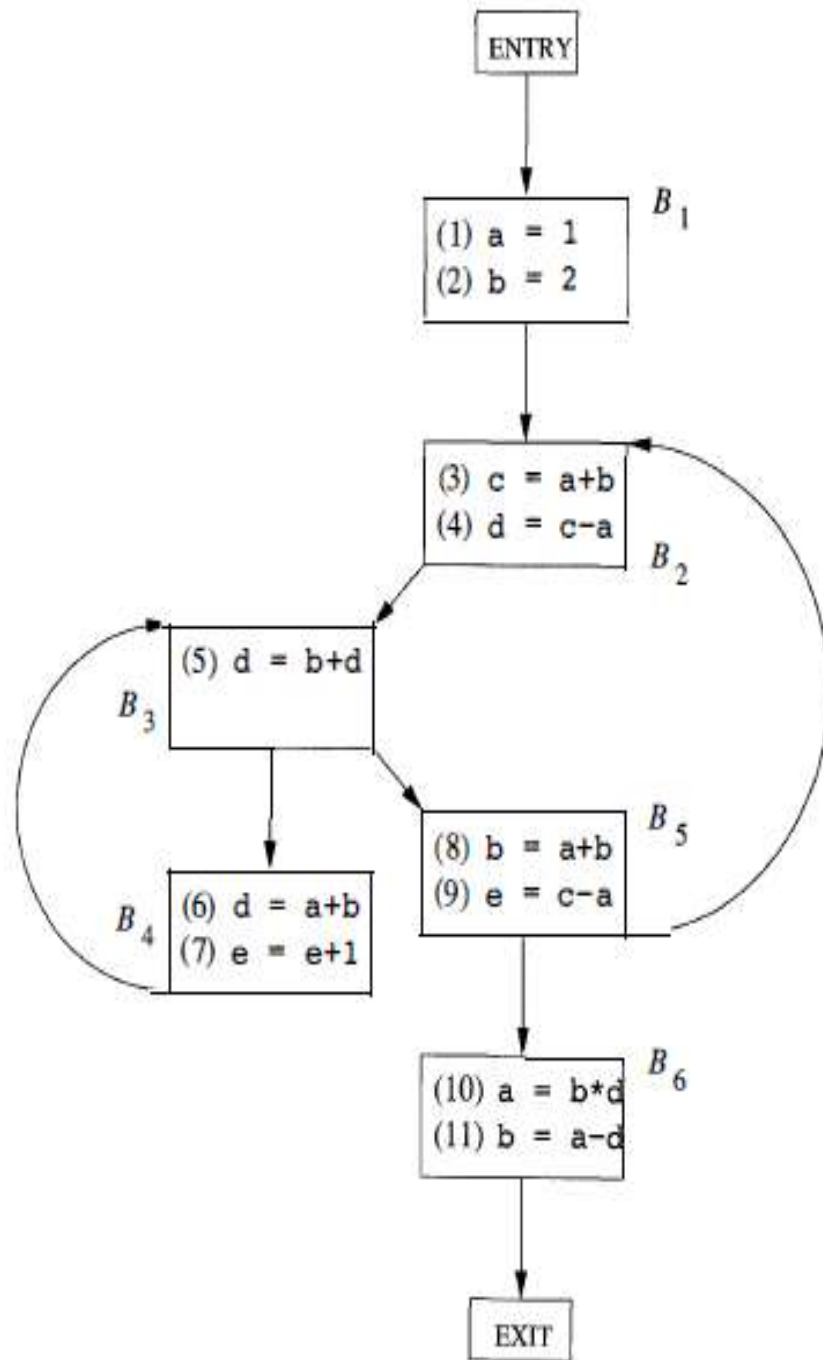
$e_killB3 = \{b+d, b*d, a-d\}$

$e_killB4 = \{e+1, b+d, b*d, a-d\}$

$e_killB5 = \{a+b, b+d, b*d, e+1\}$

$e_killB6 = \{a+b, c-a, b*d, b+d\}$

第二次迭代



$IN[B_1] = \emptyset$

$IN[B_2] = \emptyset$

$IN[B_3] = \{a+b, c-a\}$

$IN[B_4] = \{a+b, c-a\}$

$IN[B_5] = \{a+b, c-a\}$

$IN[B_6] = \{c-a\}$

$OUT[B_1] = \emptyset$

$OUT[B_2] = \{a+b, c-a\}$

$OUT[B_3] = \{a+b, c-a\}$

$OUT[B_4] = \{a+b, c-a\}$

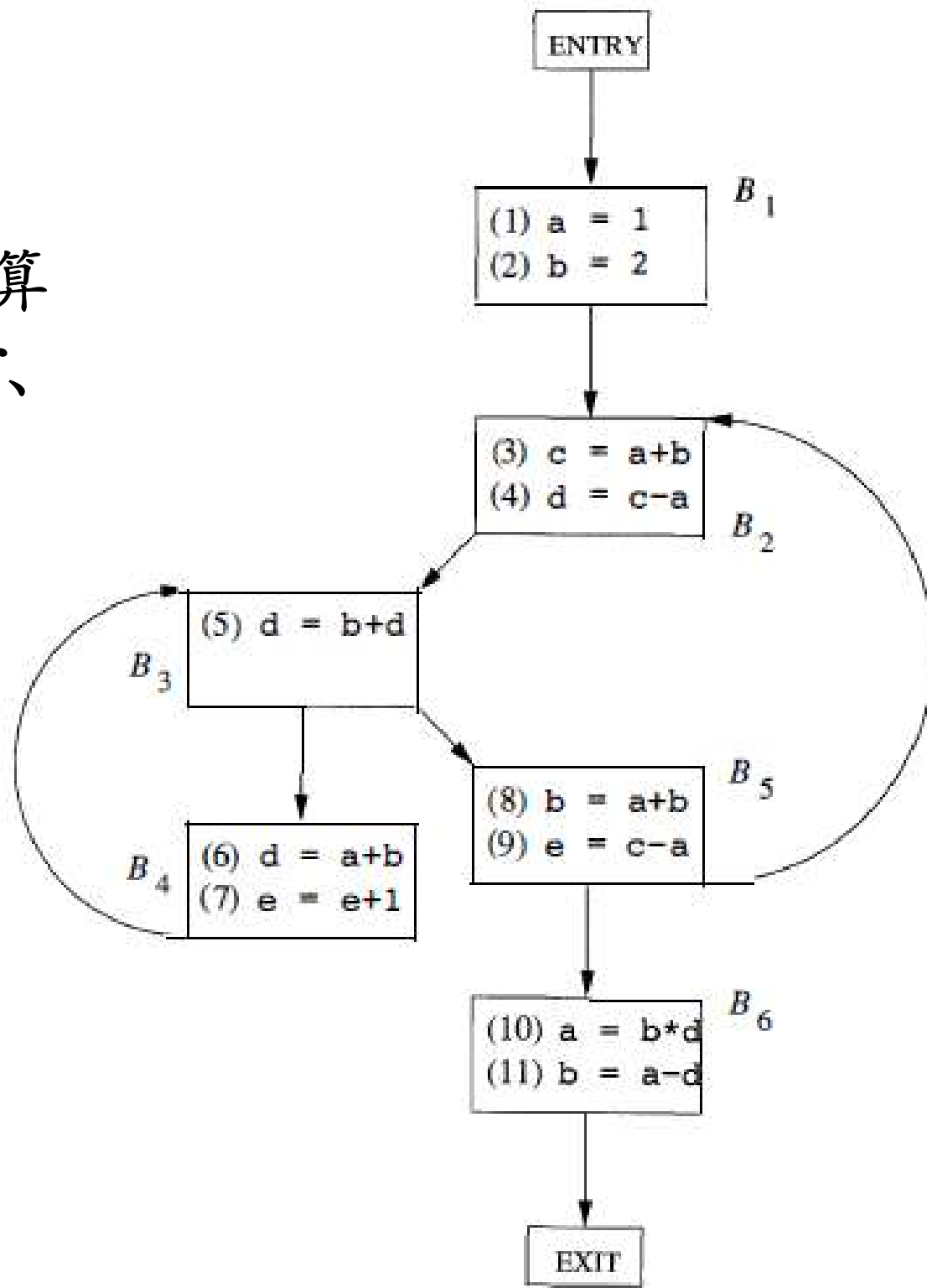
$OUT[B_5] = \{c-a\}$

$OUT[B_6] = \{c-a, a-d\}$



习题18.2

➤ 对下图中的流图，计算活跃变量分析中的def、use、IN和OUT集合。



第一轮迭代

	OUT	IN
B6		b , d
B5	b , d	a , b , c, d
B4		a , b , e
B3	a , b ,c ,d ,e	a , b ,c ,d ,e
B2	a , b ,c ,d ,e	a , b , e
B1	a , b , e	e

➤ $use_{B1} = \Phi$
 ➤ $def_{B1} = \{a, b\}$

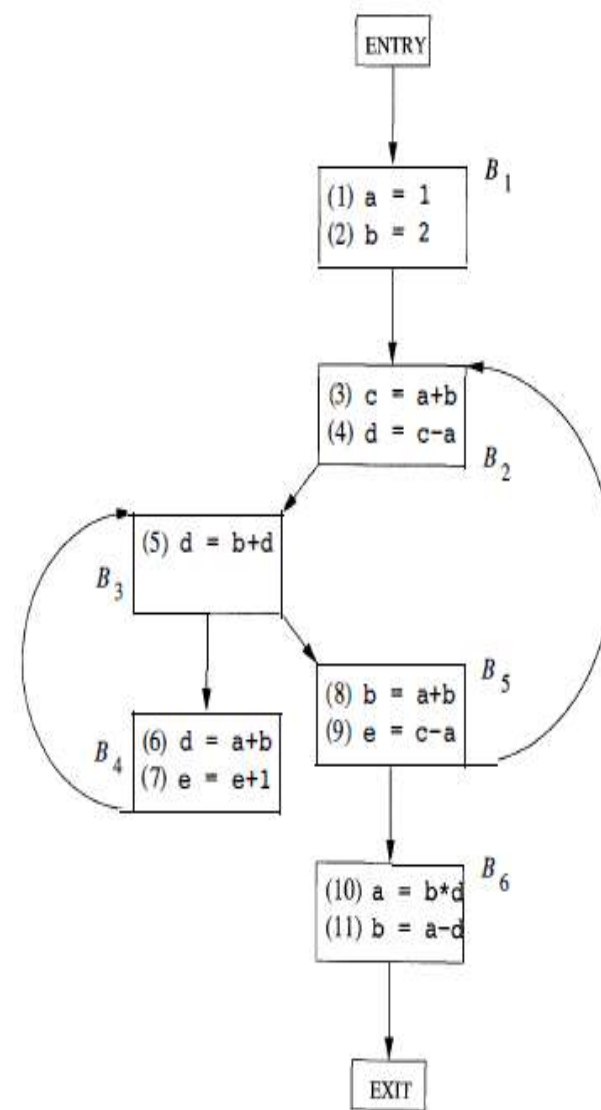
➤ $use_{B2} = \{a, b\}$
 ➤ $def_{B2} = \{c, d\}$

➤ $use_{B3} = \{b, d\}$
 ➤ $def_{B3} = \Phi$

➤ $use_{B4} = \{a, b, e\}$
 ➤ $def_{B4} = \{d\}$

➤ $use_{B5} = \{a, b, c\}$
 ➤ $def_{B5} = \{e\}$

➤ $use_{B6} = \{b, d\}$
 ➤ $def_{B6} = \{a\}$

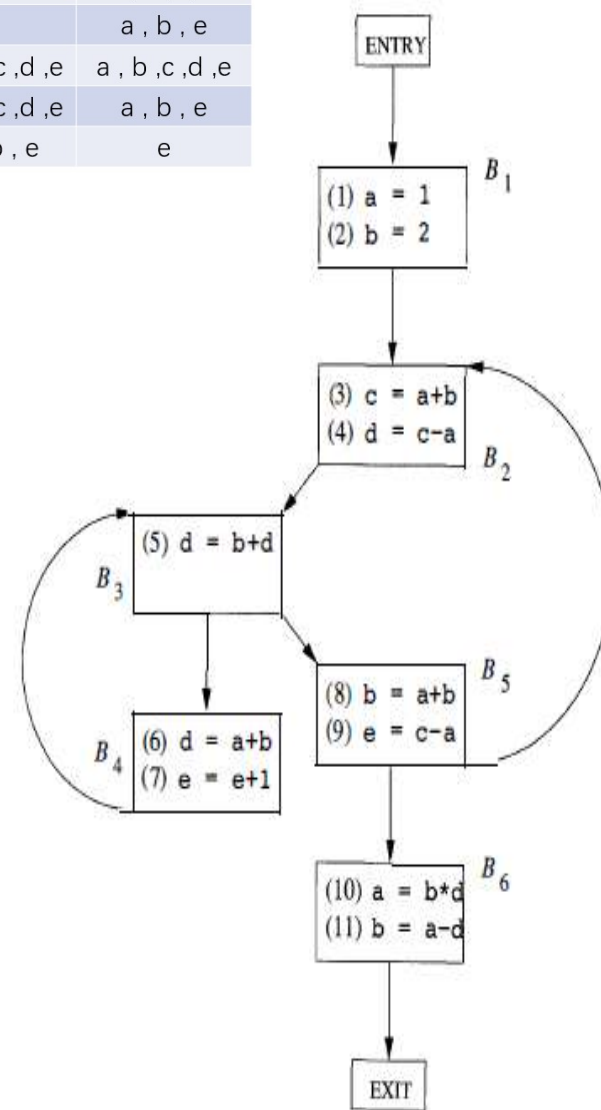


第二轮迭代

	OUT	IN
B6		b, d
B5	a, b, d, e	a, b, c, d
B4	a, b, c, d, e	a, b, c, e
B3	a, b, c, d, e	a, b, c, d, e
B2	a, b, c, d, e	a, b, e
B1	a, b, e	e

	OUT	IN
B6		b, d
B5	b, d	a, b, c, d
B4		a, b, e
B3	a, b, c, d, e	a, b, c, d, e
B2	a, b, c, d, e	a, b, e
B1	a, b, e	e

- $use_{B1} = \Phi$
- $def_{B1} = \{a, b\}$
- $use_{B2} = \{a, b\}$
- $def_{B2} = \{c, d\}$
- $use_{B3} = \{b, d\}$
- $def_{B3} = \Phi$
- $use_{B4} = \{a, b, e\}$
- $def_{B4} = \{d\}$
- $use_{B5} = \{a, b, c\}$
- $def_{B5} = \{e\}$
- $use_{B6} = \{b, d\}$
- $def_{B6} = \{a\}$



第三轮迭代

	OUT	IN
B6		b, d
B5	a, b, d, e	a, b, c, d
B4	a, b, c, d, e	a, b, c, e
B3	a , b ,c ,d ,e	a, b, c, d, e
B2	a , b ,c ,d ,e	a , b , e
B1	a , b , e	e

	OUT	IN
B6		b, d
B5	a, b, d, e	a, b, c, d
B4	a, b, c, d, e	a, b, c, e
B3	a , b ,c ,d ,e	a, b, c, d, e
B2	a , b ,c ,d ,e	a , b , e
B1	a , b , e	e

➤ $use_{B1} = \Phi$
 ➤ $def_{B1} = \{a, b\}$

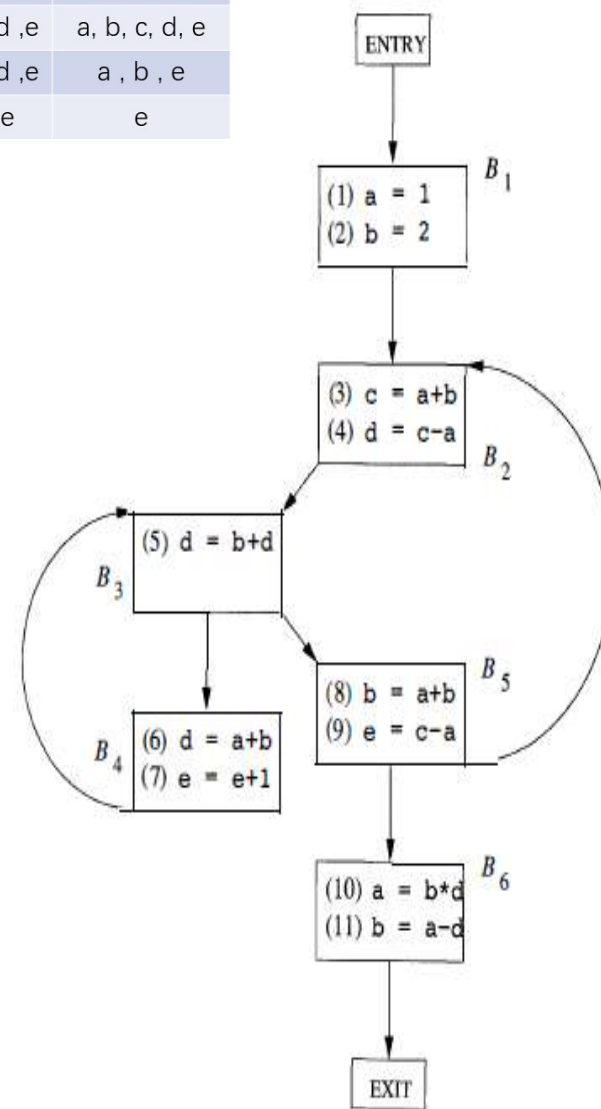
➤ $use_{B2} = \{a, b\}$
 ➤ $def_{B2} = \{c, d\}$

➤ $use_{B3} = \{b, d\}$
 ➤ $def_{B3} = \Phi$

➤ $use_{B4} = \{a, b, e\}$
 ➤ $def_{B4} = \{d\}$

➤ $use_{B5} = \{a, b, c\}$
 ➤ $def_{B5} = \{e\}$

➤ $use_{B6} = \{b, d\}$
 ➤ $def_{B6} = \{a\}$





习题18.3

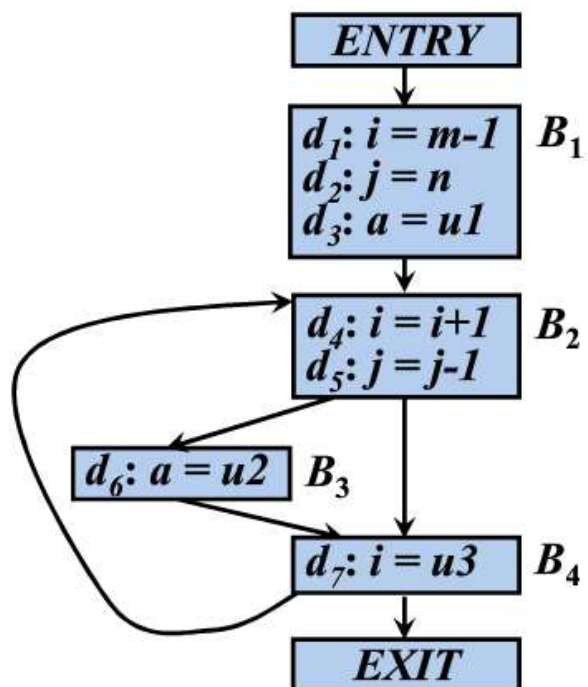
- 程序中的变量，若在第一次引用前没有置初值的话，则称它为未初始化变量。请运用数据流分析技术，给出计算程序的未初始化变量集合的方法。



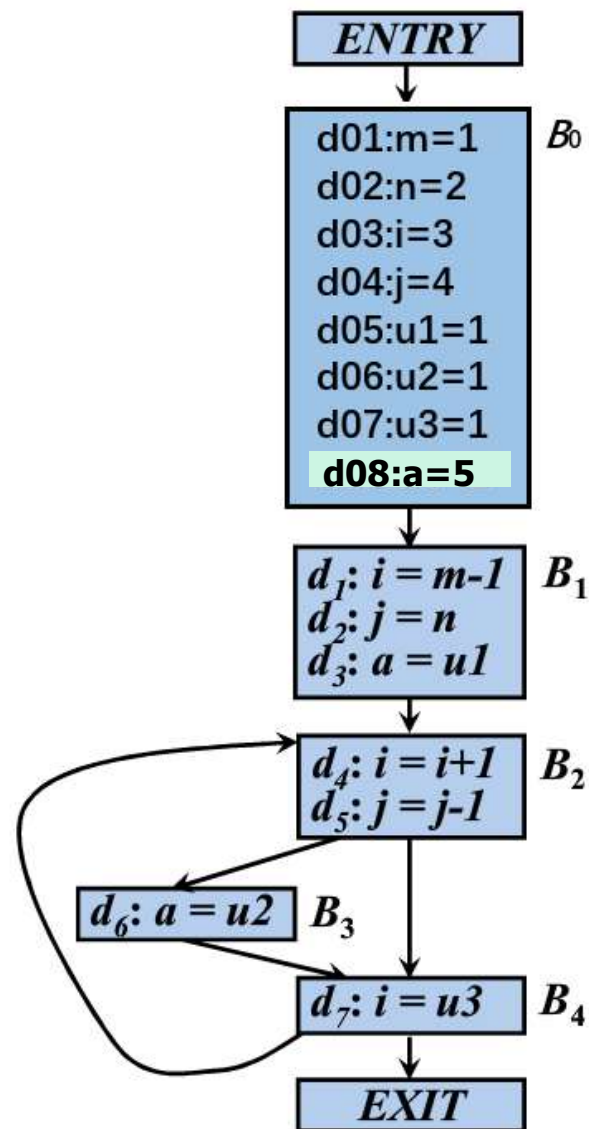
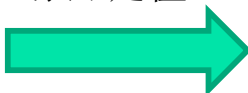
运用技术：到达定值分析

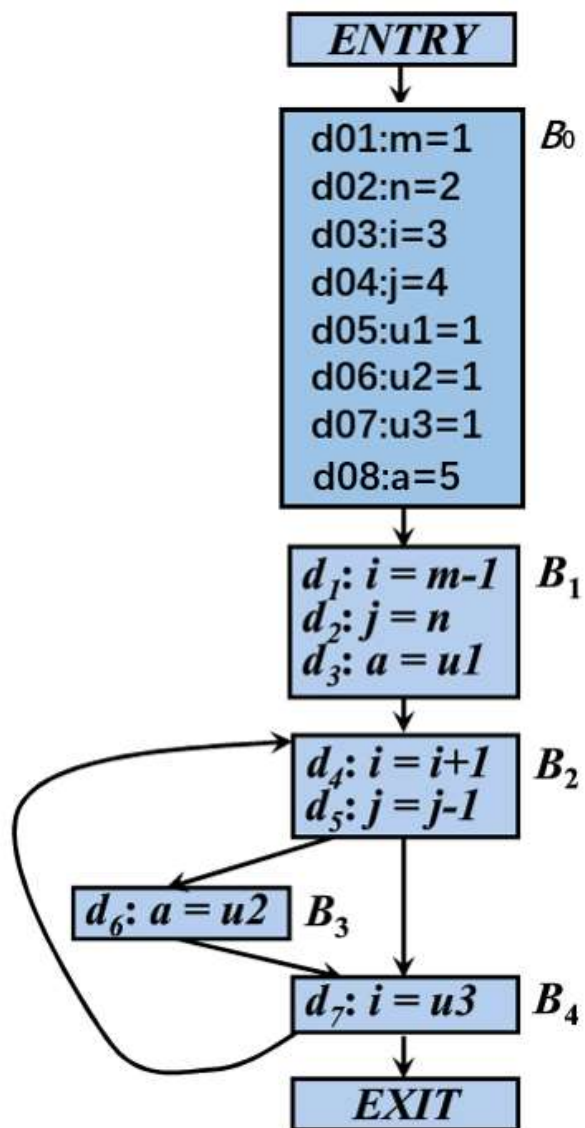
基本块的一个到达那的未
基加进量过，加是
的添后变死工变量
所有别然一个杀人该
在所分，一被有明
在量值果有只说
方法：变定如没量即
方所值析，都变
实现对初分序这定变
具体，赋值程序一个化
具之前，赋值程序一个化
之进达所就那一初

例子



添加定值





进行到达定值分析

IN[B]	B1	B2	B3	B4
d01	✓	✓	✓	✓
d02	✓	✓	✓	✓
d03	✓	×	×	×
d04	✓	×	×	×
d05	✓	✓	✓	✓
d06	✓	✓	✓	✓
d07	✓	✓	✓	✓
d08	✓	×	×	×



习题18.4

- 考虑一个类Pascal的语言，其中所有的变量都是整型（不需要显式声明），并且仅包含赋值语句、读语句、写语句、条件语句和循环语句。下面的产生式定义了该语言的语法（其中lit表示整型常量；OP的产生式没有给出，因为它和下面讨论的问题无关）

$Program \rightarrow Stmt$

$Stmt \rightarrow id := Exp$

$Stmt \rightarrow read(id)$

$Stmt \rightarrow write(Exp)$

$Stmt \rightarrow Stmt ; Stmt$

$Stmt \rightarrow if(Exp) then begin Stmt end$
 $else begin Stmt end$

$Stmt \rightarrow while(Exp) do begin Stmt end$

$Exp \rightarrow id$

$Exp \rightarrow lit$

$Exp \rightarrow Exp OP Exp$

定义*Stmt*的两个属性：*MayDef*表示它可能定值的变量集合，*MayUse*表示它可能引用的变量集合。

- (1) 写一个SDD或SDT，它计算*Stmt*的*MayDef*属性和*MayUse*属性
- (2) 基于*MayDef*和*MayUse*属性，说明*Stmt*₁ ; *Stmt*₂和*Stmt*₂ ; *Stmt*₁在什么情况下有同样的语义

产生式	语义规则
$Program \rightarrow Stmt$	print(Stmt.MayDef, Stmt.MayUse)
$Stmt \rightarrow id := Exp$	Stmt.MayDef={id.name}; Stmt.MayUse=Exp.MayUse
$Stmt \rightarrow \text{read} (id)$	Stmt.MayDef={id.name}; Stmt.MayUse= $\emptyset$
$Stmt \rightarrow \text{write}(Exp)$	Stmt.MayDef= $\emptyset$; Stmt.MayUse=Exp.MayUse
$Stmt \rightarrow Stmt ; Stmt$	Stmt.MayDef=Stmt1.MayDef $\cup$ Stmt2.MayDef; Stmt.MayUse=Stmt1.MayUse $\cup$ Stmt2.MayUse
$Stmt \rightarrow \text{if} (Exp) \text{ then begin } Stmt \text{ end else begin } Stmt \text{ end}$	Stmt.MayDef=Stmt1.MayDef $\cup$ Stmt2.MayDef; Stmt.MayUse=Stmt1.MayUse $\cup$ Stmt2.MayUse $\cup$ Exp.MayUse
$Stmt \rightarrow \text{while} (Exp) \text{ do begin } Stmt \text{ end}$	Stmt.MayDef=Stmt1.MayDef; Stmt.MayUse=Stmt1.MayUse $\cup$ Exp.MayUse
$Exp \rightarrow id$	Exp.MayUse={id.name}
$Exp \rightarrow \text{lit}$	Exp.MayUse= $\emptyset$
$Exp \rightarrow Exp \text{ OP } Exp$	Exp.MayUse=Exp1.MayUse $\cup$ Exp2.MayUse



习题18.4

定义 $Stmt$ 的两个属性： $MayDef$ 表示它可能定值的变量集合， $MayUse$ 表示它可能引用的变量集合。

➤ (2) 基于 $MayDef$ 和 $MayUse$ 属性，说明 $Stmt_1; Stmt_2$ 和 $Stmt_2; Stmt_1$ 在什么情况下有同样的语义

1. $Stmt \rightarrow stmt1; stmt2 \quad \{ Stmt.MayDef := Stmt1.MayDef \cup Stmt2.MayDef;$

$Stmt.MayUse := Stmt1.MayUse \cup Stmt2.MayUse \}$ $Stmt \rightarrow stmt2; stmt1$

$\{ Stmt.MayDef := Stmt2.MayDef \cup Stmt1.MayDef; Stmt.MayUse := Stmt2.MayUse \cup Stmt1.MayUse \}$

2. 某个语句使用的值不能在另一个语句中改变值

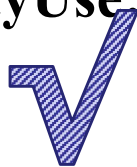
$$(Stmt1.MayDef \cap Stmt2.MayUse) \cup (Stmt2.MayDef \cap Stmt1.MayUse) = \emptyset$$

3. 不能分别在两个语句里面改变值（重复定义）

$$Stmt1.MayDef \cap Stmt2.MayDef = \emptyset$$

MayUse $\cap$ **MayUse:**

$a:=c; b:=c$



$b:=c; a=c$

MayDef $\cap$ **MayUse:**

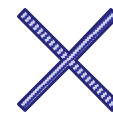
$a:=1; b:=a$



$b:=a; a=1$

MayDef $\cap$ **MayDef:**

$a:=1; a:=2$



$a:=2; a=1$



习题18.5

- 考虑一个类Pascal的语言，其中所有的变量都是整型（不需要显式声明），并且仅包含赋值语句、读语句、写语句、条件语句和循环语句。下面的产生式定义了该语言的语法（其中lit表示整型常量；OP的产生式没有给出，因为它和下面讨论的问题无关）

$Program \rightarrow Stmt$

$Stmt \rightarrow id := Exp$

$Stmt \rightarrow read(id)$

$Stmt \rightarrow write(Exp)$

$Stmt \rightarrow Stmt ; Stmt$

$Stmt \rightarrow if(Exp) then begin Stmt end$
 $else begin Stmt end$

$Stmt \rightarrow while(Exp) do begin Stmt end$

$Exp \rightarrow id$

$Exp \rightarrow lit$

$Exp \rightarrow Exp OP Exp$

定义*Stmt*的两个属性：*Def*表示*Stmt*中一定会定值且该定值前未引用的变量集合，*MayUse*表示*Stmt*中可能出现的引用前未定值的变量集合。

- (1) 写一个SDD或SDT，它计算*Stmt*的*Def*属性和*MayUse*属性
- (2) 基于上面的计算，程序可能未赋初值的变量集合从哪儿可以得到



- 1) $Program \rightarrow Stmt \{ Program.MayUse = Stmt.MayUse \}$
- 2) $Stmt \rightarrow \mathbf{id} := Exp$
 $\{ Stmt.Def = \{id.name\} - Exp.MayUse;$
 $Stmt.MayUse = Exp.MayUse \}$
- 3) $Stmt \rightarrow \mathbf{read} (id)$
 $\{ Stmt.MayUse = \{ \}; Stmt.Def = \{id.name\} \}$
- 4) $Stmt \rightarrow \mathbf{write}(Exp)$
 $\{ Stmt.Def = \{ \}; Stmt.MayUse = Exp.MayUse \}$
- 5) $Stmt \rightarrow Stmt1 ; Stmt2$
 $\{ Stmt.MayUse = Stmt1.MayUse \cup (Stmt2.MayUse - Stmt1.Def);$
 $Stmt.Def = Stmt1.Def \cup (Stmt2.Def - Stmt1.MayUse) \}$



6) $Stmt \rightarrow \text{if } (Exp) \text{ then begin } Stmt1 \text{ end}$

$\text{else begin } Stmt2 \text{ end}$

$\{ Stmt.MayUse = Stmt1.MayUse \cup Stmt2.MayUse \cup Exp.MayUse;$

$Stmt.Def = (Stmt1.Def \cap Stmt2.Def) - Exp.MayUse \}$

7) $Stmt \rightarrow \text{while } (Exp) \text{ do begin } Stmt1 \text{ end}$

$\{ Stmt.MayUse = Stmt1.MayUse \cup Exp.MayUse;$

$Stmt.Def = Stmt1.Def - Exp.MayUse \}$



8) $Exp \rightarrow \mathbf{id} \{ Exp.MayUse = \{id.name\} \}$

9) $Exp \rightarrow \mathbf{lit} \{ Exp.MayUse = \{ \} \}$

10) $Exp \rightarrow Exp1 \mathbf{OP} Exp2$

$\{ Exp.MayUse = Exp1.MayUse \cup Exp2.MayUse \}$



习题19.1

➤ 对于下图中的流图：

➤ (1) 找出流图中的循环。

➤ (2) B1中的语句 (1) 和 (2) 都是复制语句。其中a和b都被赋予了常量值。我们可以对a和b的哪些使用进行复制传播，并把对它们的使用替换为对一个常量的使用？在所有可能的地方进行这种替换。

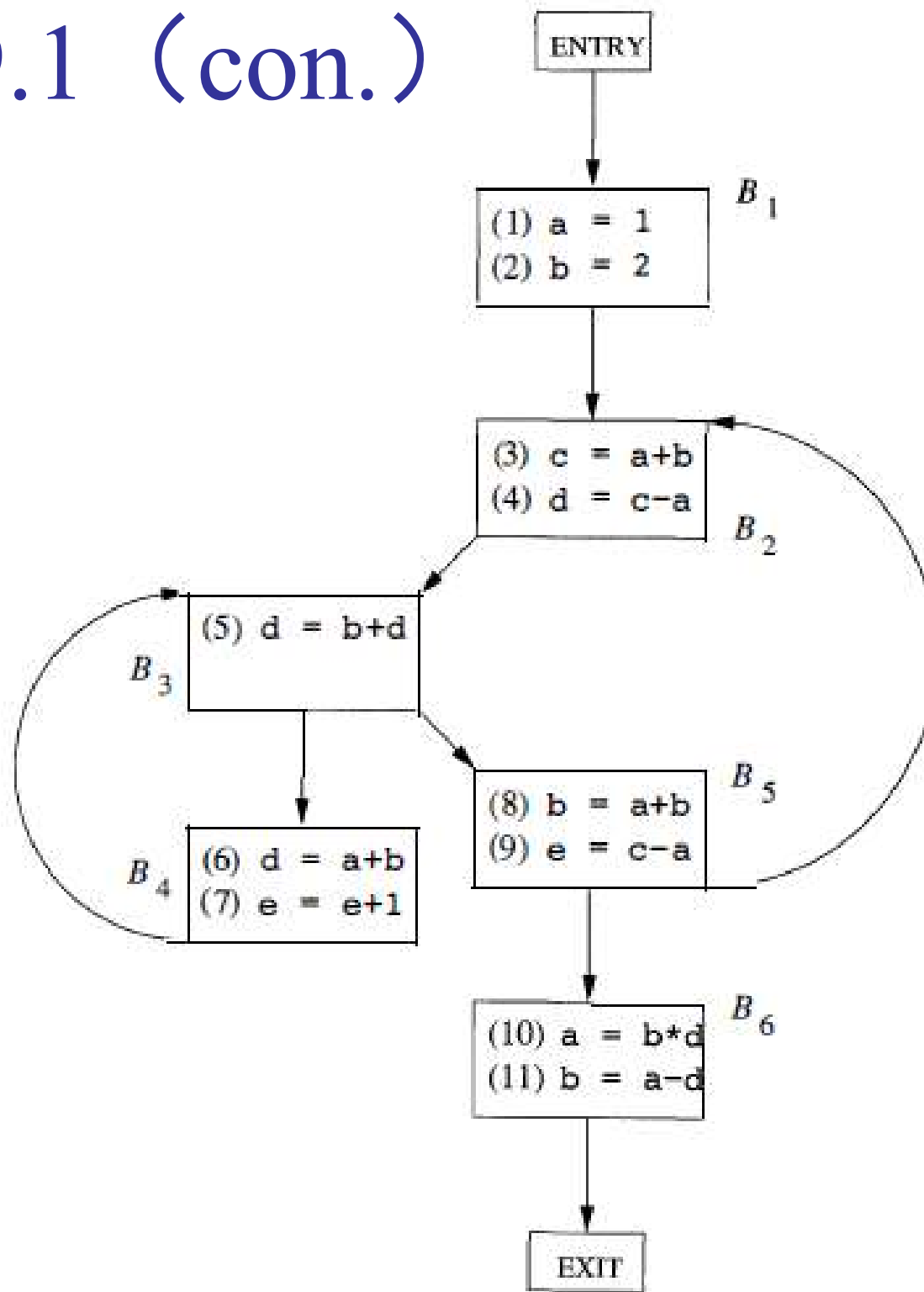
➤ (3) 对每个循环，找出所有的全局公因子表达式。

➤ (4) 寻找每个循环中的归纳变量。同时要考虑在 (2) 中引入的所有常量。

➤ (5) 寻找每个循环的全部循环不变计算。



习题19.1 (con.)





习题19.1

➤ (1)

① B_2, B_3, B_4, B_5

② B_3, B_4

➤ (2)

➤ (3) $c = 1+b$

➤ (4) $d = c-1$

➤ (6) $d = 1+b$

➤ (8) $b = 1+b$

➤ (9) $e = c-1$

➤ (3)

① B_2, B_3, B_4, B_5

➤ $a+b$

➤ $c-a$

② B_3, B_4

➤ $a+b$

➤ (4)

① B_2, B_3, B_4, B_5

➤ b, c

② B_3, B_4

➤ e

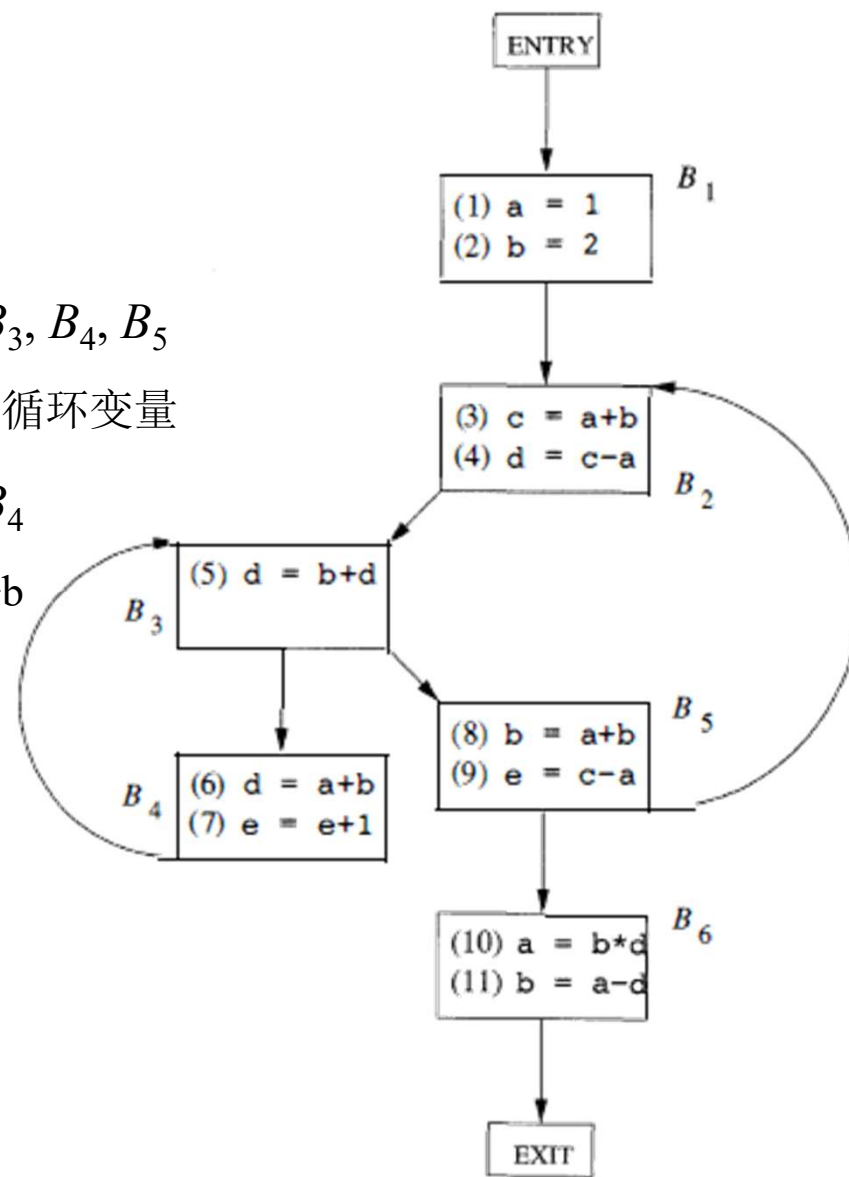
➤ (5)

① B_2, B_3, B_4, B_5

➤ 无循环变量

② B_3, B_4

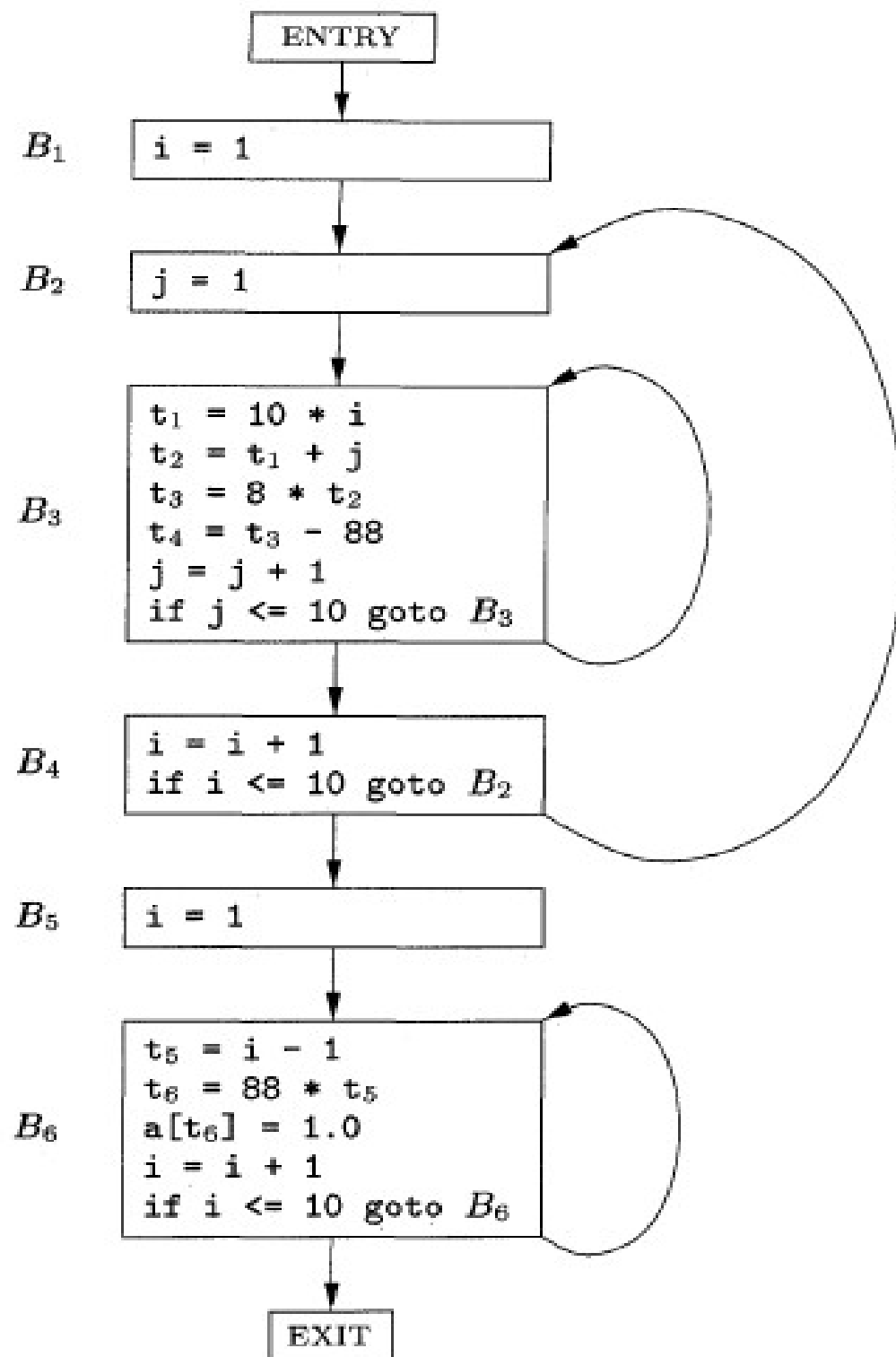
➤ $a+b$





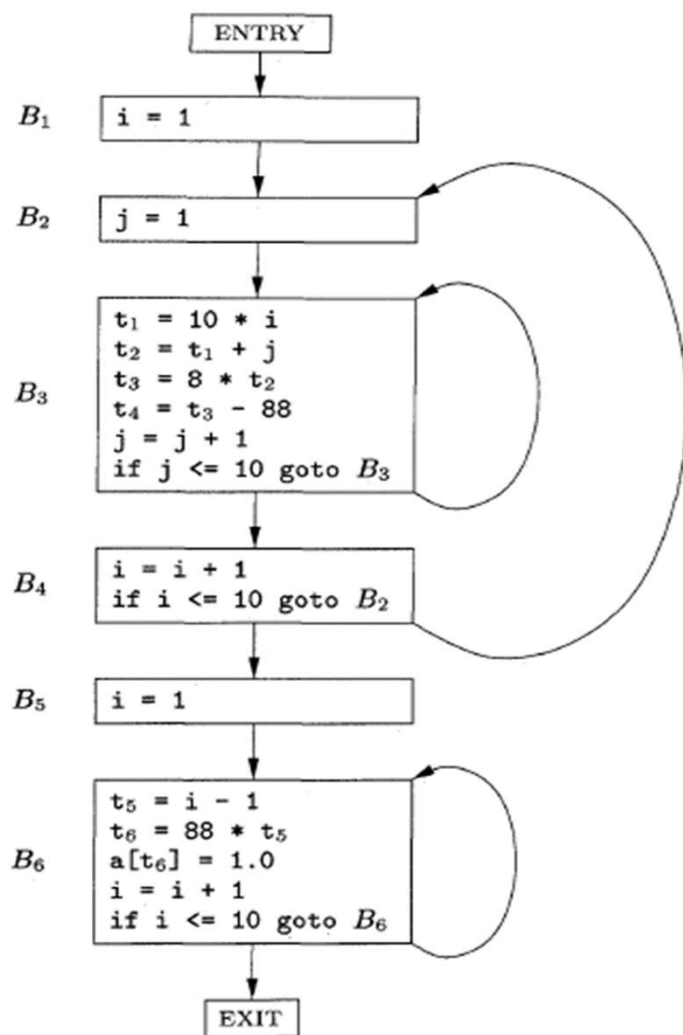
习题19.2

➤ 对于下图中的流图
进行代码优化。





全局活跃分析

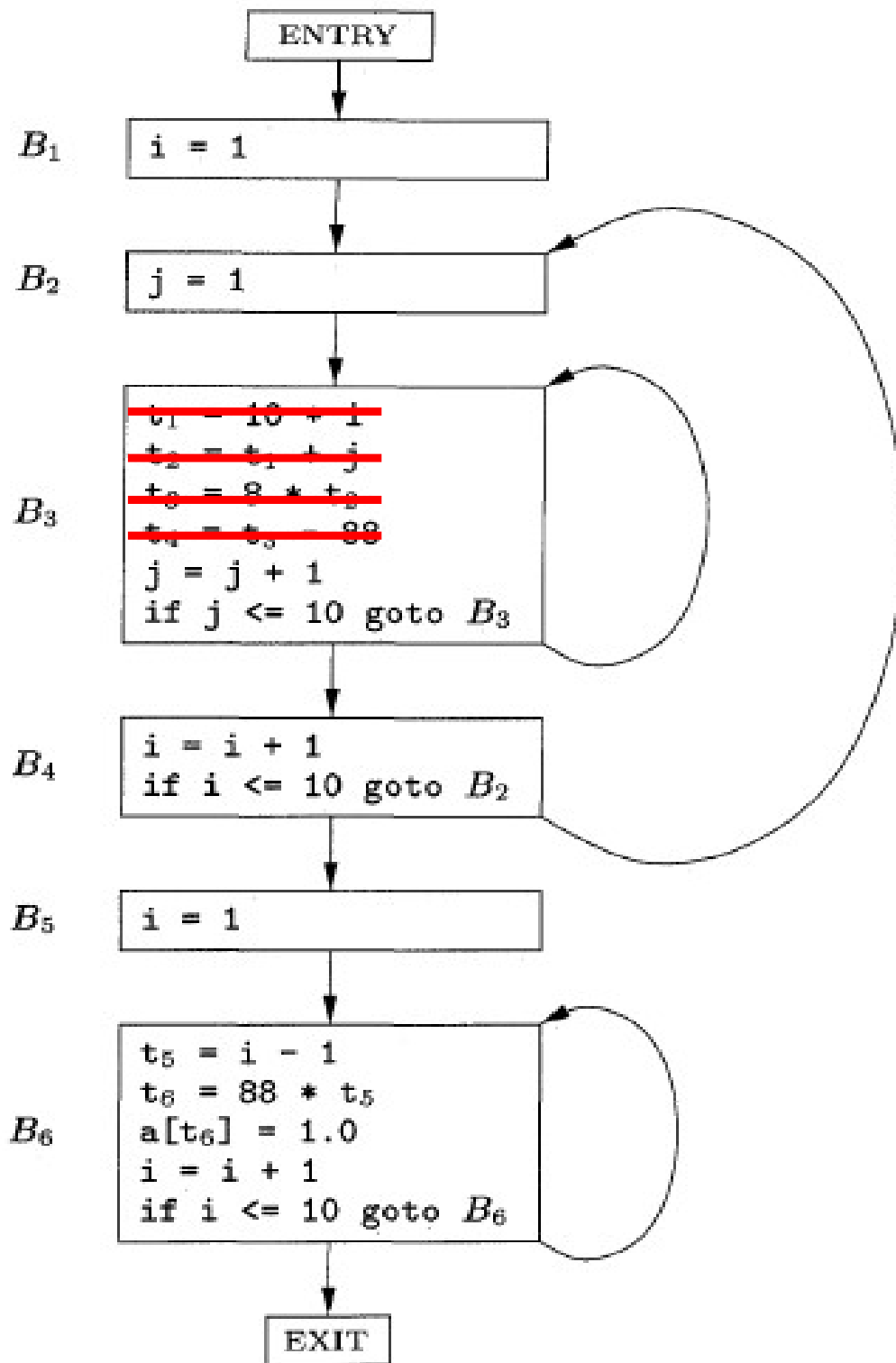


OUT[B]	B1	B2	B3	B4	B5	B6
i	✓	✓	✓	✓	✓	✓
j	×	✓	✓	×	×	×
t1	×	×	×	×	×	×
t2	×	×	×	×	×	×
t3	×	×	×	×	×	×
t4	×	×	×	×	×	×
t5	×	×	×	×	×	✓
t6	×	×	×	×	×	✓

t1-t4 全局不活跃



删除无用代码



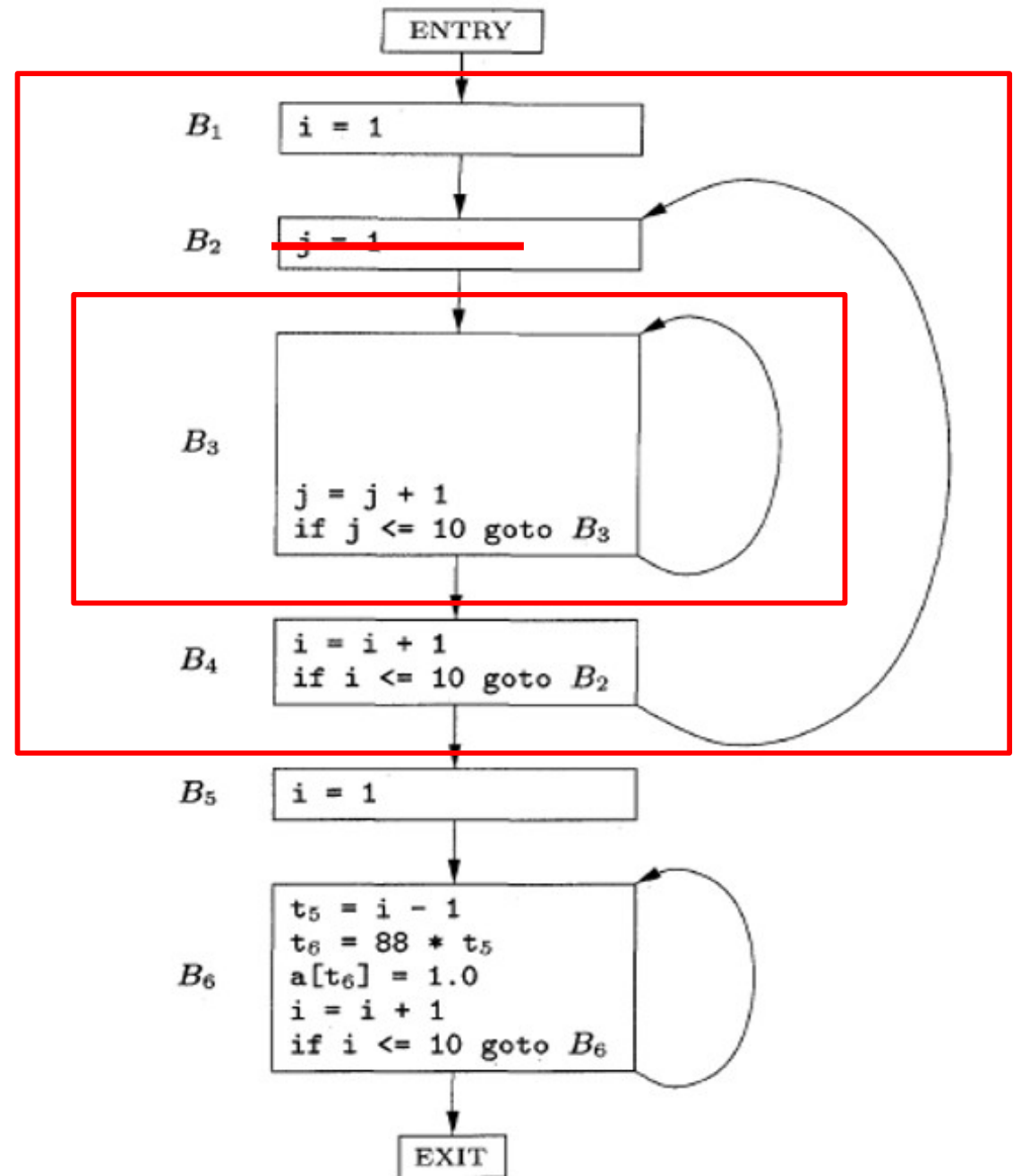


去除空循环

1. j 循环后不活跃，
3- $\rightarrow$ 3空循环可删除

2. j 变量为全局不活跃变量，可删除

3. B_2, B_3 删除后 原本的4- $\rightarrow$ 2循环为空循环，
且 $B_1 B_4$ 中的变量 i 之后进行了重新赋值，为全局不活跃变量，所以空循环可删除



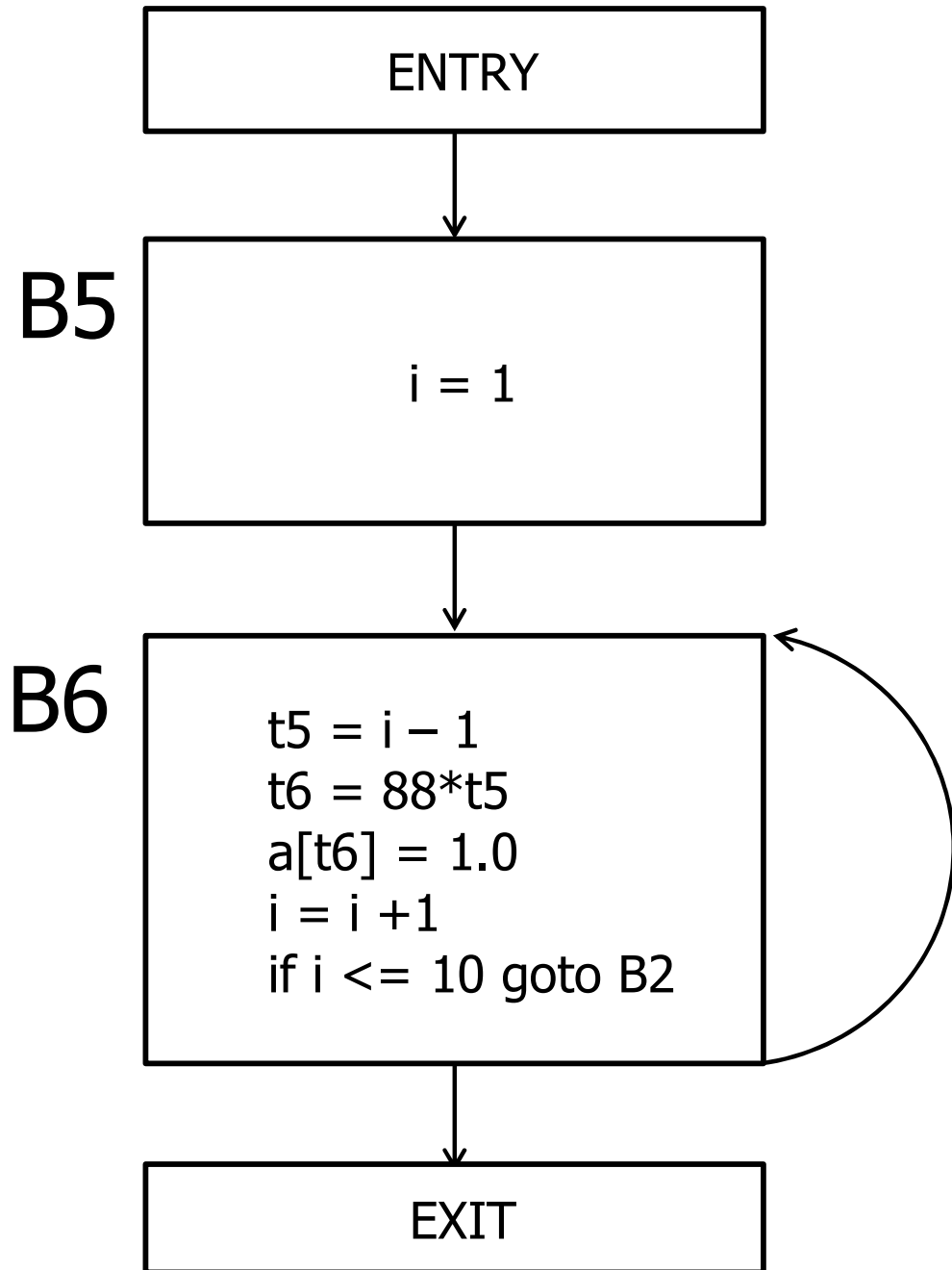


基本块优化后流图

基本变量 i

$t5(i, 1, -1)$

$t6(i, 88, -88)$



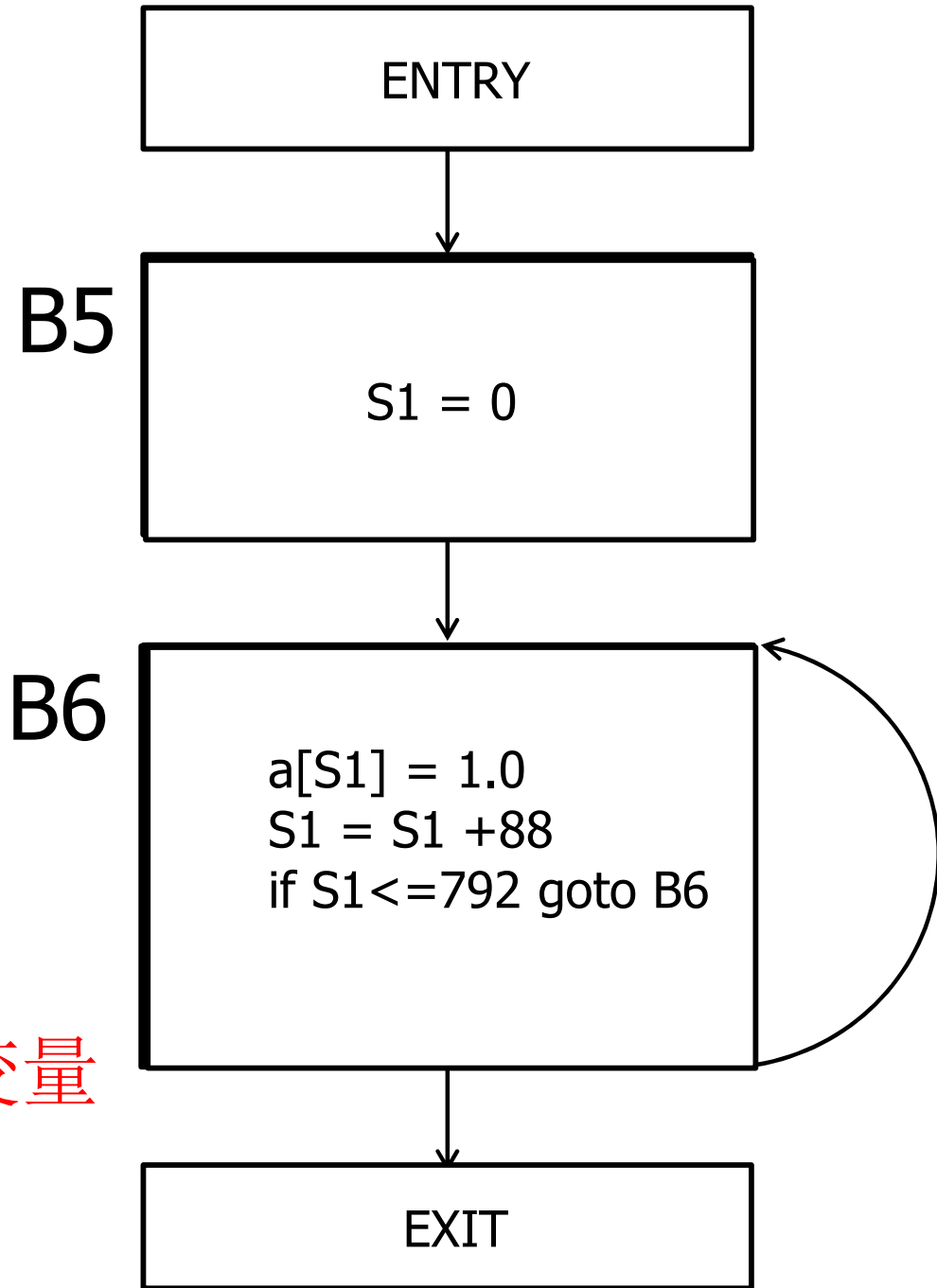


全局代码优化

1. 归纳变量削弱

2. 删除无用的复制

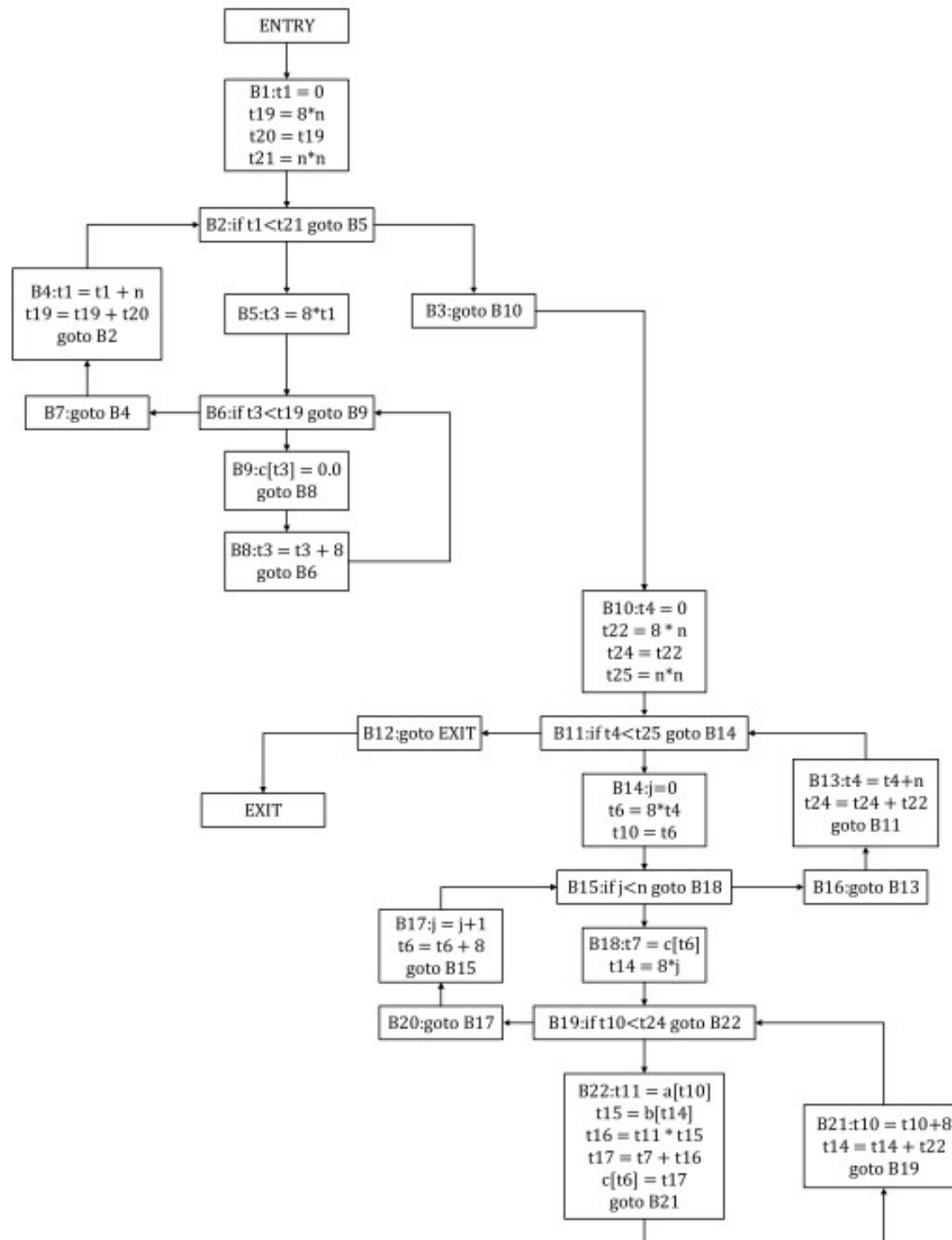
3. 删除仅用于测试的归纳变量





习题19.3

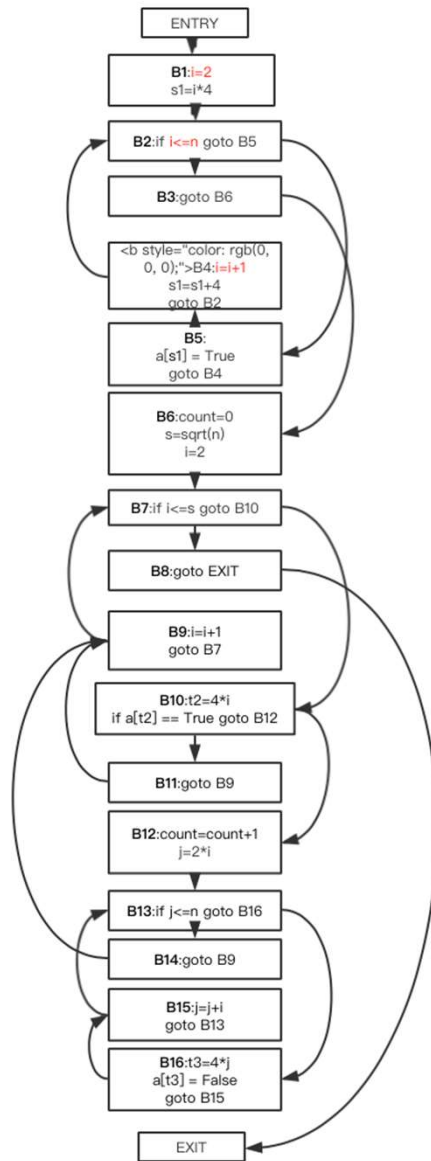
➤对习题16.1中得到的流图进行代码优化。



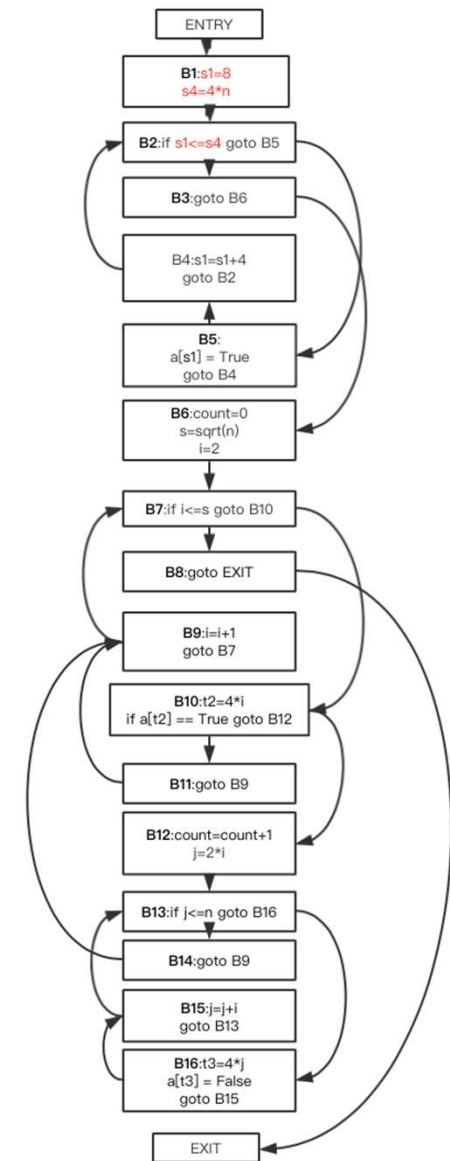


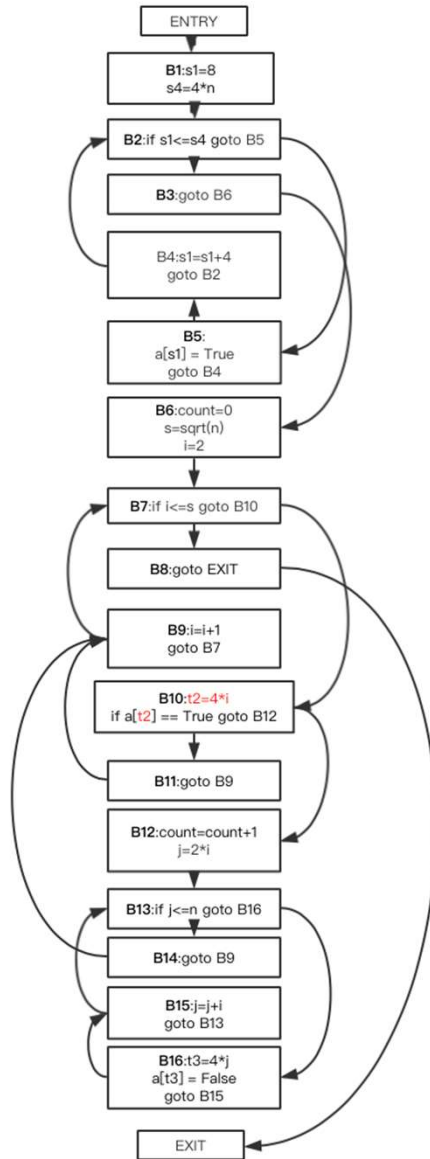
习题19.4

➤对习题16.2中得到的流图进行代码优化。

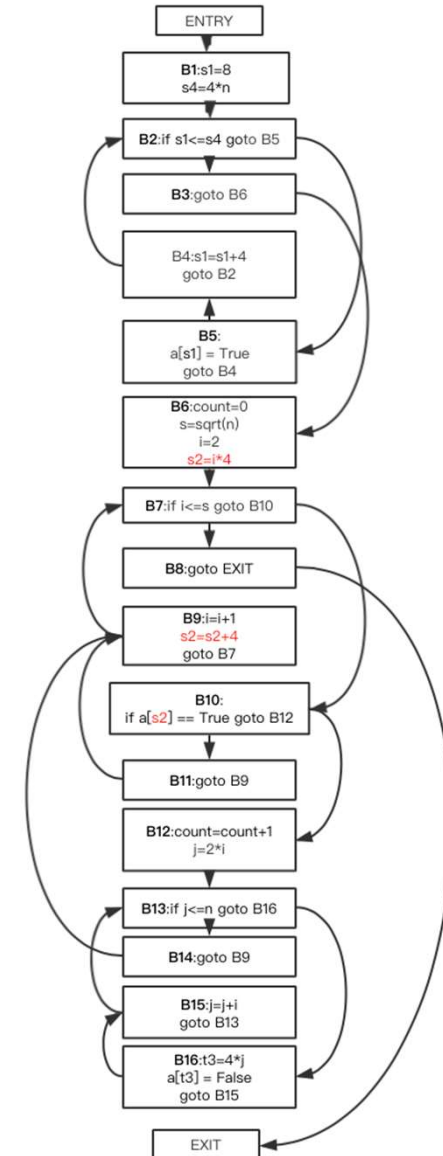


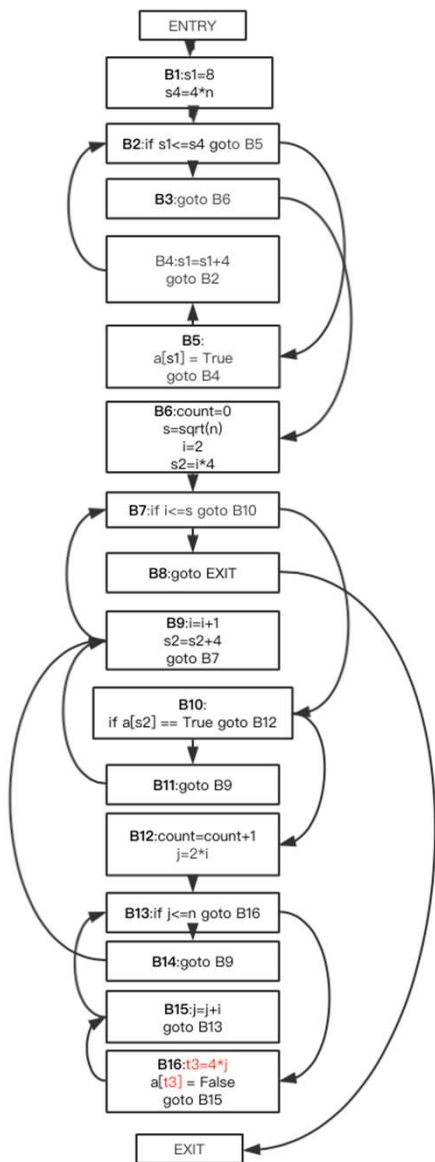
归纳变量强度削弱



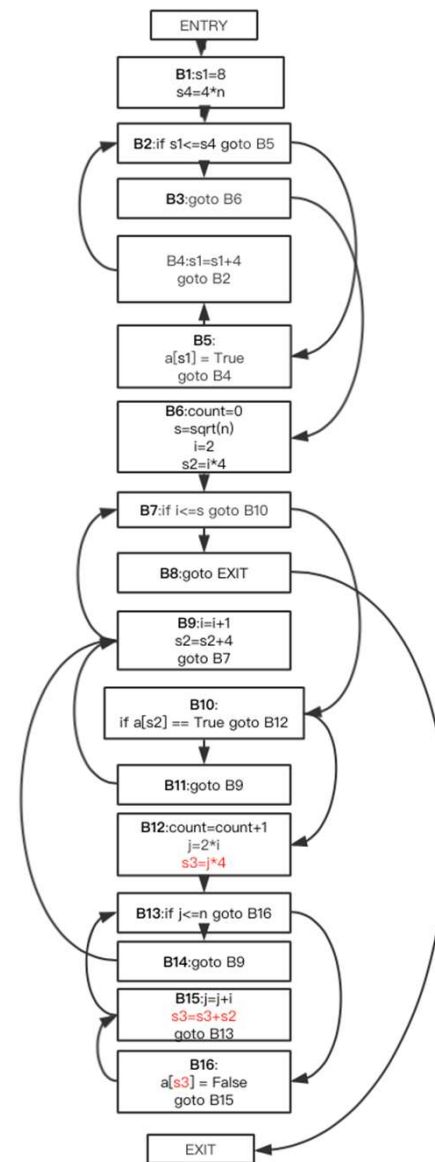


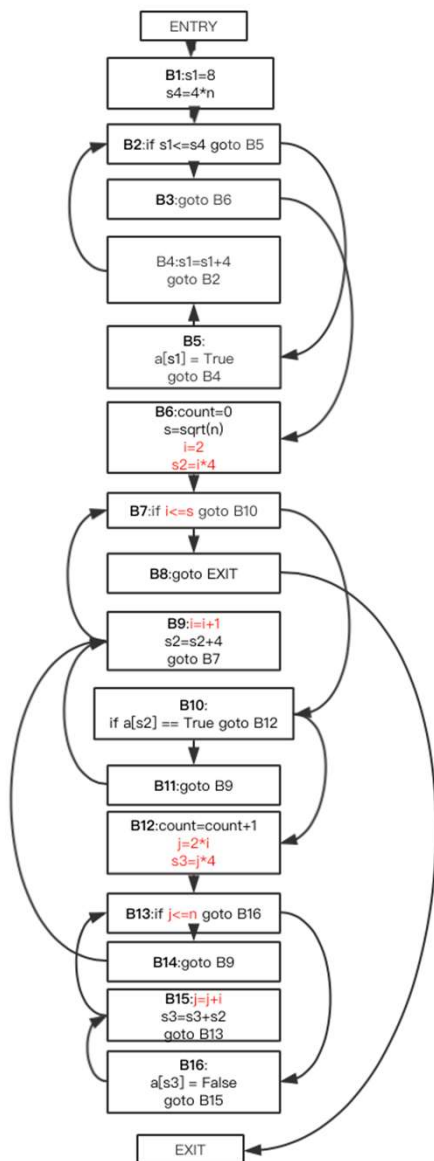
归纳变量强度削弱



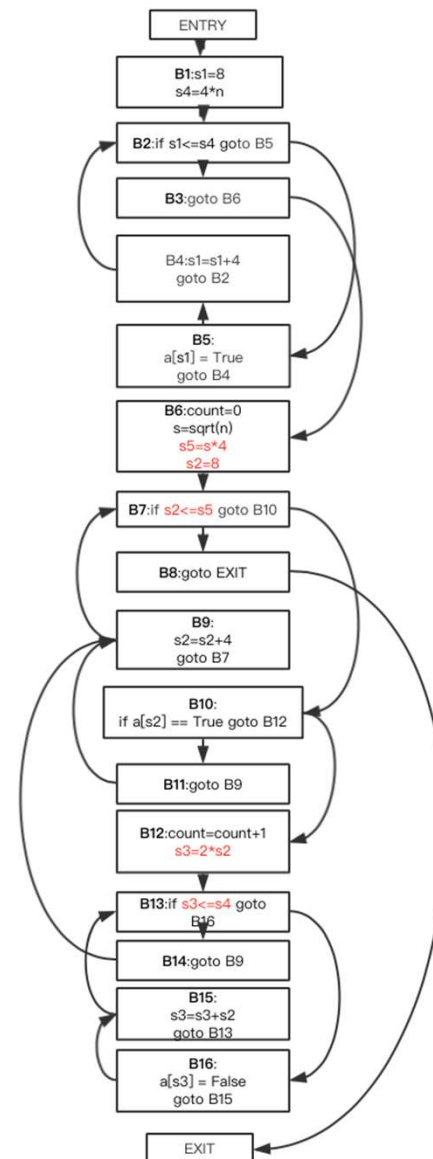


归纳变量强度削弱



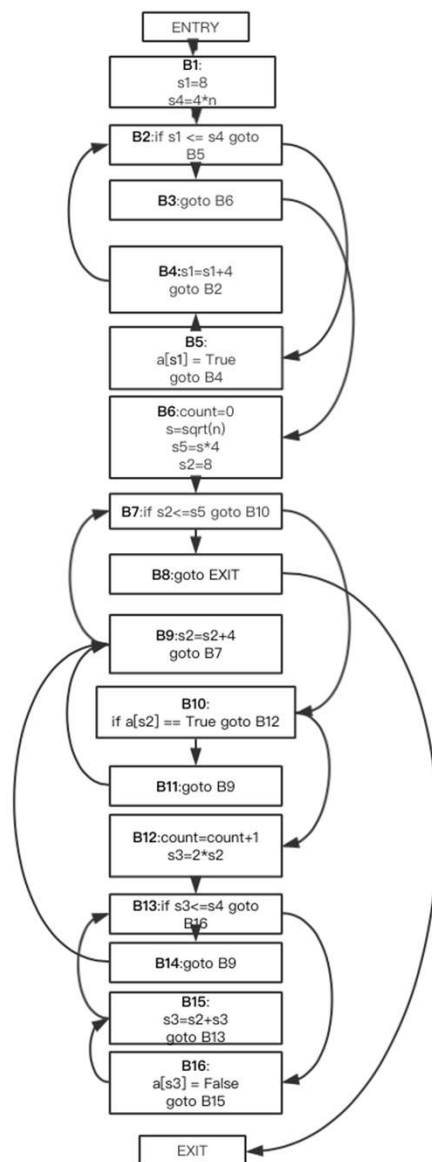


归纳变量强度削弱





答案





习题19.5

- 下图是用来计算两个向量A和B的点积的中间代码。尽你所能，通过下列方法优化这个代码：消除公共子表达式，对归纳变量进行强度消减，消除归纳变量。

dp = 0.

i = 0

L: t1 = i*8

t2 = A[t1]

t3 = i*8

t4 = B[t3]

t5 = t2*t4

dp = dp+t5

i = i+ 1

if i < n goto L



划分基本块



dp 0.

i = 0

L:

t1 = i * 8

t2 = A[t1]

t3 = i * 8

t4 = B[t3]

t5 = t2 * t4

dp = dp + t5

i = i + 1

if i < n goto L



删除归纳变量



dp 0.

t1 = 0

t6 = n*8

L: t2= A[t1]

t4 = B[t1]

t5= t2*t4

dp = dp+t5

t1 = t1 + 8

if t1 <t6 goto L



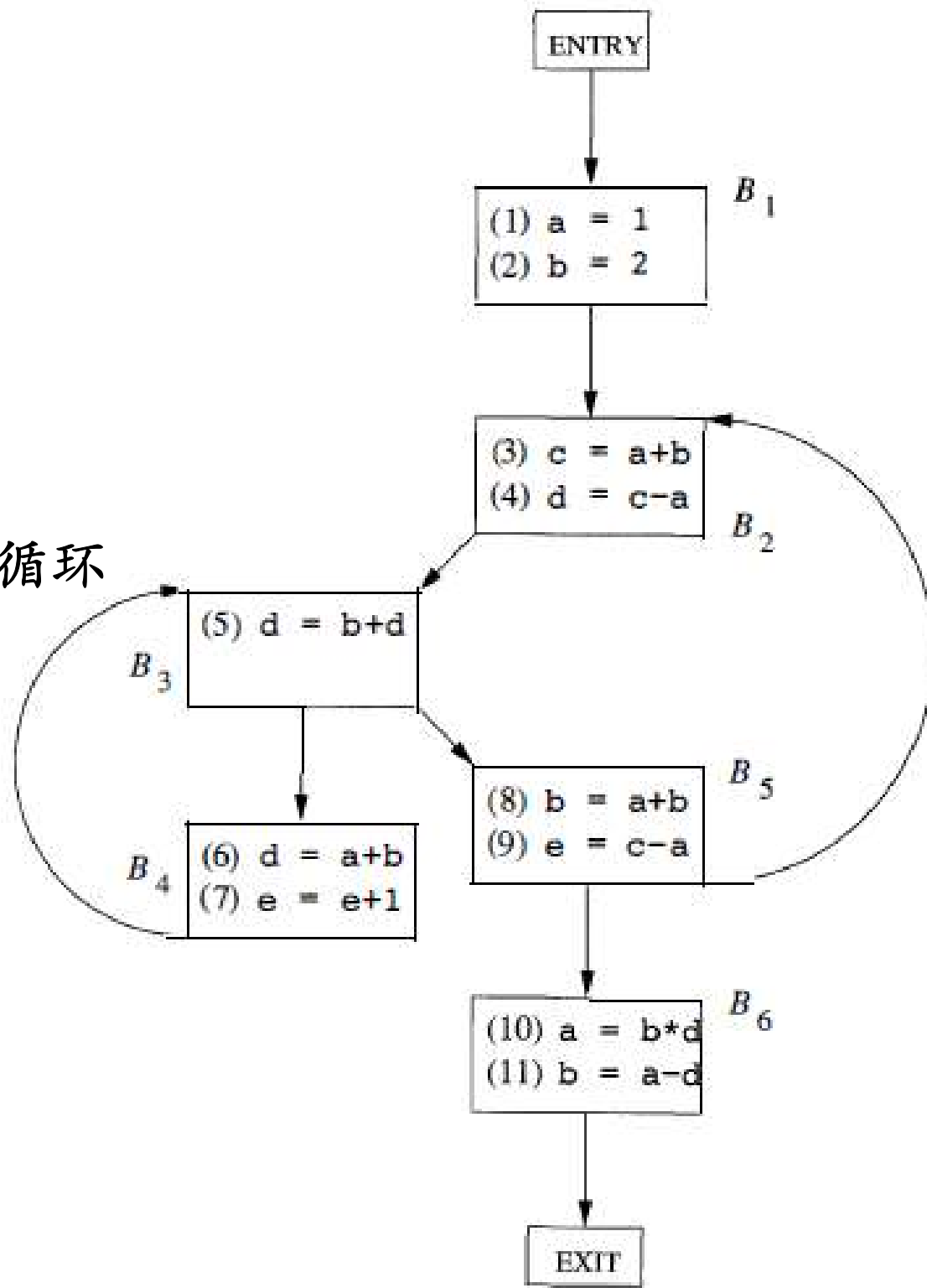
习题19.6

➤ 对下图中的流图：

➤ (1) 计算支配关系

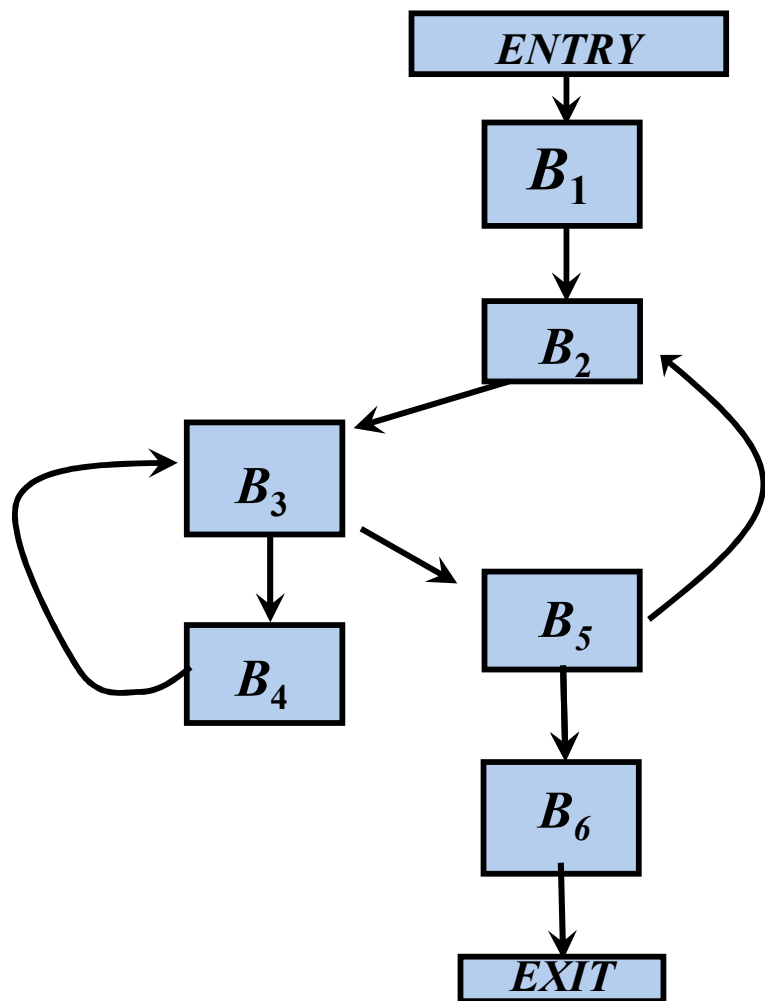
➤ (2) 构造支配节点树

➤ (3) 找出这个流图的自然循环





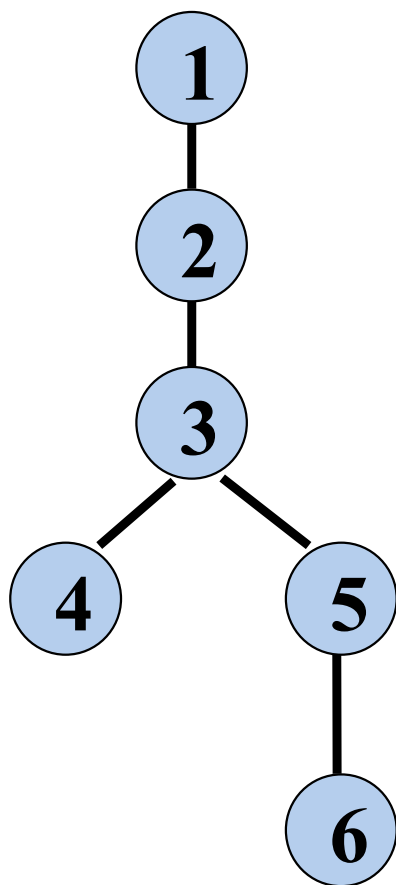
计算支配关系



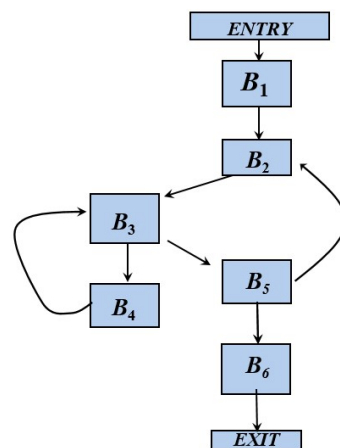
支配结点	支配对象
1	1 ~ 6
2	2 ~ 6
3	3 ~ 6
4	4
5	5 ~ 6
6	6



支配节点树、自然循环



回边	自然循环
$5 \rightarrow 2$	2 3 4 5
$4 \rightarrow 3$	3 4





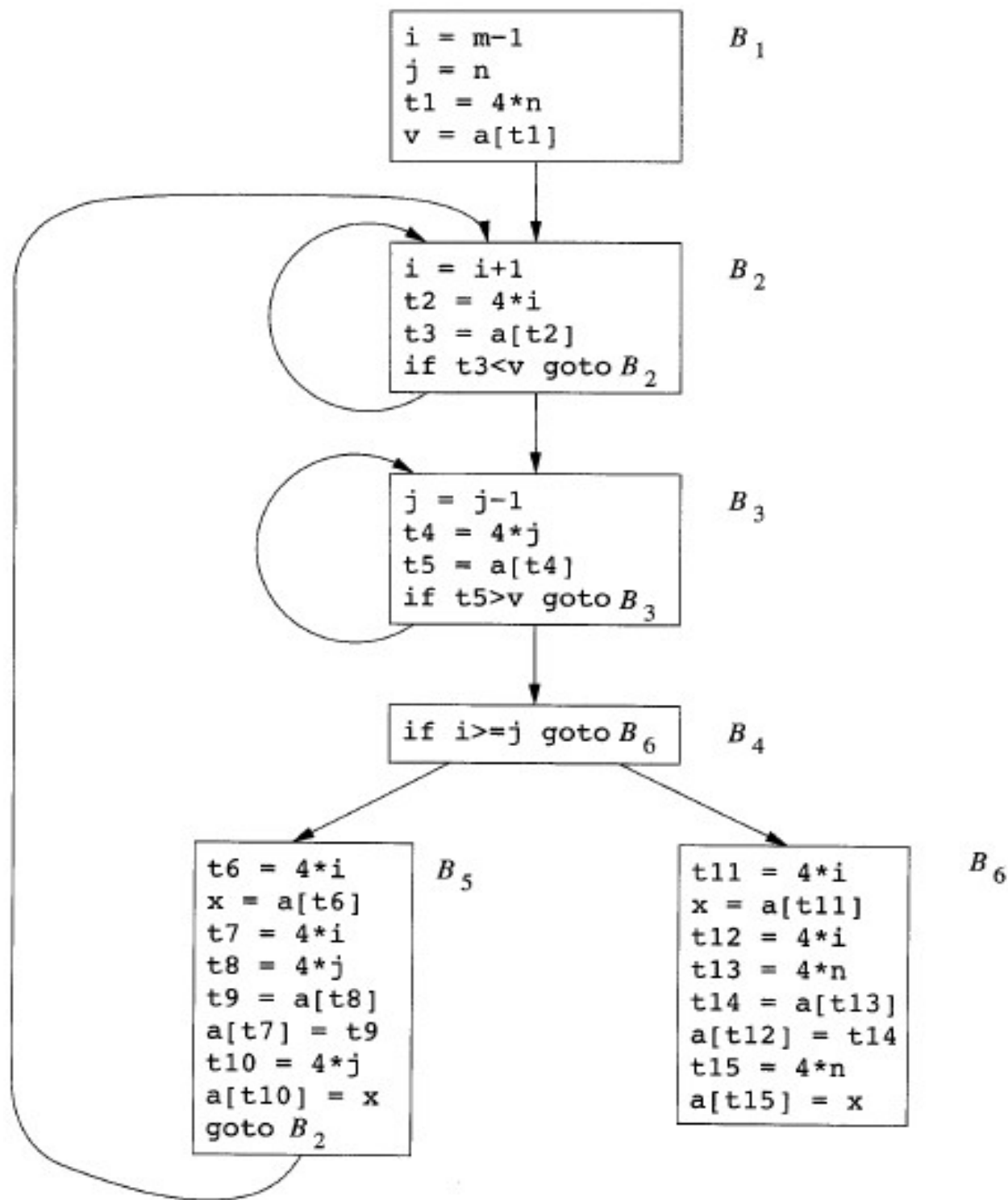
习题19.7

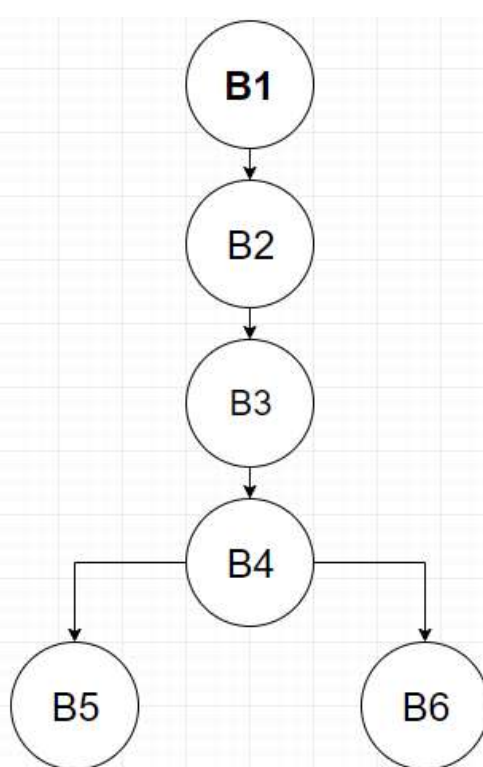
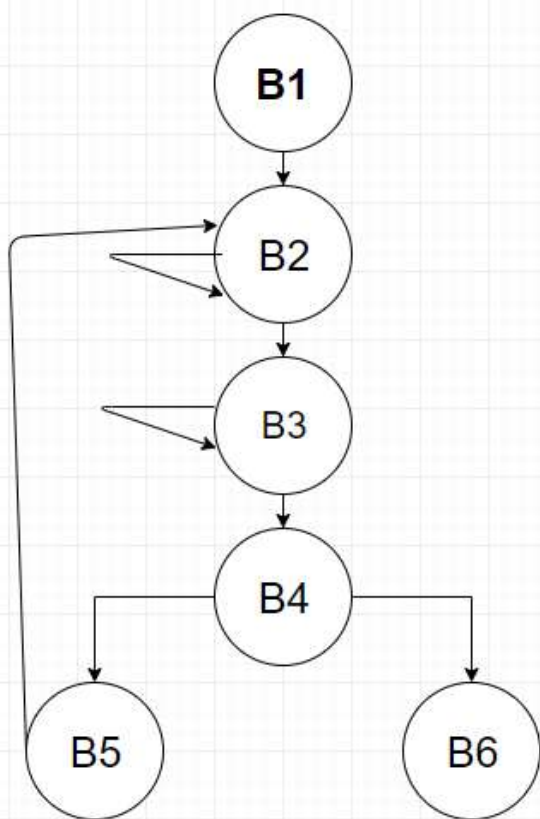
➤ 对下图中的流图：

➤ (1) 计算支配关系

➤ (2) 构造支配节点树

➤ (3) 找出这个流图的自然循环





支配节点	支配对象
1	1 2 3 4 5 6
2	2 3 4 5 6
3	3 4 5 6
4	4 5 6
5	5
6	6

	In	Out
E		
B1	{E}	{E, B1}
B2	{E, B1}	{E, B1, B2}
B3	{E, B1, B2}	{E, B1, B2, B3}
B4	{E, B1, B2, B3}	{E, B1, B2, B3, B4}
B5	{E, B1, B2, B3, B4}	{E, B1, B2, B3, B4, B5}
B6	{E, B1, B2, B3, B4}	{E, B1, B2, B3, B4, B6}



回边 自然循环

2→2

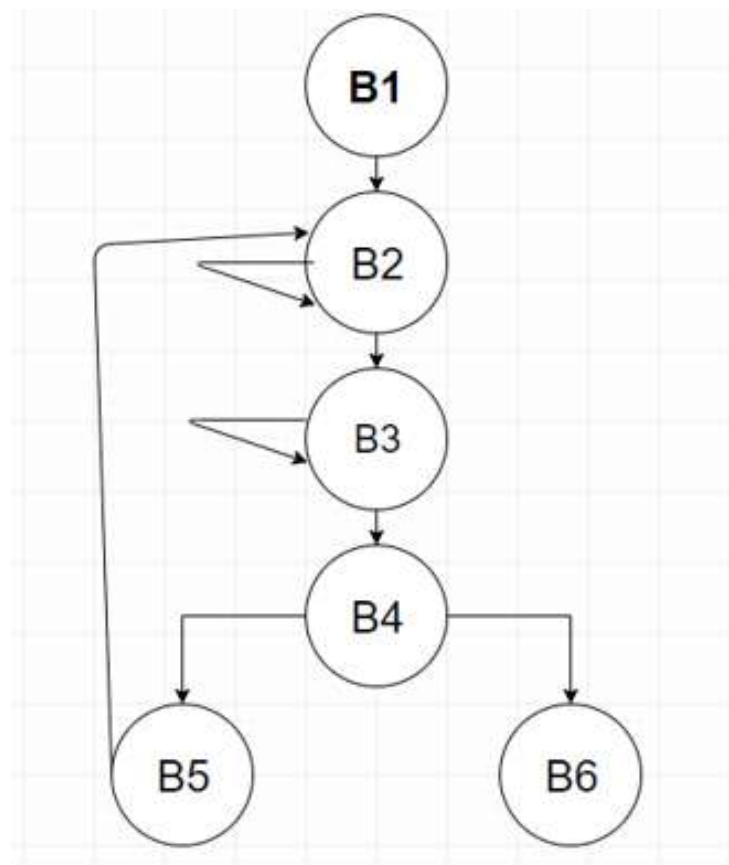
2

3→3

3

5→2

2345





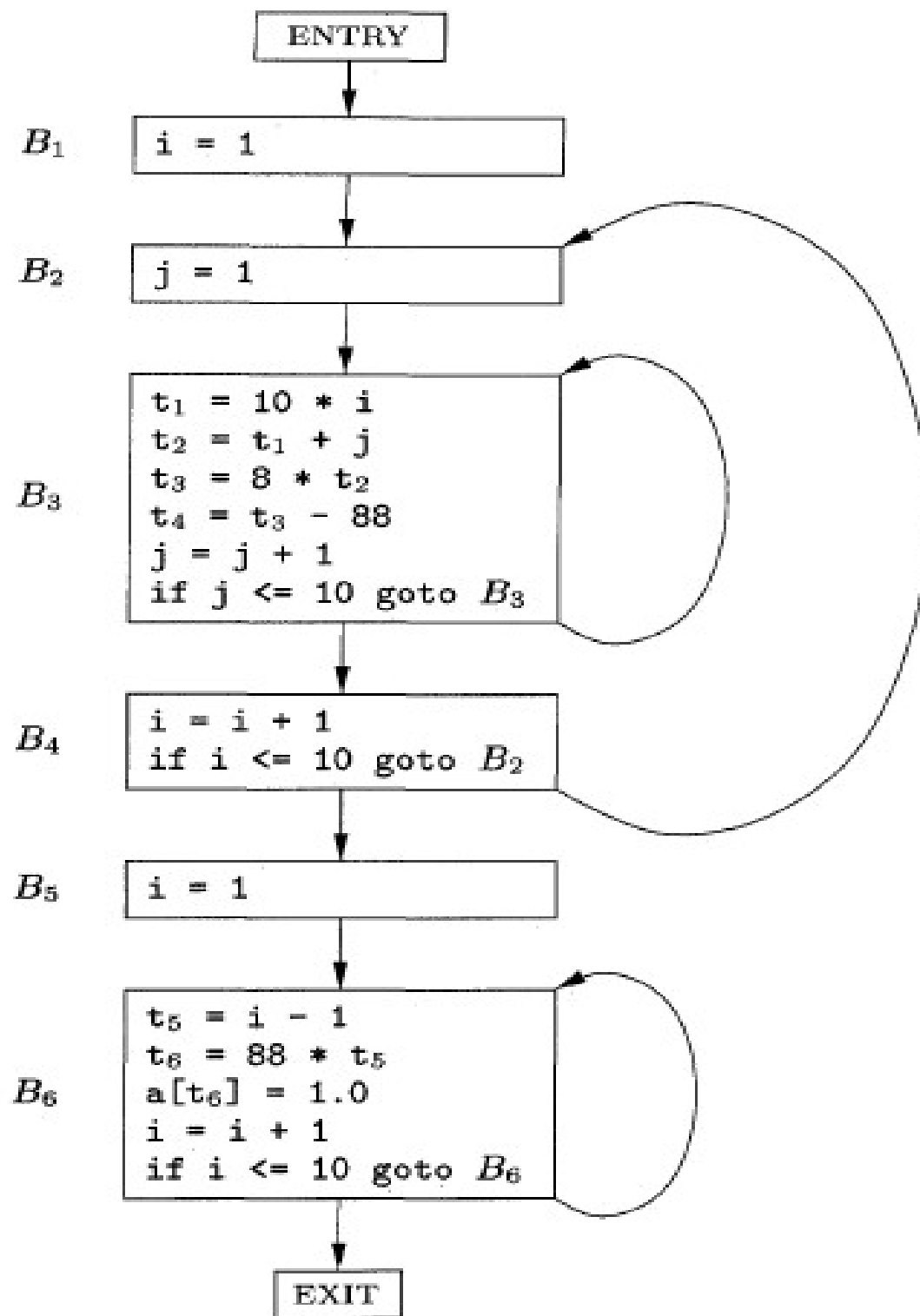
习题19.8

➤ 对下图中的流图：

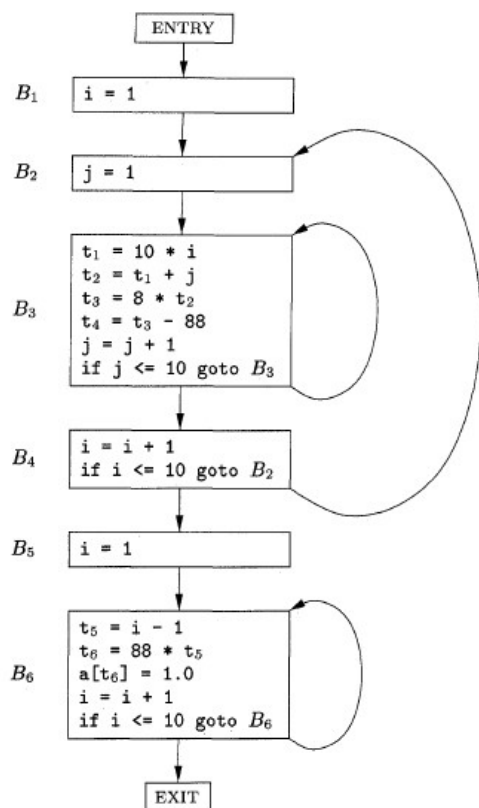
➤ (1) 计算支配关系

➤ (2) 构造支配节点树

➤ (3) 找出这个流图的自然循环



1.计算支配关系



基本块	OUT 0	IN 1	OUT 1	IN 2	OUT 2
ENTRY	E				
1	N	E	E, 1	E	E, 1
2	N	E, 1	E, 1, 2	E, 1	E, 1, 2
3	N	E, 1, 2	E, 1, 2, 3	E, 1, 2	E, 1, 2, 3
4	N	E, 1, 2, 3	E, 1, 2, 3, 4	E, 1, 2, 3	E, 1, 2, 3, 4
5	N	E, 1, 2, 3, 4	E, 1, 2, 3, 4, 5	E, 1, 2, 3, 4	E, 1, 2, 3, 4, 5
6	N	E, 1, 2, 3, 4, 5	E, 1, 2, 3, 4, 5, 6	E, 1, 2, 3, 4, 5	E, 1, 2, 3, 4, 5, 6

N

$OUT[ENTRY] = \{ENTRY\}$

for(除ENTRY之外的每个基本块B)

$OUT[B] = N$

while(某个OUT值发生了改变)

for(除ENTRY之外的每个基本块B)

$\{ IN[B] = \bigcap_{P \text{ 是 } B \text{ 的一个前驱}} OUT[P]$

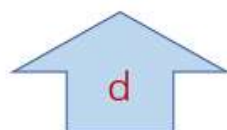
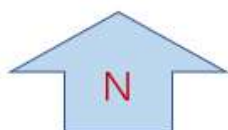
$OUT[B] = IN[B] \cup \{B\}$

}

d

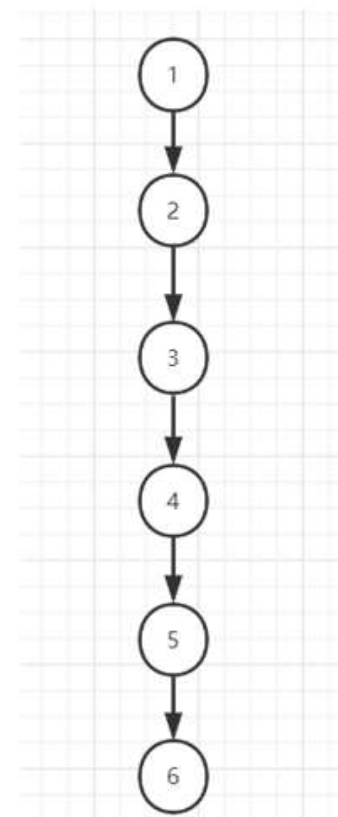
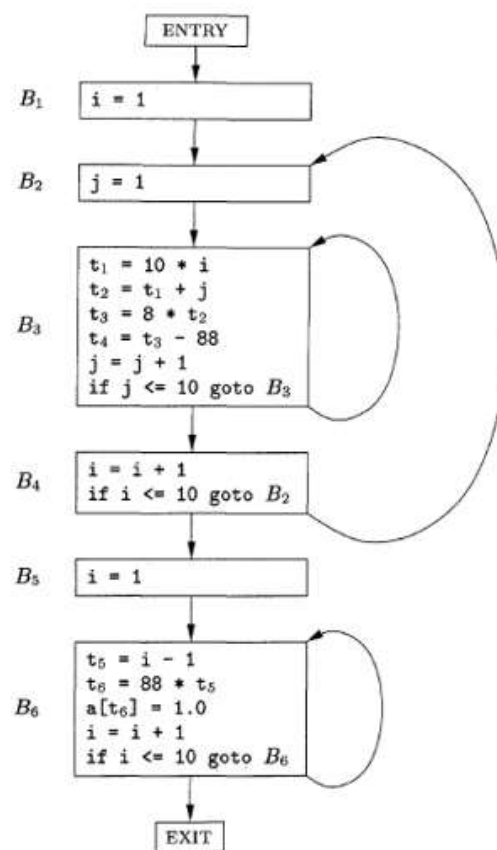
2.构造支配节点树

	OUT 2
E	
1	E, 1
2	E, 1, 2
3	E, 1, 2, 3
4	E, 1, 2, 3, 4
5	E, 1, 2, 3, 4, 5
6	E, 1, 2, 3, 4, 5, 6



➤ 直接支配结点 (Immediate Dominator)

➤ 从入口结点到达 n 的所有路径上, 结点 n 的最后一个支配结点称为直接支配结点



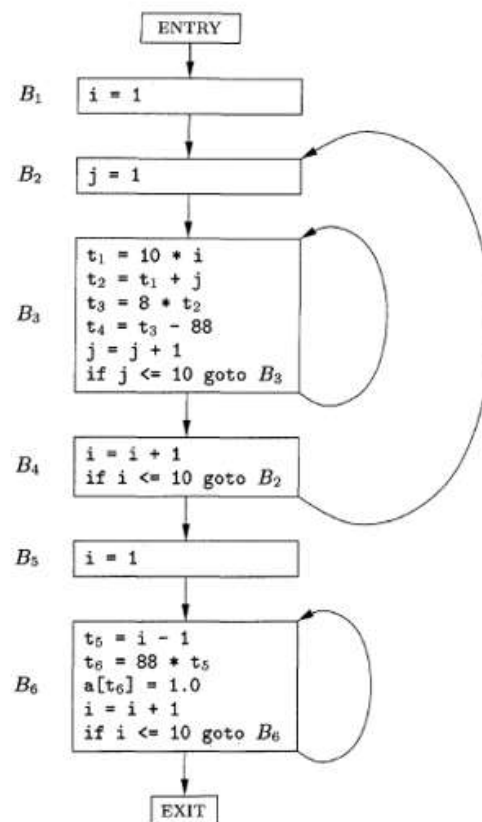
3. 找出流图的自然循环

► 假定流图中存在两个结点 d 和 n 满足 $d \text{ dom } n$ 。如果存在从结点 n 到 d 的有向边 $n \rightarrow d$ ，那么这条边称为 **回边**

► 给定一个回边 $n \rightarrow d$ ，该回边的自然循环为： d ，以及所有可以不经 d 而到达 n 的结点。 d 为该循环的首结点。

N	支配节点D	后继节点S	$D \cap S$	Back Edge
E				
1	E, 1	2	$\emptyset$	
2	E, 1, 2	3	$\emptyset$	
3	E, 1, 2, 3	3, 4	3	3 --> 3
4	E, 1, 2, 3, 4	2, 5	2	4 --> 2
5	E, 1, 2, 3, 4, 5	6	$\emptyset$	
6	E, 1, 2, 3, 4, 5, 6	6	6	6 --> 6

Back Edge	Natural Loop
3 --> 3	3
4 --> 2	2, 3, 4
6 --> 6	6





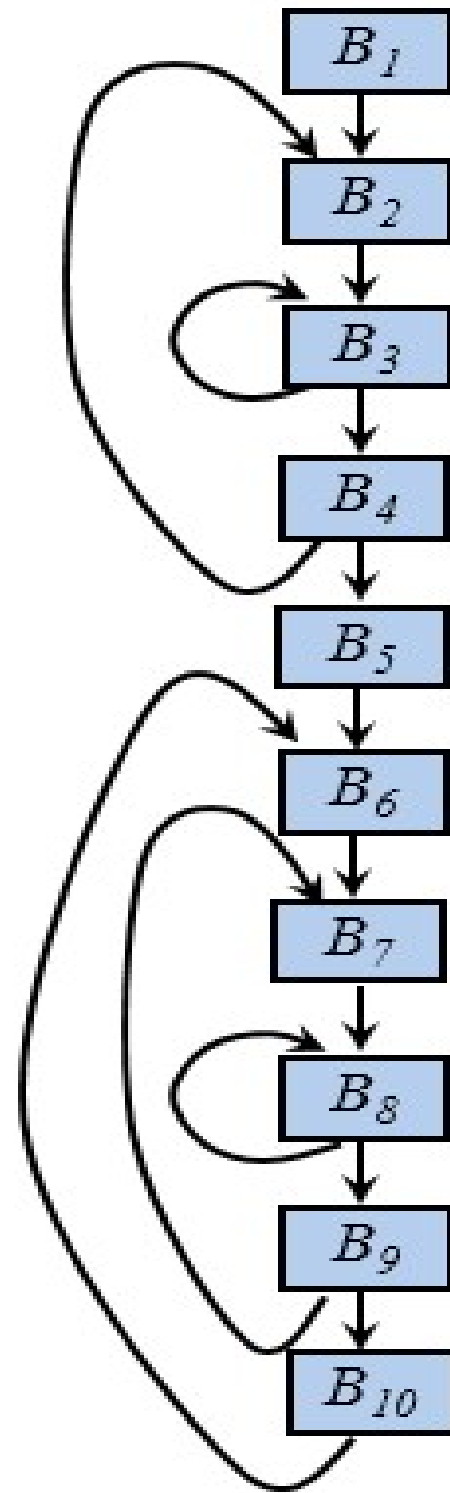
习题19.9

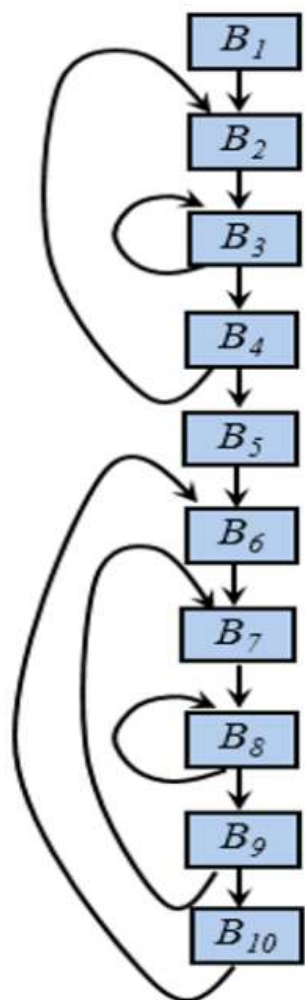
➤ 对下图中的流图：

➤ (1) 计算支配关系

➤ (2) 构造支配节点树

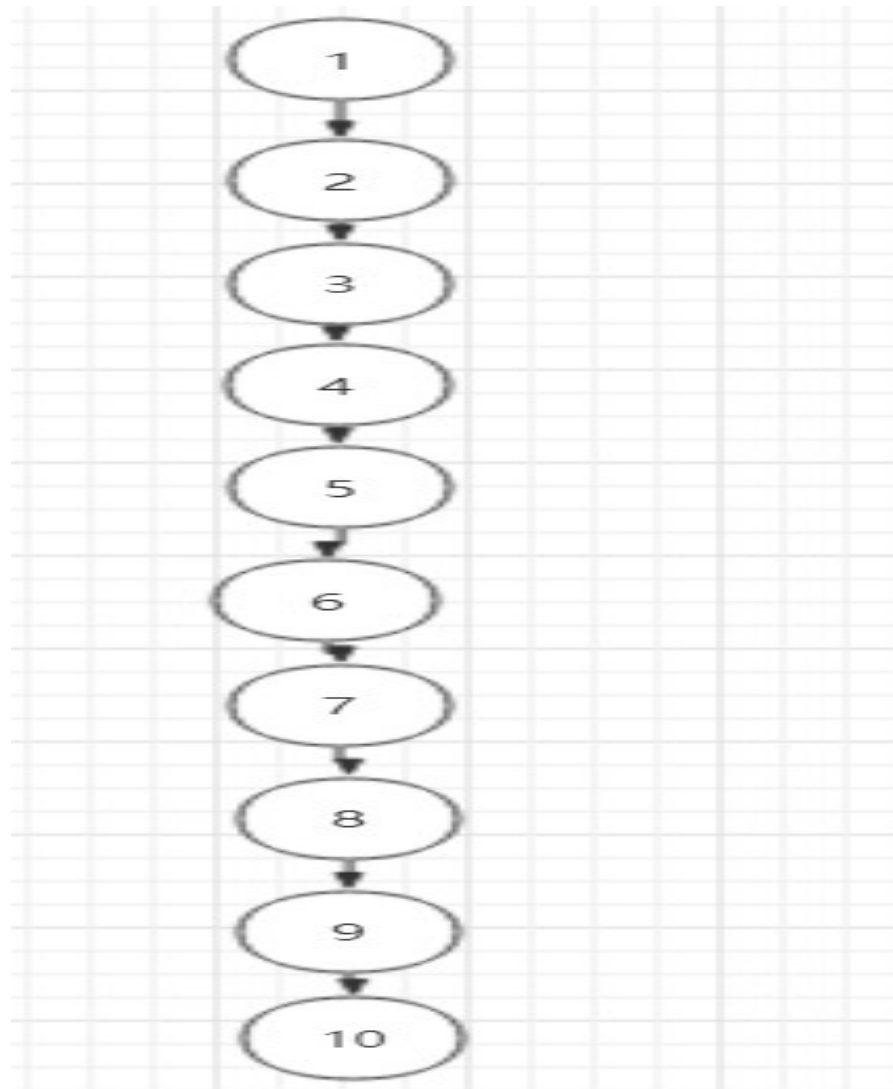
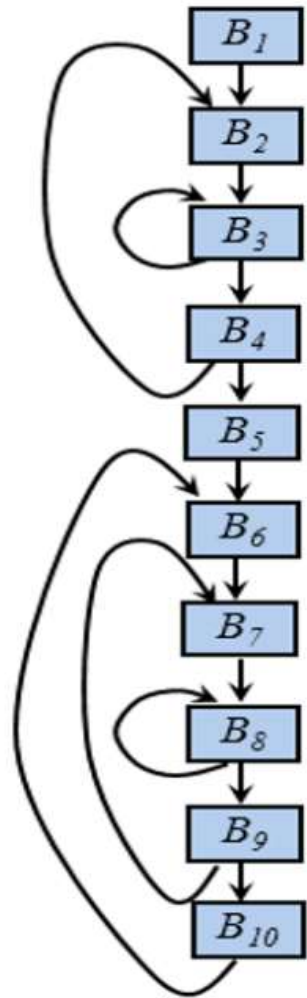
➤ (3) 找出这个流图的自然循环

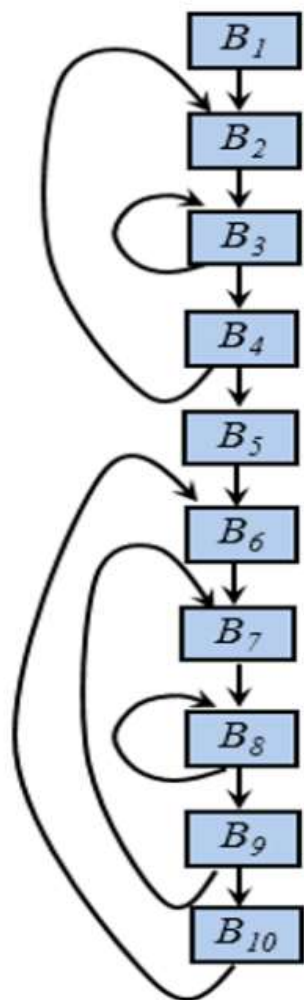




$OUT[ENTRY] = \{ENTRY\}$
for(除ENTRY之外的每个基本块B) $OUT[B] = N$
while(某个OUT值发生了改变)
 for(除ENTRY之外的每个基本块B)
 $\{ IN[B] = \bigcap_{P \text{ 是 } B \text{ 的一个前驱}} OUT[P]$
 $OUT[B] = IN[B] \cup \{B\}$
 }
 $OUT^0[B]$

	$OUT^0[B]$	$IN'[B]$	$OUT'[B]$
E	E		
1	N	E	E1
2	N	E1	E12
3	N	E12	E123
4	N	E123	E1234
5	N	E1234	E12345
6	N	E12345	E123456
7	N	E123456	E1234567
8	N	E1234567	E12345678
9	N	E12345678	E123456789
10	N	E123456789	E12345678910





➤ 给定一个回边 $n \rightarrow d$, 该回边的自然循环为: d , 以及所有可以不经过 d 而到达 n 的结点。 d 为该循环的首结点。

回边	自然循环
4->2	2 3 4
3->3	3
10->6	6 7 8 9 10
9->7	7 8 9
8->8	8



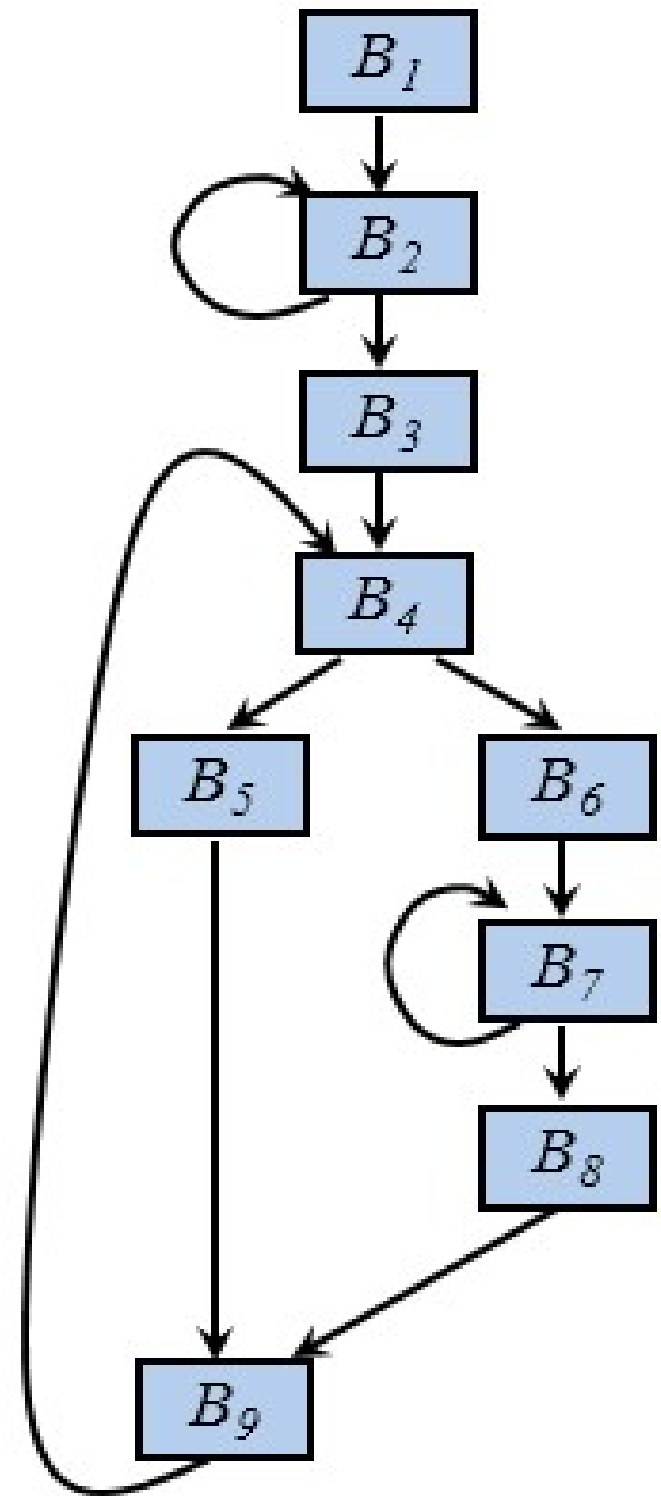
习题19.10

➤ 对下图中的流图：

➤ (1) 计算支配关系

➤ (2) 构造支配节点树

➤ (3) 找出这个流图的自然循环

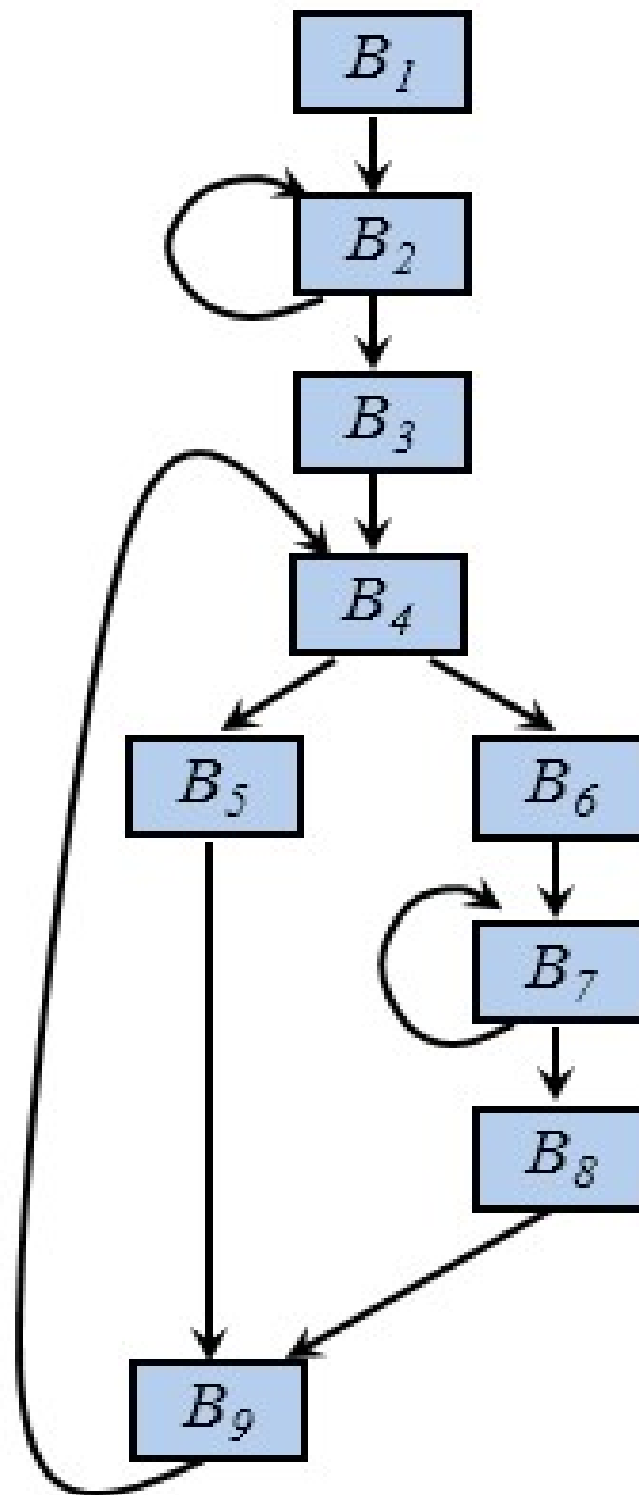




计算支配关系

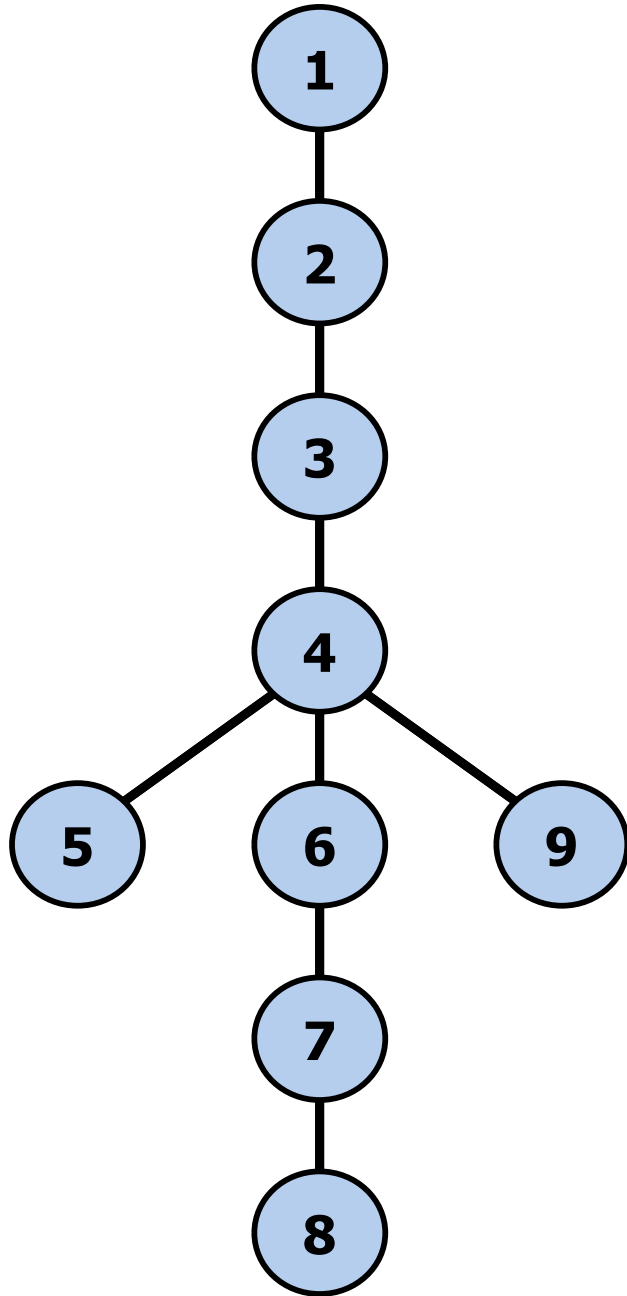
➤ 支配关系

支配节点	支配对象
1	1~9
2	2~9
3	3~9
4	4~9
5	5
6	6~8
7	7 8
8	8
9	9





构造支配节点树



支配节点	支配对象
1	1~9
2	2~9
3	3~9
4	4~9
5	5
6	6~8
7	7 8
8	8
9	9



找出自然循环

➤至此，找出了流图中所有自然循环

回边	自然循环
$2 \rightarrow 2$	2
$7 \rightarrow 7$	7
$9 \rightarrow 4$	4, 5, 6, 7, 8, 9

